

Ingeniería del Software II

Pedro R. D'Argenio

Sobre esta materia

- En esta materia estudiaremos varias técnicas de análisis (automático o asistido) de software. Algunas de las técnicas también son aplicables al análisis de hardware. Estas técnicas involucran, entre otras cosas, la especificación, verificación y validación de sistemas de software/hardware.
- Principalmente, estudiaremos dos técnicas automáticas importantes, model checking y deducción semántica a través de SAT solving y técnicas formales para la derivación de test.
- El curso será teórico-práctico, y experimentaremos en el uso de herramientas (LTSA y Alloy, fundamentalmente).

Burocráticas

- 👁 Horarios:

- 👁 Teóricos: Martes y Jueves, 9:00-11:00
- 👁 Prácticos: Martes y Jueves, 11:00-13:00

- 👁 Acceso virtual:

- 👁 Aula Virtual: <https://famaf.aulavirtual.unc.edu.ar/>
- 👁 Zulip: #ingsoftii

- 👁 La materia se evaluará según:

- 👁 **1 trabajo práctico obligatorio** en grupos de a 3 personas que involucra varias etapas.
- 👁 **2 evaluaciones parciales** sin recuperatorio.

Burocráticas

👁 Horarios:

- 👁 Teóricos: Martes y Jueves, 9:00-11:00
- 👁 Prácticos: Martes y Jueves, 11:00-13:00

- ❖ Todo el material de la materia
- ❖ Anuncio de eventos importantes
- ❖ Consultas y foro
- ❖ Videos grabados durante ASPO (2020 y 2021)

👁 Acceso virtual:

- 👁 Aula Virtual: <https://famaf.aulavirtual.unc.edu.ar/>
- 👁 Zulip: #ingsoftii

- ❖ Anuncios
- ❖ Consultas
- ❖ Foro de discusión

👁 La materia se evaluará según:

- 👁 **1 trabajo práctico obligatorio** en grupos de a 3 personas que involucra varias etapas.
- 👁 **2 evaluaciones parciales** sin recuperatorio.

IMPORTANTE

La dirección de email asociada al aula virtual debe ser una dirección **activa** y chequeada regularmente.

Toda información importante será comunicada a través de avisos del aula virtual, si es información general, y a esas direcciones de email si es información particular

Burocráticas

De acuerdo a ello se pueden conseguir alguno de los siguientes estados:

- **Promoción:** De acuerdo a lo establecido en la Ordenanza 4/2011, para obtener la promoción, el alumno deberá:
 - aprobar todas las evaluaciones parciales con una nota no menor a 6 (seis), y obteniendo un promedio no menor a 7 (siete),
 - aprobar todos los Trabajos Prácticos.
- **Regular:** De acuerdo a lo establecido en la Ordenanza 4/2011, para obtener la regularidad, el alumno deberá:
 - aprobar al menos el 60 % de los Trabajos Prácticos o de Laboratorio.

Burocráticas

De acuerdo a ello se pueden conseguir algunos estados:

Calificación final:
 $(P1+P2+TP)/3$

- **Promoción:** De acuerdo a lo establecido en la Ordenanza 4/2011, para obtener la promoción, el alumno deberá:
 - aprobar todas las evaluaciones parciales con una nota no menor a 6 (seis), y obteniendo un promedio no menor a 7 (siete),
 - aprobar todos los Trabajos Prácticos.
- **Regular:** De acuerdo a lo establecido en la Ordenanza 4/2011, para obtener la regularidad, el alumno deberá:
 - aprobar al menos el 60 % de los Trabajos Prácticos o de Laboratorio.

Aprobar el TP
con 6 o más

Charlemos fechas

Dom	Lun	Mar	Mie	Jue	Vie	Sab
23	24	25	26	27	28	29
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21

Marzo	Abril	Mayo	Junio
-------	-------	------	-------

Propuesta Proyecto

Elección Herramienta

Definición Herramienta

1er Parcial

Informe Preliminar

Presentaciones

Revisiones

2do Parcial

Informe Final

Bibliografía

Libros:

- Jeff Magee & Jeff Kramer. Concurrency: State Models & Java Programs (2nd edition). Wiley. 2006.
- Christel Baier and Joost-Pieter Katoen. Principles of Model Checking. The MIT Press. May 2008.
- Daniel Jackson. Software Abstractions: Logic, Language, and Analysis. The MIT Press. 2006.
- A. Biere, M. Heule, H. van Maaren, y T. Walsh (eds.). Handbook of Satisfiability. 2nd edition. IOS press, 2021.
- Aaron R. Bradley and Zohar Manna. The Calculus of Computation: Decision Procedures with Applications to Verification. Springer, 2007.

Bibliografía

Artículos (lista incompleta):

- B. Alpern & F. Schneider. Defining Liveness. Information Processing Letter 21:181-185. 1985.
- B. Alpern & F. Schneider. Recognizing Safety and Liveness. Distributed Computing 2(3):117-126. 1987.
- Markus Müller-Olm, David Schmidt, y Bernhard Steffen. Model-Checking: A Tutorial Introduction. En A. Cortesi, G. Filé (Eds.): SAS99, LNCS 1694, pp. 330354. Springer-Verlag 1999.

El problema de la corrección del software

Poder garantizar la corrección del software que construimos es una tarea deseable. En algunas aplicaciones, es, sin duda, crucial:

- Software para equipamiento médico
- Software para el control de vehículos
- Software para el control de procesos industriales
- (para algunos, algunas aplicaciones financieras)
- °

Estos sistemas, cuyas fallas pueden ocasionar daños de gran importancia -- incluyendo la pérdidas de vidas humanas, catástrofes ecológicas, grandes pérdidas financieras, etc. -- se denominan **sistemas críticos**.

El problema de la corrección del software

Dado que los sistemas críticos que construimos es fundamentalmente responden a comportamientos reactivos, concurrentes y/o de tiempo real, las técnicas que veremos se aplican a una gama de sistemas mucho más amplia, incluyendo, por ejemplo, sistemas de servicios web.

- (para algunos, algunas aplicaciones industriales)
- °

Estos sistemas, cuyas importancia -- incluyendo catástrofes ecológicas, grandes pérdidas financieras, etc. -- se denominan **sistemas críticos**.

Las técnicas que estudiaremos son particularmente importantes para este tipo de sistemas.

Ingeniería del Software II

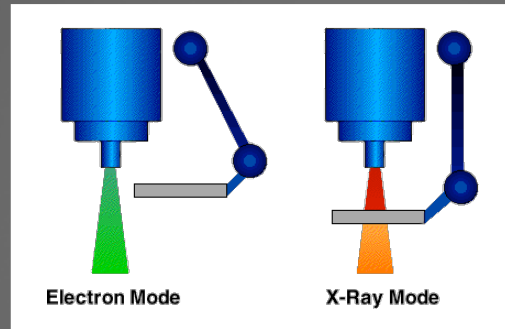
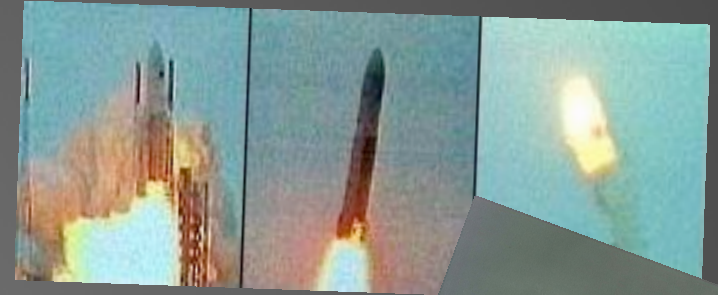
1 - Introducción

Algunos Ejemplos



Pentium:
FDIV

Ariane 5:
64 bits fp
vs 16 bits int

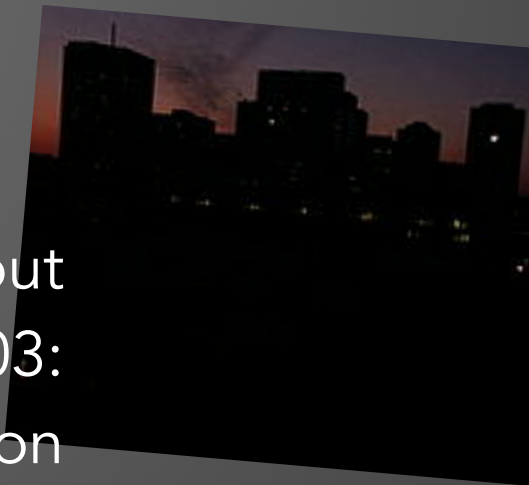


Therac-25:
Race condition



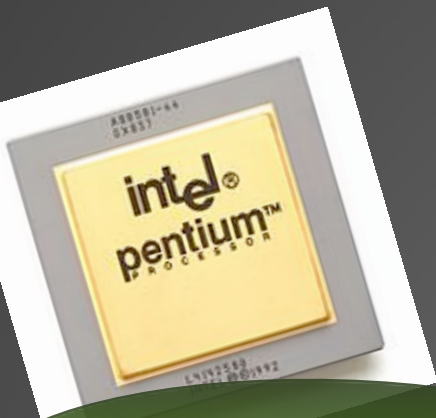
Mars Climate Orbiter:
Métrico vs Imperial

Northeast blackout
in 2003:
Race condition



Heartbleed
(Open SSL):
Buffer over-read

Algunos Ejemplos



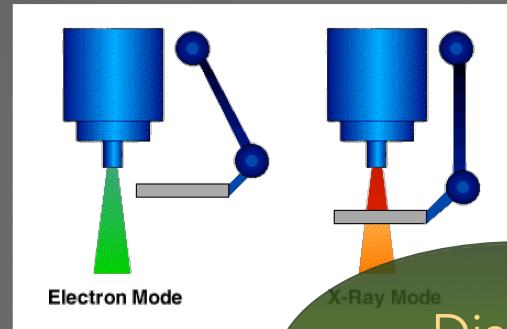
Pentium:
FDIV

$$\frac{4195835 * 3145727}{3145727} = 4195579$$

Ariane 5:
64 bits fp
vs 16 bits int



Integración



Therac-25:

Race condition

Diseño/
implementación

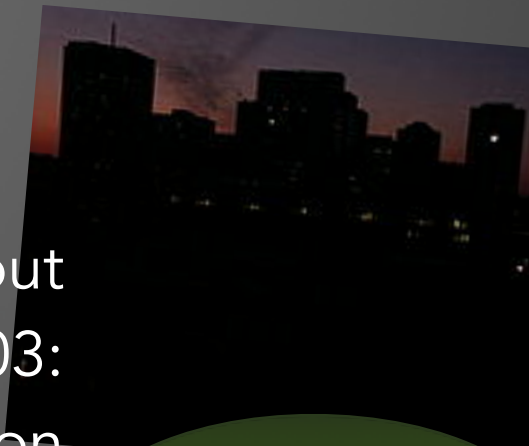
Mars Climate Orbiter:
Métrico vs Imperial



Integración

Northeast blackout
in 2003:

Race condition



Diseño/
implementación



Heartbleed
(Open SSL):
Buffer over-read

Implementación

Algunos Ejemplos

Pentium:

Ariane 5:
64 bits fo

Integración

- ❖ 1985: <https://en.wikipedia.org/wiki/Therac-25>
- ❖ 1994: https://en.wikipedia.org/wiki/Pentium_FDIV_bug
- ❖ 1996: https://en.wikipedia.org/wiki/Ariane_5_Flight_501
- ❖ 1999: https://en.wikipedia.org/wiki/Mars_Climate_Orbiter
- ❖ 2003: https://en.wikipedia.org/wiki/Northeast_blackout_of_2003
- ❖ 2014: <https://en.wikipedia.org/wiki/Heartbleed>

Implementación

Buffer over-read

Diseño/
implementación

Algunos Ejemplos



911 blackout:
MAX value
reached

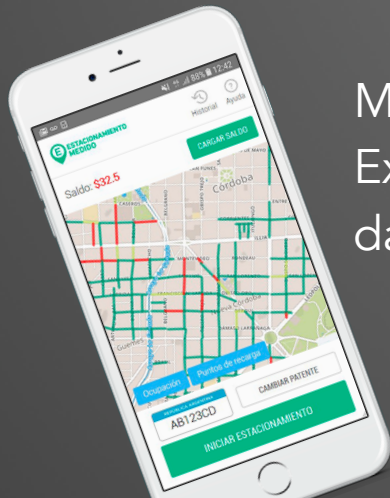
Nest Thermostat:
Drenado de
batería



Nissan airbag:
Sensado
incorrecto



Boeing 737 MAX 8:
Sensado incorrecto



Movypark:
Exposición de
datos personales

Tesla/Uber/Google
self-driving car:
aprendizaje con
limitaciones??



- ❖ <https://www.washingtonpost.com/news/the-switch/wp/2014/10/20/how-a-dumb-software-glitch-kept-6600-calls-from-getting-to-911/>
- ❖ <https://www.theverge.com/2014/10/20/7014705/coding-error-911-fcc-washington>
- ❖ <https://spectrum.ieee.org/nissan-recalls-nearly-1-million-cars-for-airbag-software-fix>
- ❖ <https://www.bbc.com/news/technology-35311447>
- ❖ <https://news.sophos.com/en-us/2016/01/18/nest-smart-thermostat-glitch-leaves-cold-feet-and-steaming-mad-customers/>
- ❖ <https://www.infobae.com/tecnologia/2019/10/22/una-falla-en-una-app-de-estacionamiento-medido-expuso-los-datos-de-miles-de-usuarios/>
- ❖ https://eldoce.tv/sociedad/exponen-datos-personales-traves-de-movypark-privacidad-error-falla-cordoba-empresa-estacionamiento-medido_91292/
- ❖ https://en.wikipedia.org/wiki/Boeing_737_MAX_groundings
- ❖ https://en.wikipedia.org/wiki/Self-driving_car#Incidents
- ❖ <https://www.craftlawfirm.com/autonomous-vehicle-accidents-2019-2024-crash-data/>
- ❖ <https://caraccidentattorney.com/blog/self-driving-car-accident-statistics/>
- ❖ <https://www.youtube.com/watch?v=E531GxfEoB8>
- ❖ <https://www.washingtonpost.com/technology/2023/06/10/tesla-autopilot-crashes-elon-musk/>
- ❖ <https://prospect.org/infrastructure/transportation/2023-06-14-elon-musk-tesla-self-driving-bloodbath/>
- ❖ <https://x.com/bdjukic/status/1740441354378051837>

Algunos Ejemplos



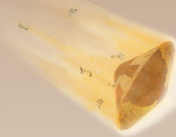
Schiaparelli enters atmosphere

Time: 0 sec
Altitude: 121 km
Speed: 21 000 km/h



Heatshield protection during atmospheric deceleration

Time of maximum heating: 1 m 12 sec
Altitude: 45 km
Speed: 19 000 km/h



Parachute deploys

Time: 3 m 21 sec
Altitude: 11 km
Speed: 1700 km/h



Front shield separates, radar turns on

Time: 4 m 1 sec
Altitude: 7 km
Speed: 320 km/h



Parachute jettisoned with rear cover

Time: 5 m 22 sec
Altitude: 1.2 km
Speed: 240 km/h



Thruster ignition

Time: 5 m 23 sec
Altitude: 1.1 km
Speed: 250 km/h



Thrusters off; freefall

Time: 5 min 52 sec
Altitude: 2 m
Speed: 4 km/h



Touchdown

Time: 5 min 53 sec
Altitude: 0 m
Speed: 10 km/h



Algunos Ejempl

WTF!!!!



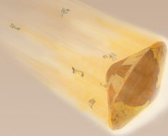
Schiaparelli enters atmosphere

Time: 0 sec
Altitude: 121 km
Speed: 21 000 km/h



Heatshield protection during atmospheric deceleration

Time of maximum heating: 1 m 12 sec
Altitude: 45 km
Speed: 19 000 km/h



Parachute deploys

Time: 3 m 21 sec
Altitude: 11 km
Speed: 1700 km/h



Front shield separates, radar turns on

Time: 4 m 1 sec
Altitude: 7 km
Speed: 320 km/h



30 segundos antes

Parachute detaches, rear cover

Time: 5 m 22 sec
Altitude: 1.2 km
Speed: 240 km/h

Thruster ignition

Time: 5 m 23 sec
Altitude: 1.1 km
Speed: 250 km/h

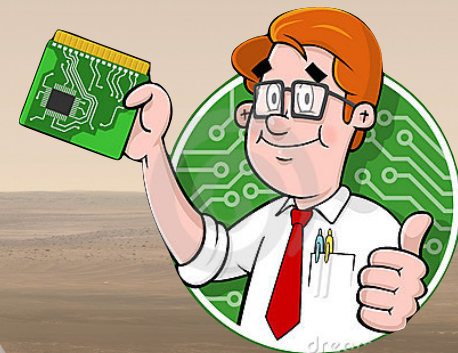
3 segundos en lugar de 30

Thrusters off; freefall

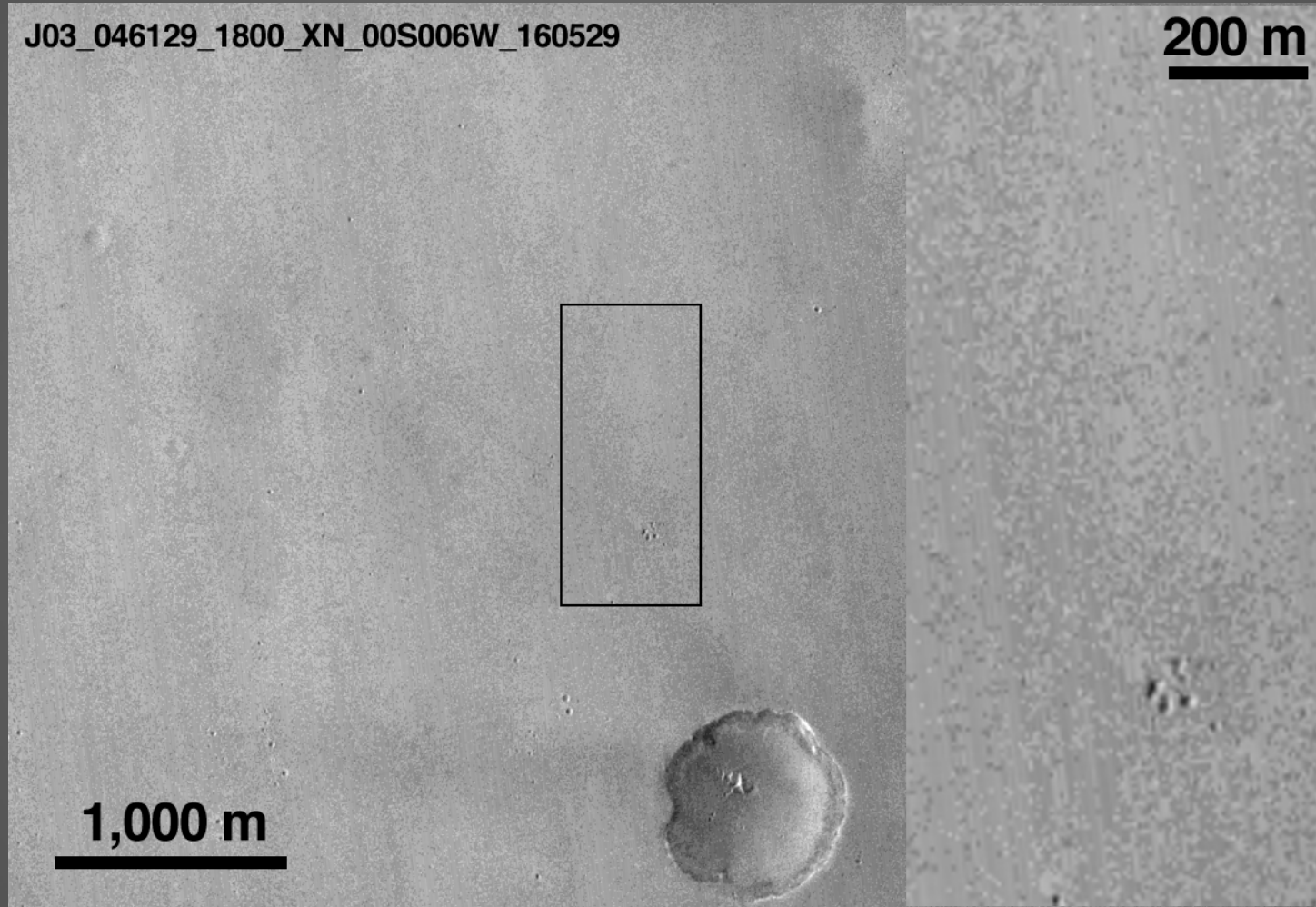
Time: 5 min 52 sec
Altitude: 2 m
Speed: 4 km/h

Touchdown

Time: 5 min 53 sec
Altitude: 0 m
Speed: 10 km/h

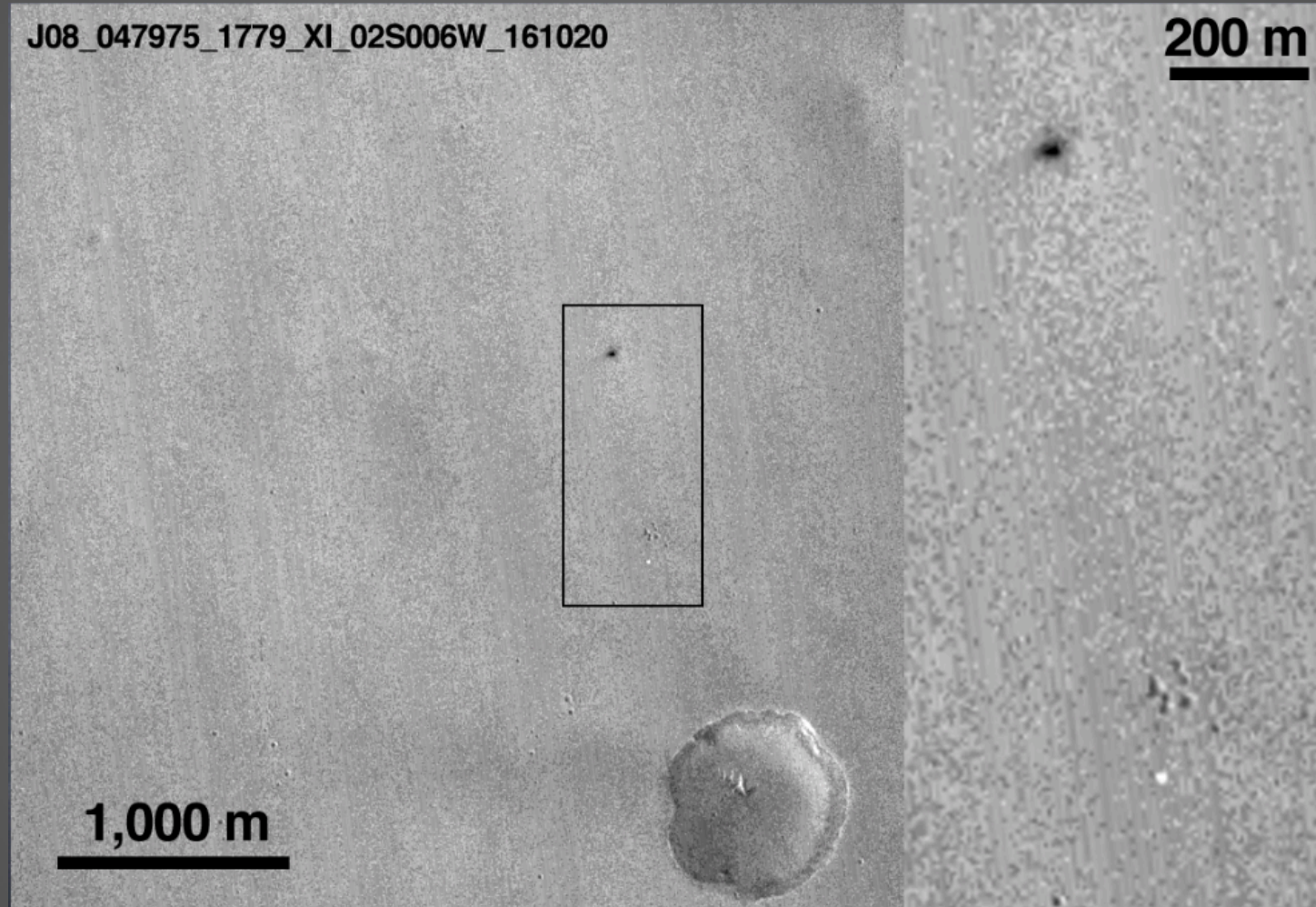


Algunos Ejemplos



https://en.wikipedia.org/wiki/Schiaparelli_EDM_lander

Algunos Ejemplos



https://en.wikipedia.org/wiki/Schiaparelli_EDM_lander

Algunos Ejemplos

Muchos más bugs en:

- The risk digest: <http://catless.ncl.ac.uk/Risks/>
- Otros:
 - <http://www.cs.tau.ac.il/~nachumd/verify/horror.html>
 - <http://www5.in.tum.de/~huckle/bugse.html>
 - http://en.wikipedia.org/wiki/List_of_notable_software_bugs

Limitaciones del testing y la simulación

- Tanto el testing como la simulación involucran experimentos previos al lanzamiento o uso masivo del software. En general, ambos métodos proveen una serie de entradas al software, y estudian el comportamiento del mismo en esos casos.
- El testing y la simulación raramente permiten garantizar la ausencia de errores. Una frase famosa al respecto: "El testing puede confirmar la presencia de errores pero nunca garantizar su ausencia".

Limitaciones del testing y la simulación

- Tanto el testing como la simulación involucran experimentos previos al lanzamiento o uso masivo del software. En general, ambos métodos proveen una serie de entradas al software, y estudian el comportamiento del mismo en esos casos.
- El testing y la simulación raramente permiten garantizar la ausencia de errores. Una frase famosa al respecto: "El testing puede confirmar la presencia de errores pero nunca garantizar su ausencia".

Es importante saber que distintos métodos dan distintas garantías y es importante saber cuáles son.

Mmmhhh...
La verificación puede hacerlo pero en teoría

Verificación (semi)automática de software

- Existen serias limitaciones en lo que respecta a la verificación automática de software. Por ejemplo, el problema de decidir si un programa dado termina o no **no es computable**.
- Sin embargo, si imponemos algunas restricciones sobre las propiedades que queremos verificar, y sobre qué sistemas, algunas tareas pueden realizarse automáticamente. Model checking es un ejemplo de esto (que estudiaremos en más detalle más adelante).

Verificación Deductiva de Programas

La verificación de programas es la tarea de comprobar matemáticamente la corrección de un programa con respecto a su especificación. En el caso de programas secuenciales, existe desde hace varias décadas una técnica propuesta por Floyd y Hoare, basada en la especificación con aserciones en primer orden, y la visión de programas con aserciones como fórmulas de una lógica:

$$\left. \begin{array}{l} \{\text{Precondición}\} \\ \text{Programa} \\ \{\text{Postcondición}\} \end{array} \right\} \text{ Corrección total}$$

Esta “fórmula” indica que, bajo cualquier estado que satisfaga la precondición inicialmente, la ejecución del programa termina, y lo hace en un estado que satisface la postcondición.

Ejemplos

```
{ $x = X$  &  $y = Y$ }  
  aux := x;  
  x := y;  
  y := aux;  
{ $x = Y$  &  $y = X$ }
```

```
{true}  
  i := 1;  
  while (i ≤ length(A)) do  
    j := i;  
    while (j > 1) do  
      if (A[j-1] > A[j]) then  
        swap(j-1, j)  
      j := j-1  
    end  
    i := i+1  
  end  
{ $\forall x \in [1.. \text{length}(A) - 1] : A[x] \leq A[x + 1]$ }
```

Programas como aserciones lógicas

Asignación

$$\frac{}{\{\psi[E/x]\} \quad x := E \quad \{\psi\}}$$

Composición en secuencia

$$\frac{\{\phi\} \quad p_1 \quad \{\eta\} \quad \{\eta\} \quad p_2 \quad \{\psi\}}{\{\phi\} \quad p_1; p_2 \quad \{\psi\}}$$

Ejecución condicional

$$\frac{\{\phi \wedge B\} \quad p_1 \quad \{\psi\} \quad \{\phi \wedge \neg B\} \quad p_2 \quad \{\psi\}}{\{\phi\} \quad \text{if } B \text{ then } p_1 \text{ else } p_2 \text{ fi} \quad \{\psi\}}$$

Iteración

$$\frac{\{\phi \wedge B\} \quad p \quad \{\phi\}}{\{\phi\} \quad \text{while } B \text{ do } p \text{ od} \quad \{\phi \wedge \neg B\}}$$

Consecuencia

$$\frac{\phi' \Rightarrow \phi \quad \{\phi\} \quad p \quad \{\psi\} \quad \psi \Rightarrow \psi'}{\{\phi'\} \quad p \quad \{\psi'\}}$$

Programas como aserciones lógicas

Regla de inferencia para iteración:

La regla de inferencia para razonar sobre programas con iteración es lo que hace interesante al sistema de inferencia. Esta regla involucra nociones importantes, como las nociones de *invariante de ciclo* Inv y *función cota* t :

Si se cumple:

- (1) $\{\phi\} \text{ Init } \{Inv\}$
- (2) $\{C \wedge Inv\} \text{ CC } \{Inv\}$
- (3) $\neg C \wedge Inv \Rightarrow \psi$
- (4) $\{t = T\} \text{ CC } \{t < T\}$
- (5) $C \Rightarrow t > 0$

entonces:

```
{  $\phi$  }  
Init  
while  $C$  do  
    CC  
od  
{  $\psi$  }
```


Características de la verificación usando lógica de Hoare

- Puede extenderse a programas concurrentes (Owicki-Gries).
- No puede automatizarse (invariantes y funciones cotas son “mágicas”).
- Introduce conceptos de gran importancia (aserciones, especificaciones, invariantes, etc.) que influyen en lenguajes, metodologías de programación, diseño de compiladores, etc.

Ingeniería del Software II

2 - Programación concurrente

Programación concurrente

• ¿Qué?

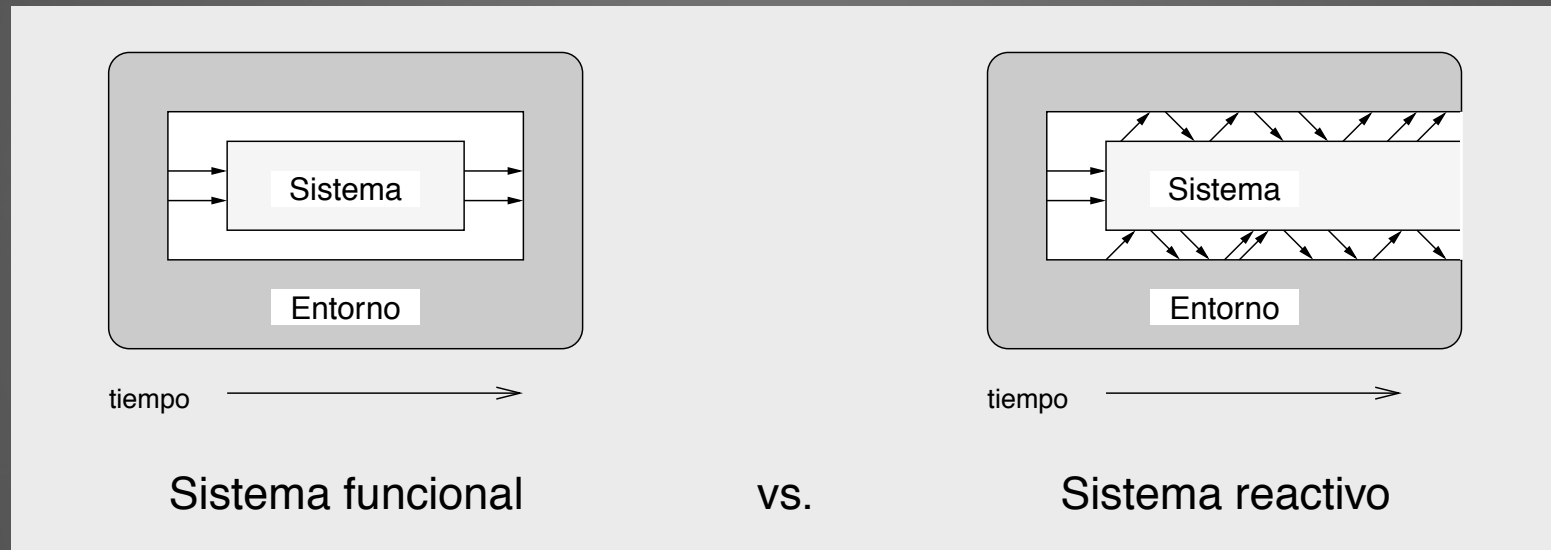
- Programación de sistemas compuestos de varios procesos que se ejecutan de manera superpuesta en un período de tiempo e interactúan entre sí.

• ¿Por qué?

- Interacción con el entorno con el fin de observar/controlar dispositivos físicos (ej: dispositivos periféricos de una computadora)
- Mejorar el tiempo de respuesta en la interacción con el usuario (ej: múltiples procesos en un desktop)
- Mejorar velocidad de cálculo
- Mejorar uso de recursos
- Comunicación
- ...

Características reactivas de los programas concurrentes

Muchos programas concurrentes suelen ser reactivos, es decir, su funcionalidad involucra la interacción permanente con el ambiente (y otros procesos).



Los sistemas reactivos tienen características diferentes a las de los programas convencionales. En muchos casos, éstos no computan resultados, y suele no requerirse que terminen.

Ejemplos: sistemas operativos, software de control, hardware, etc.

Interacción de programas concurrentes

- Los programas concurrentes están compuestos por procesos (o threads, o componentes) que necesitan interactuar. Existen varios mecanismos de interacción entre procesos. Entre éstos se encuentran la memoria compartida y el pasaje de mensajes.
- Además, los programas concurrentes deben, en general, colaborar para llegar a un objetivo común, para lo cual la sincronización entre procesos es crucial.

Algunos problemas comunes de los programas concurrentes

- **Violación de propiedades universales** (invariantes)
- **Starvation** (inanición): Uno o más procesos quedan esperando indefinidamente un mensaje o la liberación de un recurso
- **Deadlock**: dos o más procesos esperan mutuamente el avance del otro
- **Problemas de uso no exclusivo** de recursos compartidos
- **Livelock**: Dos o más procesos no pueden avanzar en su ejecución porque continuamente responden a los cambios en el estado de otros procesos

Concurrencia: un ejemplo

```
int y1 = 0;  
int y2 = 0;  
short in_critical = 0;
```

```
active proctype process_1() {  
    do  
        :: true ->  
            y1 = y2+1;  
            ((y2==0) || (y1<=y2));  
            in_critical++;  
            in_critical--;  
            y1 = 0;  
    od  
}
```

```
active proctype process_2() {  
    do  
        :: true ->  
            y2 = y1+1;  
            ((y1==0) || (y2<y1));  
            in_critical++;  
            in_critical--;  
            y2 = 0;  
    od  
}
```

¿Qué hace este programa?

Concurrencia: un ejemplo

Se bloquea
hasta que la aserción
se haga verdadera

```
int y1 = 0;
int y2 = 0;
short in_critical = 0;
```

```
active proctype process_1() {
  do
    :: true ->
      y1 = y2+1;
      ((y2==0) || (y1<=y2));
      in_critical++;
      in_critical--;
      y1 = 0;
  od
}
```

```
active proctype process_2() {
  do
    :: true ->
      y2 = y1+1;
      ((y1==0) || (y2<y1));
      in_critical++;
      in_critical--;
      y2 = 0;
  od
}
```

¿Qué hace este programa?

¿Garantiza
exclusión mutua?

Semántica de programas concurrentes

Una semántica típica para programas concurrentes está basada en **sistemas de transición de estados**. Un sistema de transición de estados es un grafo dirigido en el cual:

- los **nodos** son los estados del sistema (posiblemente infinitos estados)
- las **aristas** son las transiciones atómicas de estados en estados, dadas por las sentencias del sistema.
- un nodo distinguido que reconoceremos como el **estado inicial**.

Semántica de programas concurrentes

Una semántica típica para programas concurrentes está basada en **sistemas de transición de estados**. Un sistema de transición de estados es un grafo dirigido en el cual:

$(S, s_0, \rightarrow)$

↓
Relación de
transición

↓
Estado inicial

↓
Conjunto de estados

Ejemplo

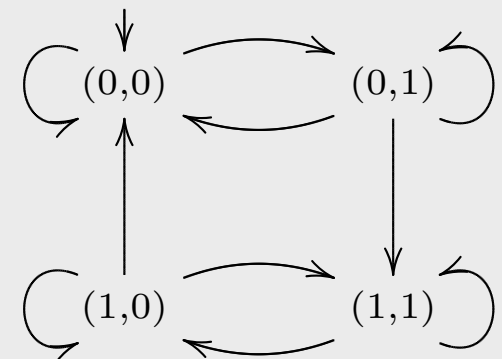
$$S = \{0, 1\} \times \{0, 1\}$$

$$s_0 = (0, 0)$$

$$(x, y) \rightarrow (x, 0)$$

$$(x, y) \rightarrow (x, 1)$$

$$(x, y) \rightarrow (y, y)$$

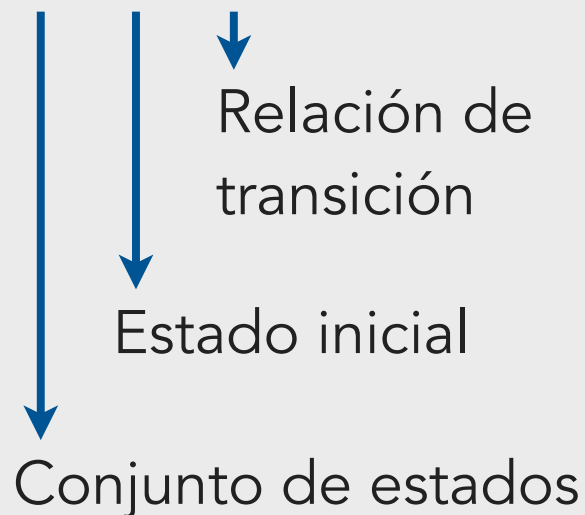


Semántica de programas concurrentes

Una semántica típica para programas concurrentes está basada en **sistema de transición de estados**. Un sistema de transición de estados es un grafo dirigido en el cual:

Muchas veces la transición aparece etiquetada con el evento que la origina.

$(S, s_0, \rightarrow)$



Ejemplo

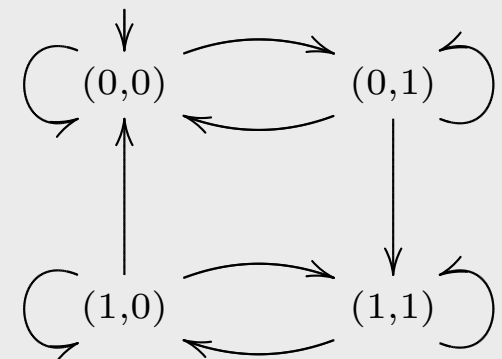
$$S = \{0, 1\} \times \{0, 1\}$$

$$s_0 = (0, 0)$$

$$(x, y) \rightarrow (x, 0)$$

$$(x, y) \rightarrow (x, 1)$$

$$(x, y) \rightarrow (y, y)$$



Ejecuciones de un programa

Una **ejecución** es una secuencia de estados $s_0s_1s_2\cdots$, tal que:

- s_0 es el estado inicial,
- puede llegarse desde s_i a s_{i+1} por alguna sentencia atómica (i.e. transición) del sistema.

El conjunto de todas las ejecuciones determina el comportamiento de un programa concurrente modelado con un sistema de transición de estados.



$(0, 0)(0, 0)(0, 1)(1, 1)(1, 0)(1, 1)(1, 0)(1, 0)(1, 1)(1, 0)(0, 0)(0, 1)(0, 1)(1, 1) \cdots$



$(0, 1)(0, 0)(0, 1)(1, 1)(1, 0)(1, 1)(1, 0)(1, 0)(1, 1)(1, 0)(0, 0)(0, 1)(0, 1)(1, 1) \cdots$

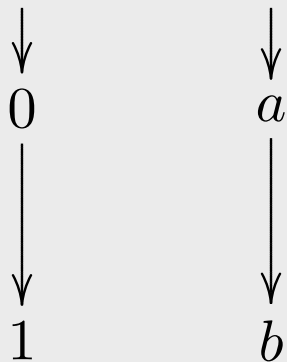


$(0, 0)(0, 1)(1, 0)(1, 1)(0, 0)(1, 1)(1, 0)(1, 0)(1, 1)(1, 0)(0, 0)(1, 1)(0, 1)(1, 1) \cdots$

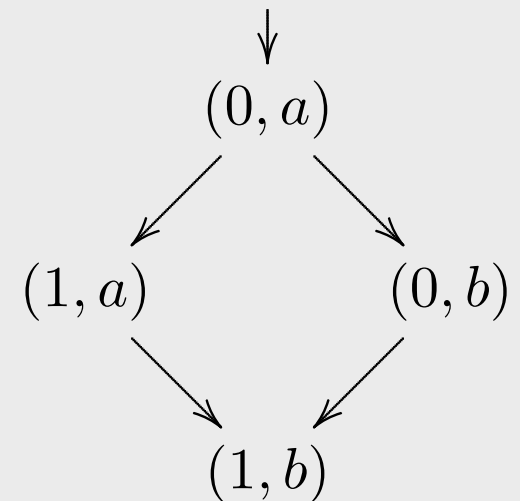
¿Cómo se ejecutan los procesos concurrentes?

De acuerdo al modelo computacional descrito, los procesos concurrentes se ejecutan **intercalando** las acciones atómicas que los componen. Llamamos a esto, **interleaving**.

El orden en que se ejecutan las acciones atómicas no puede decidirse en general, y un mismo par de procesos puede tener diferentes ejecuciones debido al **no determinismo** en la elección de las acciones atómicas a ejecutar.



La composición paralela de los STE de la izquierda, da como resultado el STE de la derecha.



Volviendo al ejemplo...

Volviendo a nuestro ejemplo anterior, el conjunto de estados está dado por la combinación de todos los valores posibles de las variables globales `y1`, `y2` e `in_critical`, además de dos variables implícitas: los `program counters` (`pc`) de los dos procesos.

El estado inicial es aquel en el cual las tres variables valen 0 y cada `pc` se sitúa al inicio de cada proceso.

Por cada sentencia atómica tenemos una transición. Por ejemplo, la sentencia

`in_critical--`

define un conjunto de transiciones donde cada una relaciona todos los estados con aquellos en los cuales `y1` e `y2` no cambian su valor, e `in_critical` se decrementa en uno (y el `pc` del proceso correspondiente también se incrementa).

```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

```

```

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

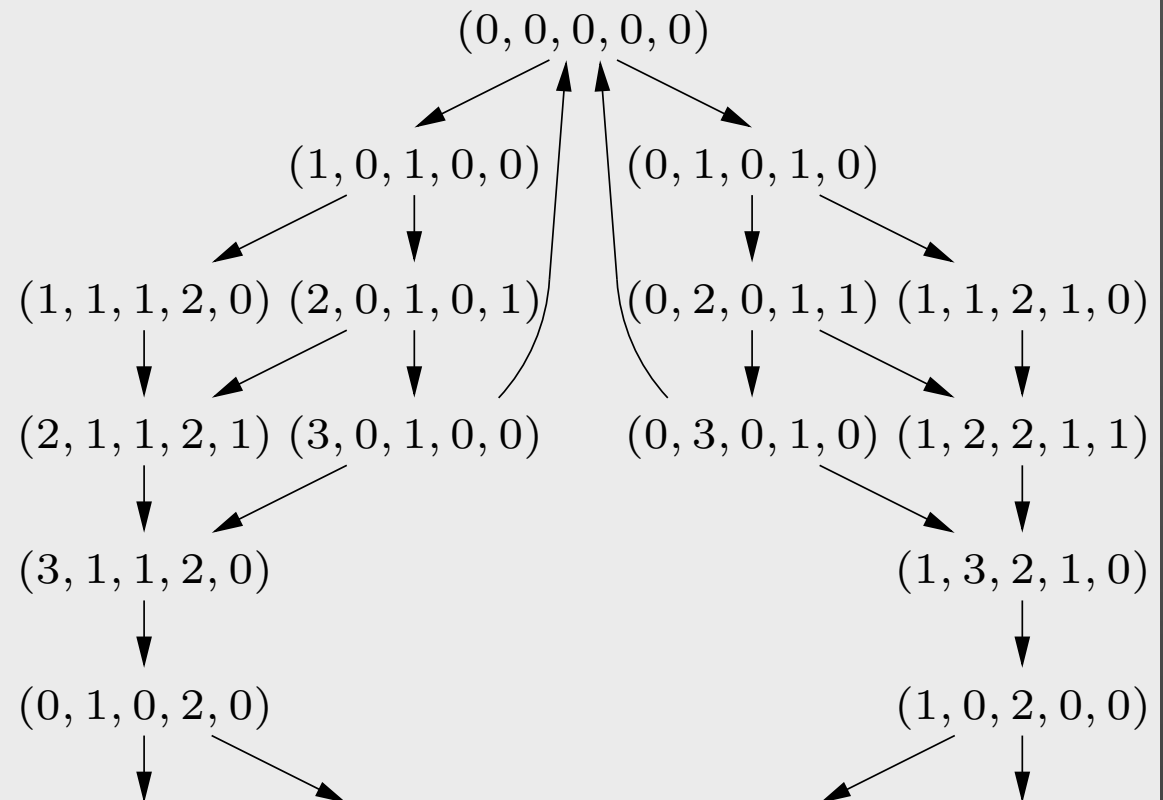
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

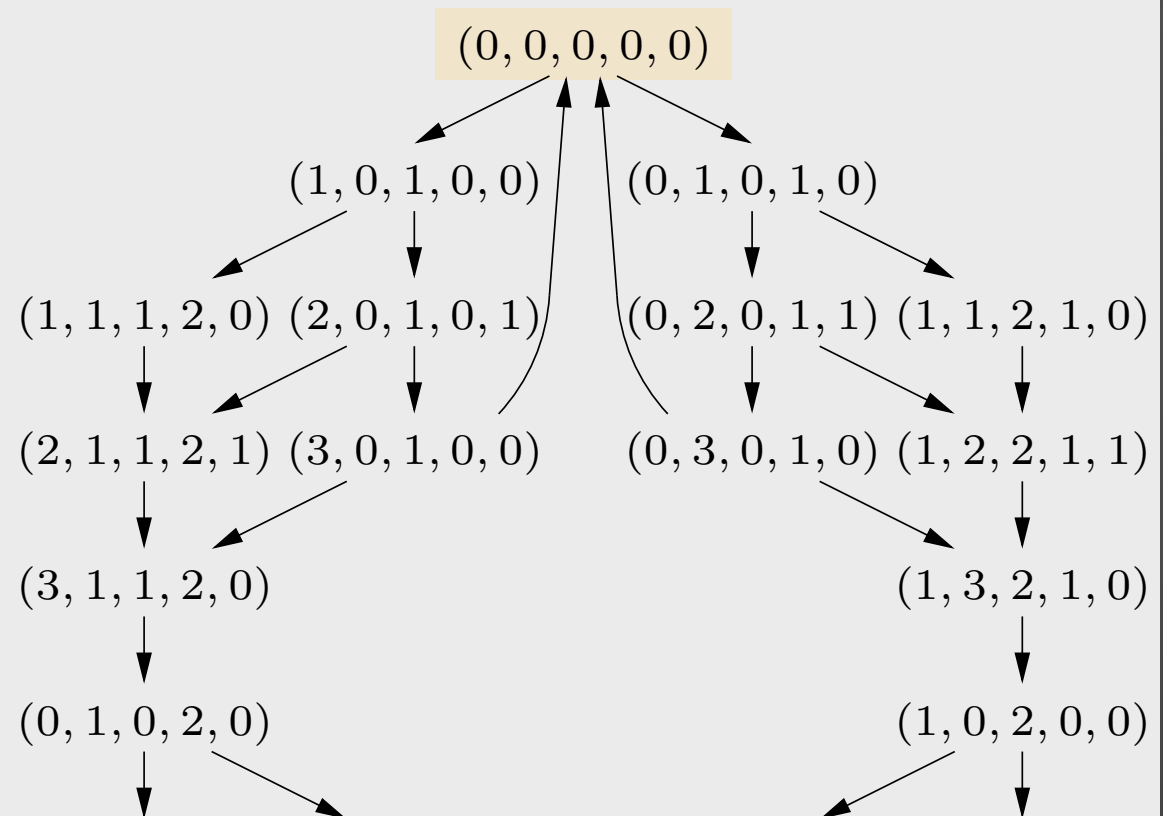
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

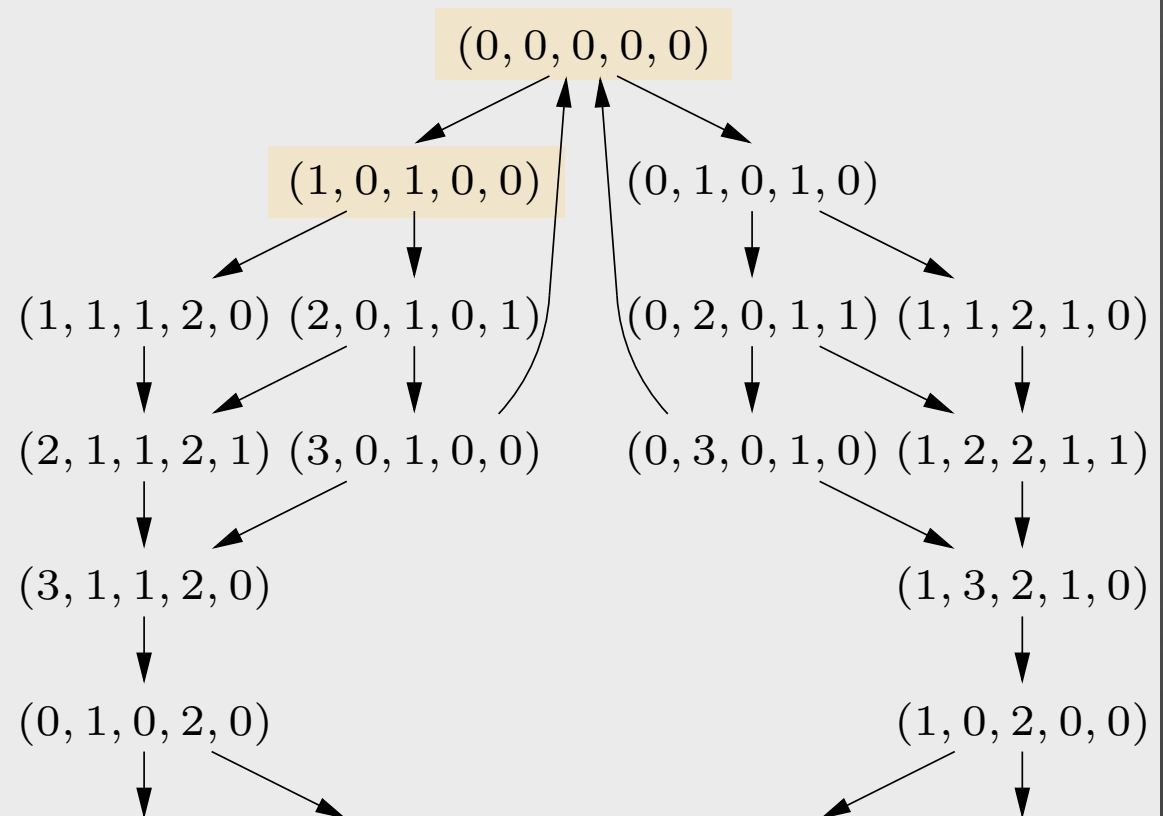
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

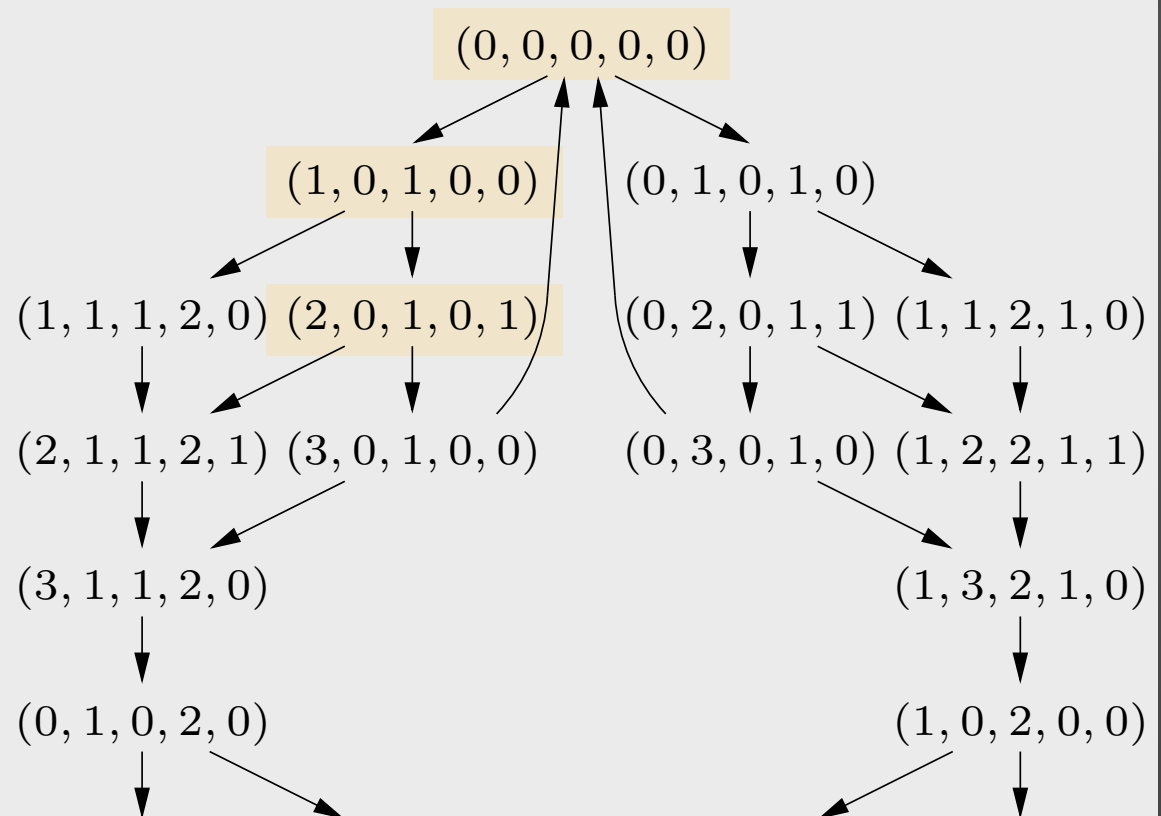
```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```

Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
  do
    :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
  od
}

```

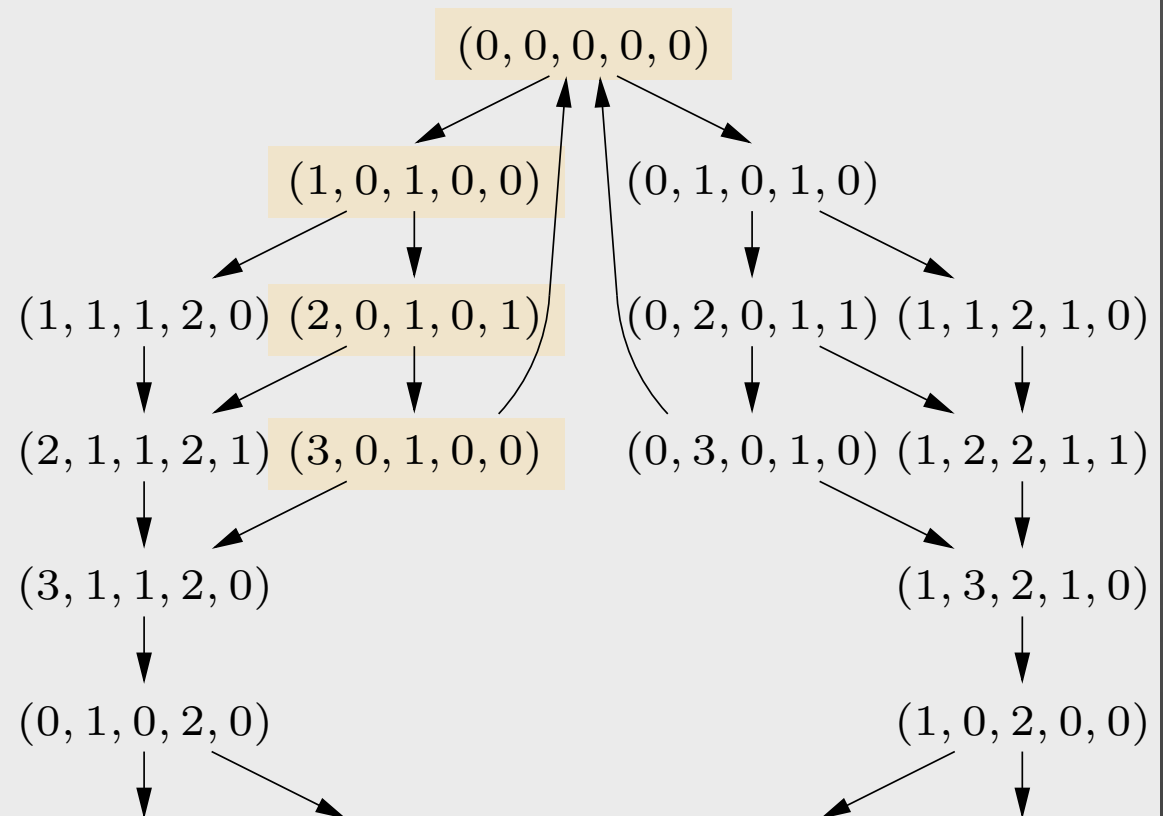
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
  do
    :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
  od
}

```




```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

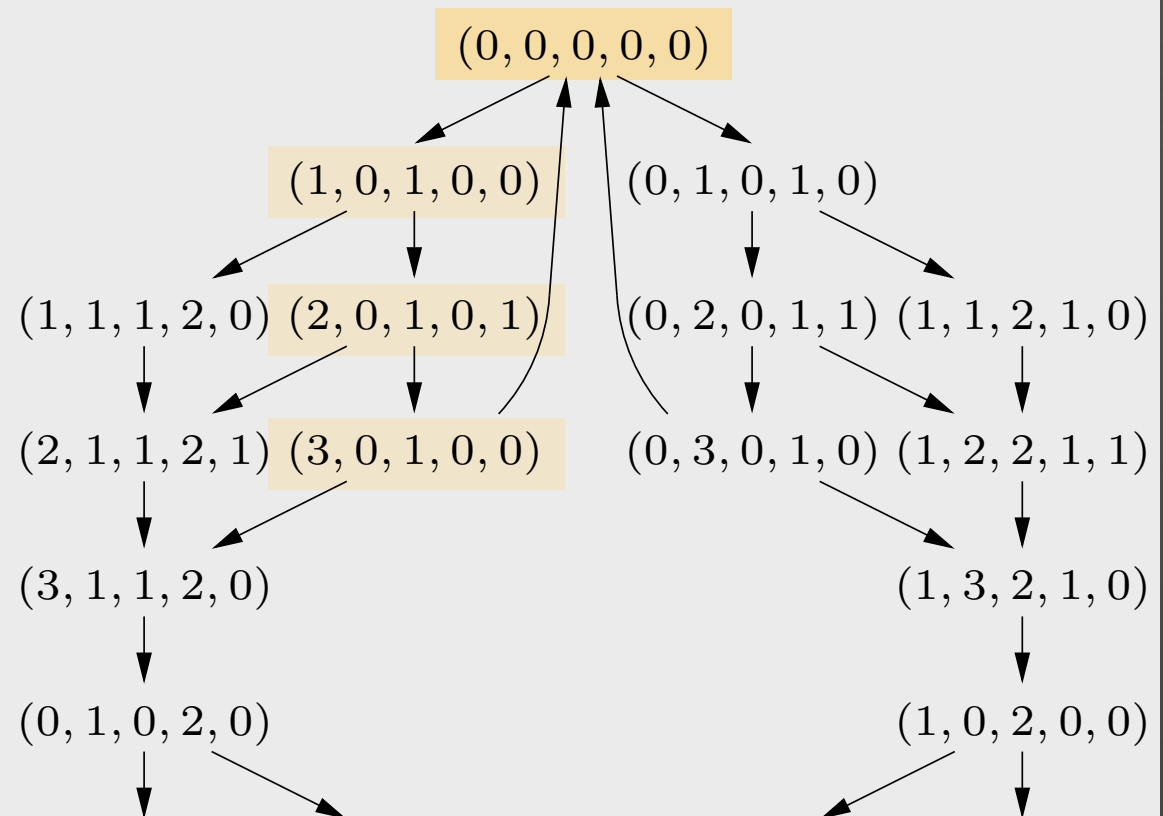
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

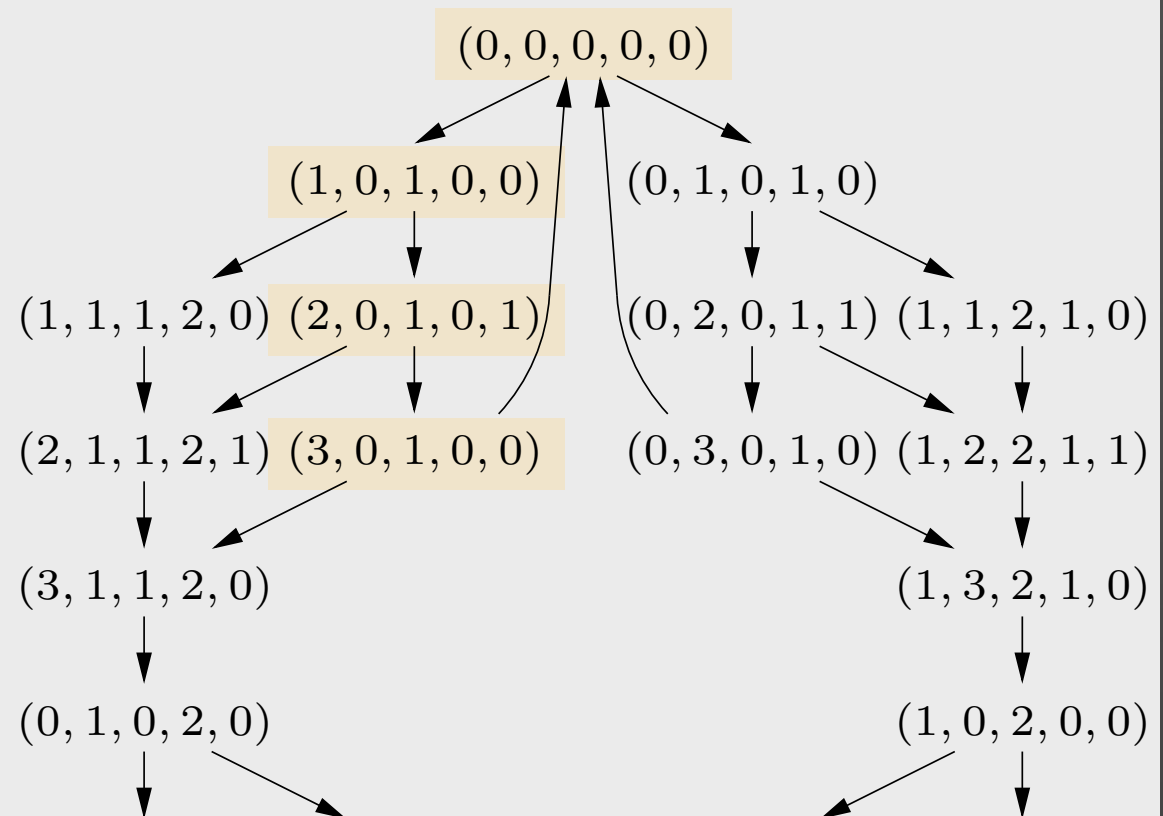
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
  do
    :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
  od
}

```

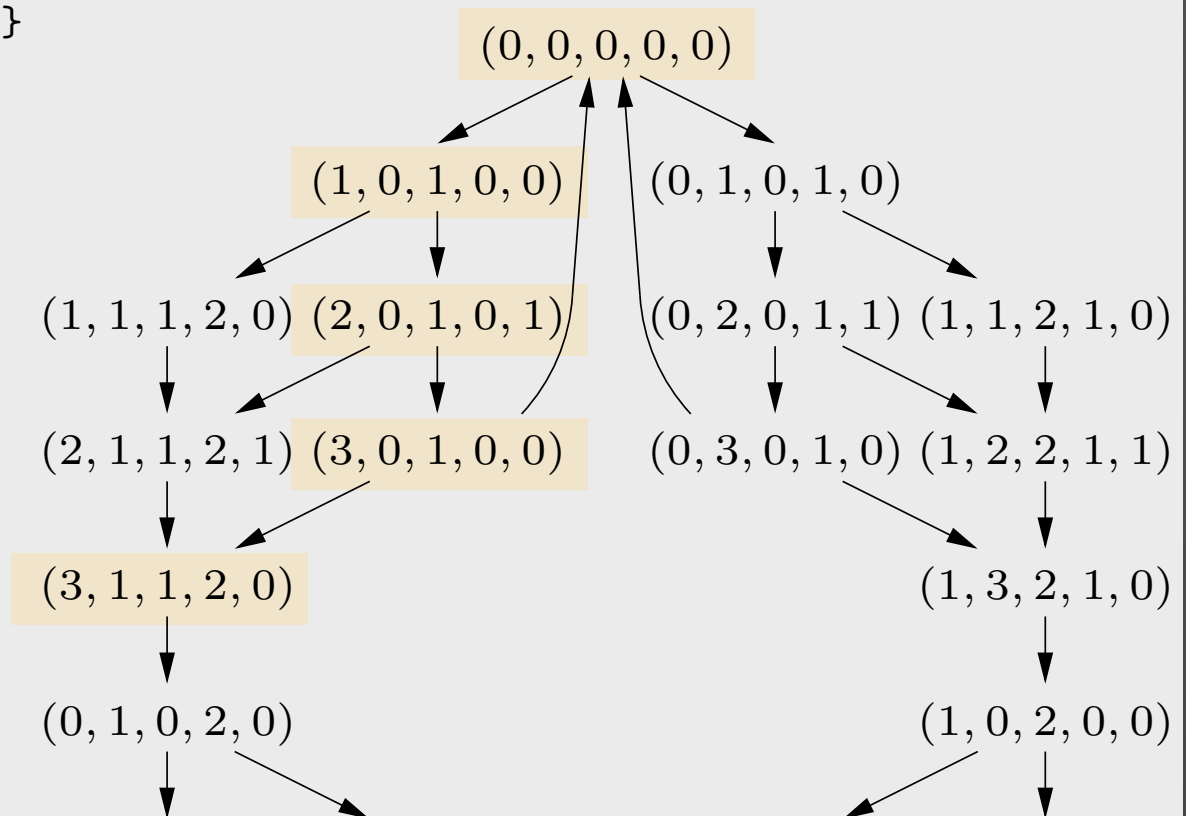
```

active proctype process_2() {
  do
    :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
  od
}

```

Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

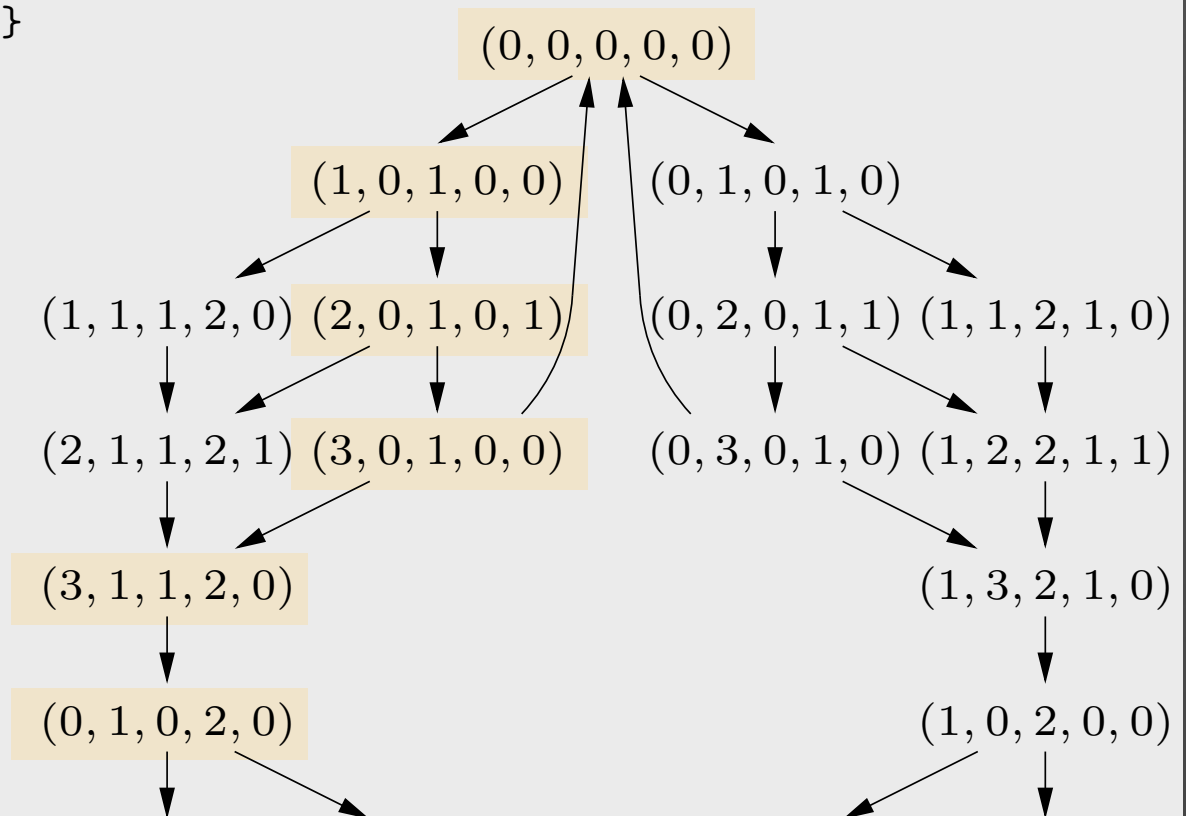
```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```

Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

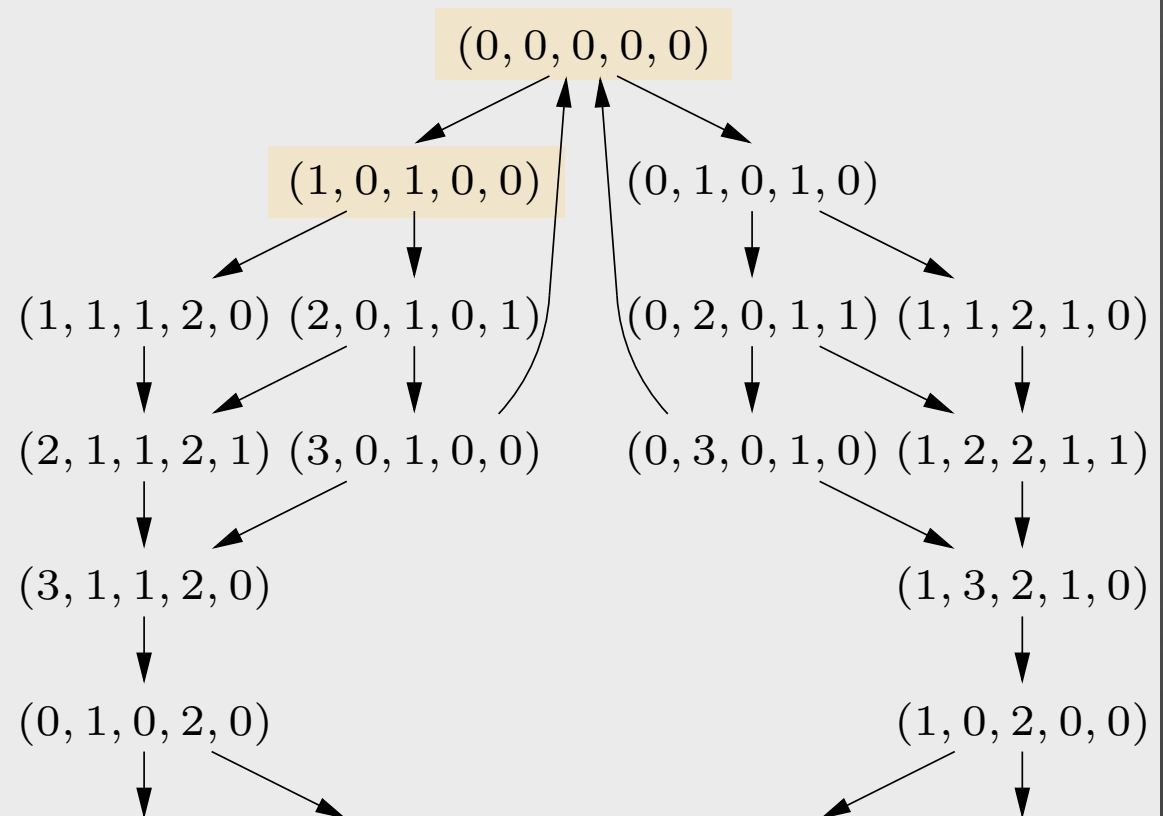
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



```

int y1 = 0;
int y2 = 0;
short in_critical = 0;

active proctype process_1() {
    do
        :: true ->
0:      y1 = y2+1;
1:      ((y2==0) || (y1<=y2));
        in_critical++;
2:      in_critical--;
3:      y1 = 0;
    od
}

```

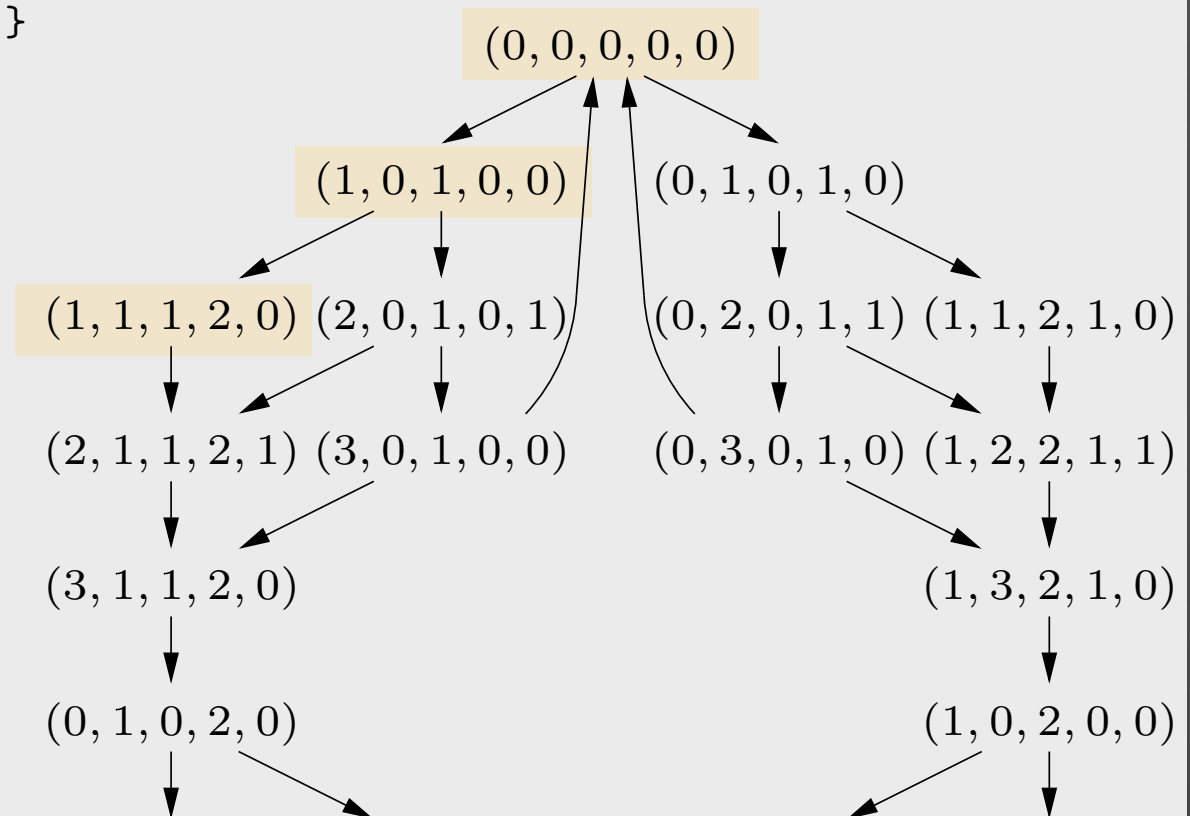
Estructura del estado:

$(pc_1, pc_2, y1, y2, in_critical)$

```

active proctype process_2() {
    do
        :: true ->
0:      y2 = y1+1;
1:      ((y1==0) || (y2<y1));
        in_critical++;
2:      in_critical--;
3:      y2 = 0;
    od
}

```



Razonamiento sobre programas concurrentes

En general, es muy difícil razonar sobre programas concurrentes. Luego, garantizar que un programa concurrente es correcto es también muy difícil.

Una de las razones tiene que ver con que diferentes interleavings de acciones atómicas pueden llevar a diferentes resultados o comportamientos de los sistemas concurrentes.

El número de interleavings posibles, por su parte, es en general muy grande, lo que hace que el testing difícilmente pueda brindarnos confianza de que nuestros programas concurrentes funcionan correctamente.

Razonamiento sobre programas concurrentes

En general, es muy difícil razonar sobre programas concurrentes. Luego, garantizar que un programa concurrente es correcto es también muy difícil.

Además, los tests
no pueden controlar el
no determinismo

Una de las razones es que diferentes interleavings de acciones atómicas producen diferentes resultados o comportamientos de los sistemas concurrentes.

El número de interleavings posible es muy grande, lo que hace que el testing pierda confianza de que nuestros programas funcionan correctamente.

El espacio de estados
crece exponencialmente con
el número de componentes y
variables.

En el ejemplo anterior es prácticamente imposible realizar el test que lleve al overflow.

Abstracción:

Modelos de programas concurrentes

Una forma de aliviar, en parte, el problema de razonar sobre programas concurrentes es considerar **representaciones abstractas** de éstos. Estas representaciones abstractas, llamadas **modelos**, nos permiten concentrarnos en las características particulares que queremos analizar.

Las álgebras de procesos (CSP, CCS, ACP, LOTOS, etc.) permiten construir estos modelos, concentrándose en las propiedades funcionales de sistemas concurrentes. Para esto, es importante considerar los **eventos** en los cuales cada proceso puede estar involucrado, y los **patrones de ocurrencia** que éstos siguen.

El lenguaje FSP

El lenguaje que utilizaremos en la primera parte de la asignatura es FSP (Finite State Processes). FSP es una variante simple de CSP, que incluye, entre otras cosas:

- Prefijos de acciones:

$x \rightarrow P$

Evento

Proceso

El lenguaje FSP

El lenguaje que utilizaremos en la primera parte de la asignatura es FSP (Finite State Processes). FSP es una variante simple de CSP, que incluye, entre otras cosas:

- Prefijos de acciones:

$x \rightarrow P$

Construyen un
nuevo proceso

El lenguaje FSP

El lenguaje que utilizaremos en la primera parte de la asignatura es FSP (Finite State Processes). FSP es una variante simple de CSP, que incluye, entre otras cosas:

- Prefijos de acciones:

$$x \rightarrow P$$

- Definiciones recursivas:

$$\text{OFF} = (\text{on} \rightarrow (\text{off} \rightarrow \text{OFF}))$$

- Elección:

$$(x \rightarrow P \mid y \rightarrow Q)$$

- Elección no determinista:

$$(x \rightarrow P \mid x \rightarrow Q)$$

¿Cuál es la diferencia?

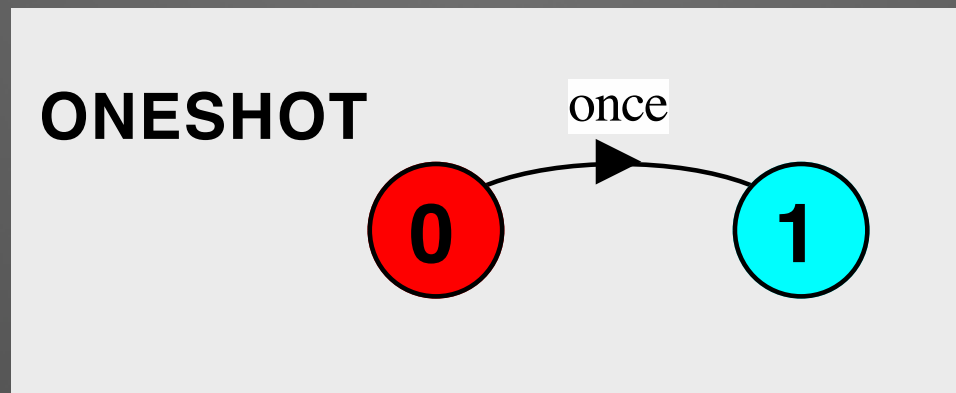
Semántica de procesos

La semántica de los procesos FSP está dada en términos de sistemas de transición de estados y trazas. En particular, los procesos pueden verse gráficamente como sistemas de transición de estados.

Por ejemplo, el proceso

ONESHOT = **once** $\rightarrow$ **STOP**.

puede visualizarse como:



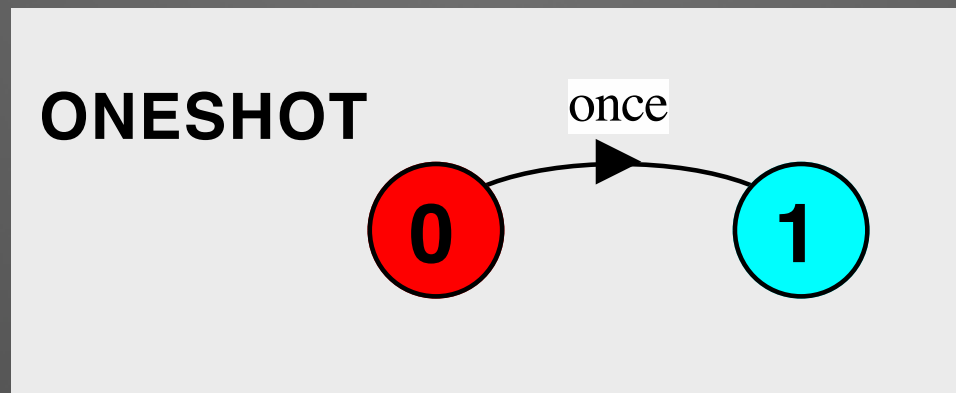
Semántica de procesos

La semántica de los procesos FSP está dada en términos de sistemas de transición de estados y trazas. En particular, los procesos pueden verse gráficamente como sistemas de transición de estados.

Por ejemplo, el proceso

ONESHOT = *once*

puede visualizarse como:



Los gráficos están hechos con LTSA que es la herramienta que soporta a FSP

FSP: Prefijos de acciones

Uno puede definir un proceso que, luego de realizar un evento o acción atómica x , se comporta exactamente como cierto proceso P usando prefijos:

$$(x \rightarrow P)$$

Esto es utilizado, por ejemplo, en el proceso ONESHOT visto anteriormente.

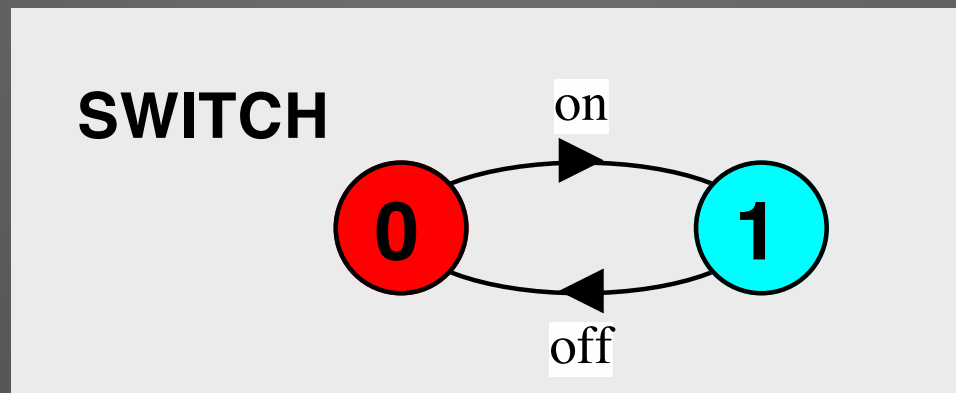
FSP: Recursión

El comportamiento de un proceso también puede definirse usando recursión, donde tenemos algunas restricciones sintácticas:

Ejemplo:

```
SWITCH = OFF,  
OFF = (on -> ON),  
ON = (off -> OFF).
```

Por supuesto, estos procesos pueden verse gráficamente como sistemas de transición de estados (y la herramienta LTSA lo hace por nosotros!).

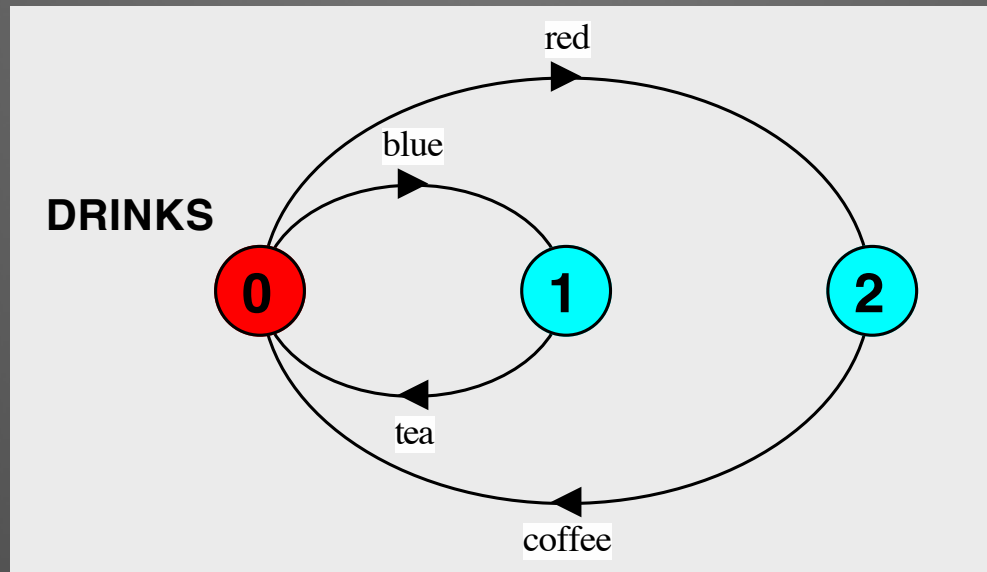


FSP: Elección

La ramificación en el flujo de ejecución de un proceso se describe mediante la elección.

Ejemplo: $\text{DRINKS} = (\text{red} \rightarrow \text{coffee} \rightarrow \text{DRINKS} \mid \text{blue} \rightarrow \text{tea} \rightarrow \text{DRINKS}) .$

Y, nuevamente, estos procesos pueden verse gráficamente como sistemas de transición de estados.



FSP: Elección no determinista

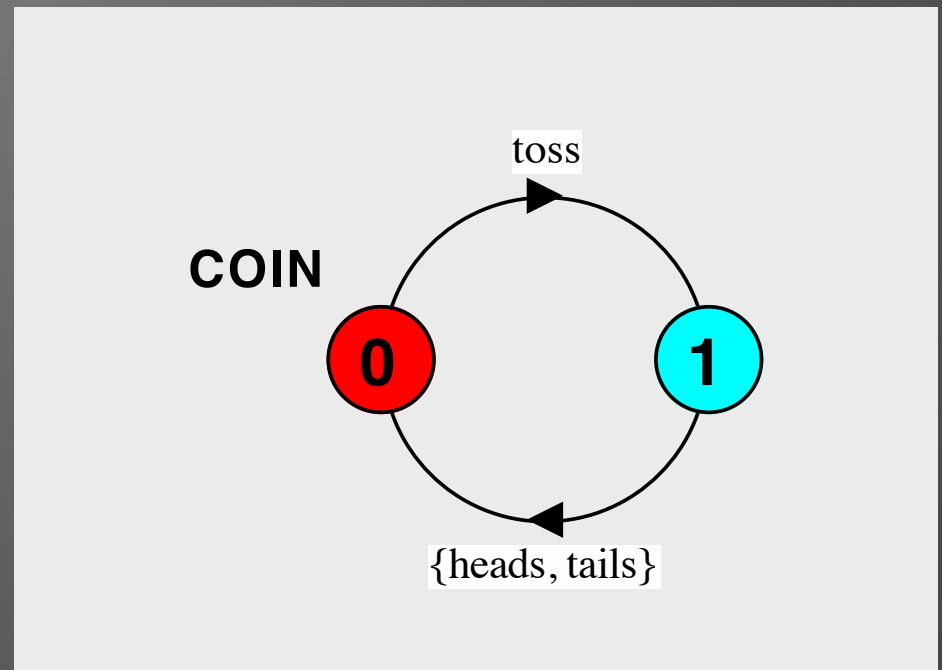
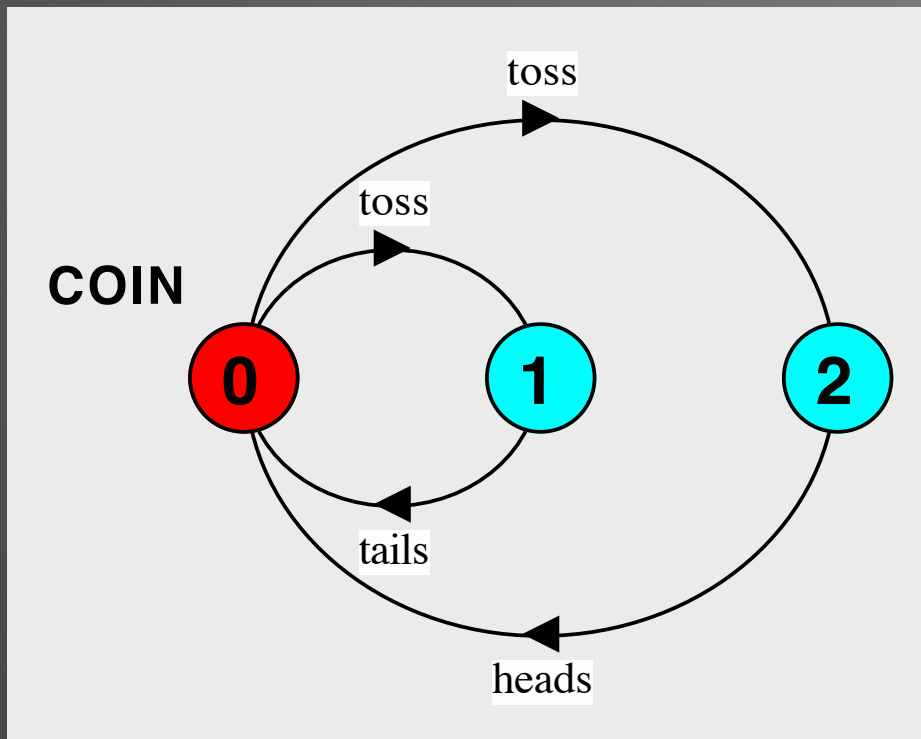
Es simplemente un caso particular de elección.

Ejemplo:

```
COIN = ( toss -> heads -> COIN
        | toss -> tails -> COIN
        ).
```

Comparar:

```
COIN = ( toss -> ( heads -> COIN
                  | tails -> COIN
                  )
        ).
```



FSP: Procesos y acciones indexados

Cuando se necesita modelar procesos que tomen un número grande de posibles valores, pueden utilizarse acciones (y procesos) indexadas, donde el rango del índice debe ser finito.

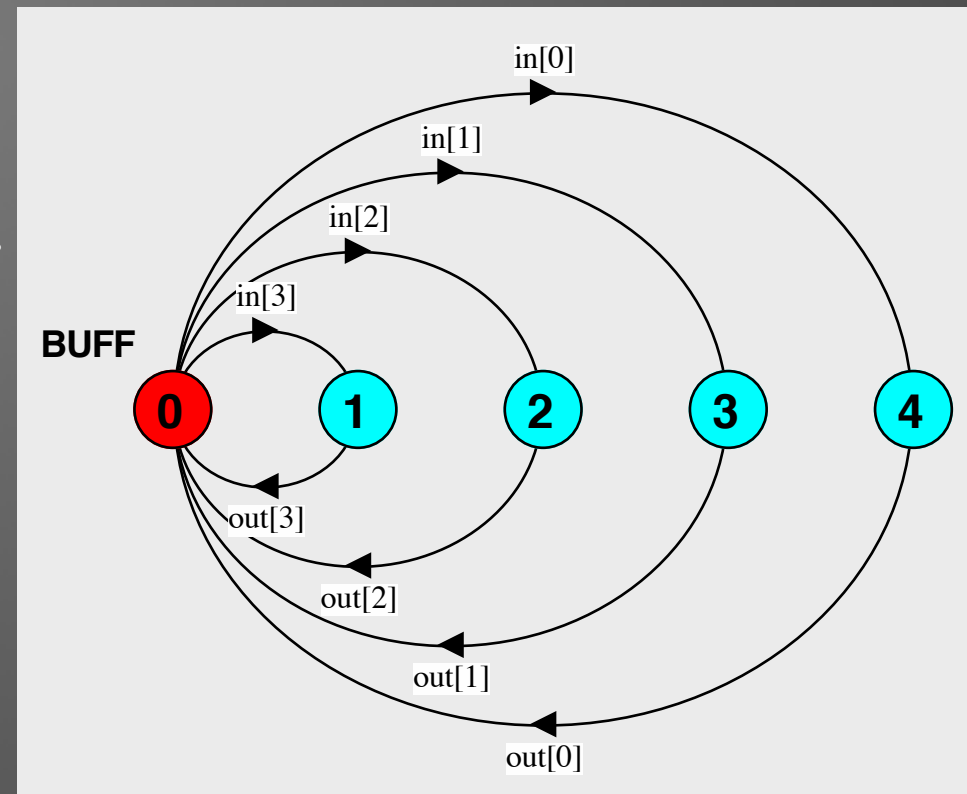
Esta facilidad tiene múltiples usos. En particular, puede emplearse para modelar estado explícito.

Ejemplo:

$\text{BUFF} = (\text{in}[i:0..3] \rightarrow \text{out}[i] \rightarrow \text{BUFF}).$

Es equivalente a escribir:

```
BUFF = ( in[0] -> out[0] -> BUFF
        | in[1] -> out[1] -> BUFF
        | in[2] -> out[2] -> BUFF
        | in[3] -> out[3] -> BUFF ).
```



FSP: Procesos y acciones indexados

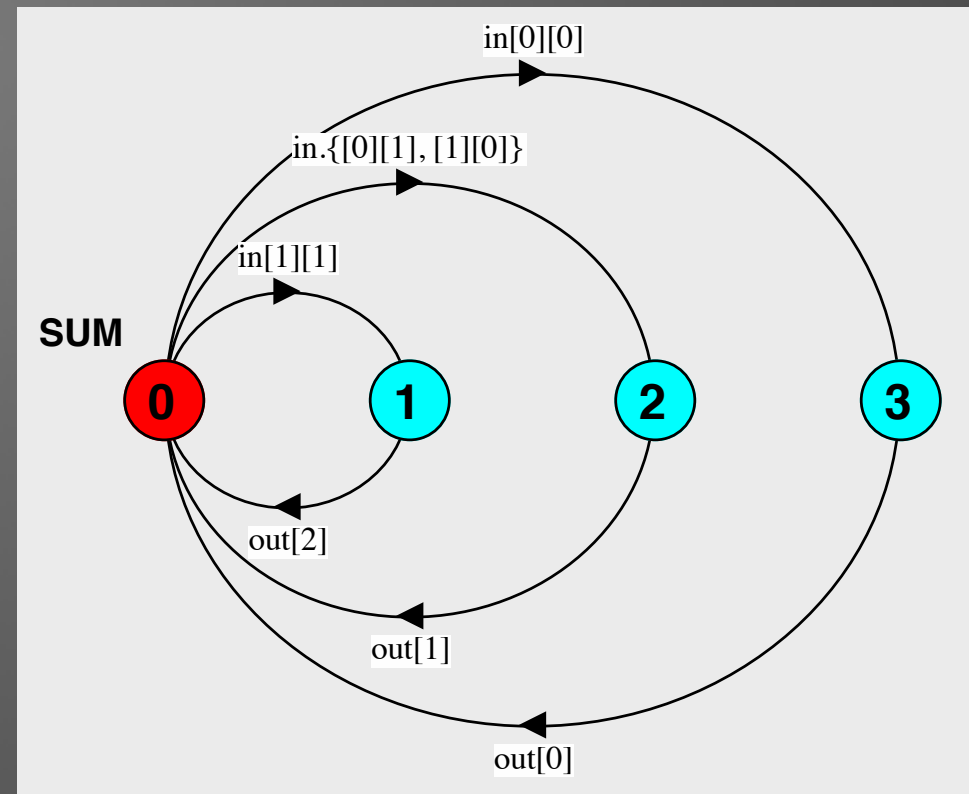
Cuando se necesita modelar procesos que tomen un número grande de posibles valores, pueden utilizarse acciones (y procesos) indexadas, donde el rango del índice debe ser finito.

Esta facilidad tiene múltiples usos. En particular, puede emplearse para modelar estado explícito.

Ejemplo:

Intenten con otros valores de N

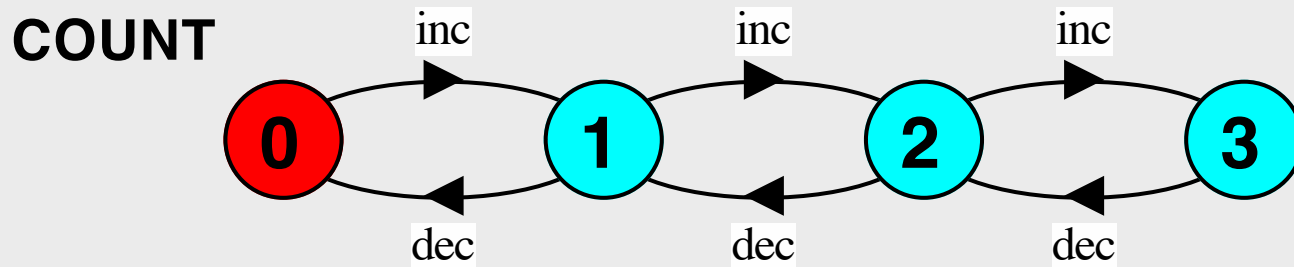
```
const N = 1
range T = 0..N
range R = 0..2*N
SUM = (in[a:T][b:T] -> TOTAL[a+b]),
TOTAL[s:R] = (out[s] -> SUM).
```



FSP: Acciones con guardas

Es en general útil contar con acciones que se ejecuten condicionalmente, con respecto al estado de la máquina o sistema modelados. Esto puede expresarse usando la notación "when" en FSP.

Ejemplo: $\text{COUNT } (N=3) = \text{COUNT}[0],$
 $\text{COUNT}[i:0..N] = (\text{when}(i < N) \text{ inc} \rightarrow \text{COUNT}[i+1]$
 $\quad \quad \quad | \text{when}(i > 0) \text{ dec} \rightarrow \text{COUNT}[i-1]$
 $\quad \quad \quad).$

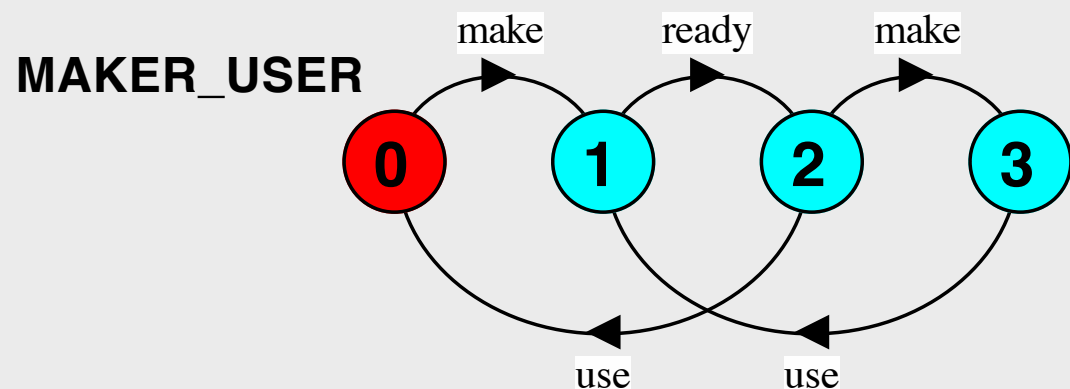
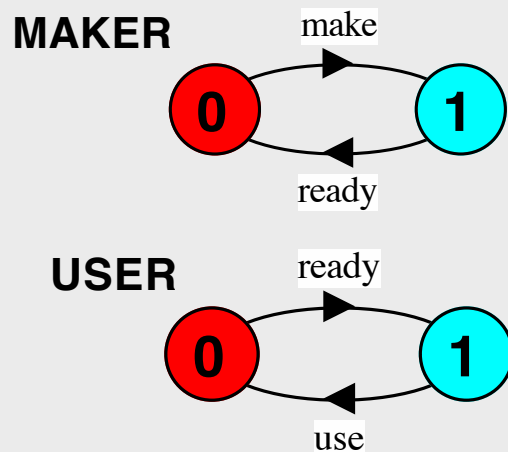


FSP: Composición paralela

Hasta el momento, ninguna de las construcciones vistas nos permite modelar concurrencia. La construcción que nos permite hacer esto, y la más compleja de comprender, es la composición paralela de procesos.

Dados dos procesos P y Q , $P \parallel Q$ denota la composición paralela de estos procesos.

Ejemplo: $\text{MAKER} = (\text{make} \rightarrow \text{ready} \rightarrow \text{MAKER}).$
 $\text{USER} = (\text{ready} \rightarrow \text{use} \rightarrow \text{USER}).$
 $\parallel \text{MAKER_USER} = (\text{MAKER} \parallel \text{USER}).$

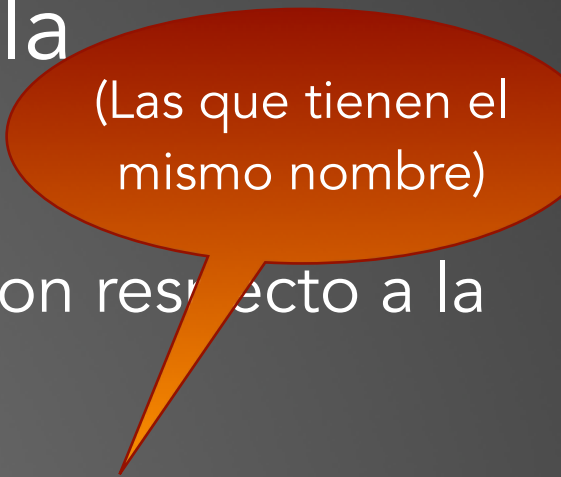


FSP: Composición paralela

Hay varios puntos importantes por recordar con respecto a la composición paralela en FSP:

- La sincronización se realiza en las acciones comunes (y puede involucrar a más de dos procesos).
- No se puede identificar explícitamente al proceso “activo” y al proceso “pasivo” en la sincronización de acciones comunes. Esta interpretación corre por cuenta del diseñador (i.e. ustedes).
- El modelo de concurrencia es interleaving, donde las acciones atómicas independientes de diferentes procesos pueden ejecutarse en interleavings arbitrarios.

FSP: Composición paralela



(Las que tienen el mismo nombre)

Hay varios puntos importantes por recordar con respecto a la composición paralela en FSP:

- La sincronización se realiza en las acciones comunes (y puede involucrar a más de dos procesos).
- No se puede identificar explícitamente al proceso "activo" y al proceso "pasivo" en la sincronización de acciones comunes. Esta interpretación corre por cuenta del diseñador (i.e. ustedes).
- El modelo de concurrencia es interleaving, donde las acciones atómicas independientes de diferentes procesos pueden ejecutarse en interleavings arbitrarios.

FSP: Otros conceptos y operaciones

- 👁 **Alfabeto de un proceso:** Conjunto de acciones al cual este proceso debe responder. Es importante para saber como se sincronizan los procesos.
- 👁 **Etiquetado de procesos:** Utilizado para generar diferentes copias de un mismo proceso.
- 👁 **Reetiquetado:** Utilizado usualmente para asegurar que procesos compuestos sincronicen en las acciones deseadas.
- 👁 **Ocultamiento:** Las acciones ocultas no pueden compartirse con otros procesos (desaparecen del alfabeto del proceso). Utiliza una acción distinguida **tau** (denominada acción sigilosa o invisible) que no tiene permitido sincronizar.

FSP: Eliminación de operadores

Todo proceso (finito) puede escribirse usando sólo:

- prefijo
- elección, y
- recursión

Estas son las **operaciones básicas**

Ingeniería del Software II

3 - Sincronización de procesos concurrentes

Recursos compartidos

En ámbitos donde la concurrencia es útil, suele ocurrir que diferentes procesos necesitan interactuar mediante el uso de recursos comunes.

Luego de experimentar un poco con programas concurrentes (o modelos de éstos), resulta evidente que el uso de recursos comunes por diferentes procesos puede dar lugar a actualizaciones incorrectas en el estado de estos recursos. Este problema es conocido como **interferencia**.

Para manejar el problema, se debe asegurar que, mientras uno de los procesos utiliza un recurso compartido, el resto no puede acceder al mismo (**exclusion mutua**).

Recursos compartidos

En ámbitos donde la concurrencia es útil, suele ocurrir que diferentes procesos necesitan interactuar con recursos comunes.

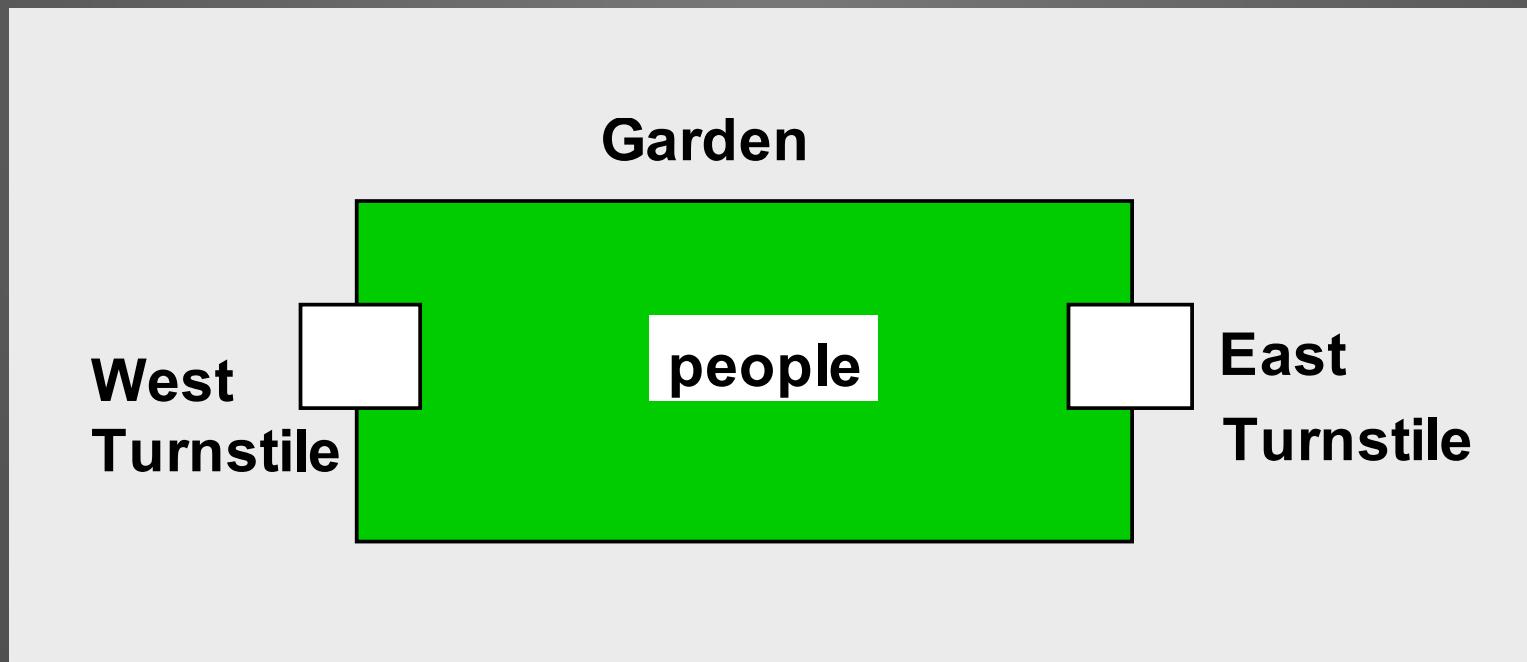
... o **Race Condition**

Luego de experimentar un poco con programas concurrentes (o modelos de éstos), resulta evidente que el uso de recursos comunes por diferentes procesos puede dar lugar a actualizaciones incorrectas en el estado de estos recursos. Este problema es conocido como **interferencia**.

Para manejar el problema, se debe asegurar que, mientras uno de los procesos utiliza un recurso compartido, el resto no puede acceder al mismo (**exclusion mutua**).

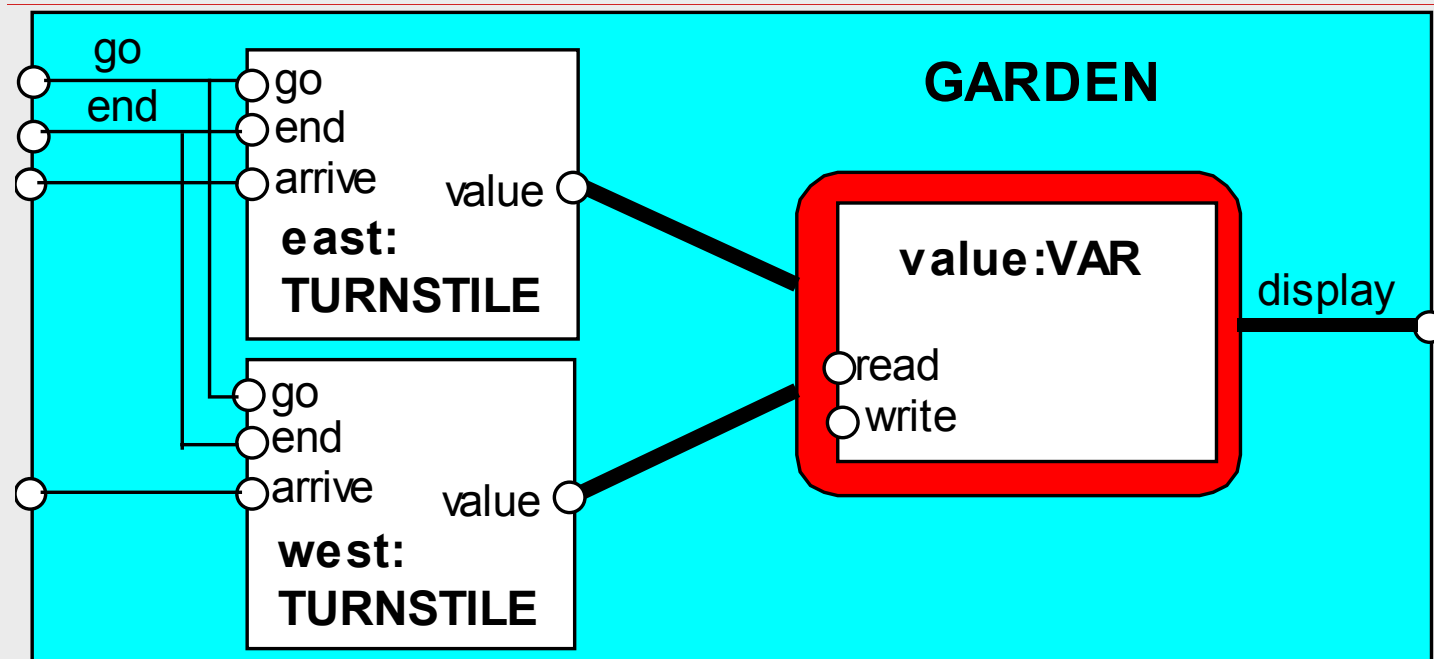
Un ejemplo de interferencia

Consideremos el problema del jardín ornamental. Tenemos un jardín ornamental con dos entradas (este y oeste), con sendas puertas giratorias. Se desea contar el número de personas que visitaron el jardín.



Un ejemplo de interferencia

Un modelo simple del problema consiste de tres procesos: dos para controlar (independientemente) las puertas y un contador para el número de personas (recurso compartido).




```
const N = 4
range T = 0..N
set VarAlpha = {value.{read[T],write[T]}}
```

```
VAR          = VAR[0],
VAR[u:T]     = (read[u]    ->VAR[u]
                |write[v:T]->VAR[v])).
```

```
TURNSTILE = (go      -> RUN),
RUN        = (arrive -> INCREMENT
              |end    -> TURNSTILE),
INCREMENT  = (value.read[x:T]
              -> value.write[x+1] -> RUN)
              +VarAlpha.
```

```
||GARDEN = (east:TURNSTILE || west:TURNSTILE
            || {east,west,display}::value:VAR)
            /{go /{east,west}.go,
              end/{east,west}.end}.
```

```
const N = 4
range T = 0..N
set VarAlpha = {value.{read[T],write[T]}}
```

El alfabeto de la variable compartida se declara explícitamente ...

```
VAR      = VAR[0],
VAR[u:T] = (read[u]    ->VAR[u]
            |write[v:T]->VAR[v]).
```

```
TURNSTILE = (go      -> RUN),
RUN        = (arrive -> INCREMENT
            |end      -> TURNSTILE),
INCREMENT  = (value.read[x:T]
            -> value.write[x+1] -> RUN)
            +VarAlpha.
```

```
||GARDEN = (east:TURNSTILE || west:TURNSTILE
|| {east,west,display}::value:VAR)
/{go /{east,west}.go,
  end/{east,west}.end}.
```

... para extender el alfabeto del proc. TURNSTILE y así asegurar que VAR no realiza acciones autonomamente (ej. `value.write[0]`). Todas las acciones de la var. compartida deben ser controladas

Detección de errores

Para poder comprobar si el modelo anterior funciona de la manera esperada o no, podemos combinarlo con un proceso que detecte la actualización incorrecta del recurso compartido:

```
TEST          = TEST[0] ,
TEST[v:T]     = (when (v<N){east.arrive,west.arrive}->TEST[v+1]
                 | end->CHECK[v]
                 ) ,
CHECK[v:T]    = (display.value.read[u:T] ->
                 (when (u==v) right -> TEST[v]
                  |when (u!=v) wrong -> ERROR)
                 )+{display.VarAlpha}.
```

Detección de errores

Para poder comprobar si el modelo anterior funciona de la manera esperada o no, podemos combinarlo con un proceso que detecte la actualización incorrecta del recurso compartido:

```
TEST          = TEST[0] ,
TEST[v:T]    = (when (v<N){east.arrive,west.arrive}->TEST[v+1]
                | end->CHECK[v]
                ) ,
CHECK[v:T]   = (display.value.read[u:T] ->
                (when (u==v) right -> TEST[v]
                 |when (u!=v) wrong -> ERROR)
                )+{display.VarAlpha}.
```

Detección de errores

```
Trace to property violation in TEST
go
east.arrive
east.value.read.0
west.arrive
west.value.read.0
east.value.write.1
west.value.write.1
end
display.value.read.1
wrong
```

Modelado de la exclusión mutua

Una forma simple de manejar exclusion mutua es mediante una "traba":

```
LOCK = (acquire -> release -> LOCK).
```

Para el caso de nuestro ejemplo, esta llave puede usarse para garantizar el acceso exclusivo al recurso compartido VAR:

```
...  
LOCK = (acquire->release->LOCK).  
||LOCKVAR = (LOCK || VAR).  
  
TURNSTILE = (go      -> RUN),  
RUN        = (arrive-> INCREMENT  
              |end    -> TURNSTILE),  
INCREMENT  = (value.acquire  
              -> value.read[x:T] -> value.write[x+1]  
              -> value.release -> RUN  
              )+VarAlpha.  
...
```

Modelado de la exclusión mutua

Una forma simple de manejar exclusion mutua es mediante una "traba":

```
LOCK = (acquire -> release -> LOCK).
```

Para el caso de nuestro ejemplo, esta llave puede usarse para garantizar el acceso exclusivo al recurso compartido VAR:

```
...  
LOCK = (acquire->release->LOCK).  
||LOCKVAR = (LOCK || VAR).  
  
TURNSTILE = (go      -> RUN),  
RUN        = (arrive-> INCREMENT  
              |end    -> TURNSTILE),  
INCREMENT  = (value.acquire  
              -> value.read[x:T] -> value.write[x+1]  
              -> value.release -> RUN  
              )+VarAlpha.  
  
...
```

Monitores y otros TAD de sincronización

Los **monitores** [Brinch Hansen 72, Hoare 74] encapsulan datos que solo pueden modificarse y observarse a través de procedimientos.

Además, garantizan que sólo uno de tales procedimientos a la vez accede a los datos ocultos en estas estructuras.

Otros TADs para sincronizar procesos son:

- los **semáforos** y
- los **buffers**.

Estos podrán verse como monitores particulares (pero no necesariamente lo son).

Monitores y otros TAD de sincronización

[https://es.wikipedia.org/wiki/Monitor_\(concurrency\)](https://es.wikipedia.org/wiki/Monitor_(concurrency))

Los **monitores** [Brinch Hansen 72, Hoare 74] encapsulan datos que solo pueden modificarse y observarse a través de procedimientos.

Además, garantizan que sólo uno de tales procedimientos a la vez accede a los datos ocultos en estas estructuras.

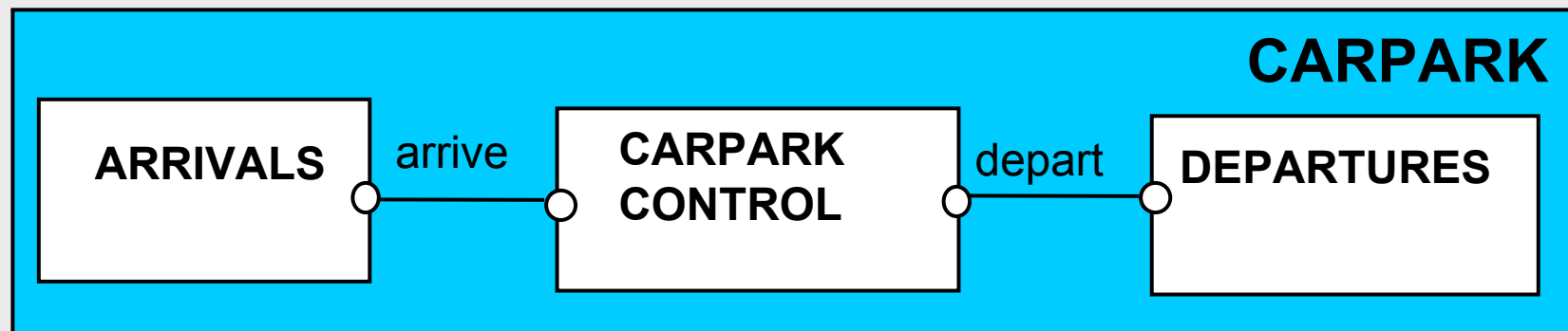
Otros TADs para sincronizar procesos son:

- los **semáforos** y
- los **buffers**.

Estos podrán verse como monitores particulares (pero no necesariamente lo son).

Sincronización condicional

Ejemplo: Se requiere un controlador para un estacionamiento de automóviles que sólo permita entrar un auto si el estacionamiento no está lleno y (obviamente) no permita salir autos si no hay ninguno estacionado.



Invariante de un monitor

Un **invariante para un monitor** es una aserción sobre las variables que éste encapsula.

Esta aserción **debe** valer siempre que ningún hilo esté ejecutando algún procedimiento del monitor.

- 👁 ¿Cuál es el invariante del controlador?
- 👁 ¿Para qué sirve este invariante?

Invariante de un monitor

El invariante del monitor nos permite derivar las condiciones de sincronización.

Invariante: $0 \leq \textit{espacio_libre} \leq \textit{capacidad}$

En el modelo, **N** es la capacidad del estacionamiento.

Invariante del monitor  En el modelo son **i** y **N** respectivamente.

El invariante del monitor nos permite derivar las condiciones de sincronización.

Invariante: $0 \leq \text{espacio_libre} \leq \text{capacidad}$

```
CARPARKCONTROL(N=4) = SPACES[N] ,  
SPACES[i:0..N] = (when(i>0) arrive->SPACES[i-1]  
                  |when(i<N) depart->SPACES[i+1]  
                  ).
```

```
ARRIVALS = (arrive->ARRIVALS) .
```

```
DEPARTURES = (depart->DEPARTURES) .
```

```
||CARPARK = (ARRIVALS || CARPARKCONTROL(4) || DEPARTURES) .
```

En el modelo, **N** es la capacidad del estacionamiento.

Relación con los verdaderos monitores

```
monitor carparkcontrol {
  const N=4
  int i
  condition free_space
  condition car_parked

  procedure arrive() {                                // when(i>0) arrive->SPACES[i-1]
    if not(i>0) then wait(free_space)
    i := i-1
    notify(car_parked)
  }

  procedure depart() {                                // when(i<N) depart->SPACES[i+1]
    if not(i<N) then wait(car_parked)
    i := i+1
    notify(free_space)
  }
}
```

Semáforos [Dijkstra 68]

- Es uno de los primeros mecanismos propuestos para sincronización de procesos.
- Un semáforo es un TAD que consta de una variable protegida que toma valores enteros no negativos y que sólo se puede acceder a través de dos operaciones: down y up (o P y V):

- $down(s)$: **when** $s > 0$ **do** $s--$
- $up(s)$: $s++$

(Importante: la operación **when . . . do . . .** es bloqueante.)

¿Cuál es el invariante de un semáforo?

Modelo del semáforo

```
const Max = 3
range Int = 0..Max

SEMAPHORE(N=0) = SEMA[N] ,
SEMA[v: Int]   = (up->SEMA[v+1]
                  |when(v>0) down->SEMA[v-1]
                  ) .
```

Invariante: $0 \leq s$

Es conveniente considerar un invariante mas fuerte dado que usualmente los semáforos están asociados a un tipo de recurso y cuentan cuantos de estos quedan disponibles.

Modelo del semáforo (cont.)

```
const Max = 3  
range Int = 0..Max
```

```
SEMAPHORE(N=0) = SEMA[N] ,  
SEMA[v:Int]    = (up->SEMA[v+1]  
                  |when(v>0) down->SEMA[v-1]  
                  ) ,  
SEMA[Max+1]    = ERROR.
```

Invariante: $0 \leq s \leq \text{Max}$

Modelo del semáforo (cont.)

```
const Max = 3
range Int = 0..Max

SEMAPHORE(N=0) = SEMA[N] ,
SEMA[v:Int]    = (up->SEMA[v+1]
                  |when(v>0) down->SEMA[v-1]
                  ) ,
SEMA[Max+1]    = ERROR.
```

Produce un estado
erróneo y LTSA avisa de ello.

Invariante: $0 \leq s \leq \text{Max}$

Notar que la especificación explícita del estado **ERROR** en FSP no es necesaria dado que éste (y en consecuencia, el compilador a través de la herramienta LTSA) mapea todo estado indefinido al estado **ERROR**. Por consiguiente, la especificación previa bastará.

Buffers acotados

Es una estructura de datos utilizada para regular la velocidad de transferencia entre el productor de la información y el consumidor de esta.

- Ejemplos de uso?

- Invariante?

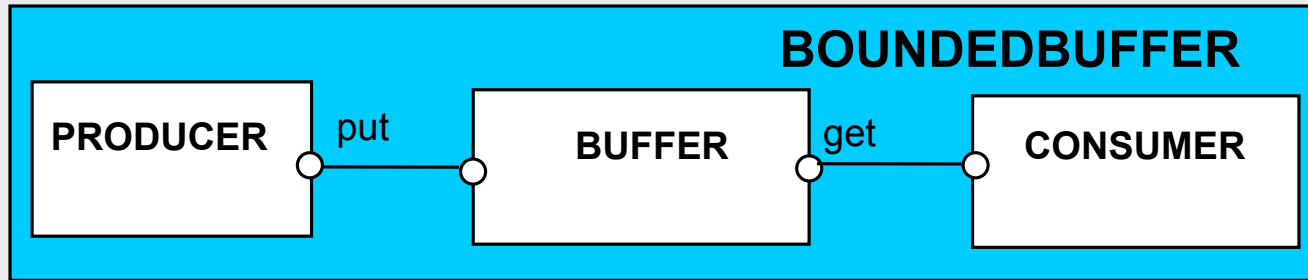
Ejemplo: Productor/Consumidor (ya lo conocen)

Observación para un modelado apropiado: El comportamiento del productor, del consumidor y del buffer mismo no es afectado por el valor de los datos manipulados; es decir, ninguna de las componentes testea o controla estos datos para su funcionamiento correcto.

Este comportamiento se dice **independiente de los datos**

=> abstraer los datos

Buffers acotados



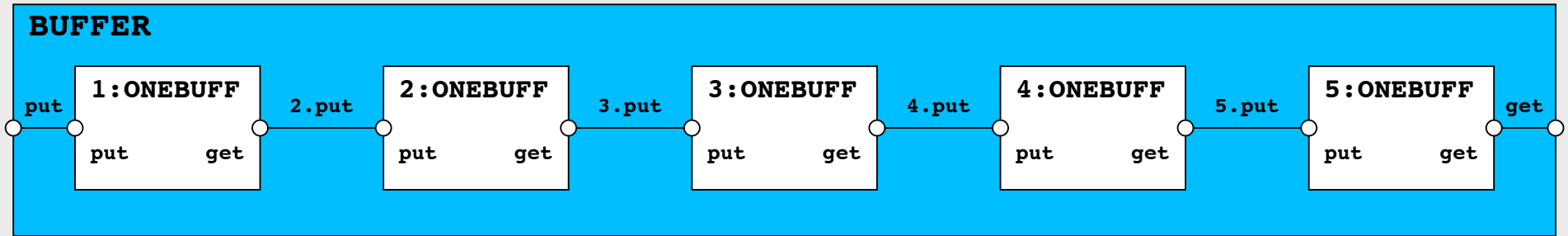
```
BUFFER(N=5) = COUNT[0],  
COUNT[i:0..N]  
    = (when (i<N) put->COUNT[i+1]  
       |when (i>0) get->COUNT[i-1]  
       ).
```

```
PRODUCER = (put->PRODUCER).  
CONSUMER = (get->CONSUMER).
```

```
||BOUNDED BUFFER = (PRODUCER||BUFFER(5)||CONSUMER).
```

Notar las condiciones respecto del invariante.

Buffers acotados



$\text{ONEBUFF} = (\text{put} \rightarrow \text{get} \rightarrow \text{ONEBUFF}).$

$$\begin{aligned} ||\text{BUFFER}(N=5) = & ([1..N]:\text{ONEBUFF} \\ &)/\{\text{put}/[1].\text{put}, \\ & [i:2..N].\text{put}/[i-1].\text{get}, \\ & \text{get}/[N].\text{get} \\ & \}\@ \{\text{put}, \text{get}\}. \end{aligned}$$

$\text{PRODUCER} = (\text{put} \rightarrow \text{PRODUCER}).$

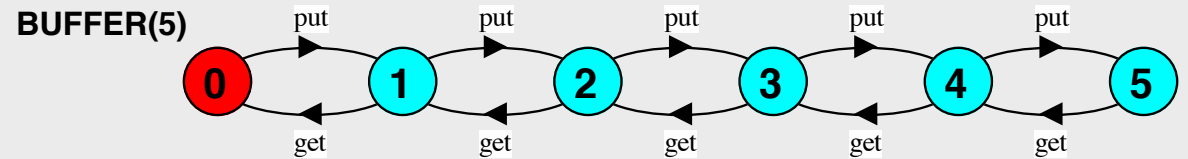
$\text{CONSUMER} = (\text{get} \rightarrow \text{CONSUMER}).$

$||\text{BOUNDEDBUFFER} = (\text{PRODUCER} || \text{BUFFER}(5) || \text{CONSUMER}).$

```

BUFFER(N=5) = COUNT[0],
COUNT[i:0..N]
  = (when (i<N) put->COUNT[i+1]
    |when (i>0) get->COUNT[i-1]
    ).

```



```

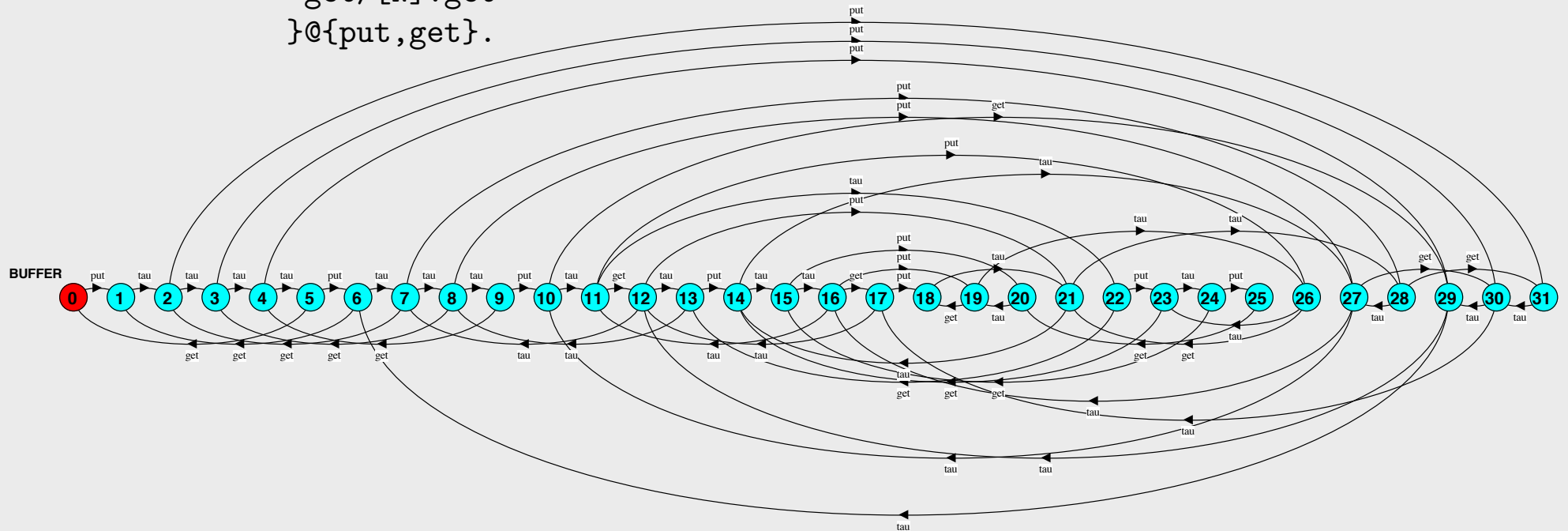
ONEBUFF = (put -> get -> ONEBUFF).

```

```

|| BUFFER(N=5) = ([1..N]:ONEBUFF
  )/{put/[1].put,
    [i:2..N].put/[i-1].get,
    get/[N].get
  }@{put,get}.

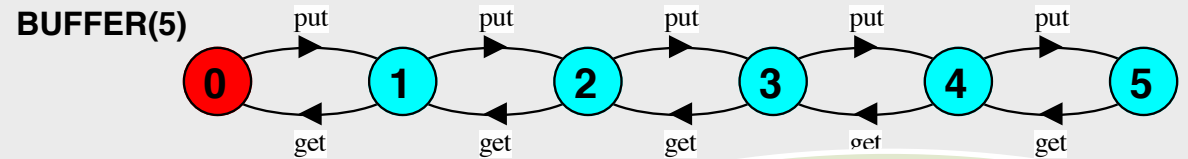
```



```

BUFFER(N=5) = COUNT[0],
COUNT[i:0..N]
  = (when (i<N) put->COUNT[i+1]
    |when (i>0) get->COUNT[i-1]
    ).

```



```

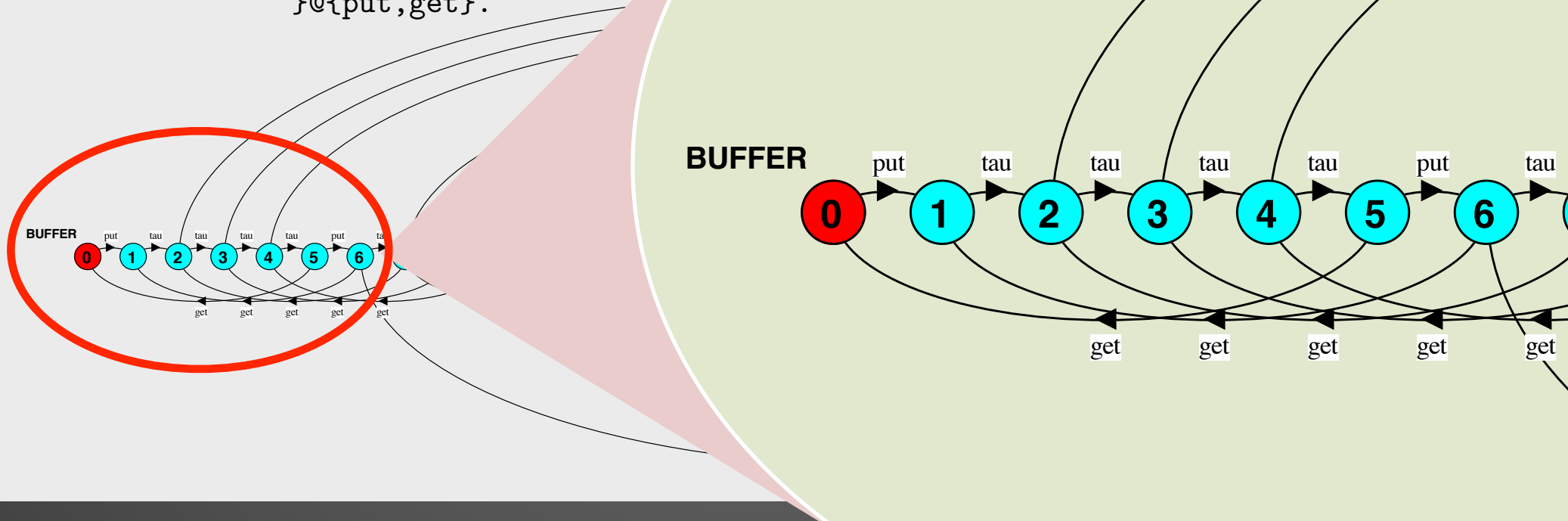
ONEBUFF = (put -> get -> ONEBUFF).

```

```

|| BUFFER(N=5) = ([1..N]:ONEBUFF
  )/{put/[1].put,
    [i:2..N].put/[i-1].get,
    get/[N].get
  }@{put,get}.

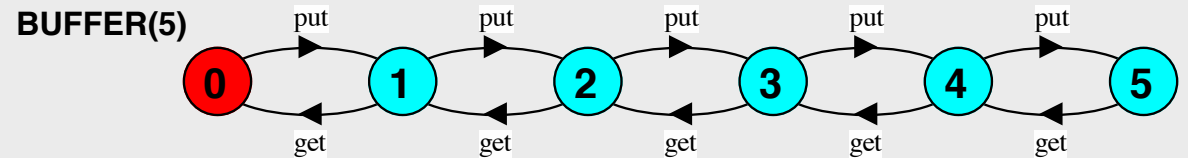
```



```

BUFFER(N=5) = COUNT[0],
COUNT[i:0..N]
  = (when (i<N) put->COUNT[i+1]
    |when (i>0) get->COUNT[i-1]
    ).

```



```

ONEBUFF = (put -> get -> ONEBUFF).

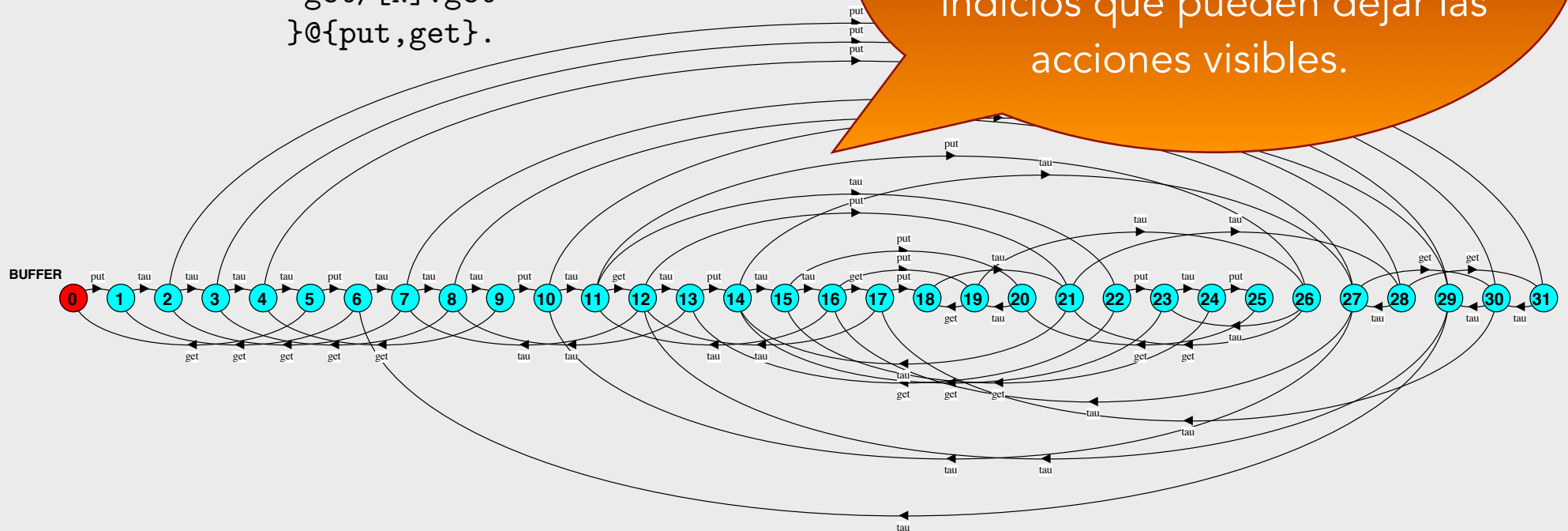
```

```

|| BUFFER(N=5) = ([1..N]:ONEBUFF
  )/{put/[1].put,
    [i:2..N].put/[i-1].get,
    get/[N].get
    }@{put,get}.

```

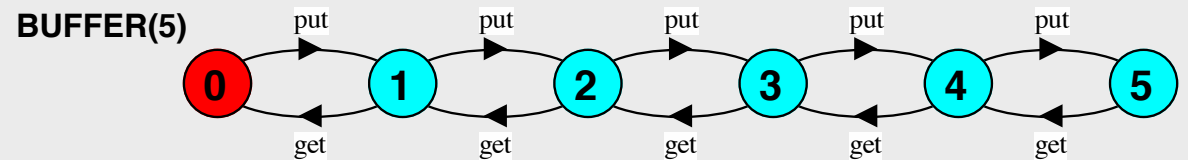
Recordar que las acciones tau son eventos internos que no pueden observarse sino a través de indicios que pueden dejar las acciones visibles.




```

BUFFER(N=5) = COUNT[0],
COUNT[i:0..N]
  = (when (i<N) put->COUNT[i+1]
    |when (i>0) get->COUNT[i-1]
    ).

```



```

ONEBUFF = (put -> get -> ONEBUFF).

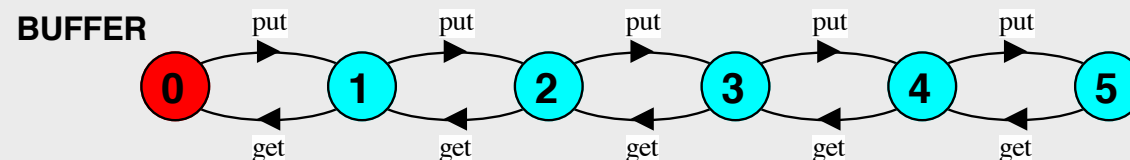
```

```

|| BUFFER(N=5) = ([1..N]:ONEBUFF
  )/{put/[1].put,
    [i:2..N].put/[i-1].get,
    get/[N].get
    }@{put,get}.

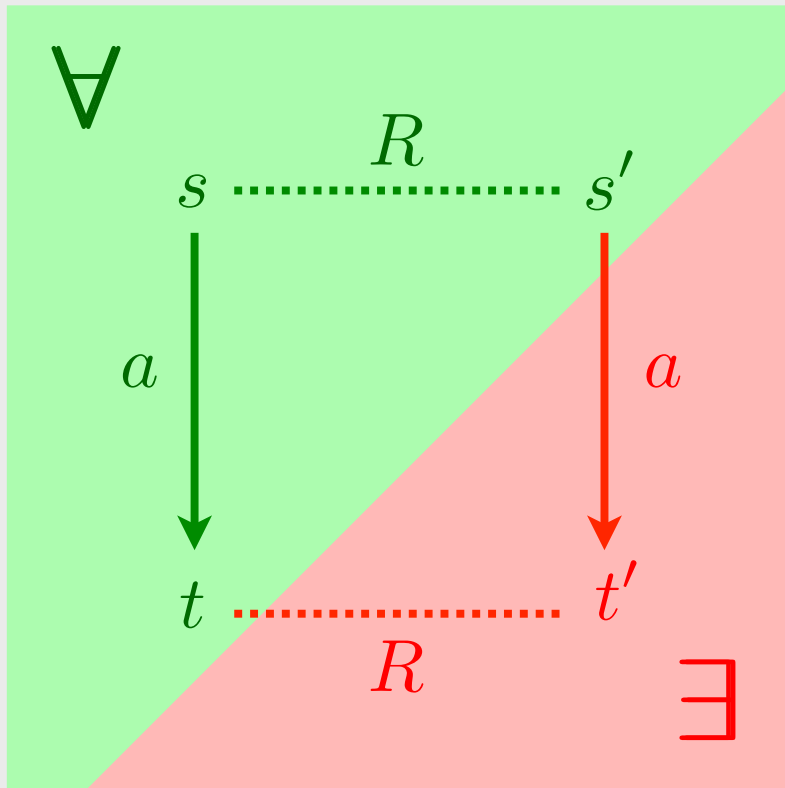
```

Luego de **abstraer** estos eventos internos, el LTS resulta isomorfo a la especificación anterior!!!



En LTSA, tal abstracción es una **minimización** que preserva la relación de equivalencia denominada **bisimulación débil**.

Simulación en LTS

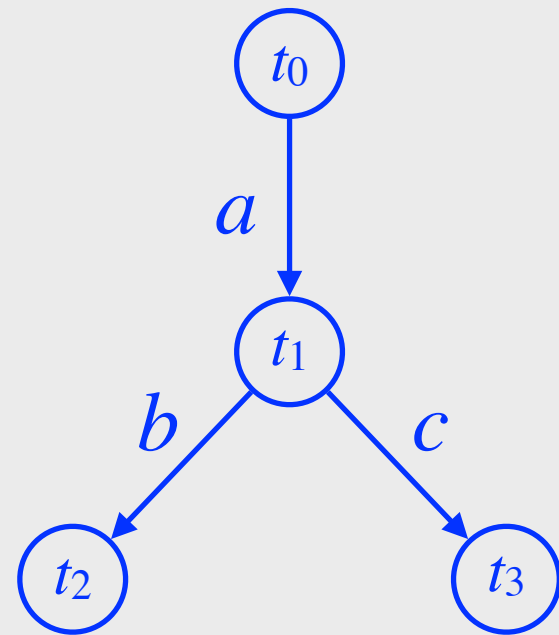
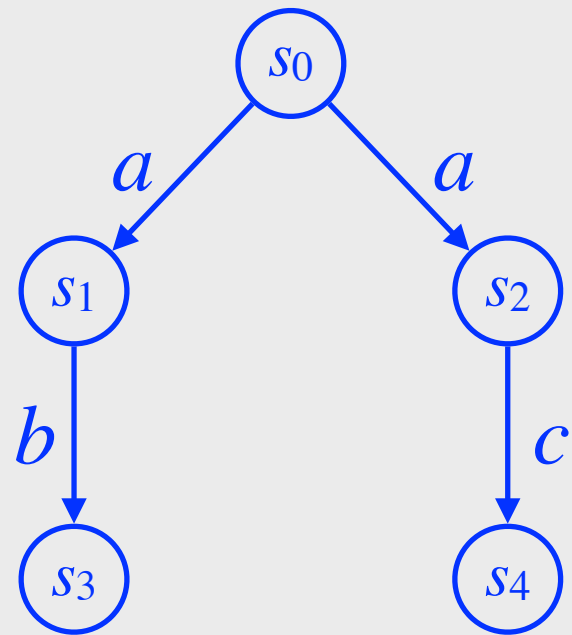


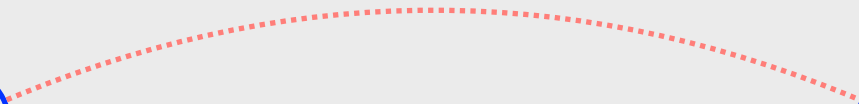
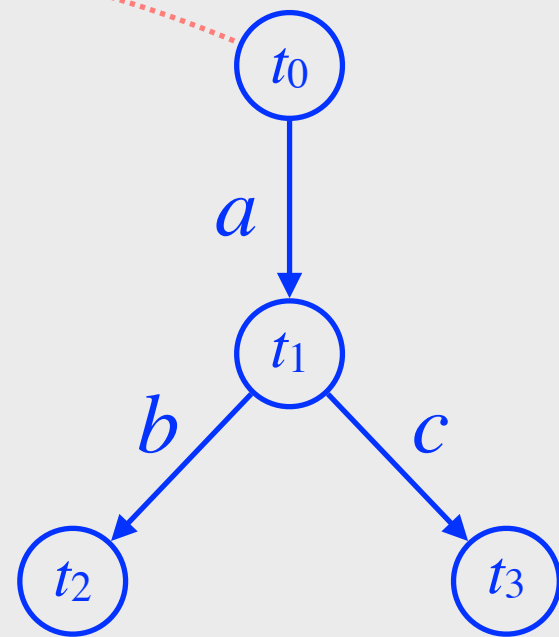
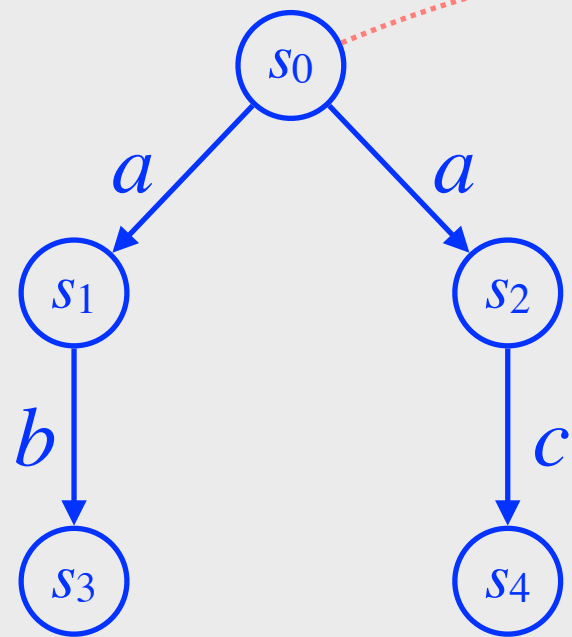
Simulación en LTS

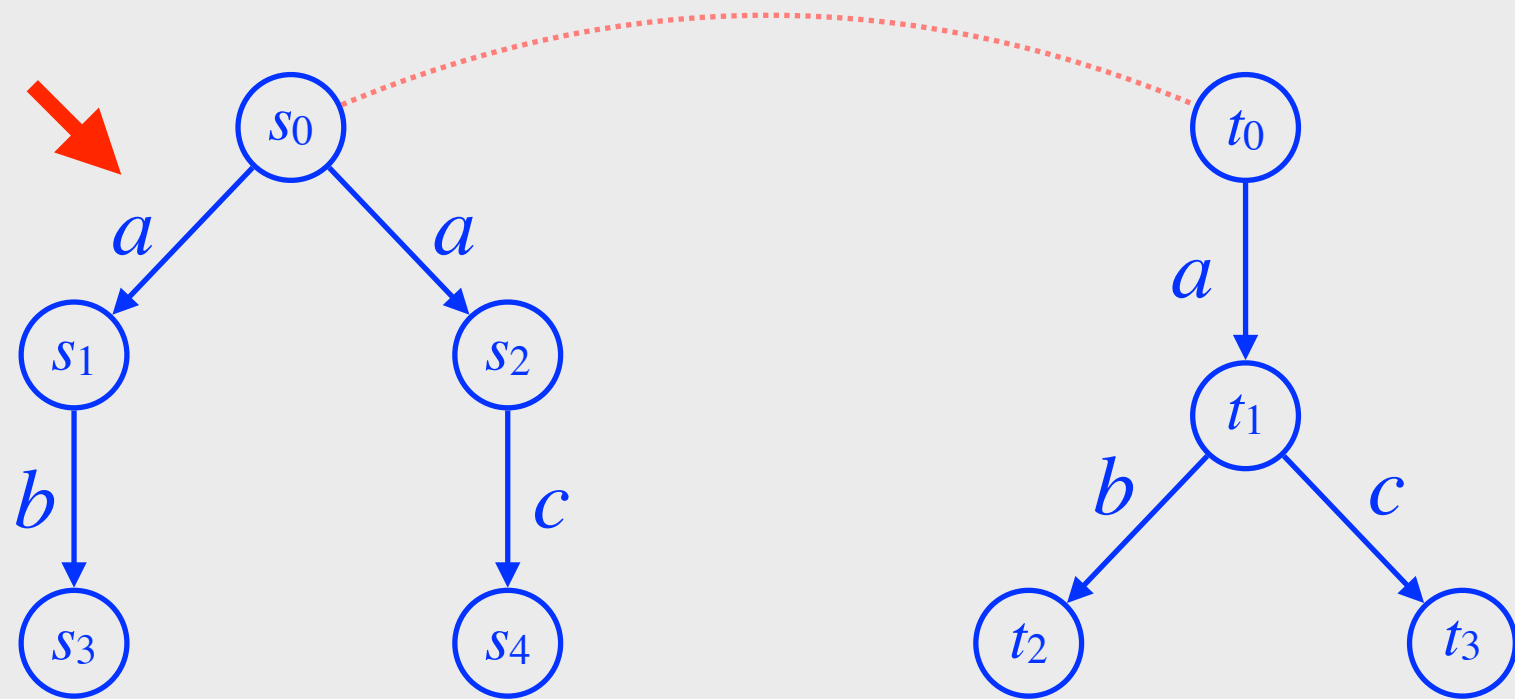
Sea $\mathcal{S} = (S, s_0, E, \rightarrow)$ un *sistema de transiciones etiquetadas*, i.e., $\rightarrow \subseteq S \times E \times S$.

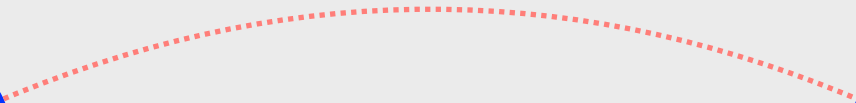
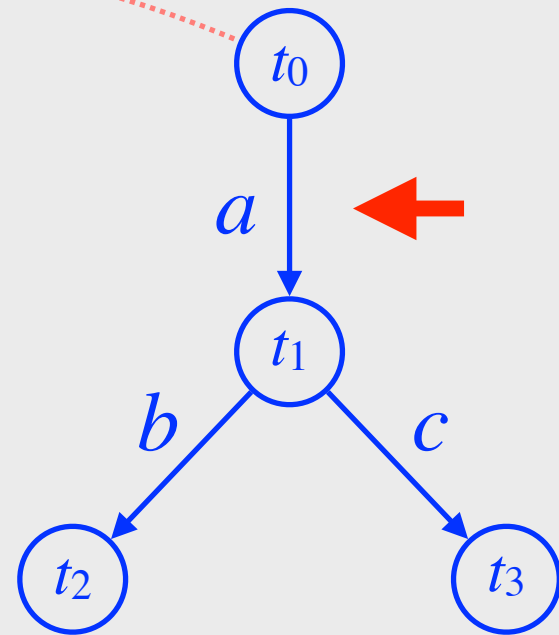
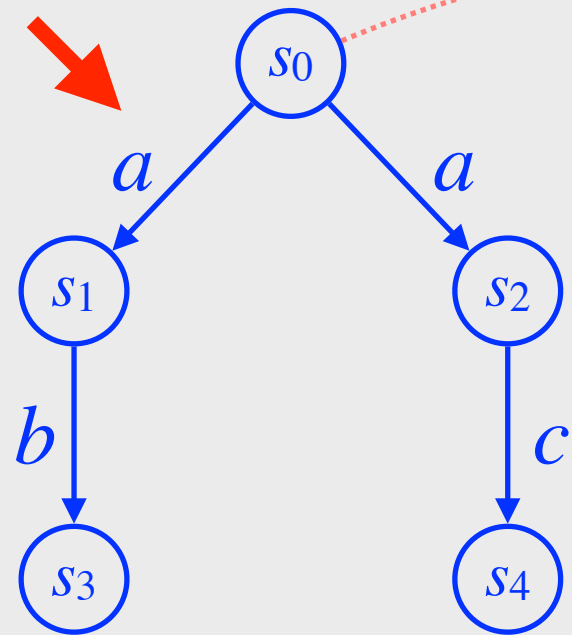
Una relación $R \subseteq S \times S$ es una *simulación* si para todo par $(s, t) \in R$ y evento $a \in E$,

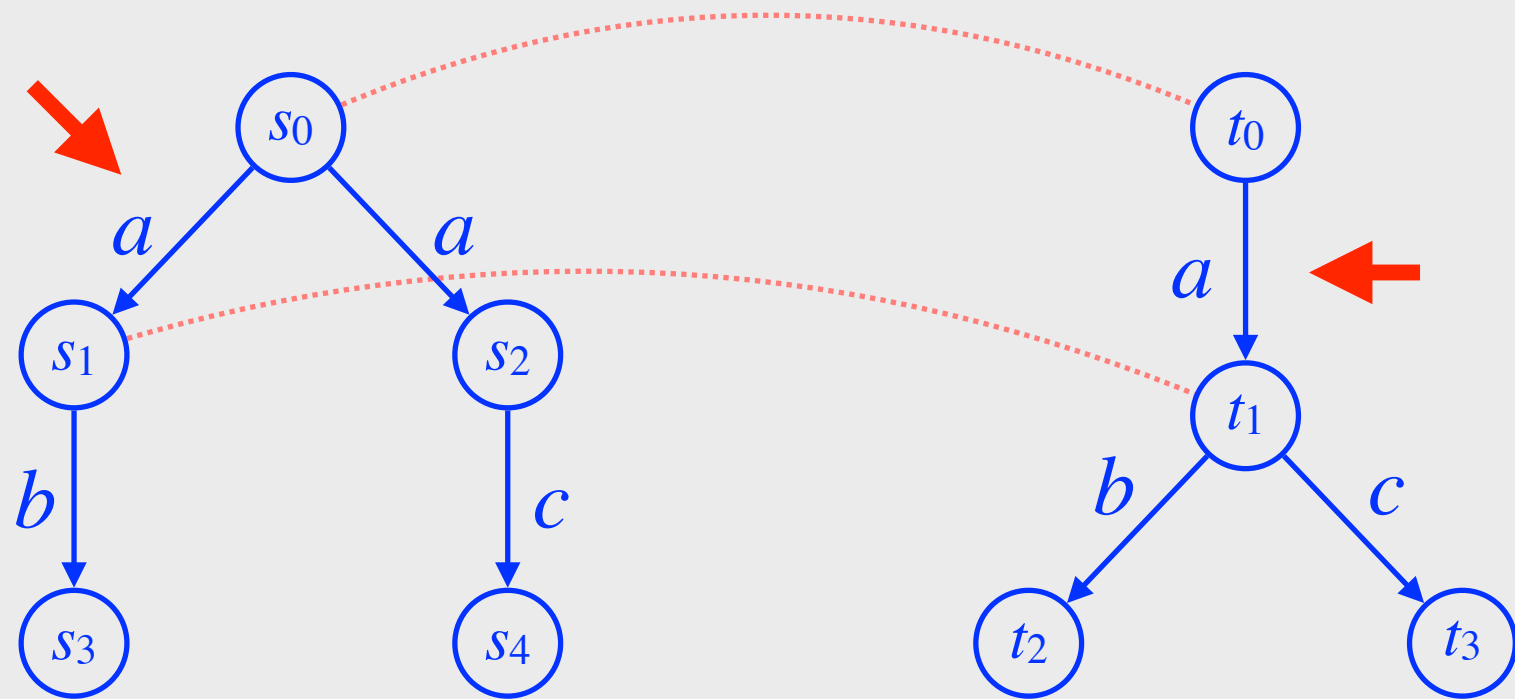
$$\forall s' : s \xrightarrow{a} s' \text{ implica } \exists t' : t \xrightarrow{a} t' \text{ y } (s', t') \in R$$

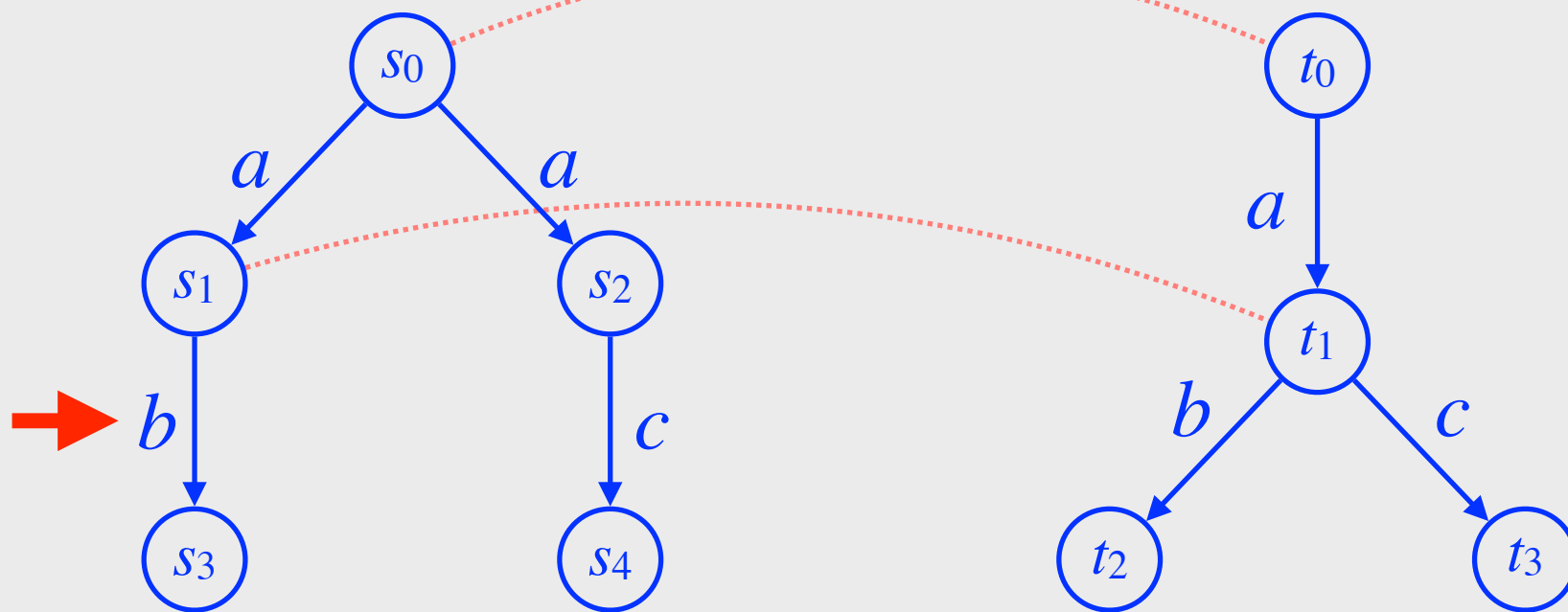


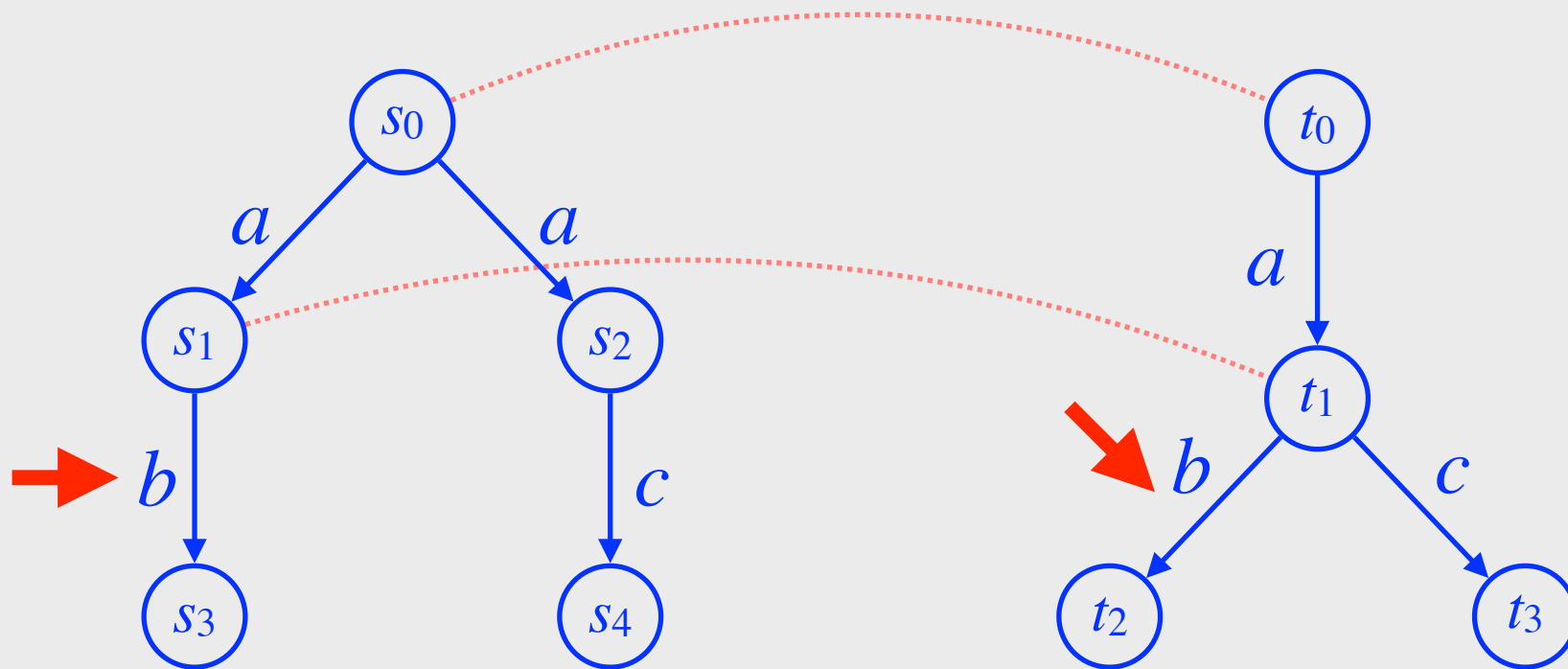


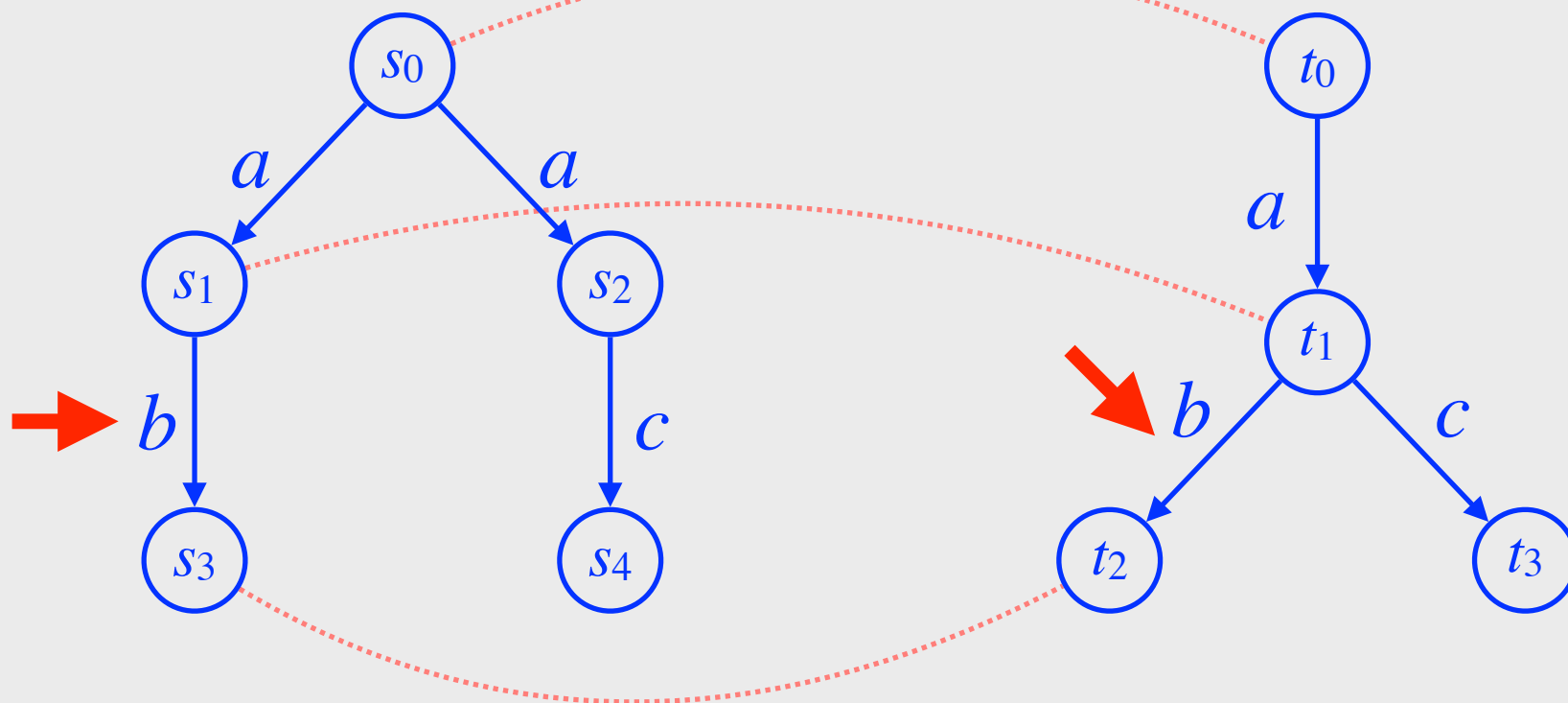


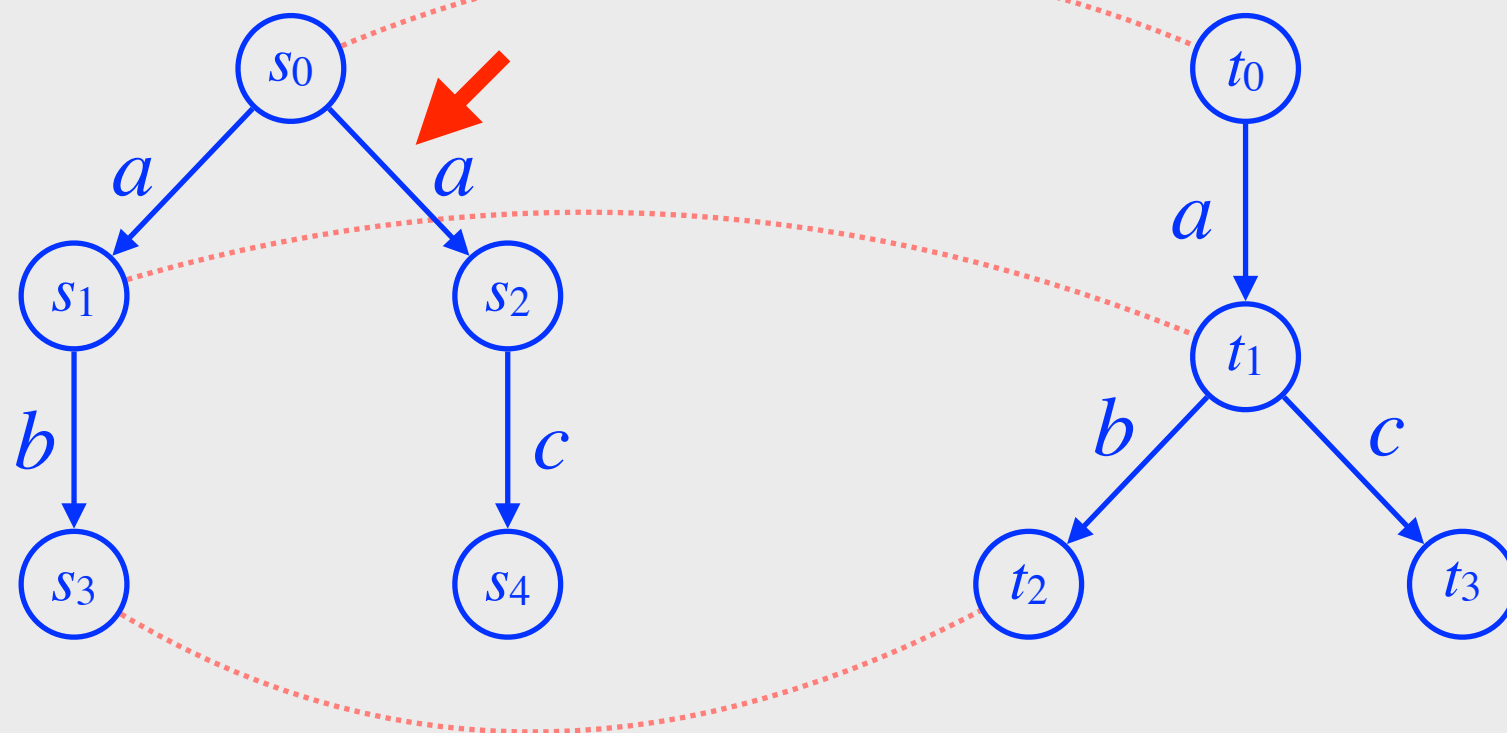


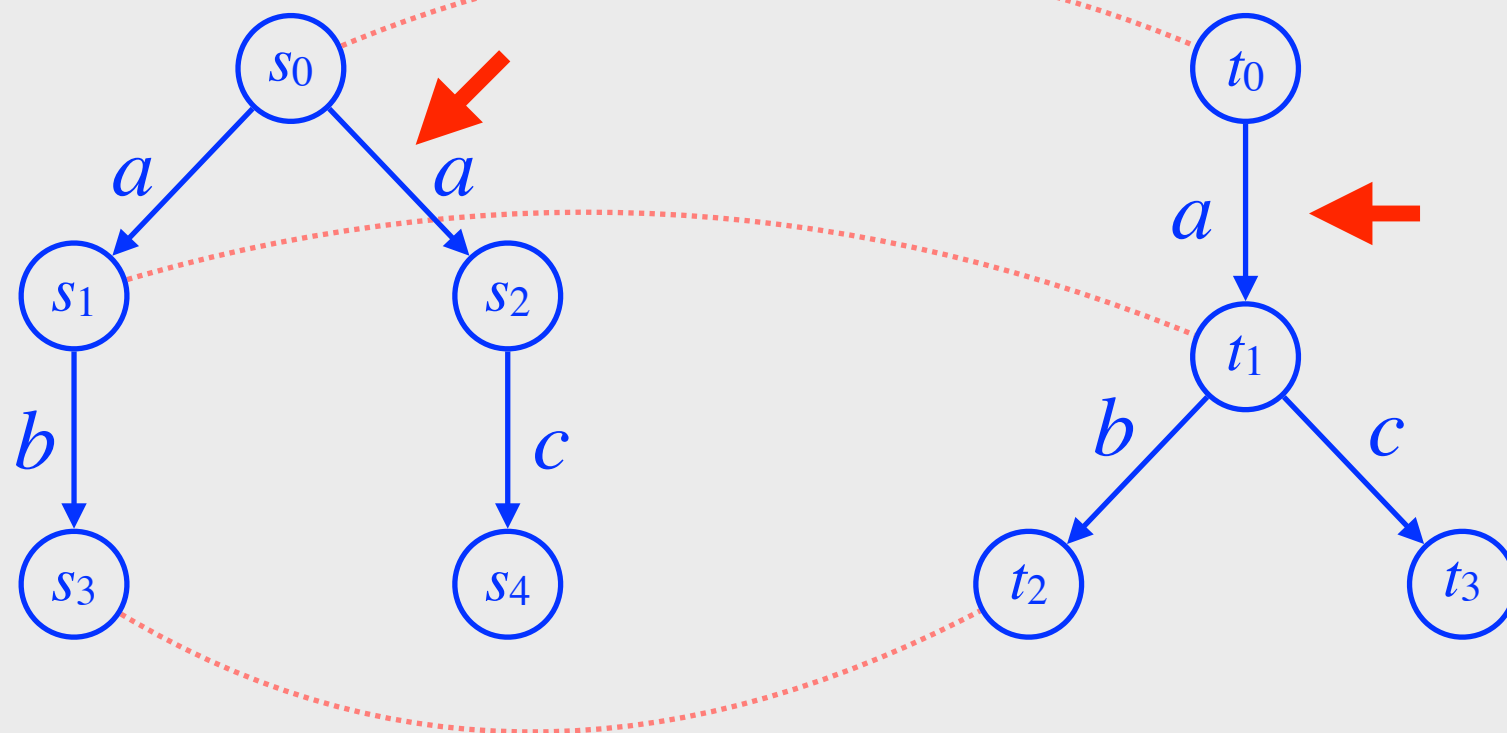


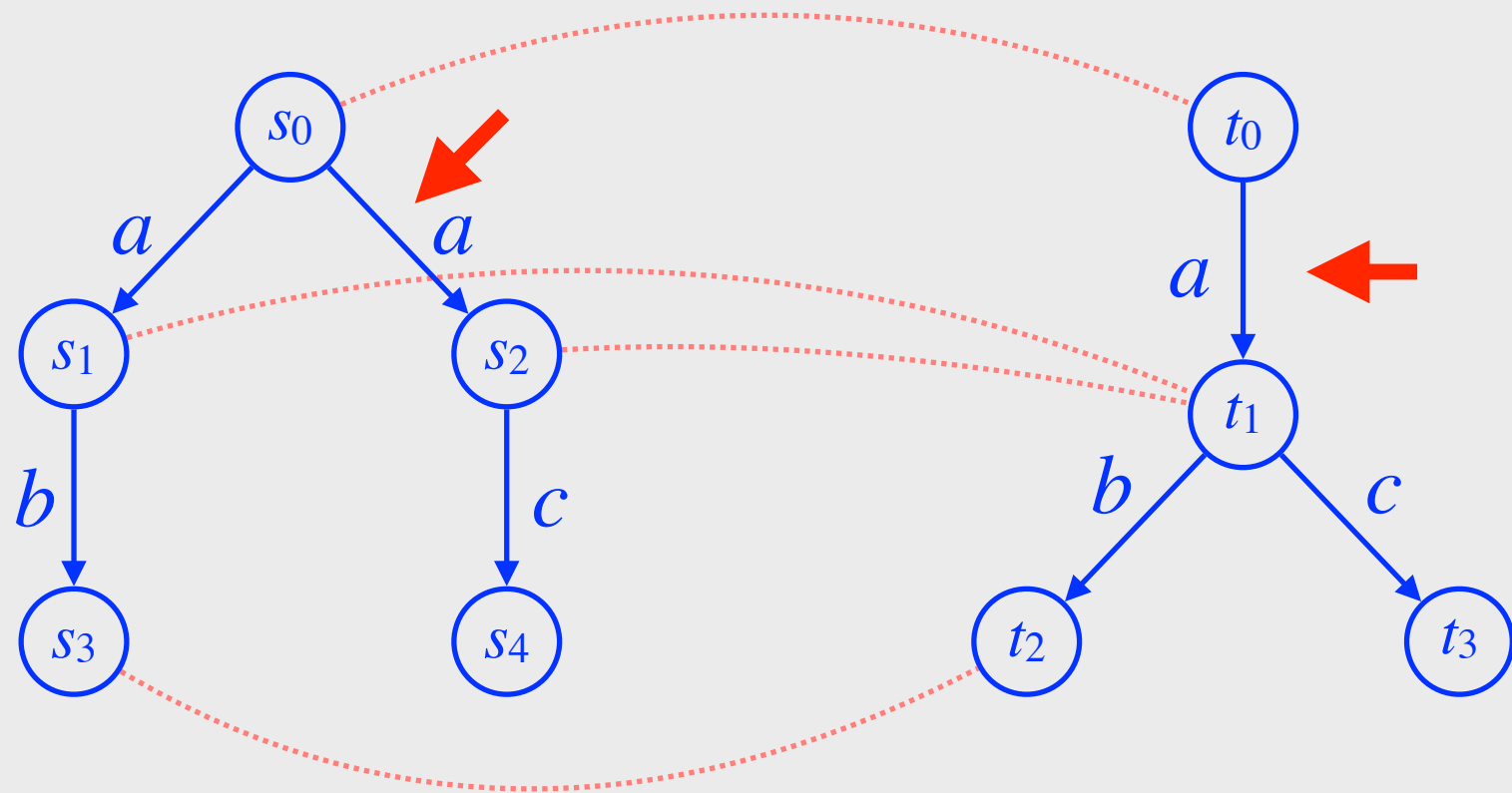


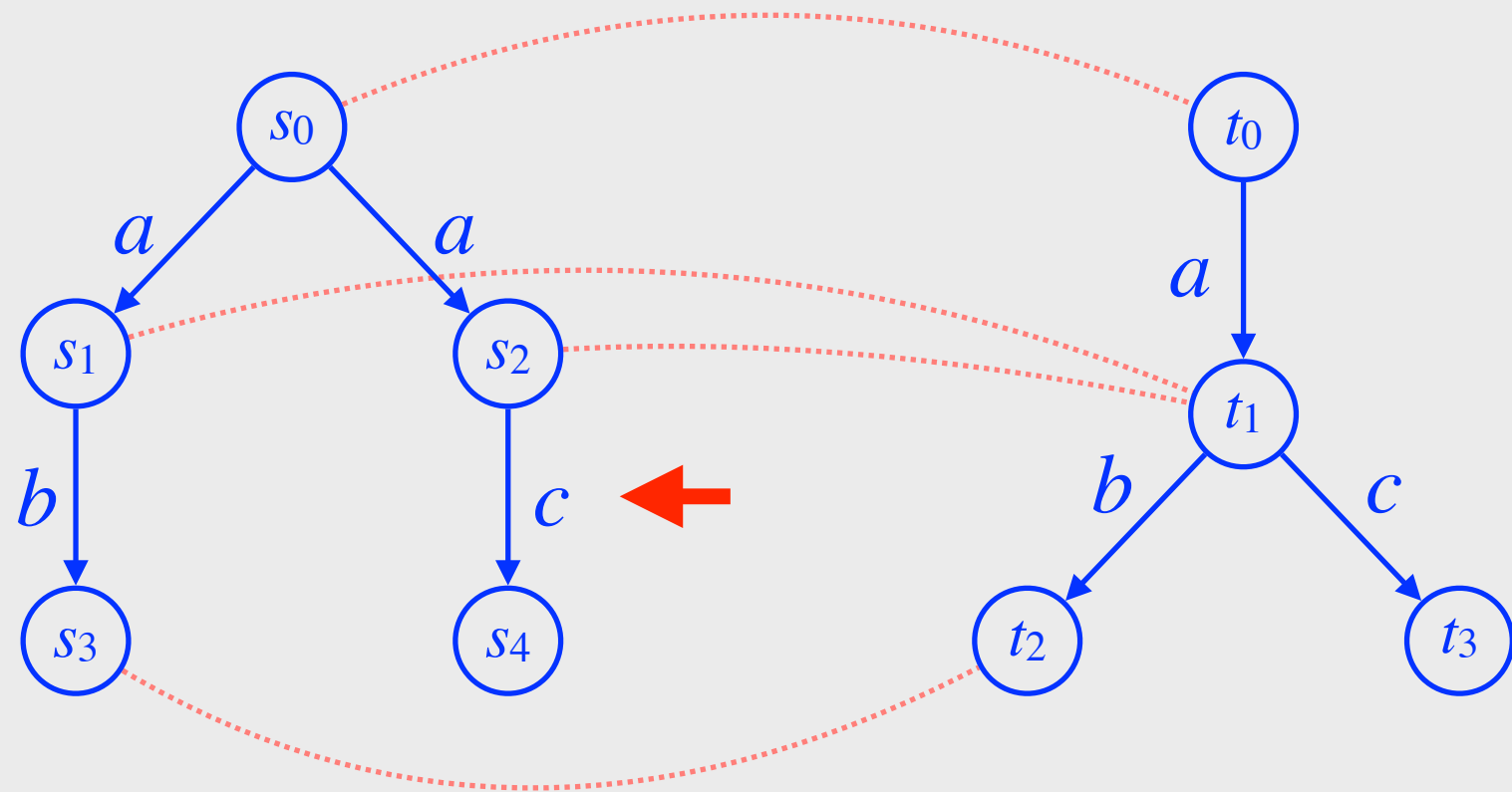


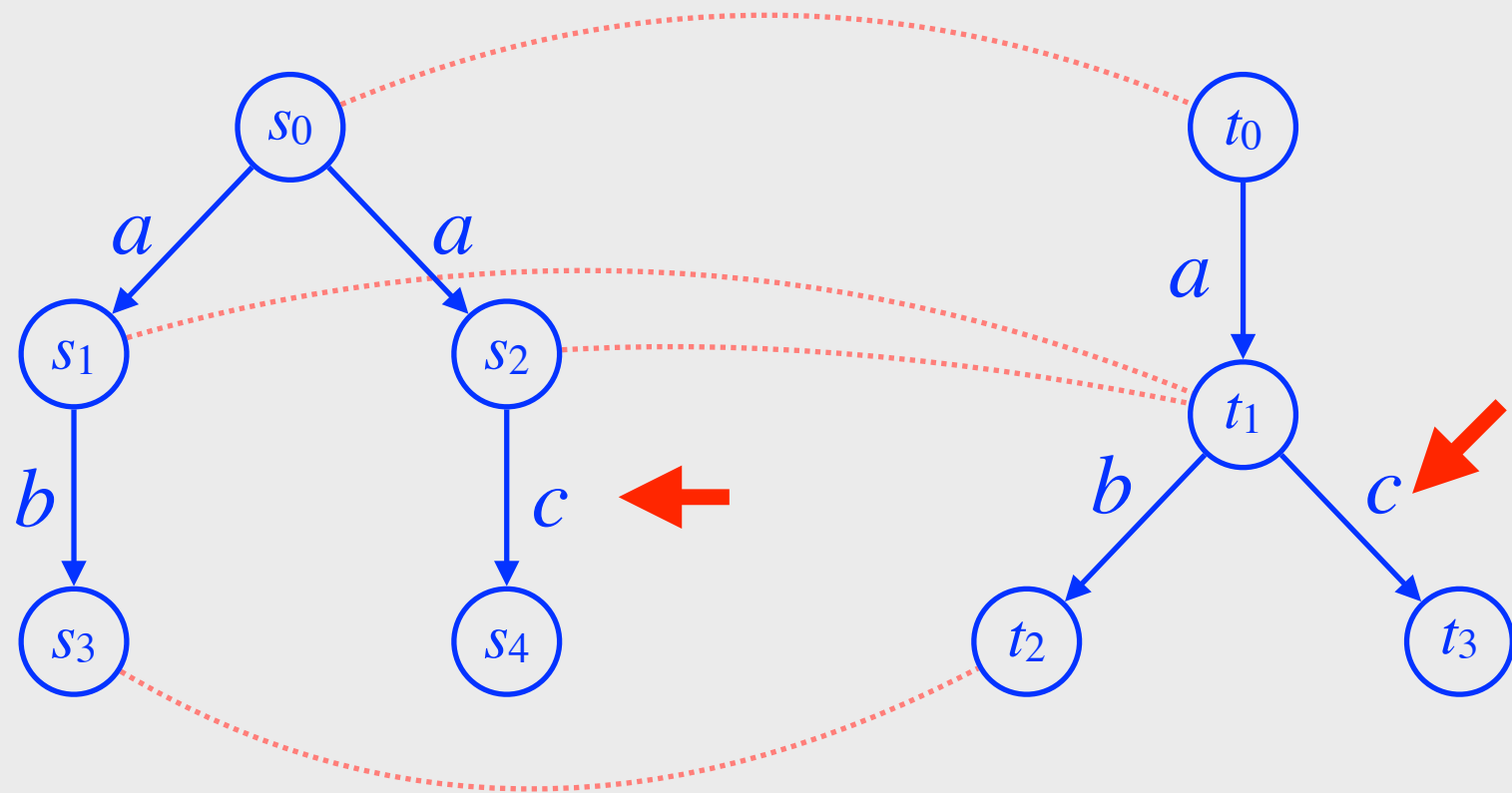


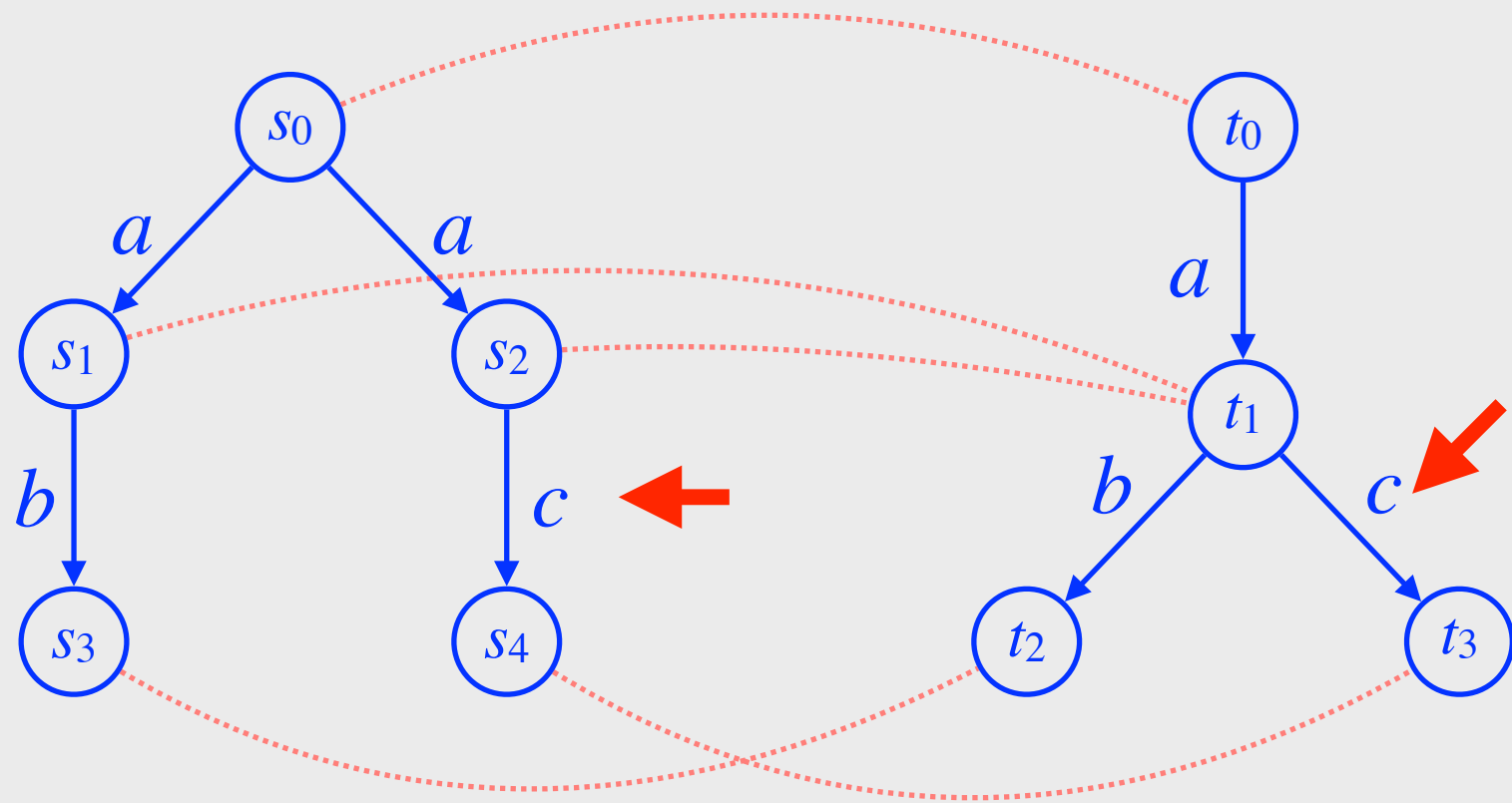


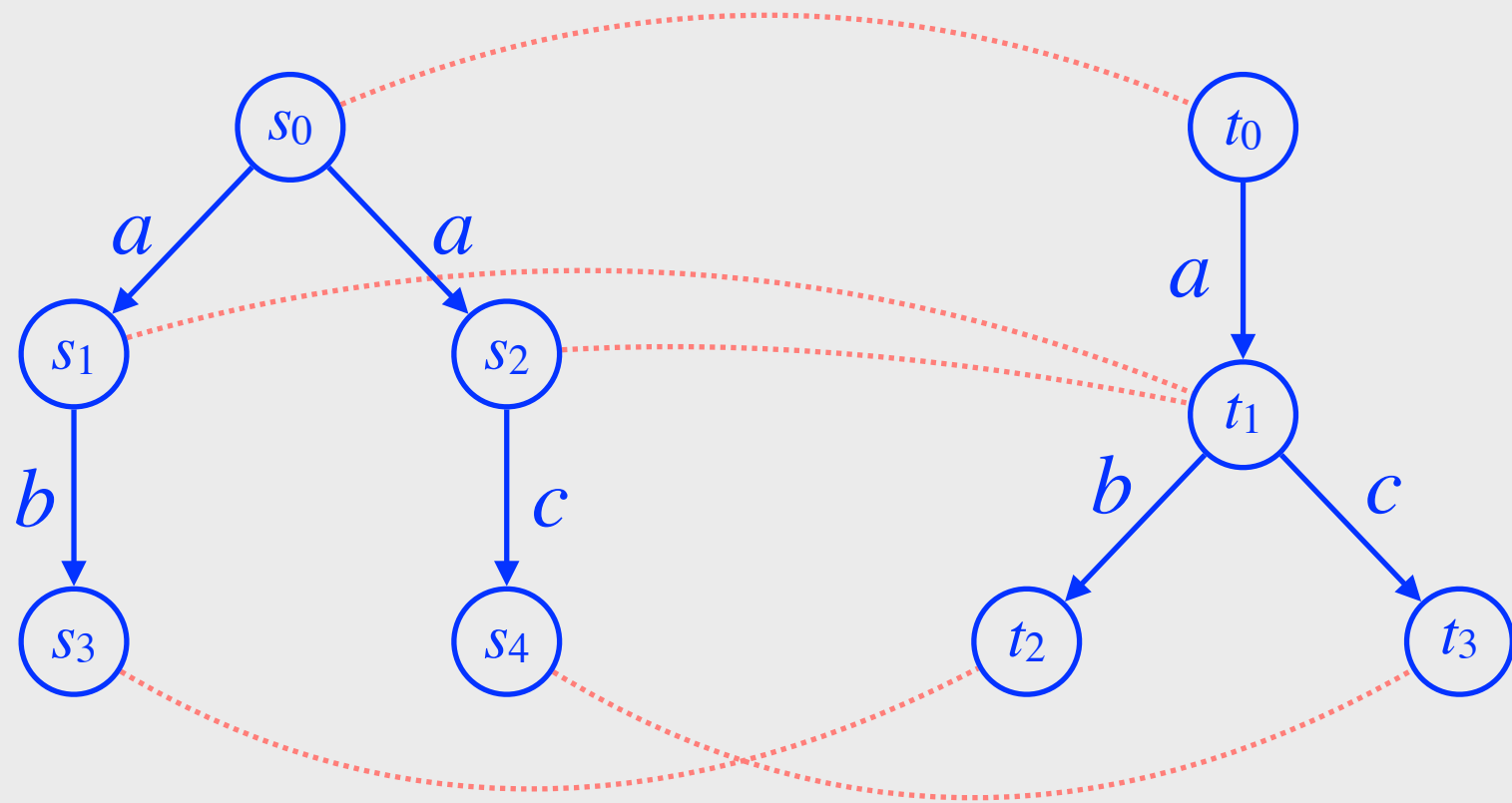








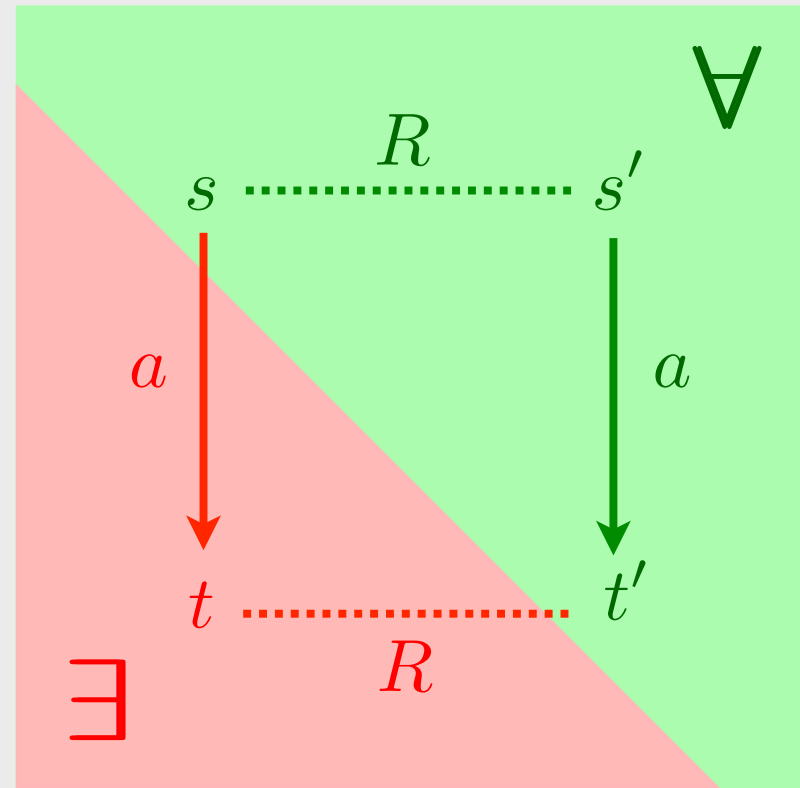
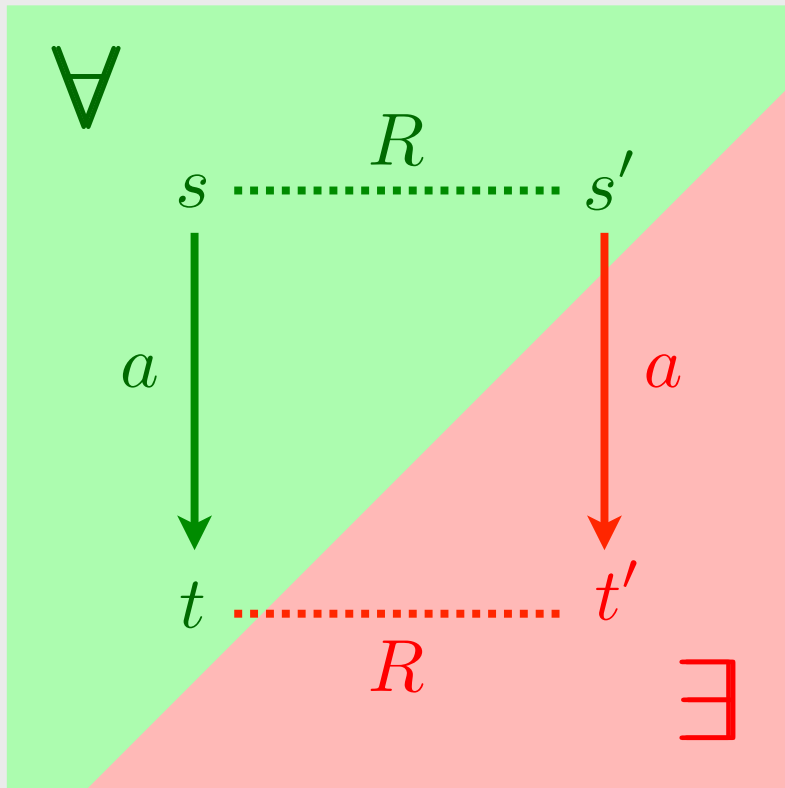




t_0 simula a s_0



Simulación en LTS



R^{-1} también debe
satisfacer esta propiedad



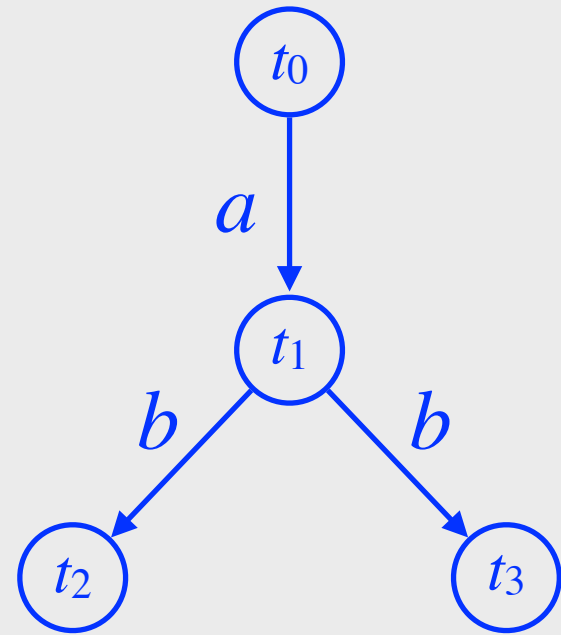
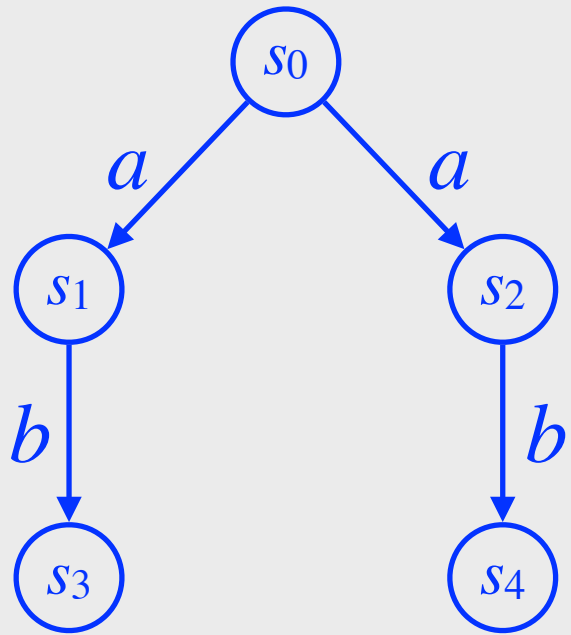
Simulación en LTS

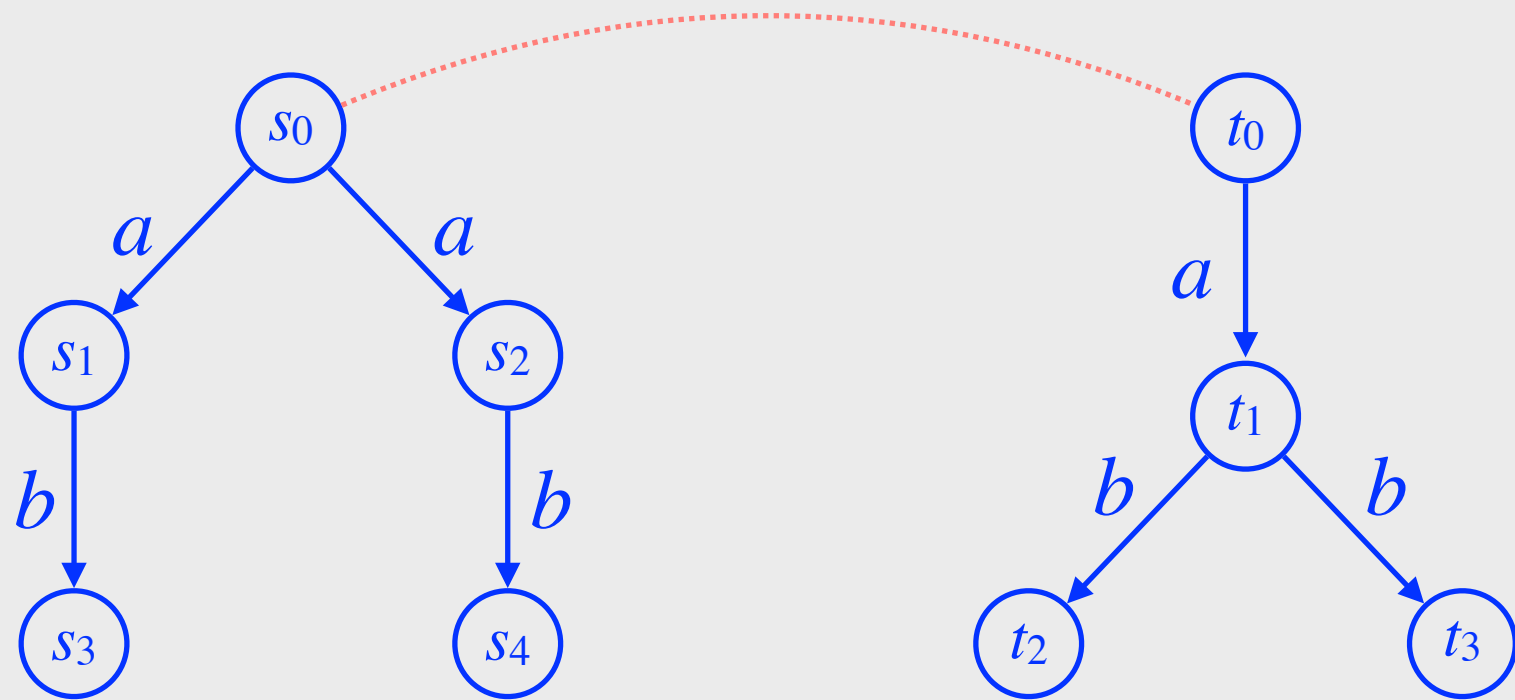
Sea $\mathcal{S} = (S, s_0, E, \rightarrow)$ un *sistema de transiciones etiquetadas*, i.e., $\rightarrow \subseteq S \times E \times S$.

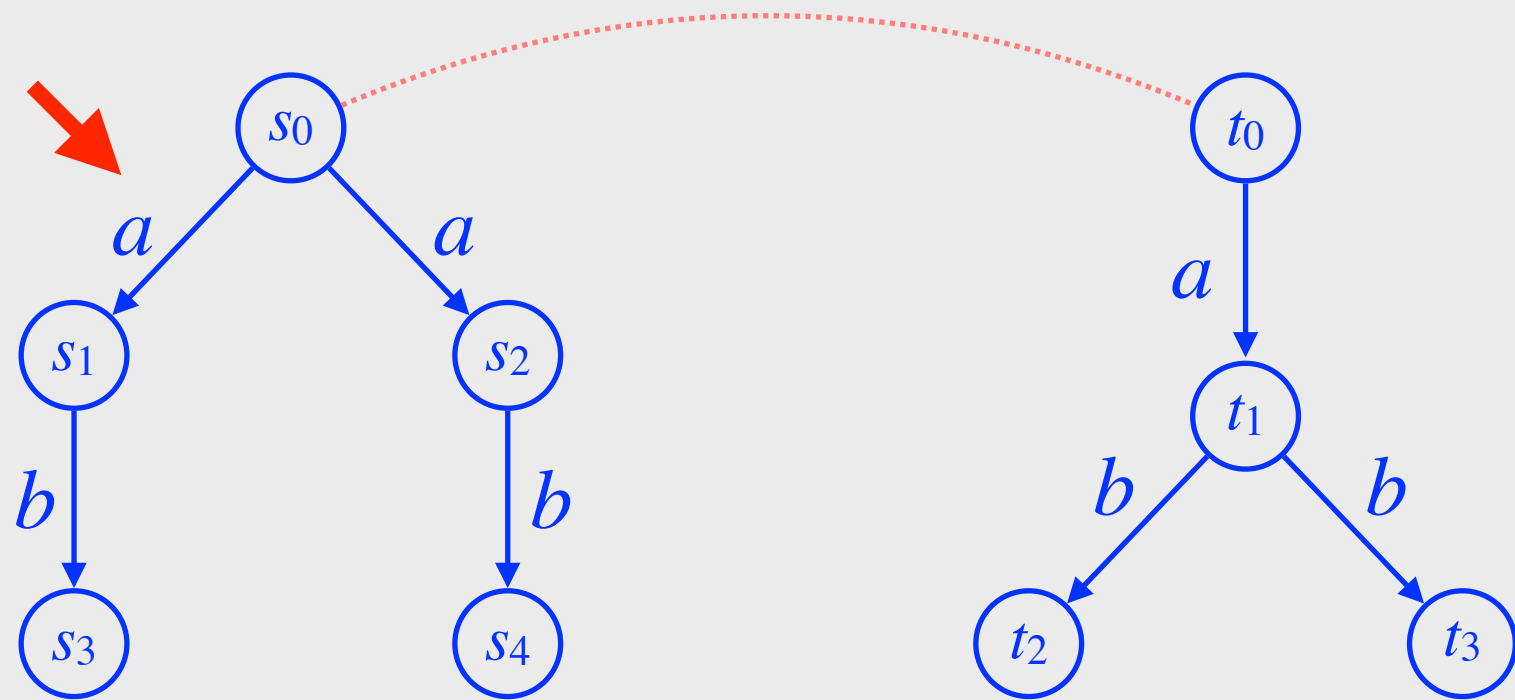
Una relación $R \subseteq S \times S$ es una *simulación* si para todo par $(s, t) \in R$ y evento $a \in E$,

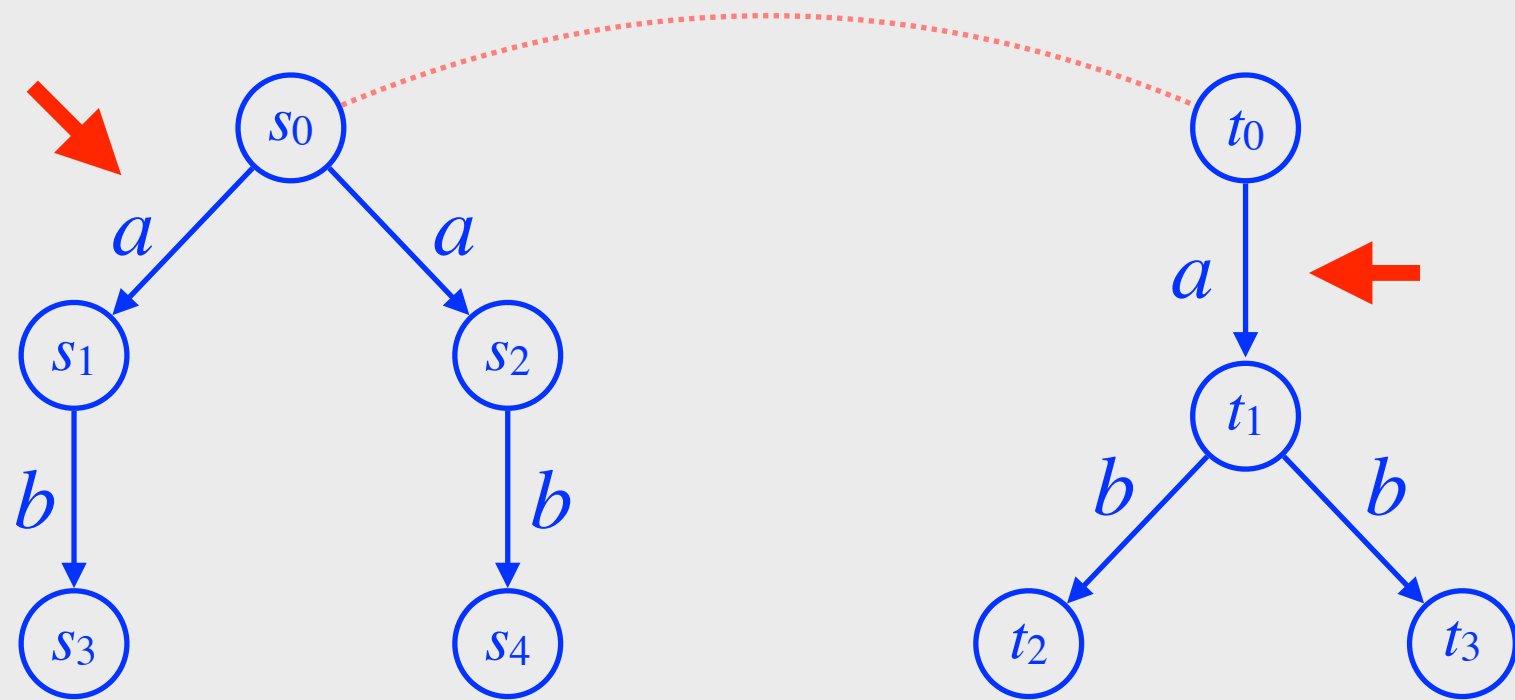
$$\forall s' : s \xrightarrow{a} s' \text{ implica } \exists t' : t \xrightarrow{a} t' \text{ y } (s', t') \in R$$

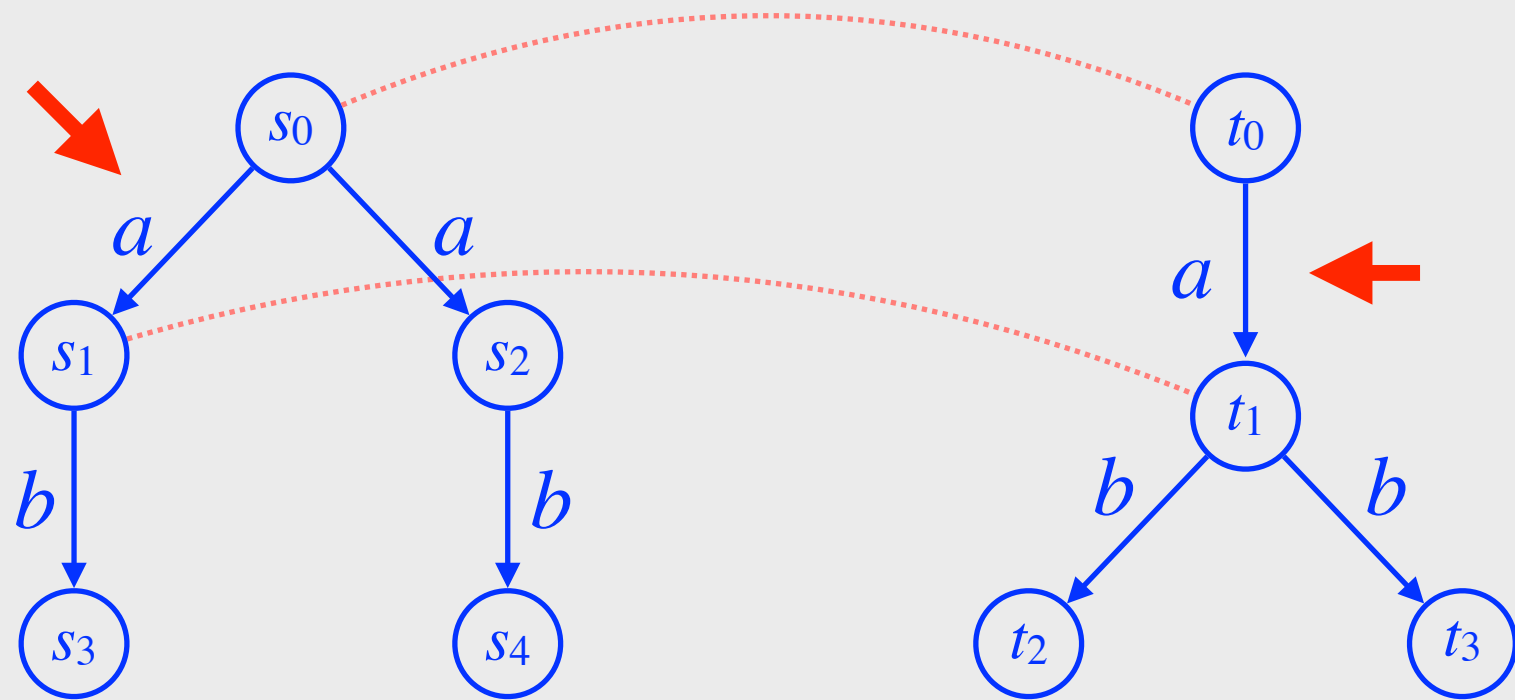
$R \subseteq S \times S$ es una *bisimulación* si R y R^{-1} son ambas simulaciones.

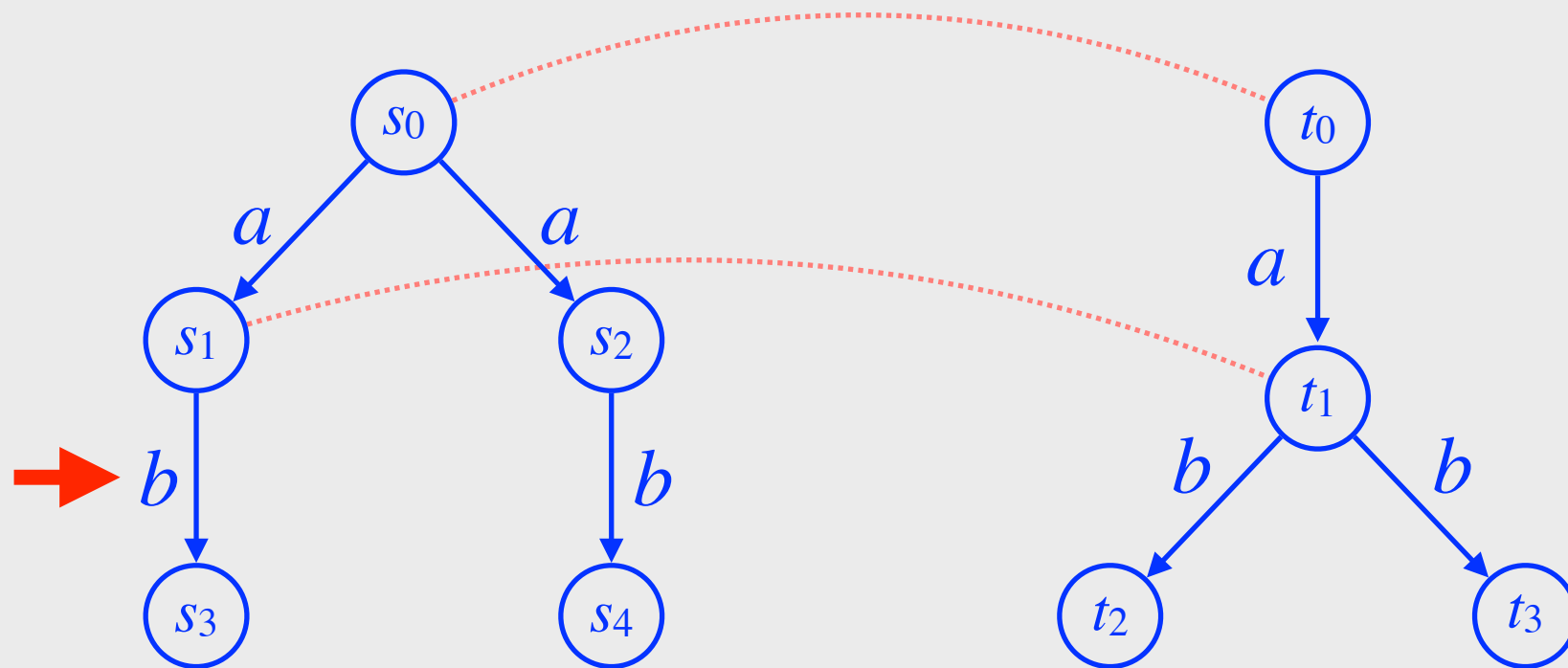


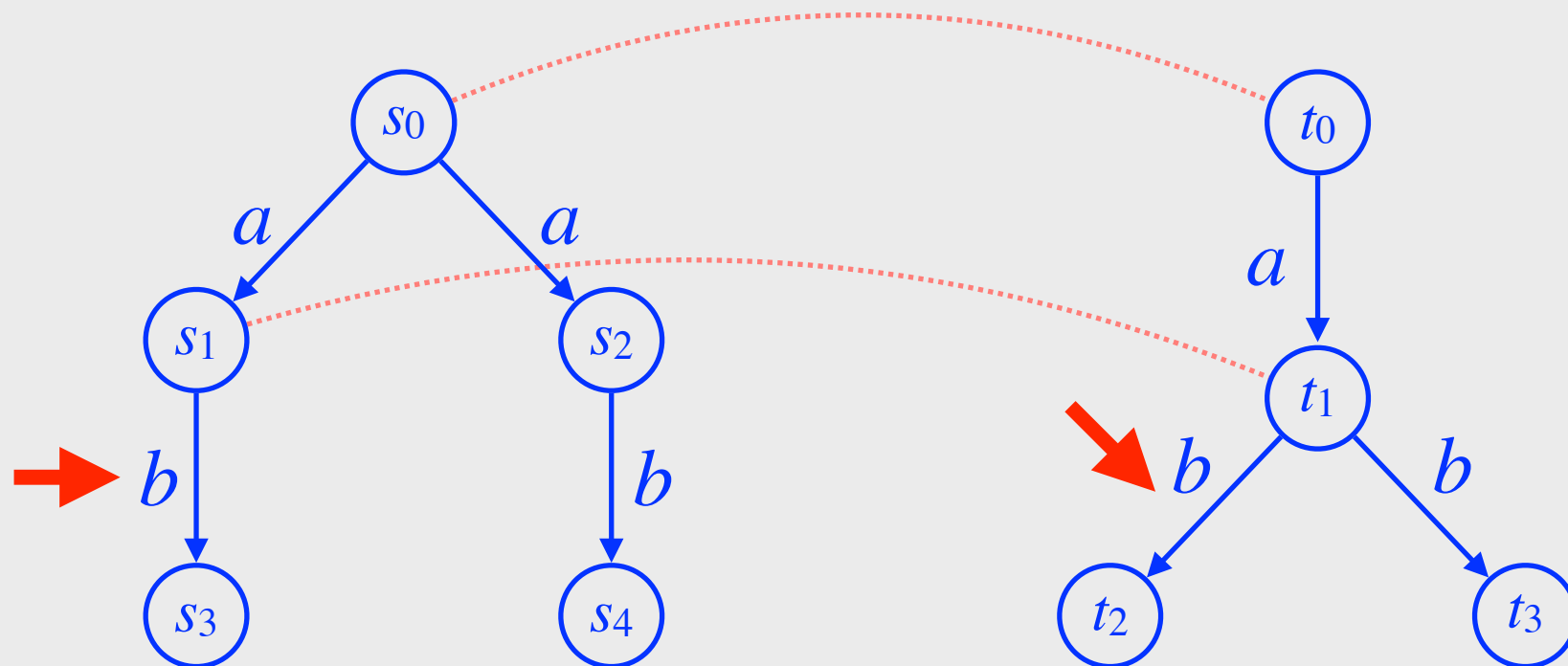


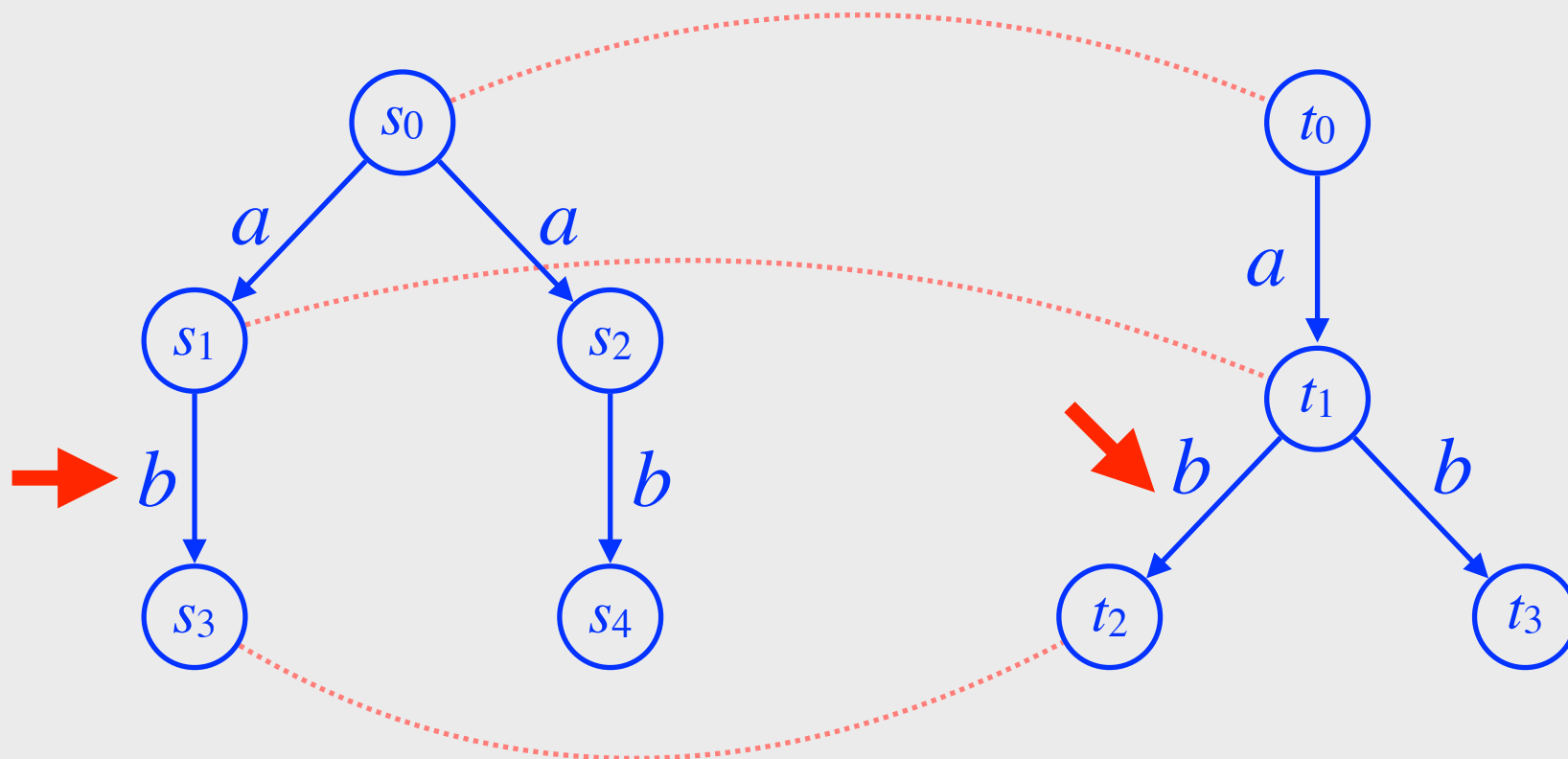


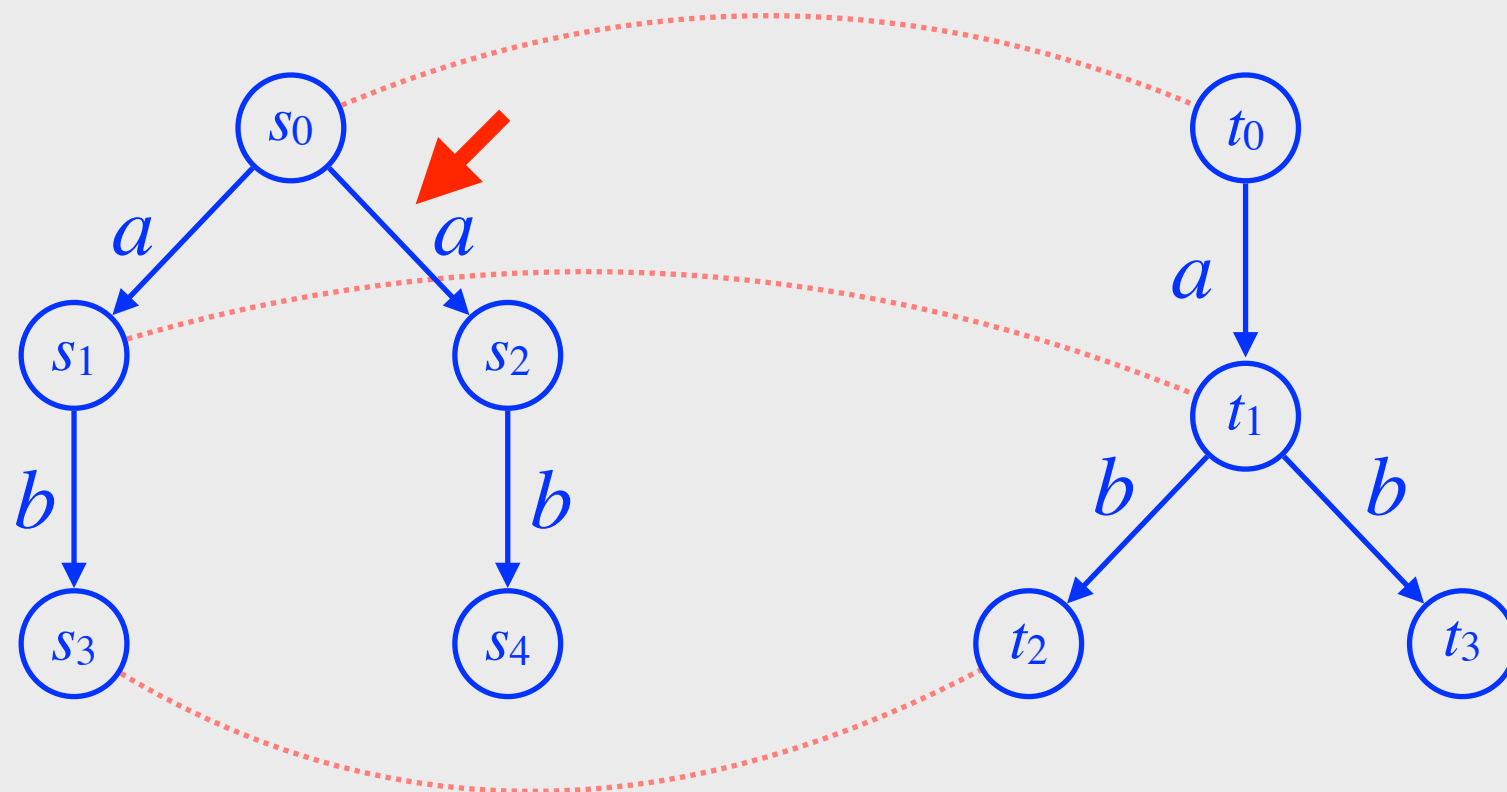


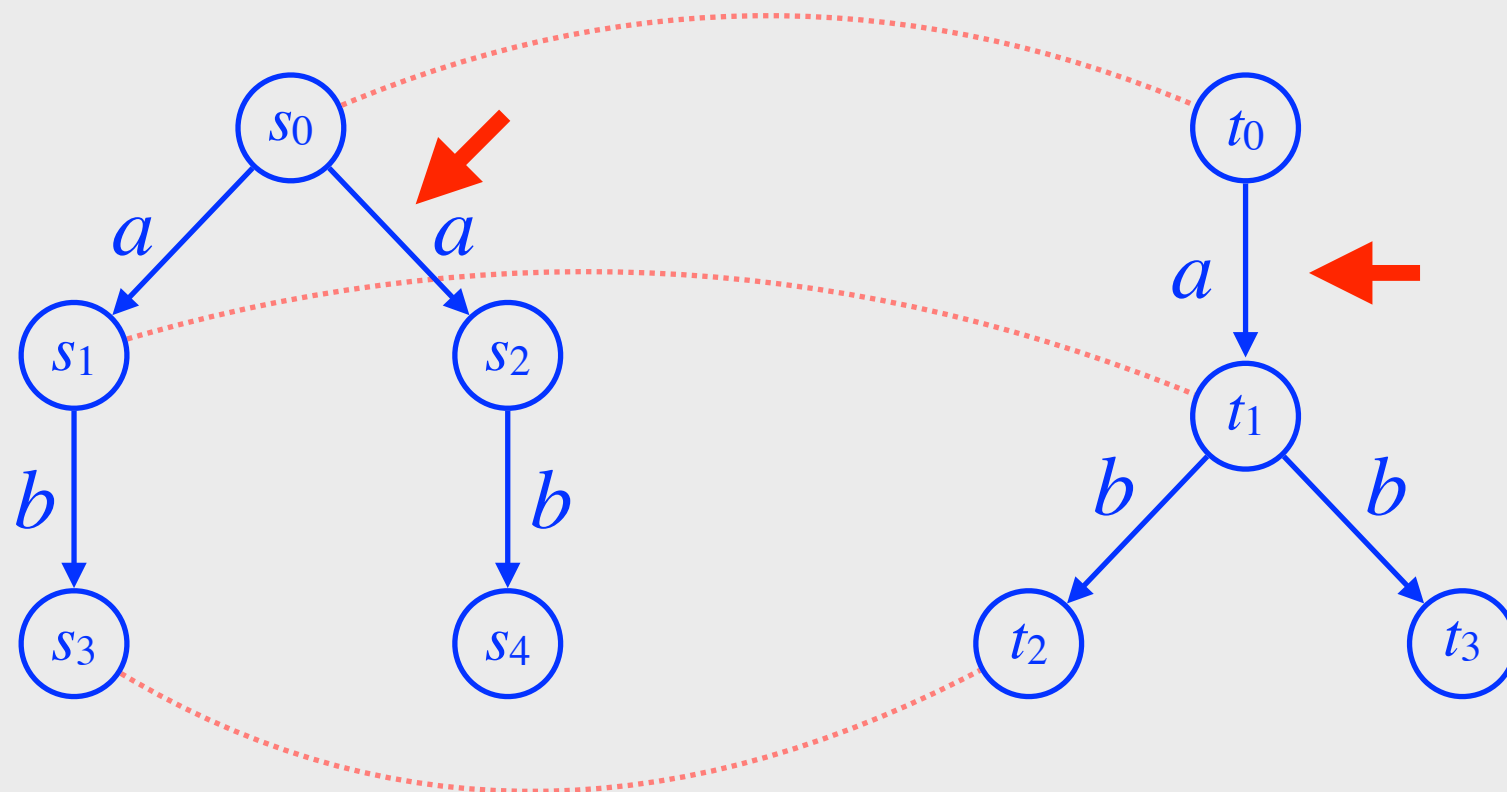


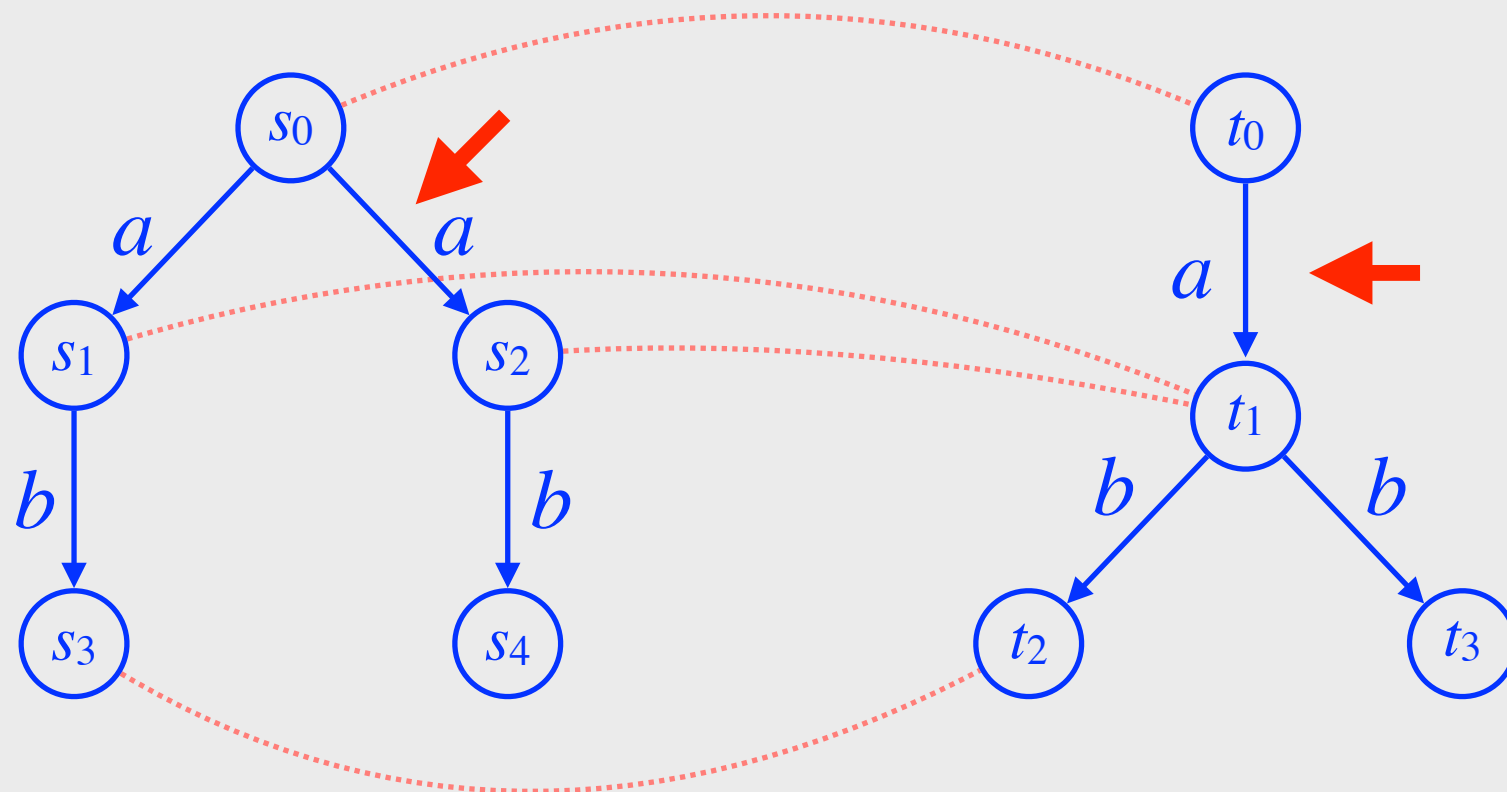


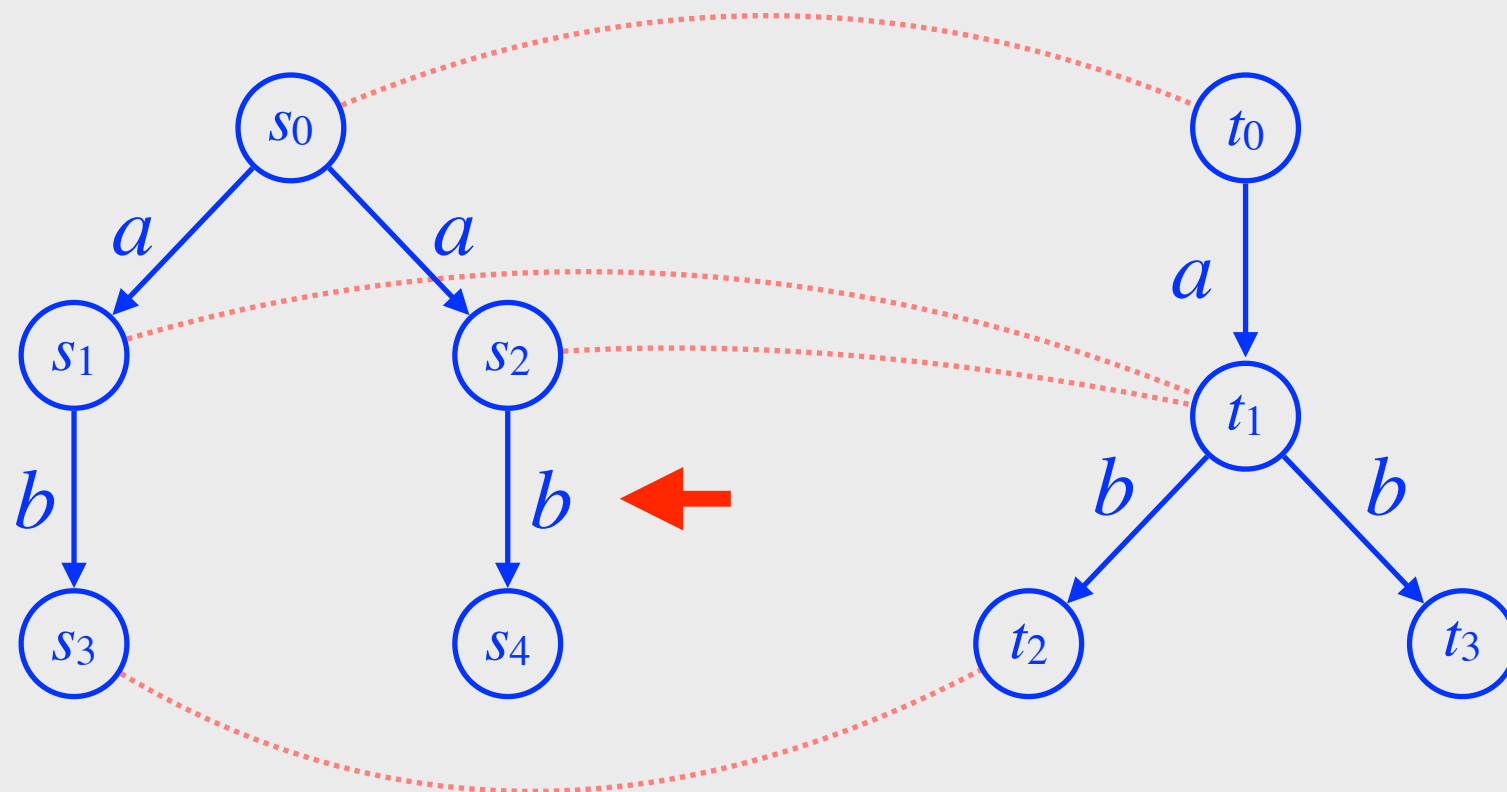


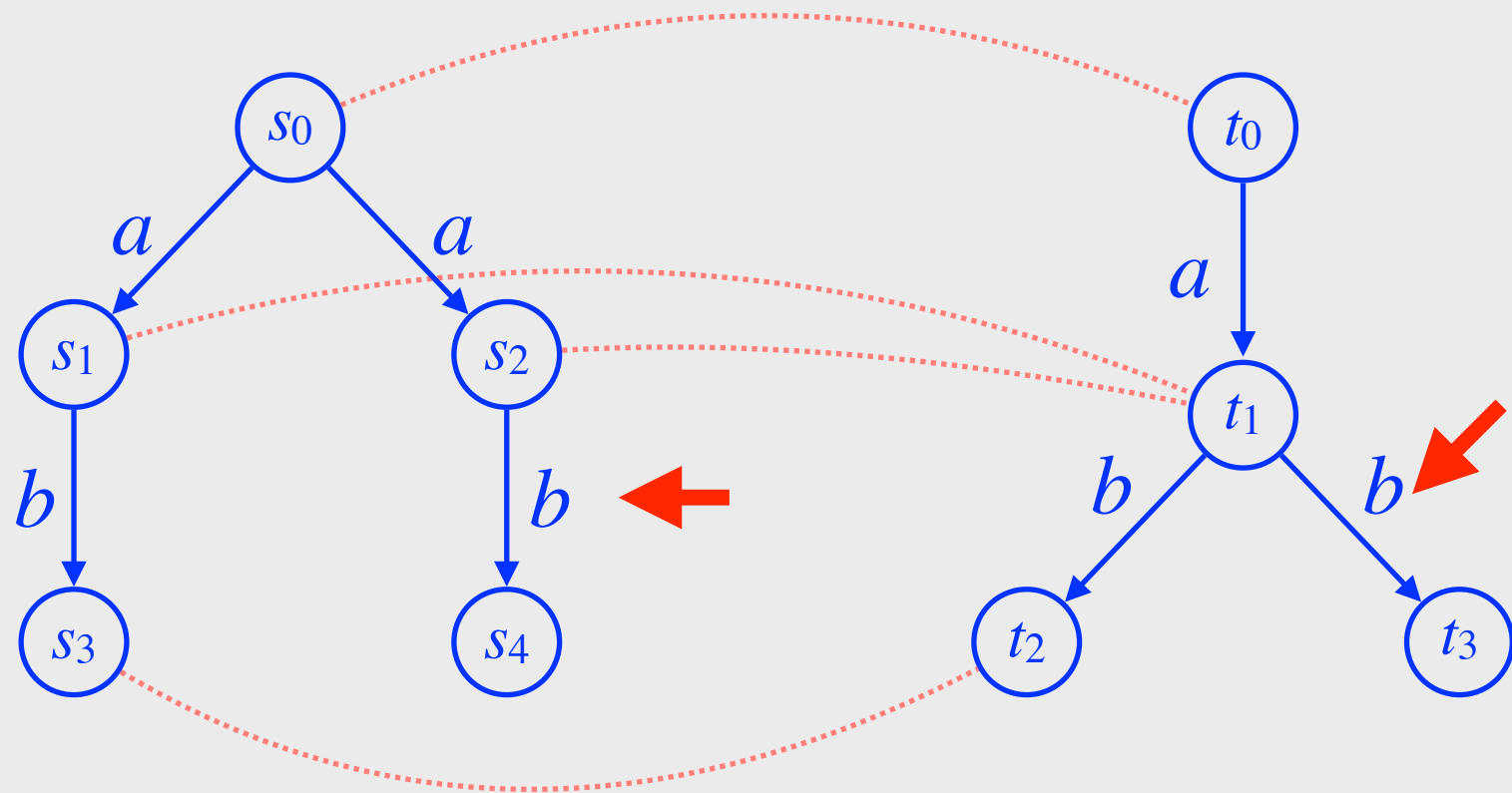


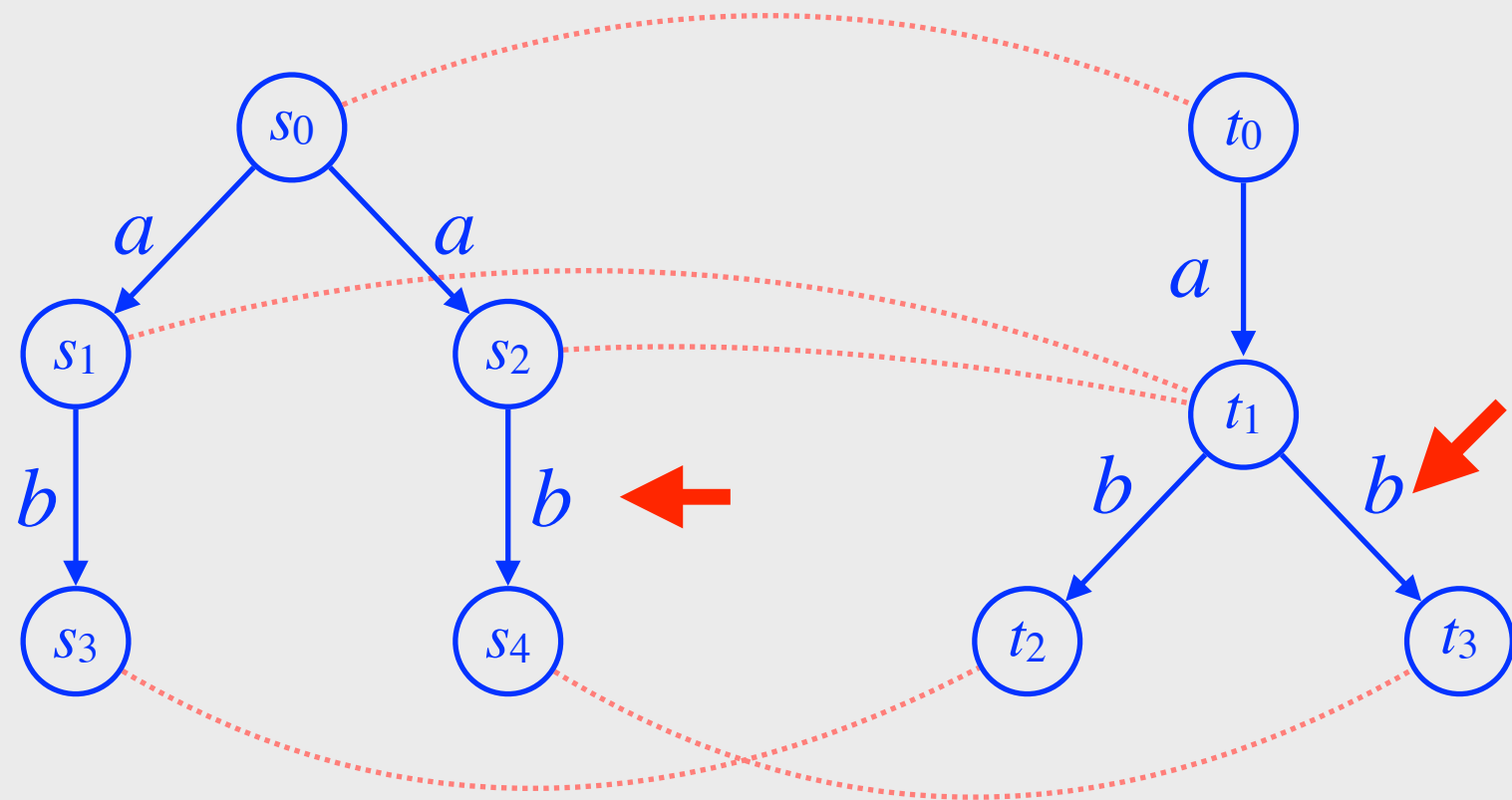


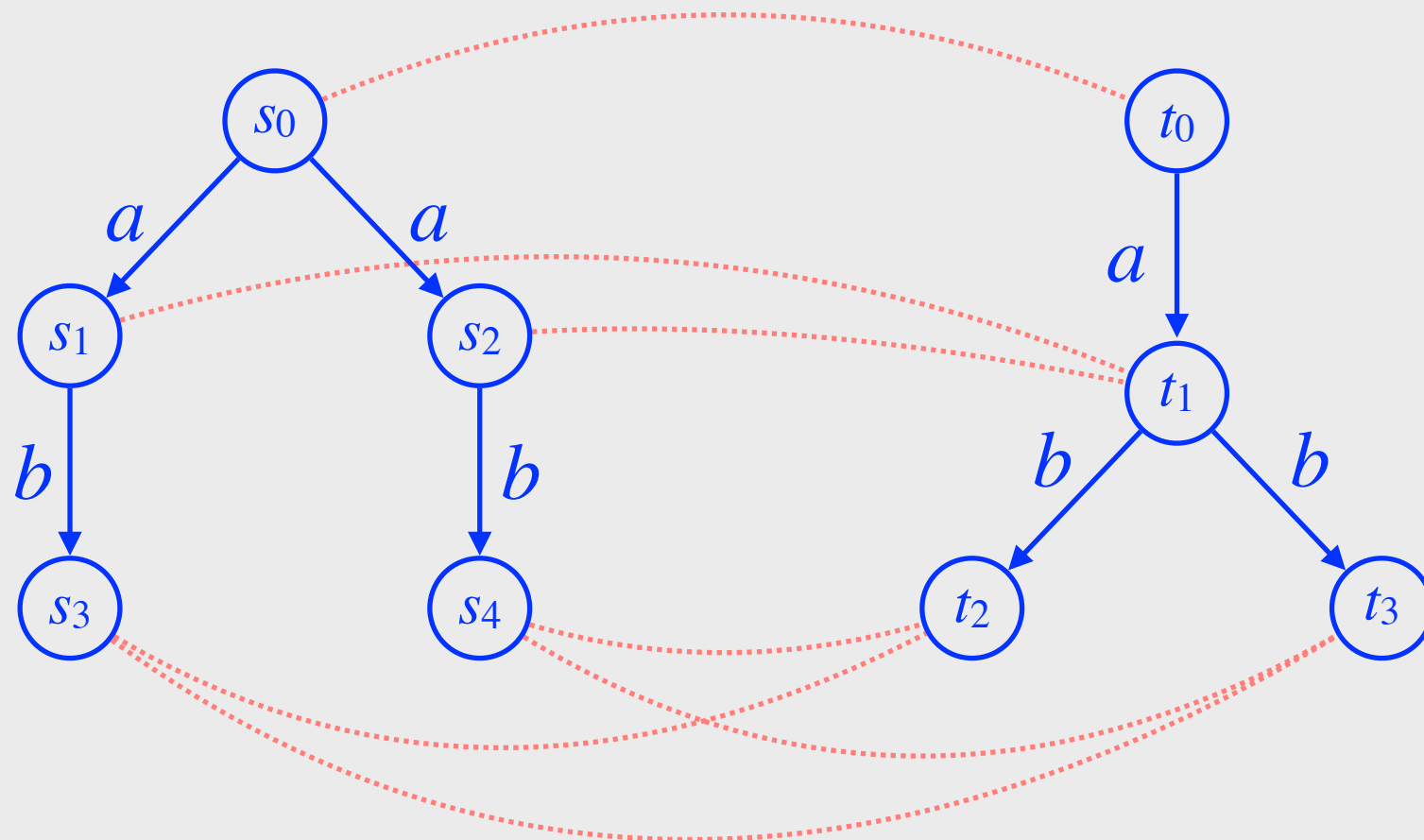


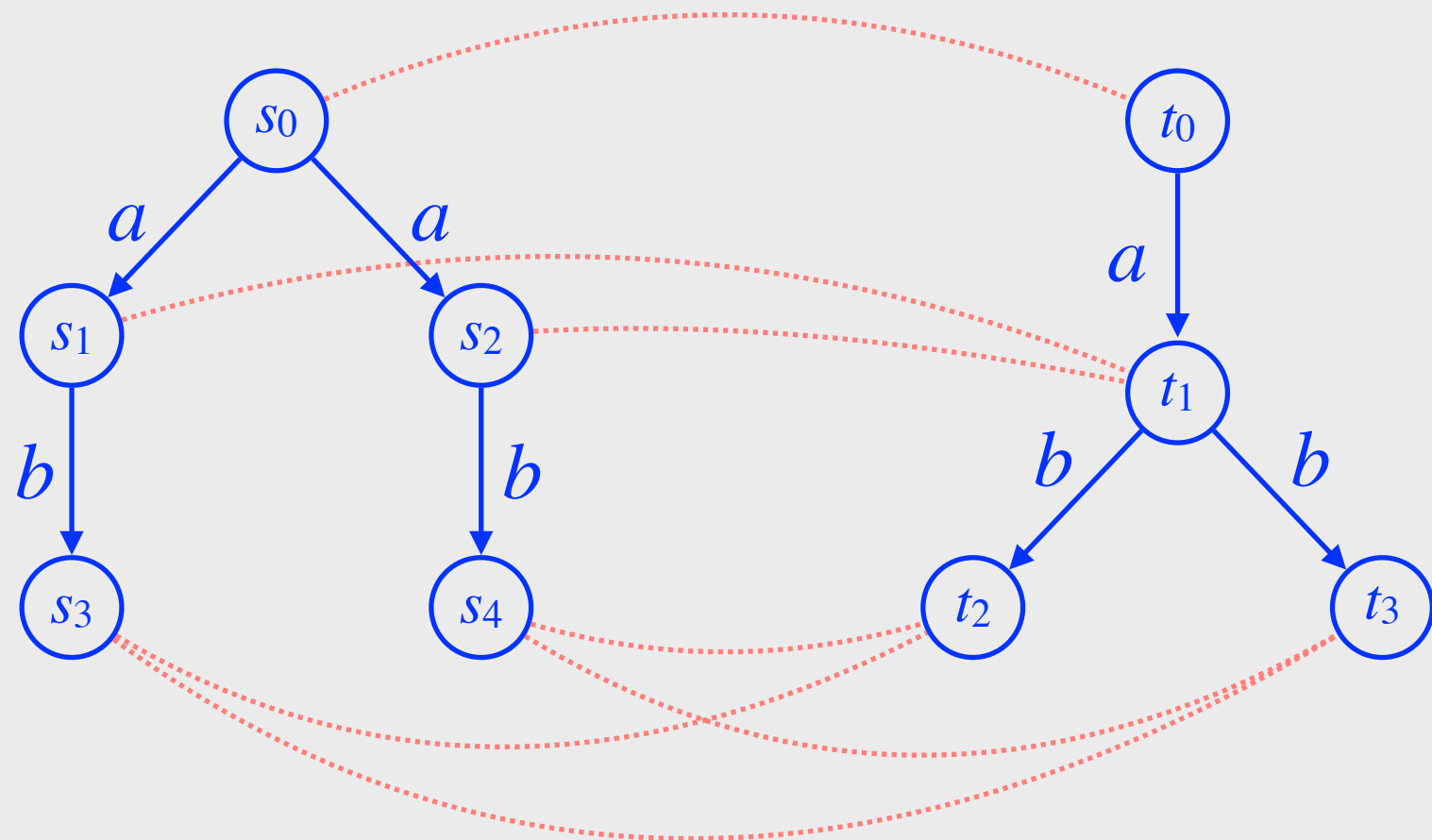




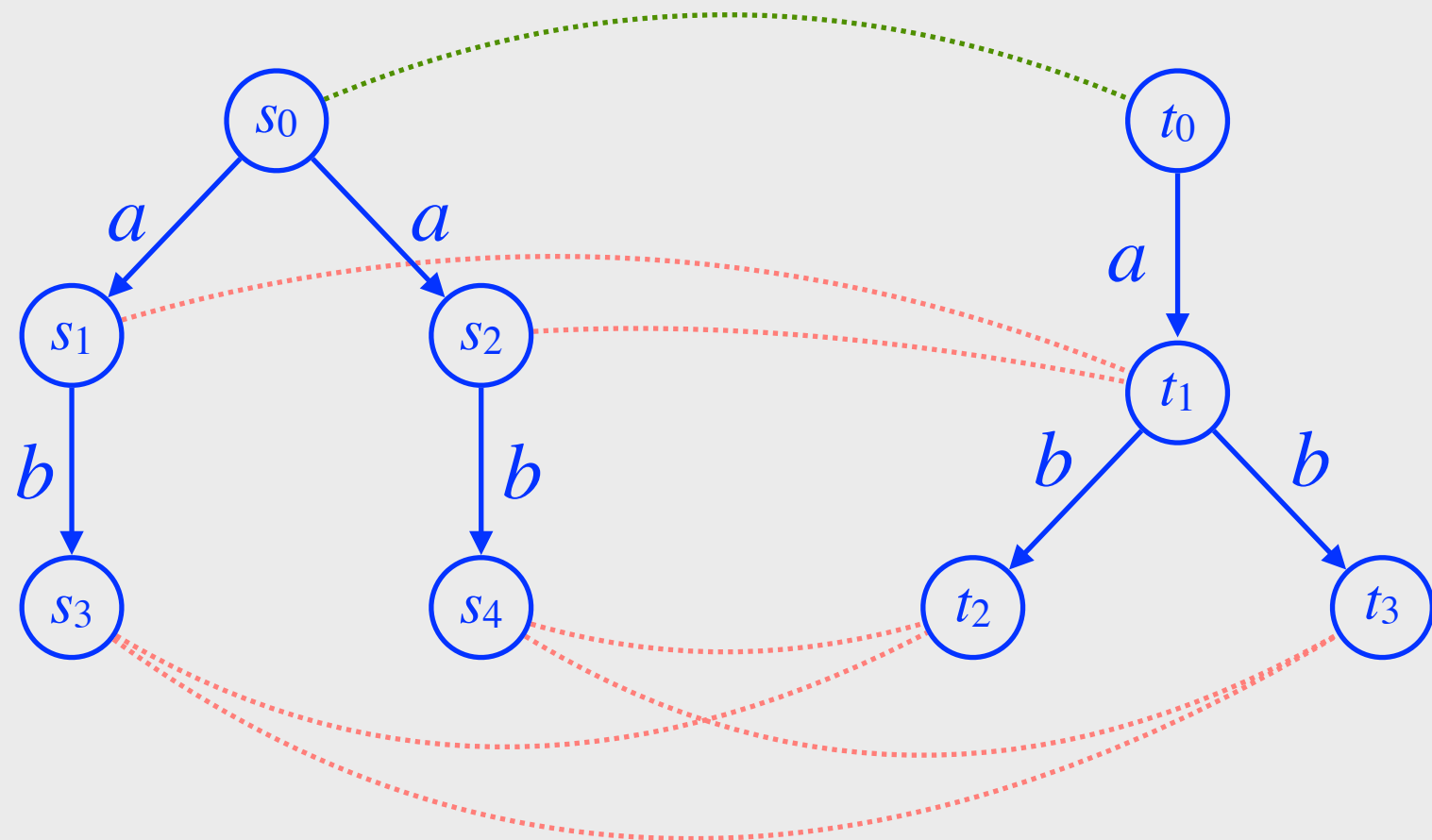




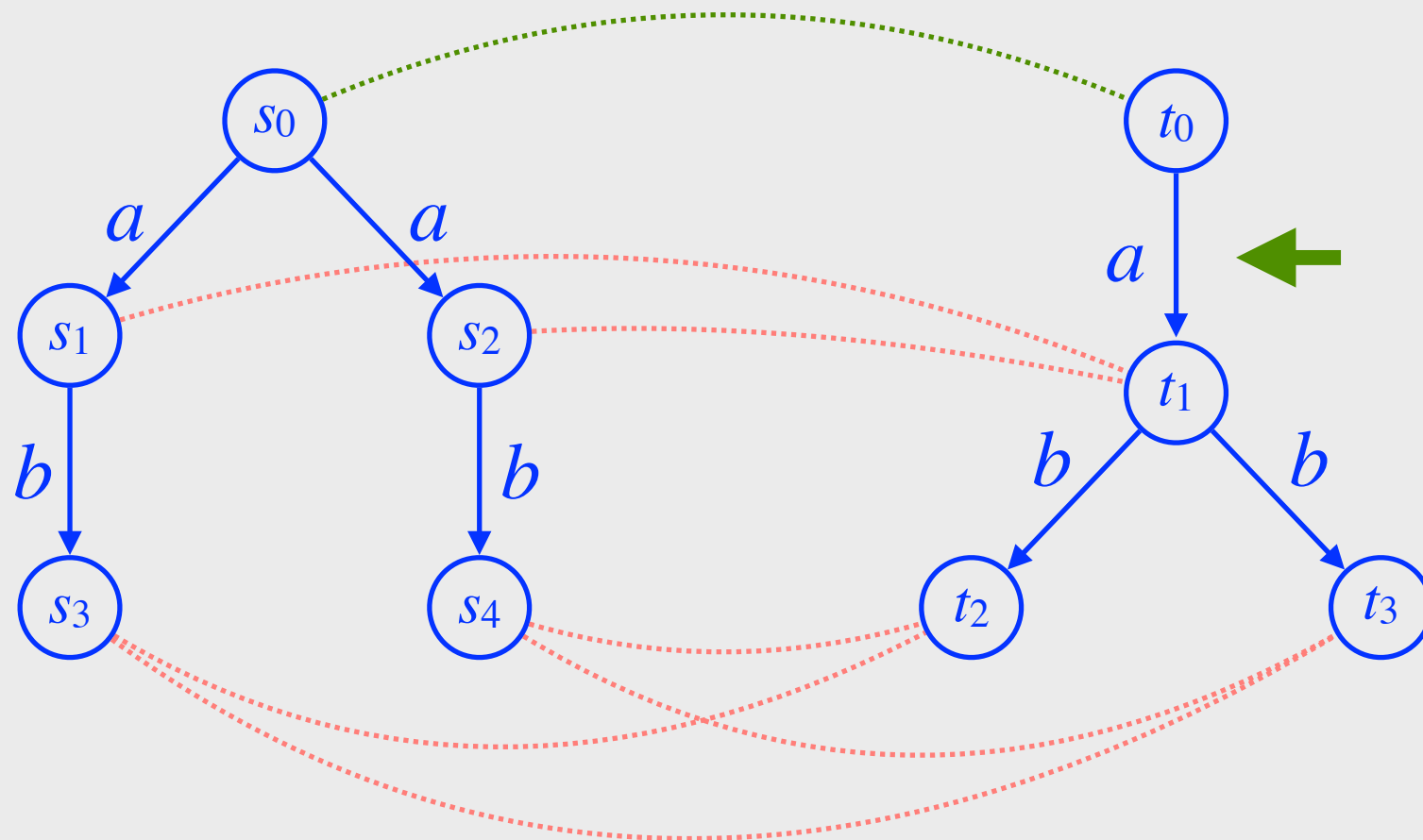




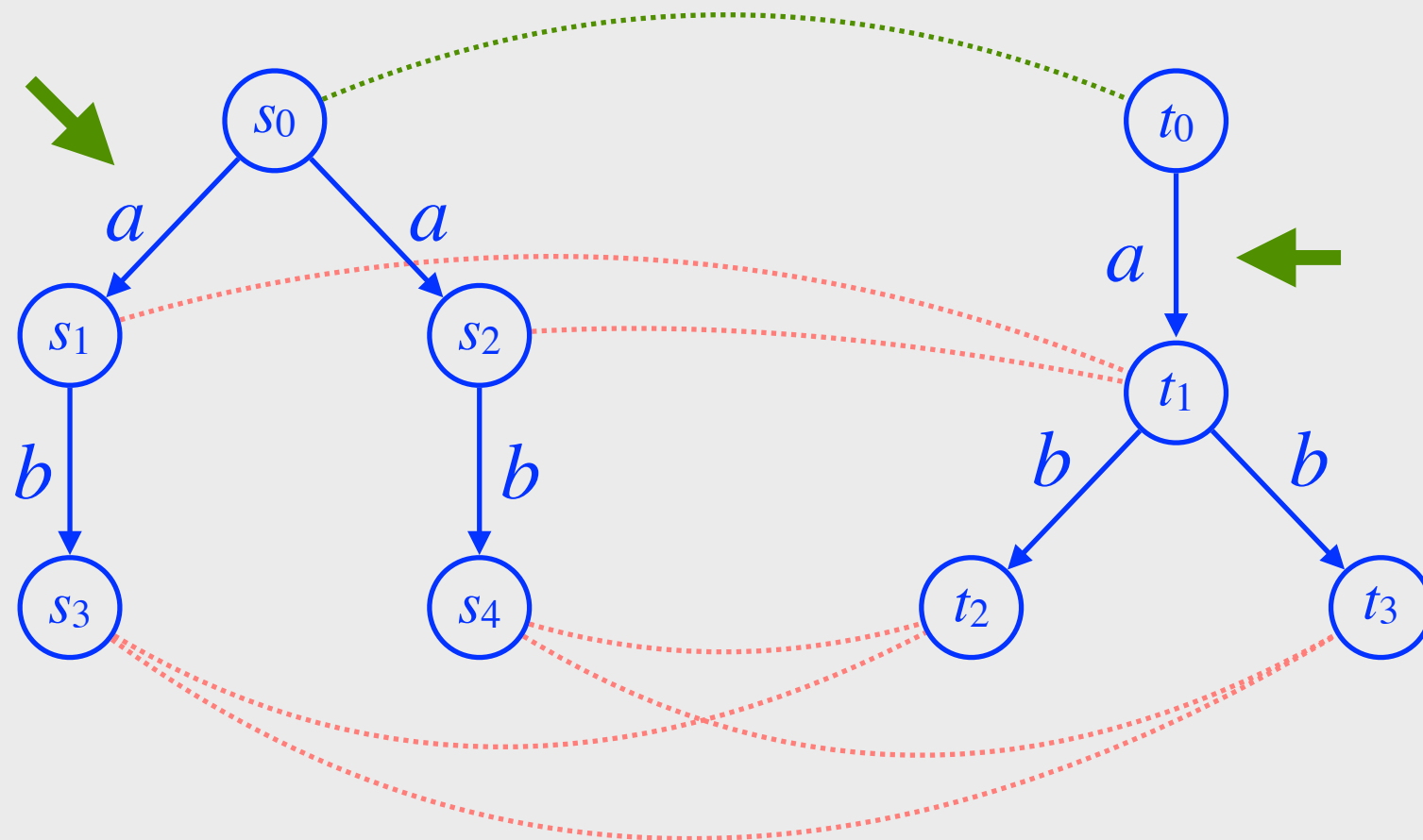
R es una simulación



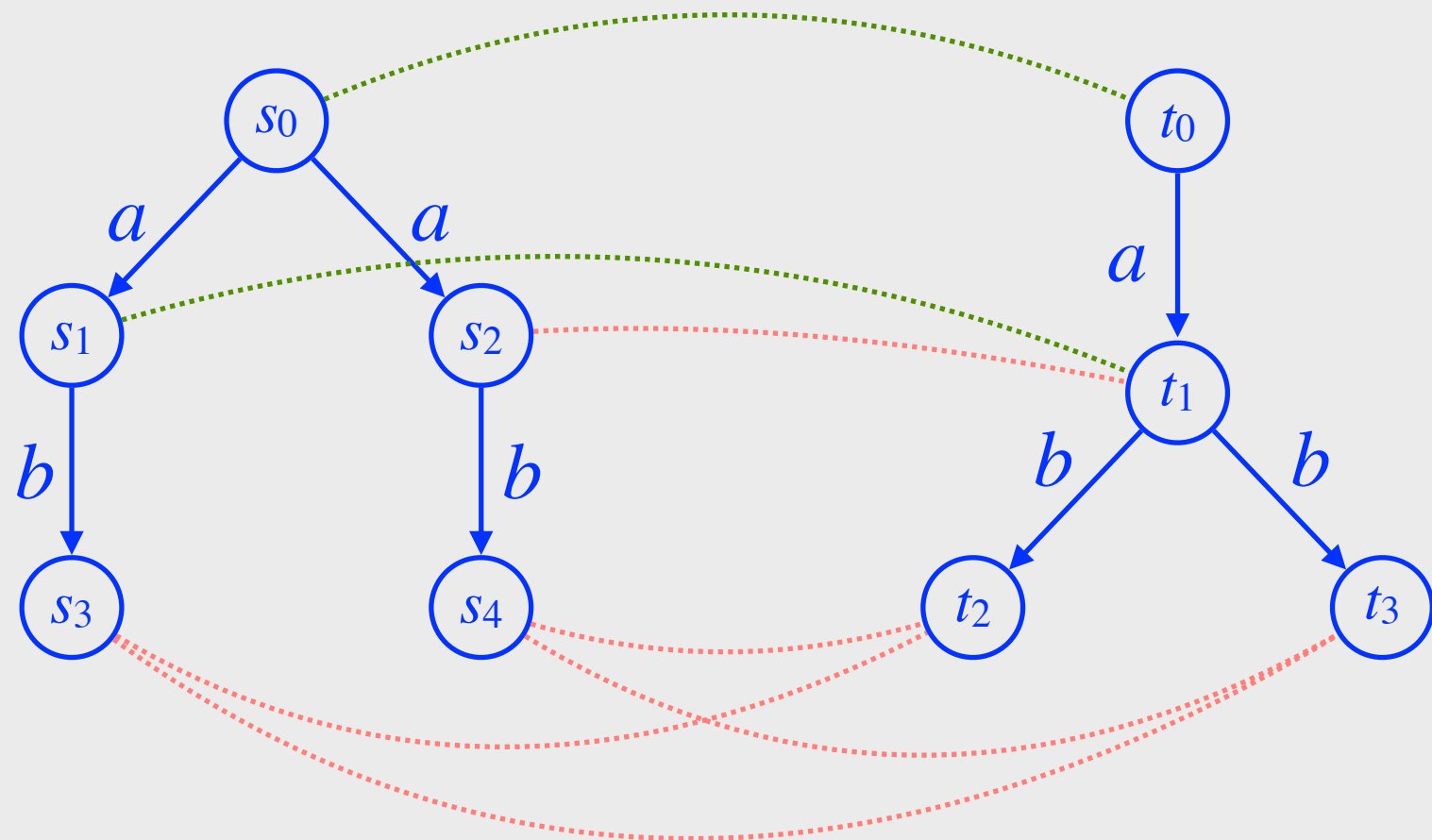
R es una simulación



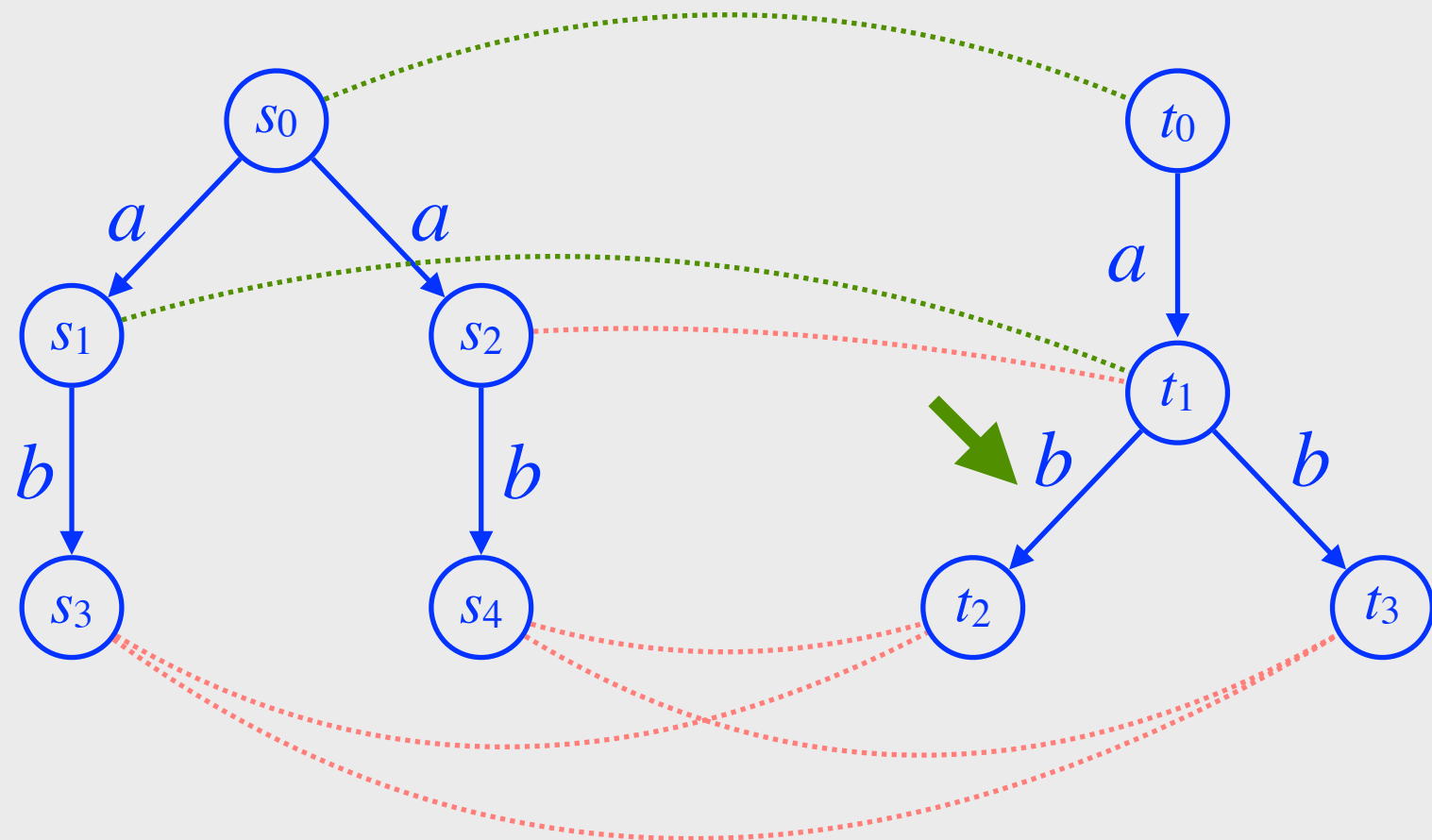
R es una simulación



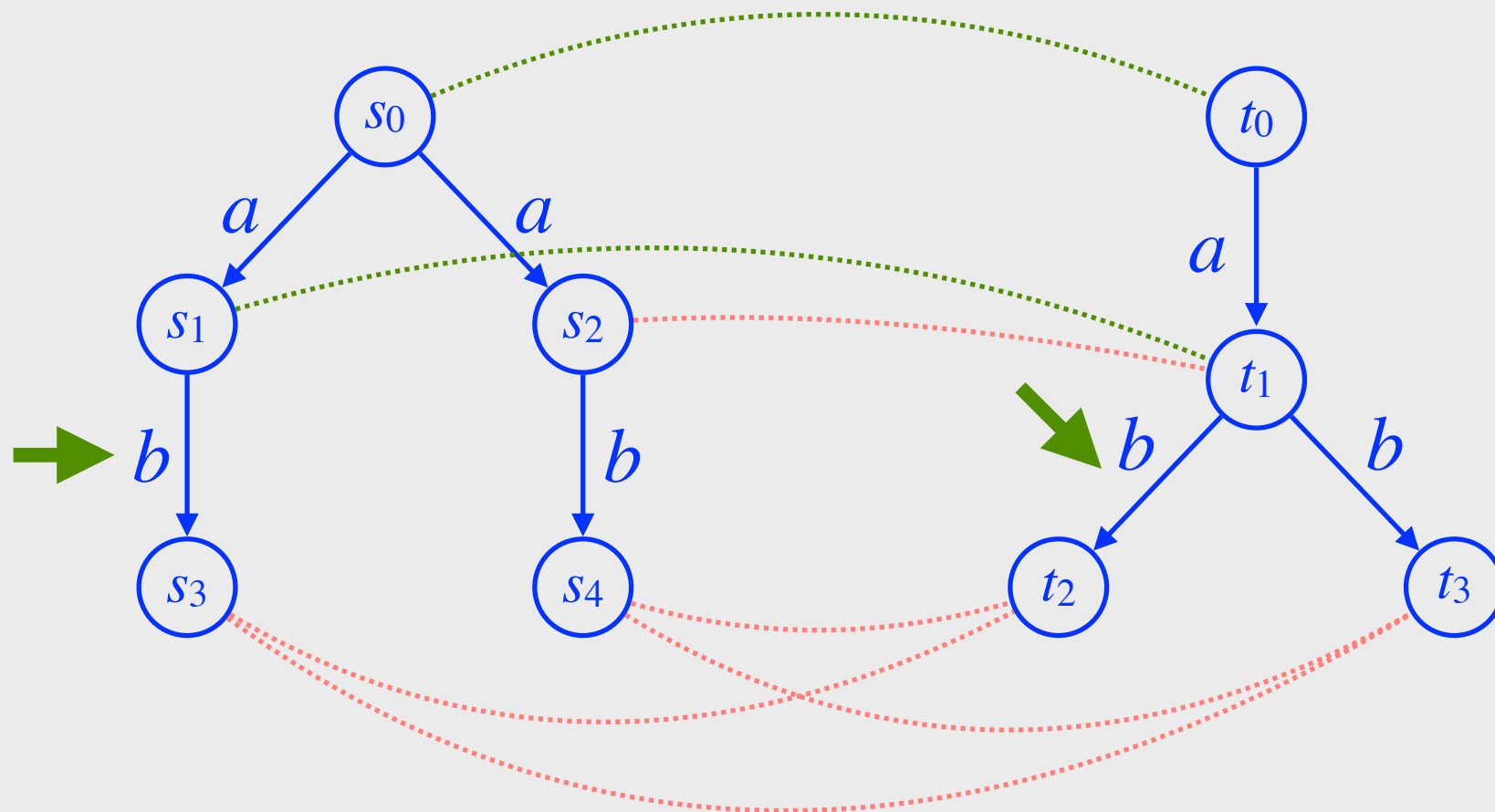
R es una simulación



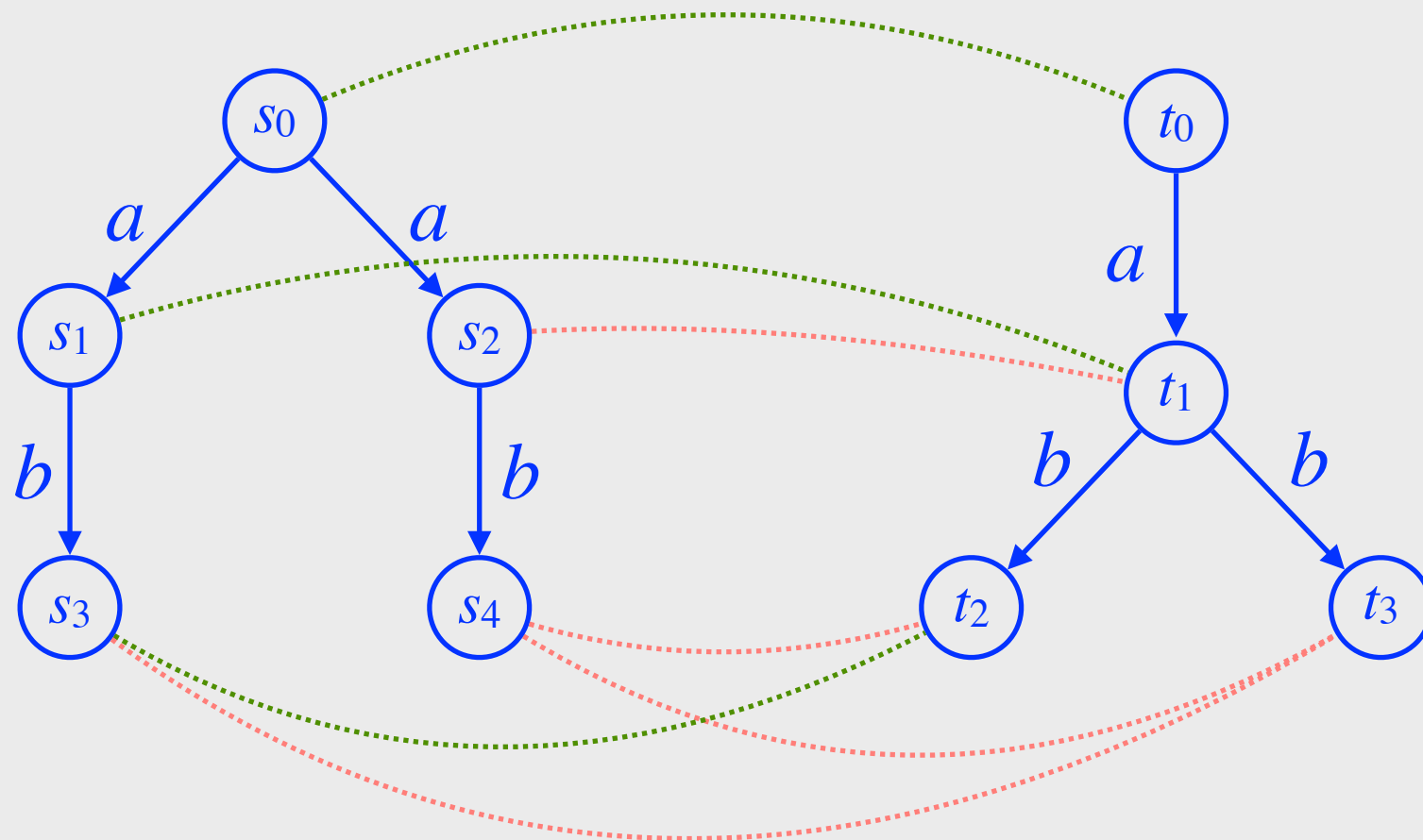
R es una simulación



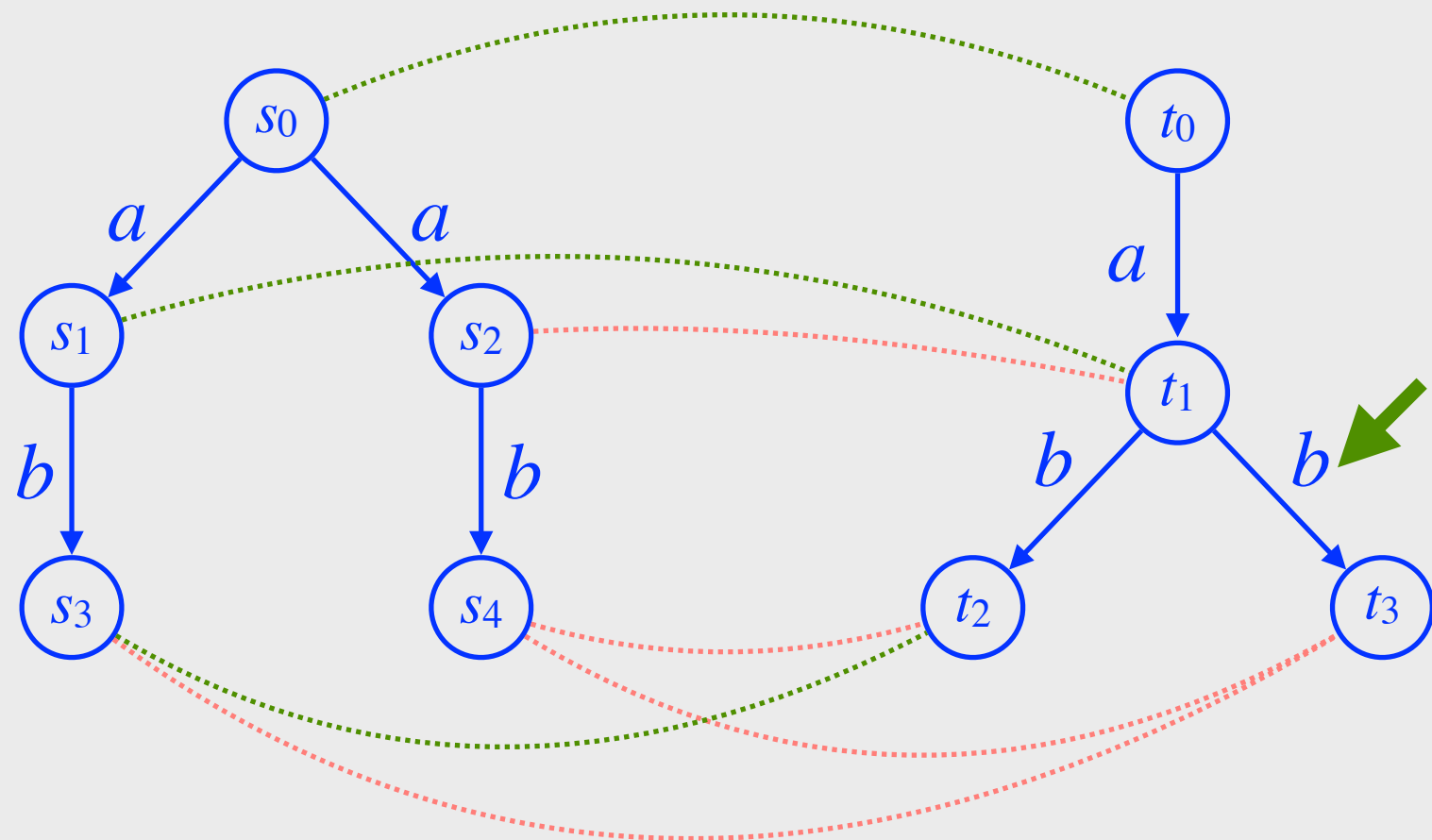
R es una simulación



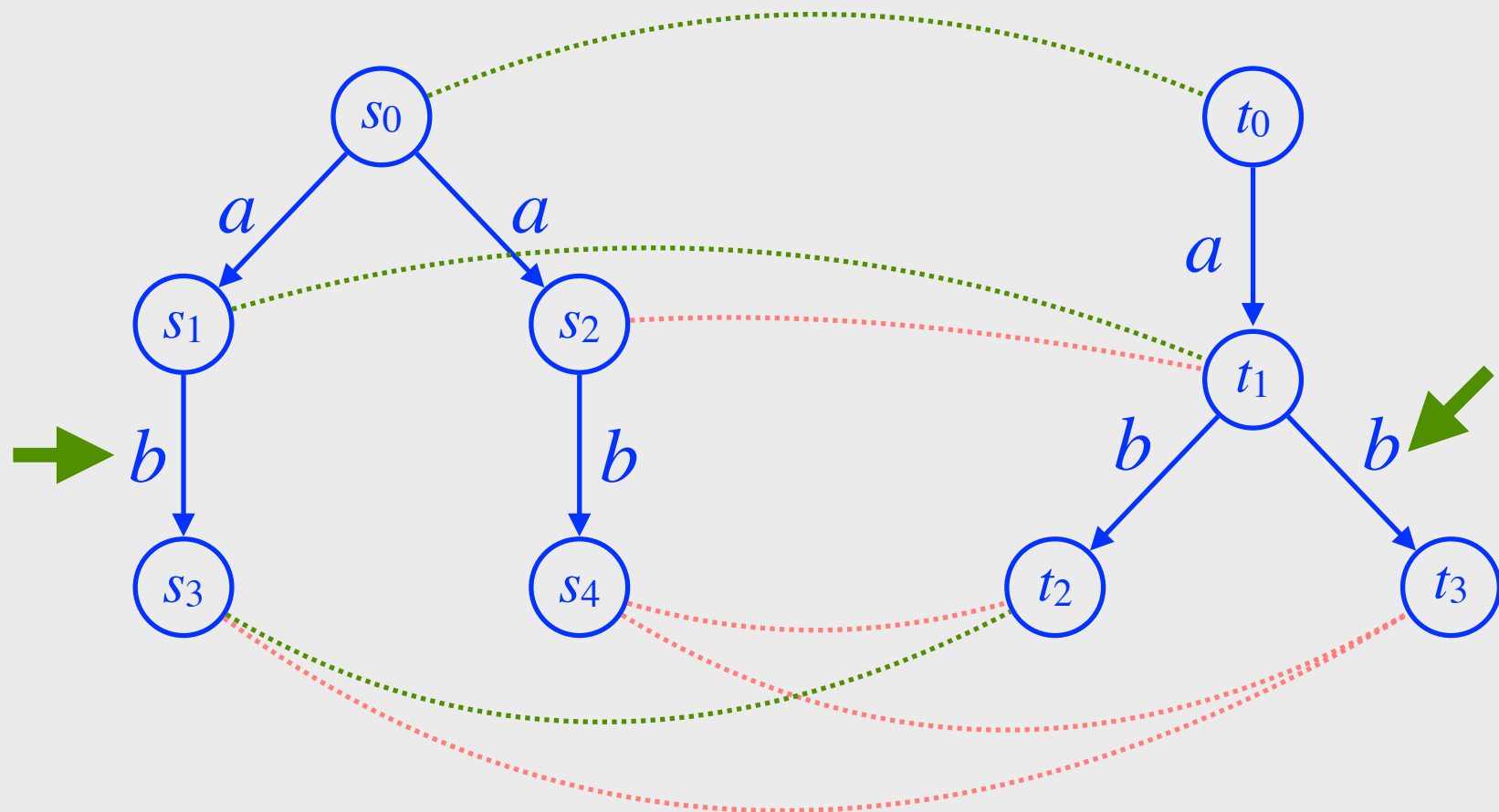
R es una simulación



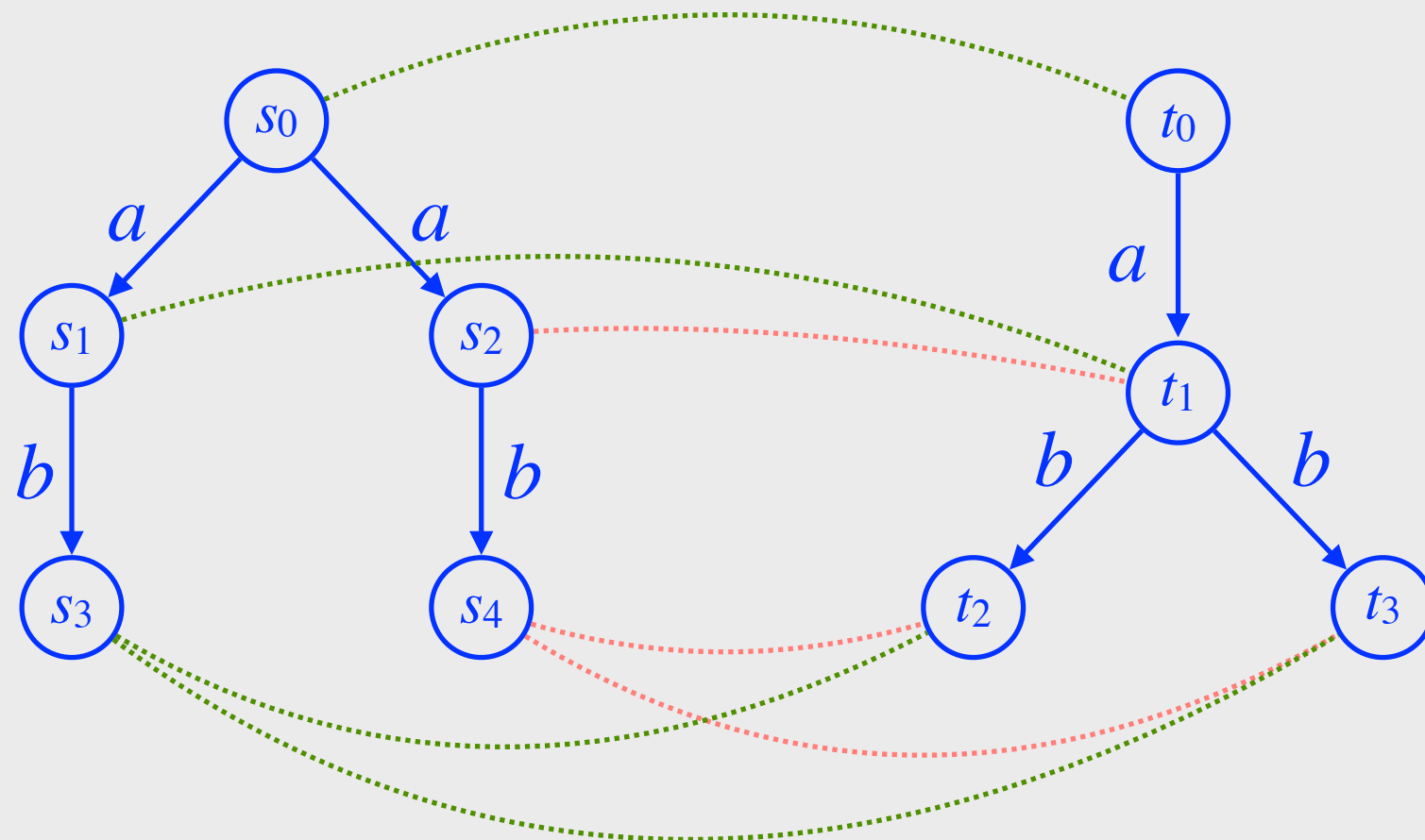
R es una simulación



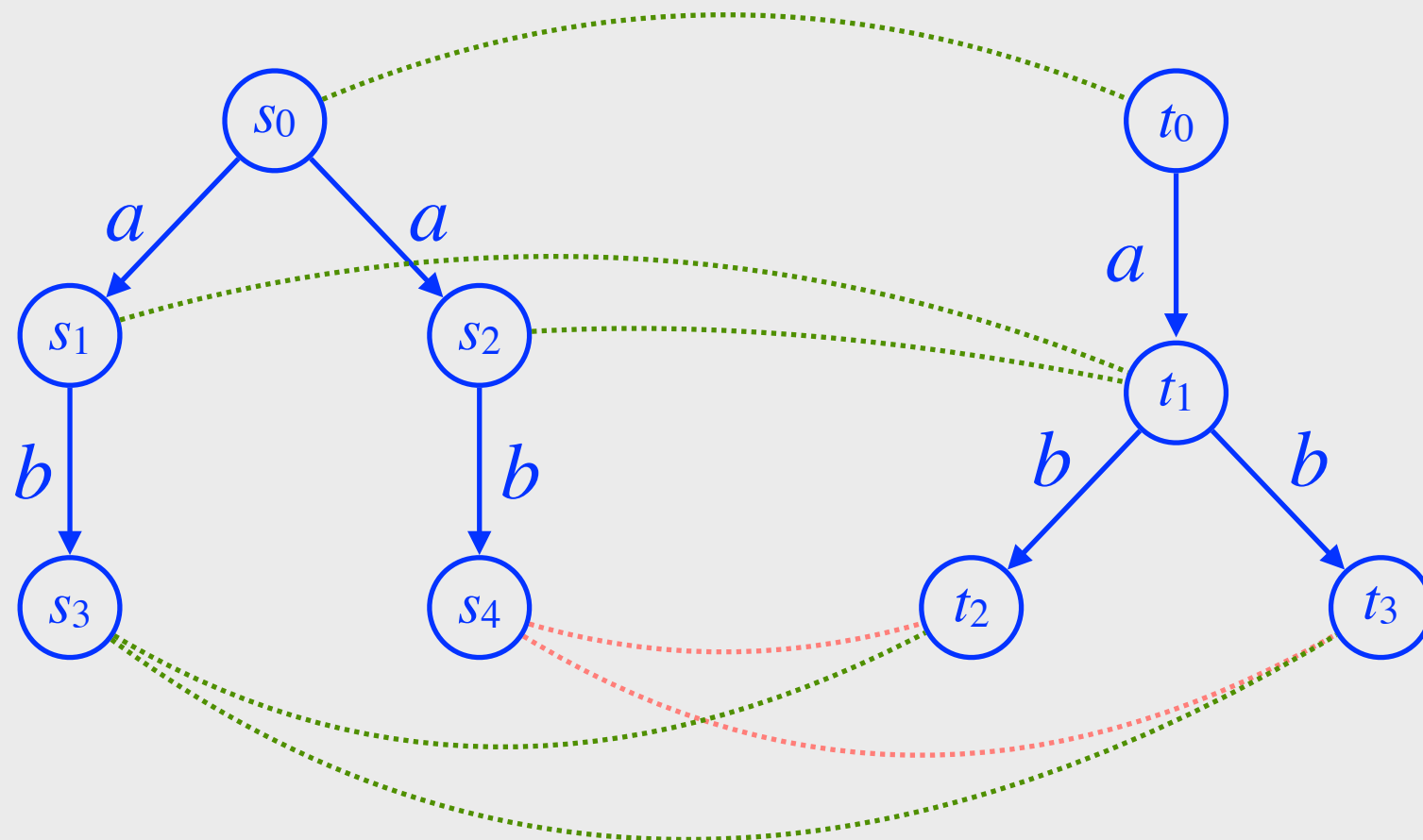
R es una simulación



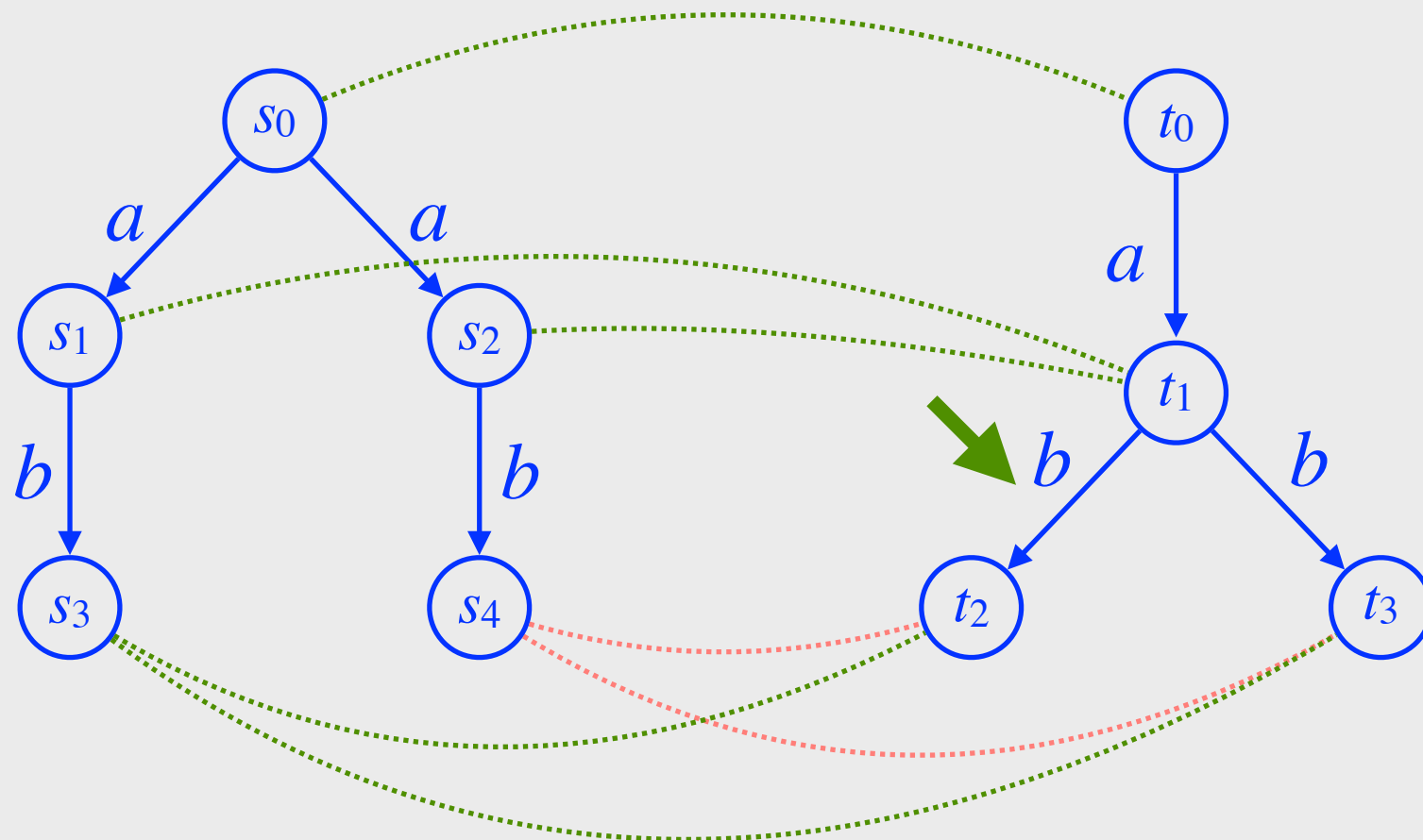
R es una simulación



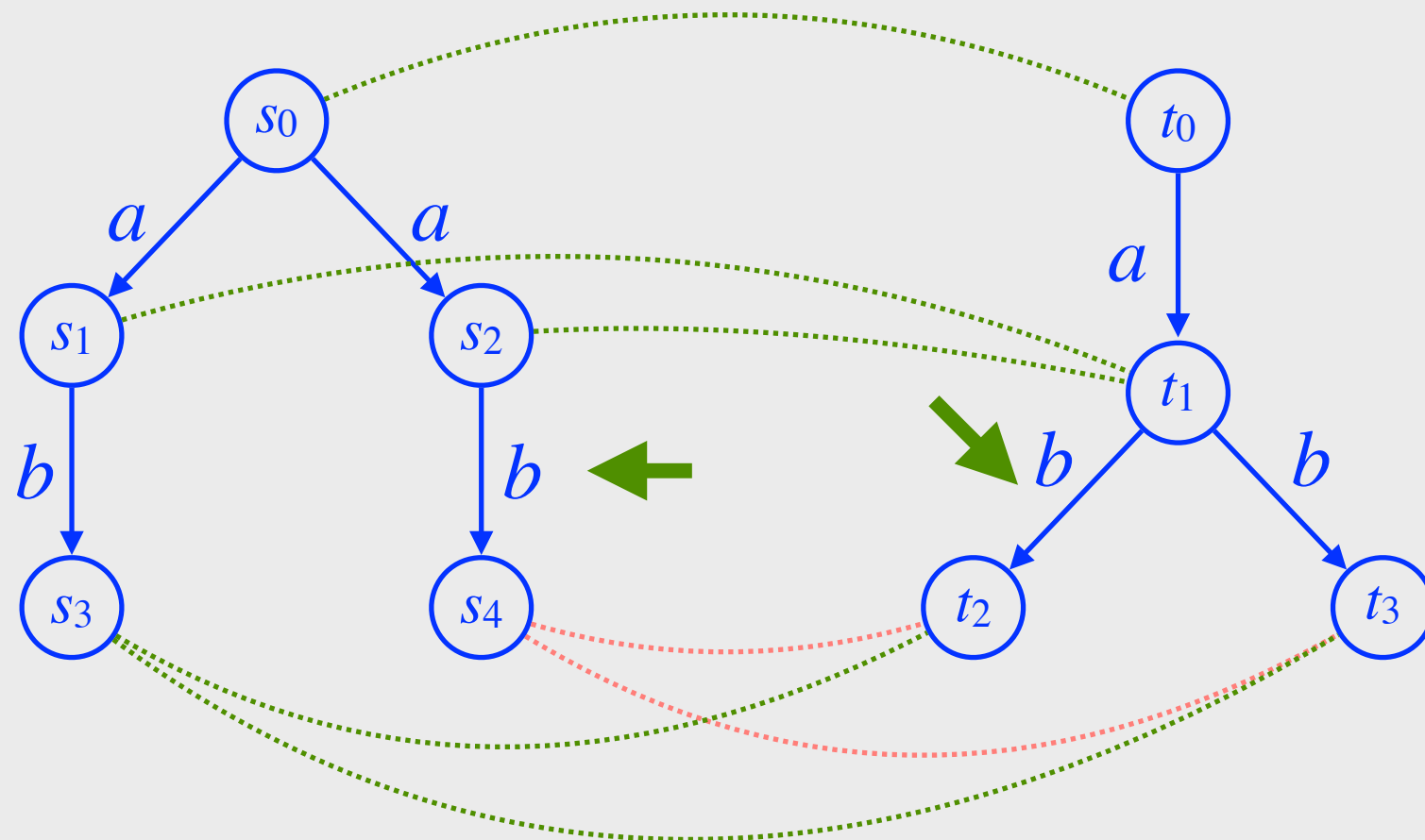
R es una simulación



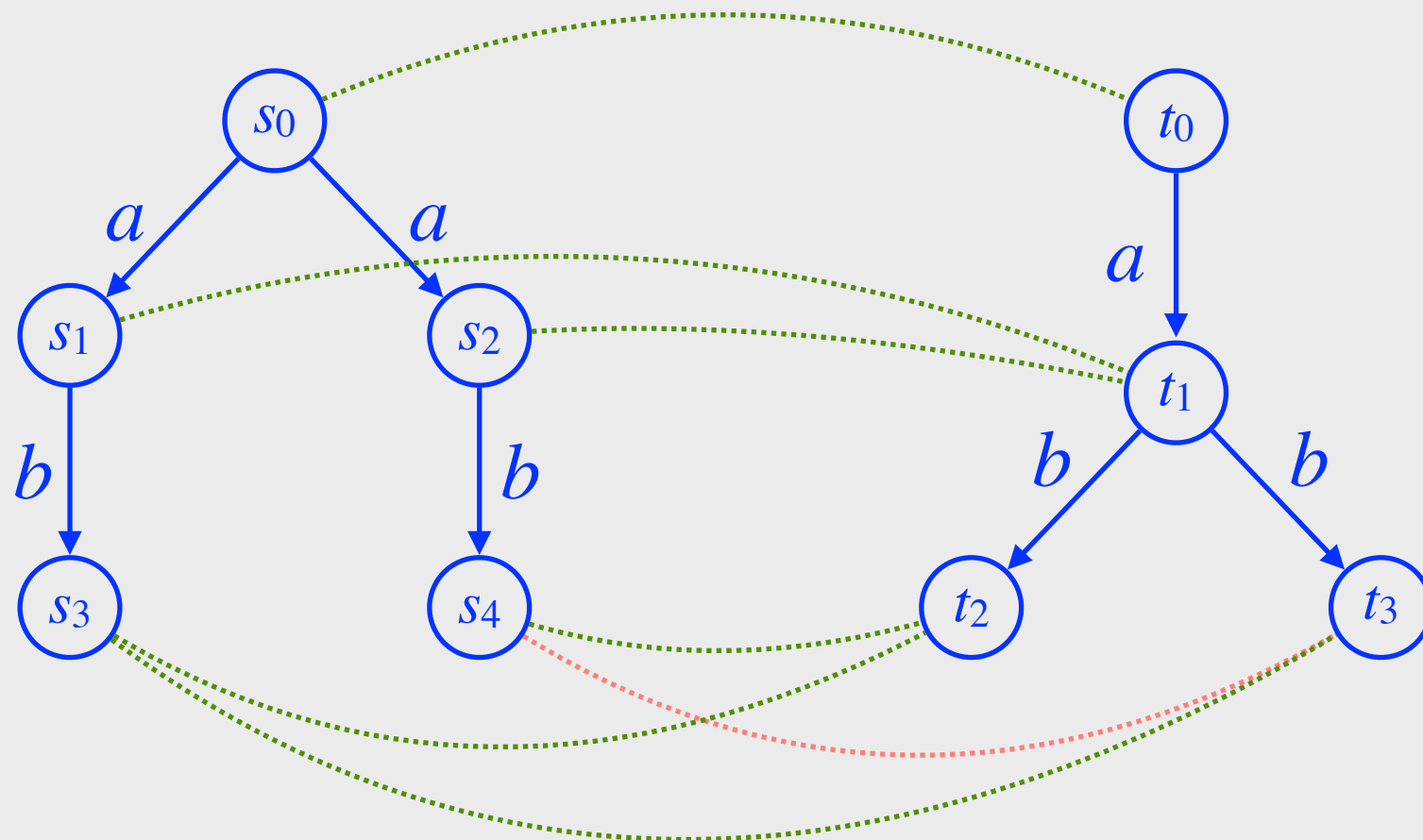
R es una simulación



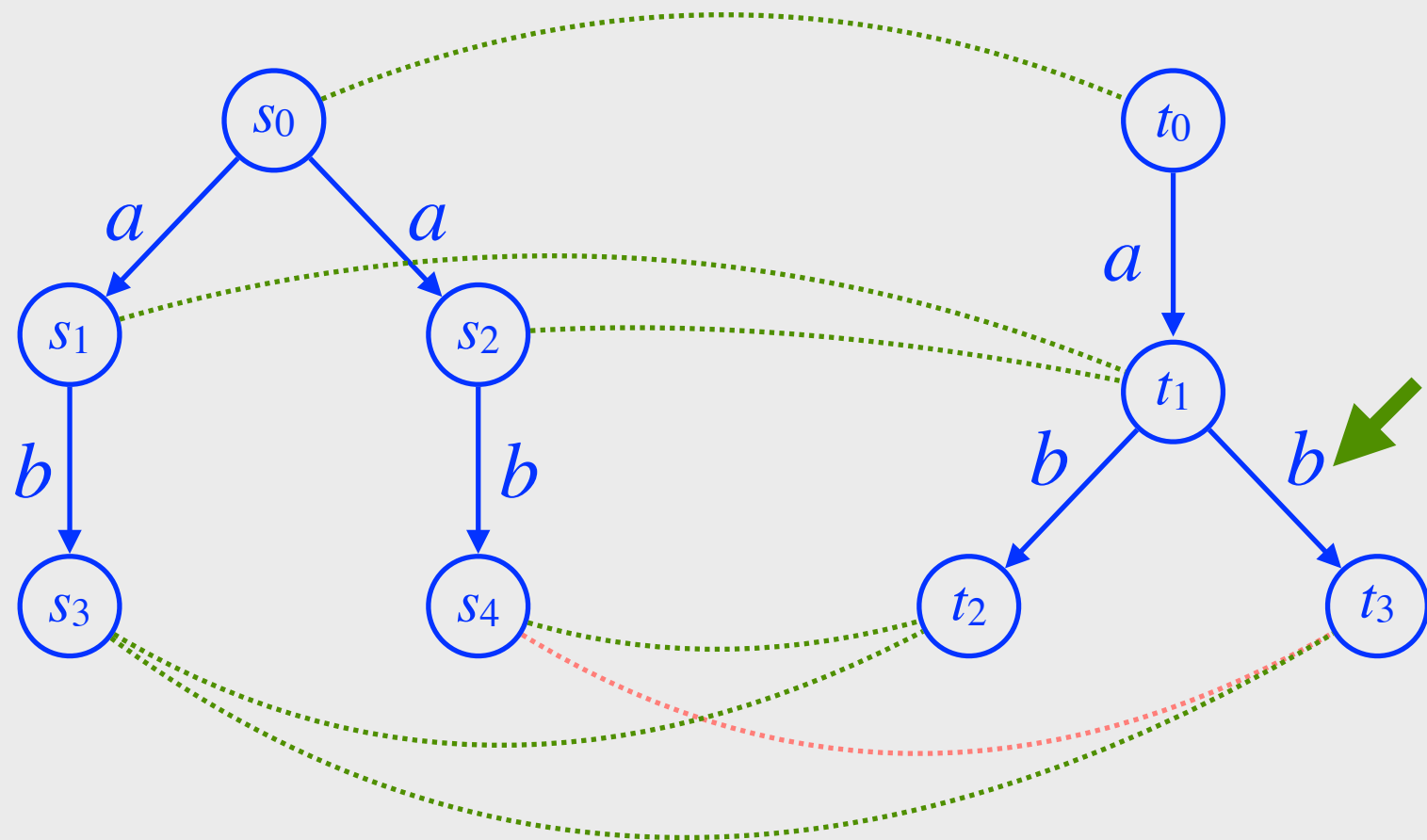
R es una simulación



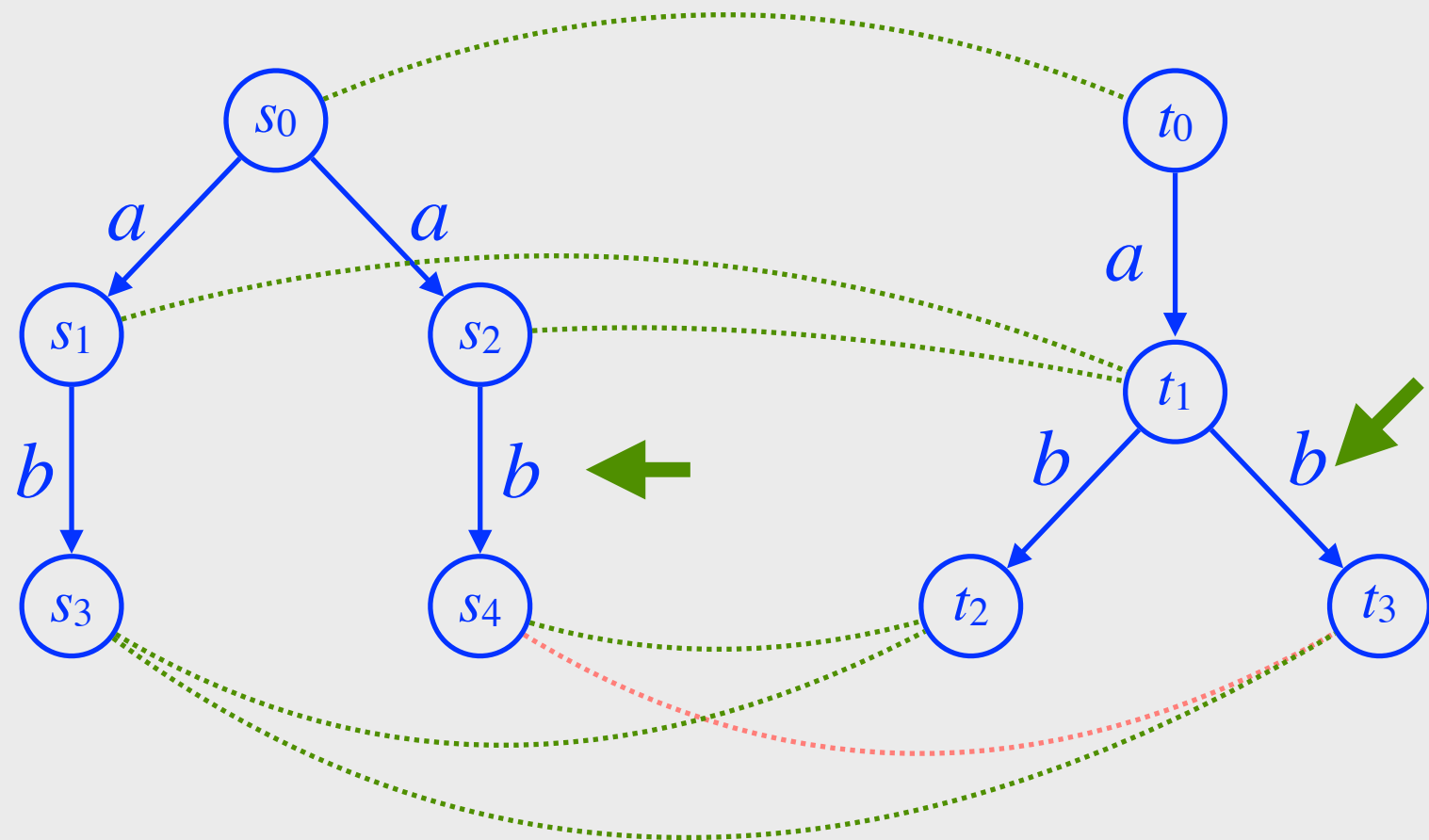
R es una simulación



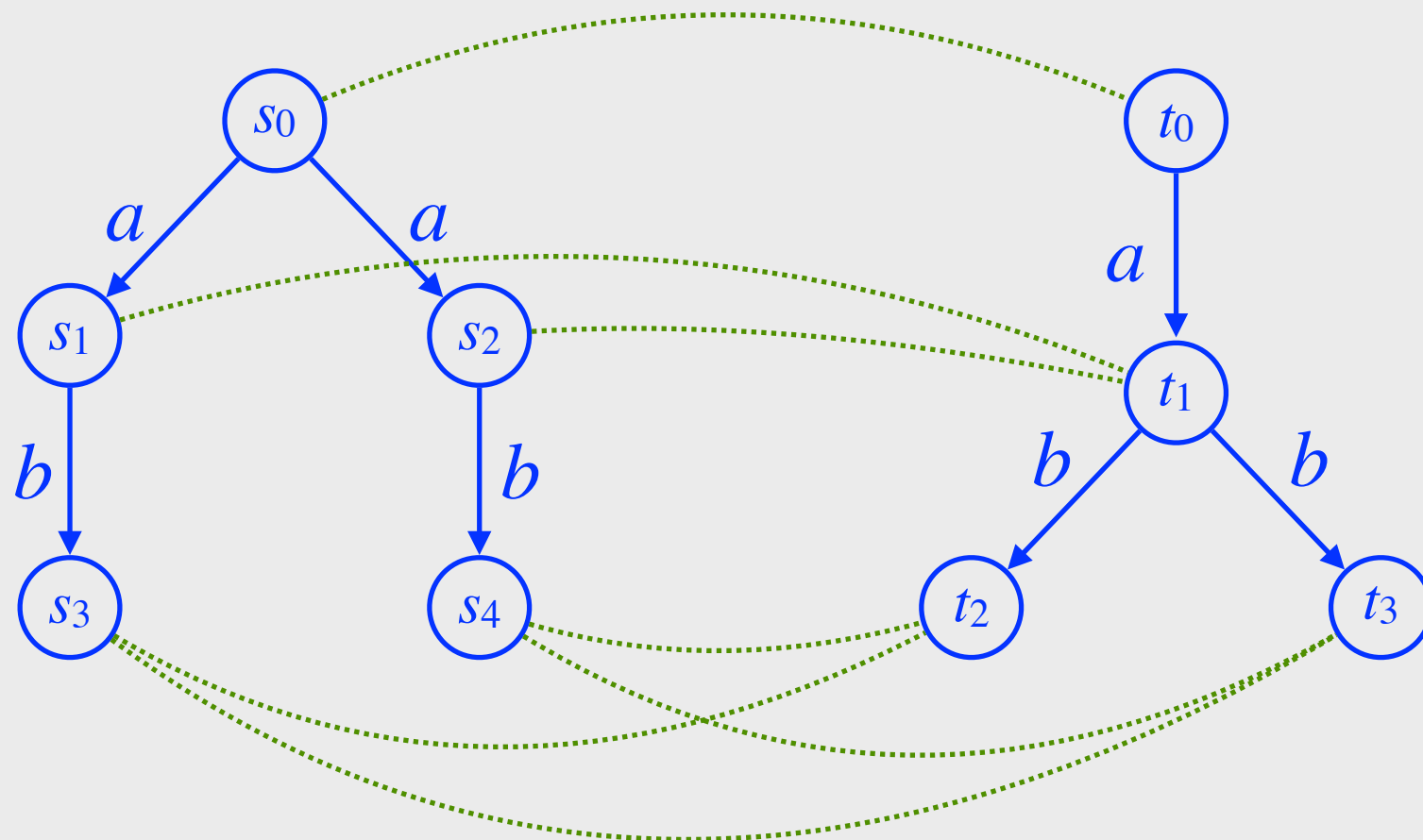
R es una simulación



R es una simulación



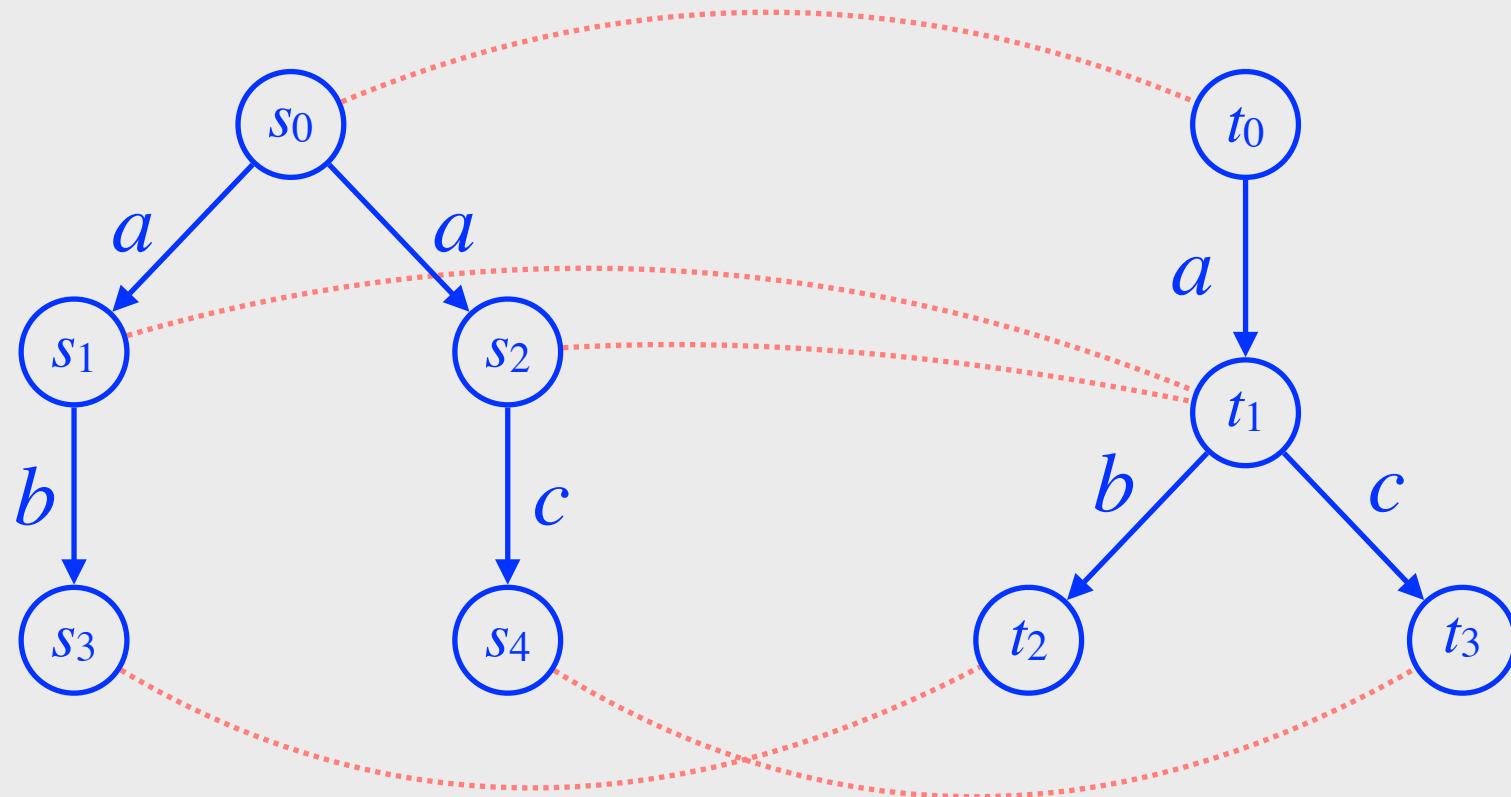
R es una simulación



R es una simulación y R^{-1} es una simulación

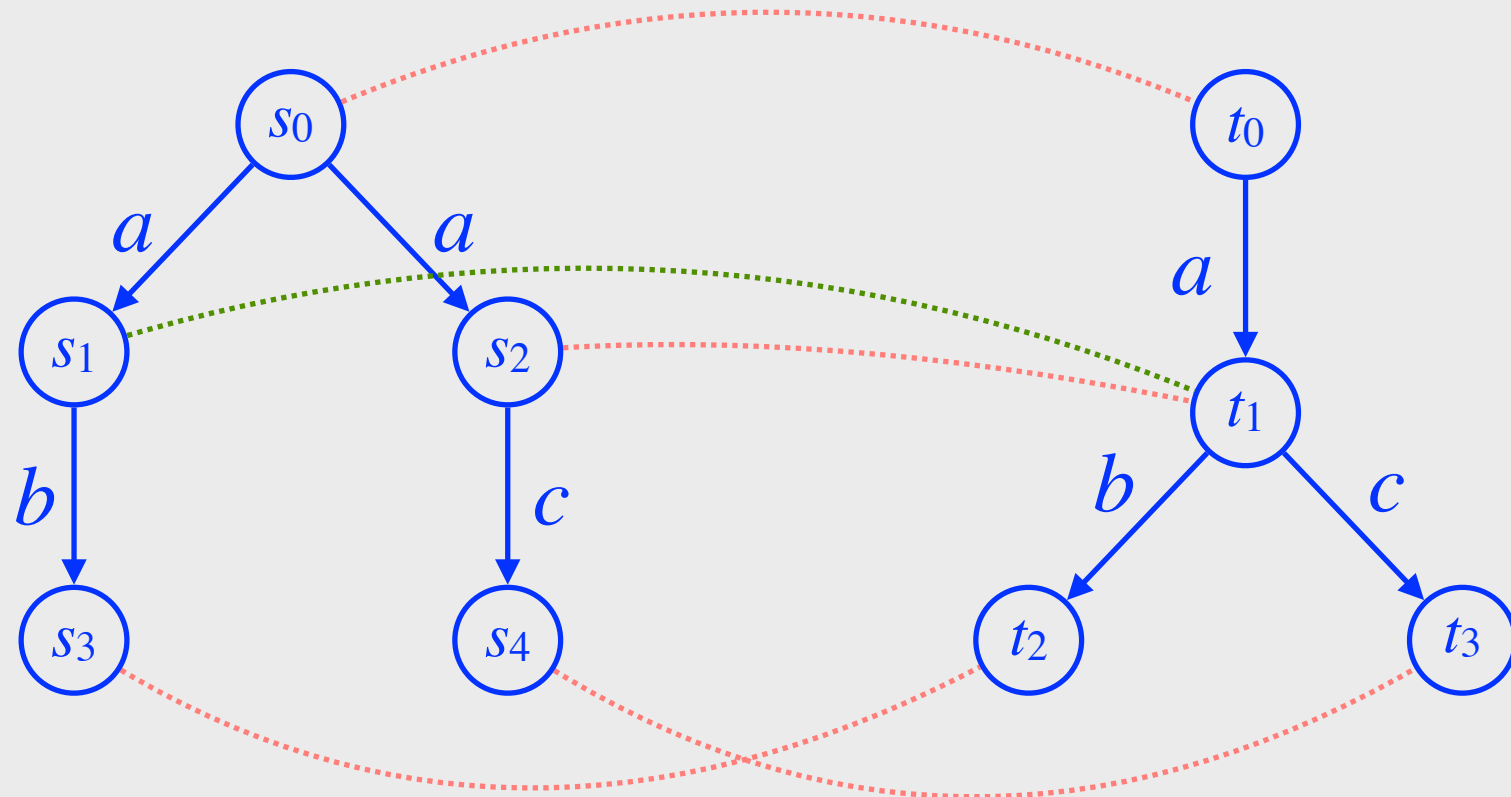
Entonces R es una bisimulación

Sabemos que esta R es una simulación



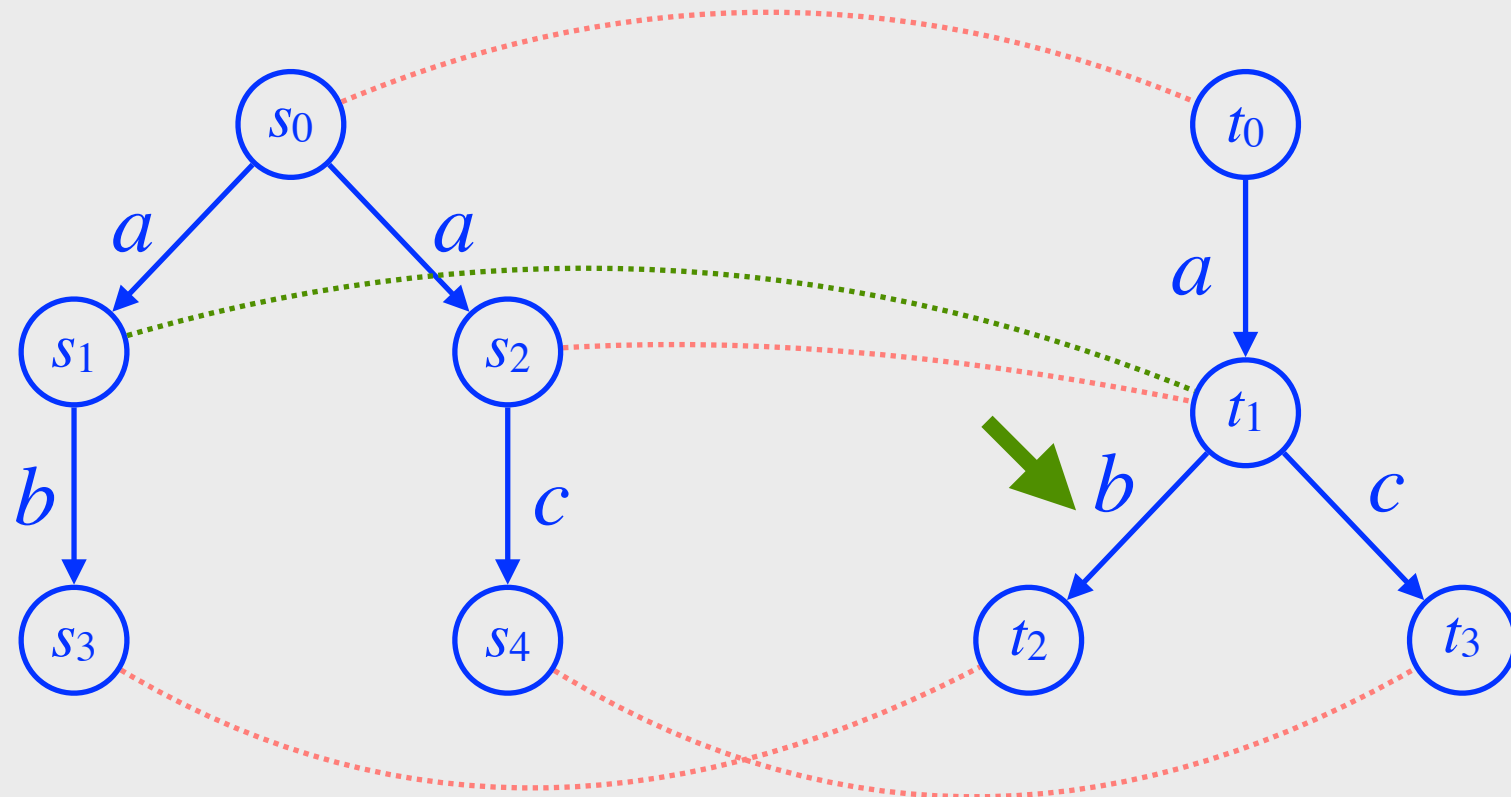
¿Es R^{-1} también una simulación?

Sabemos que esta R es una simulación



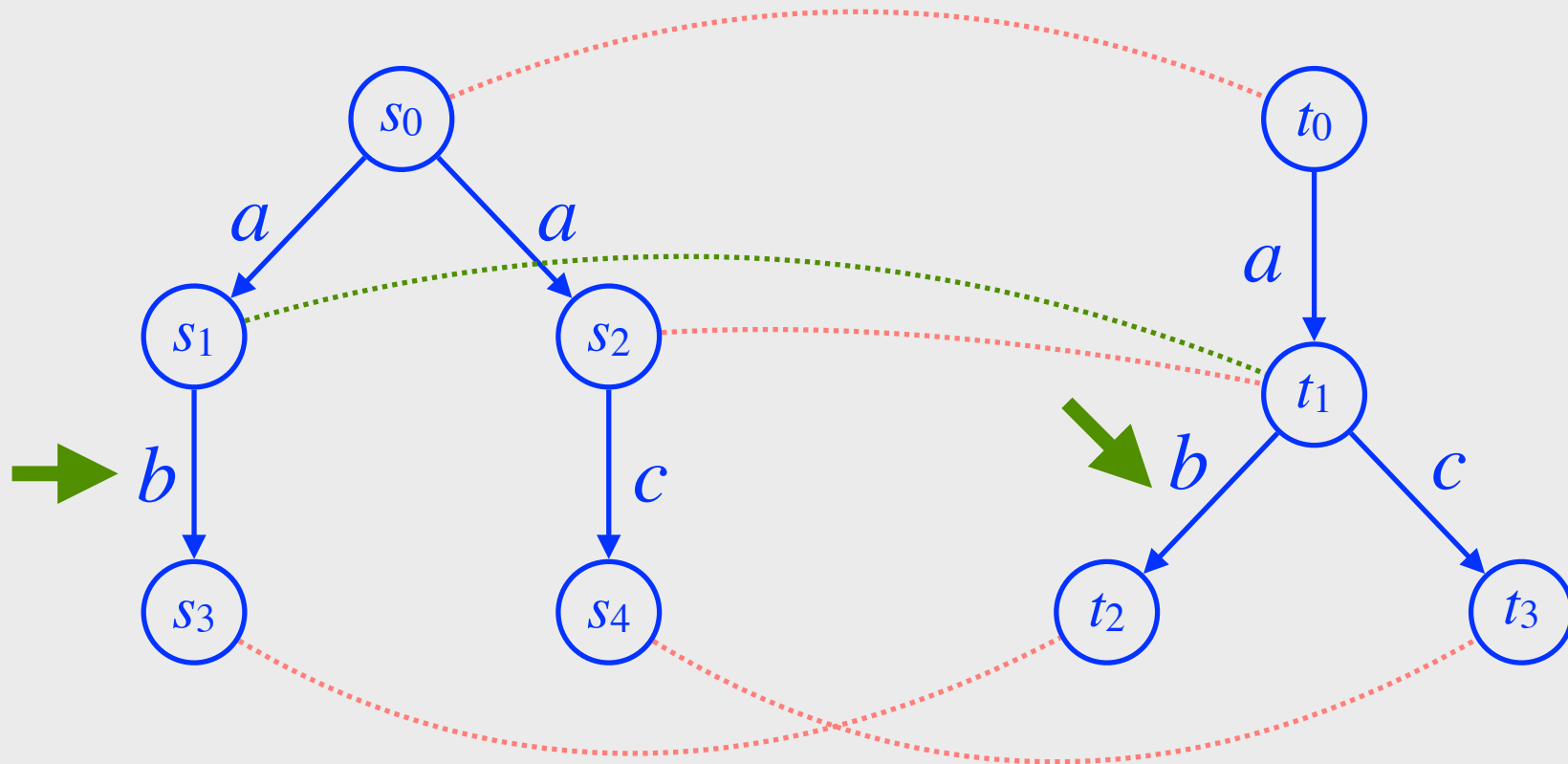
¿Es R^{-1} también una simulación?

Sabemos que esta R es una simulación



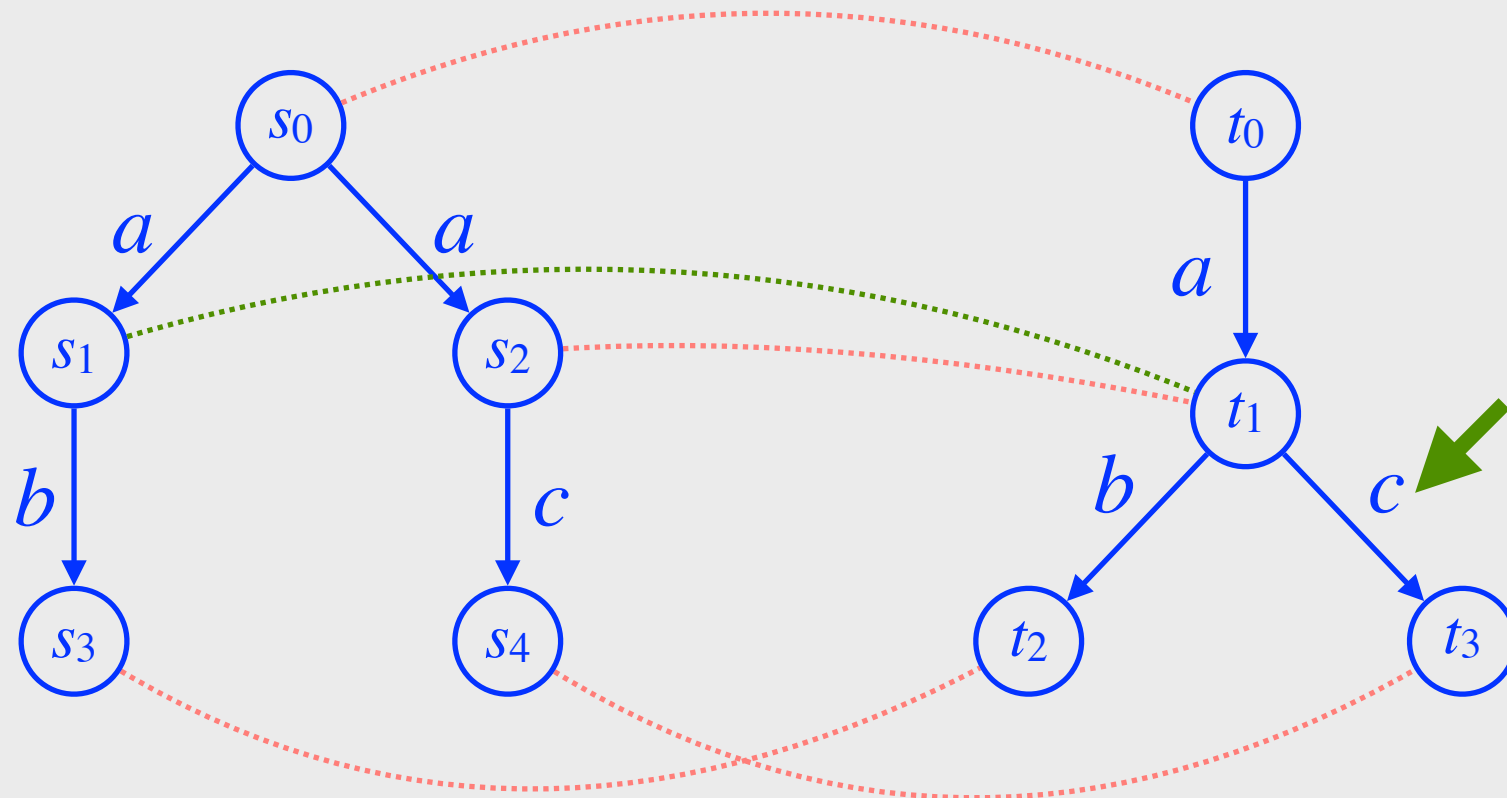
¿Es R^{-1} también una simulación?

Sabemos que esta R es una simulación



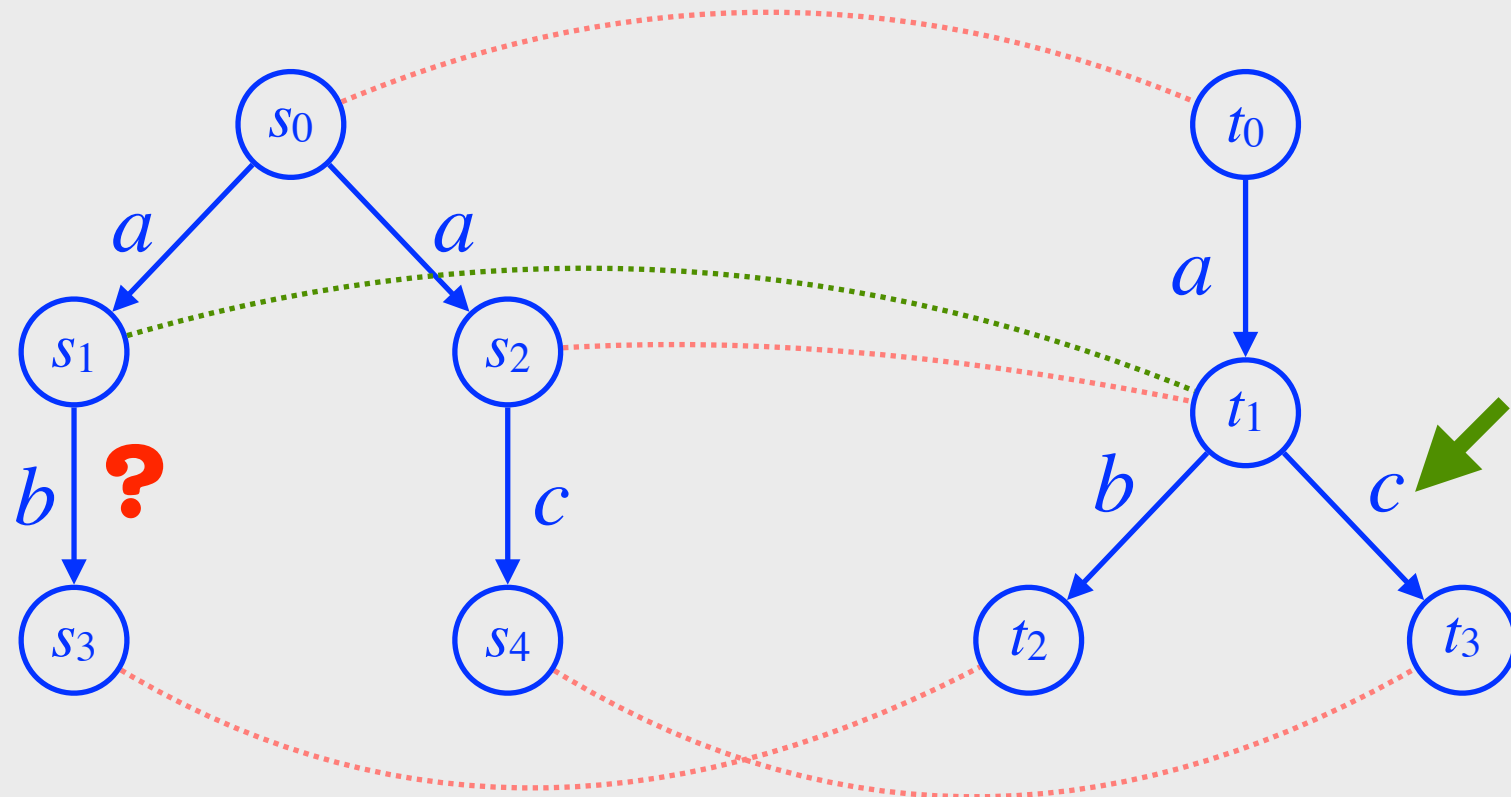
¿Es R^{-1} también una simulación?

Sabemos que esta R es una simulación



¿Es R^{-1} también una simulación?

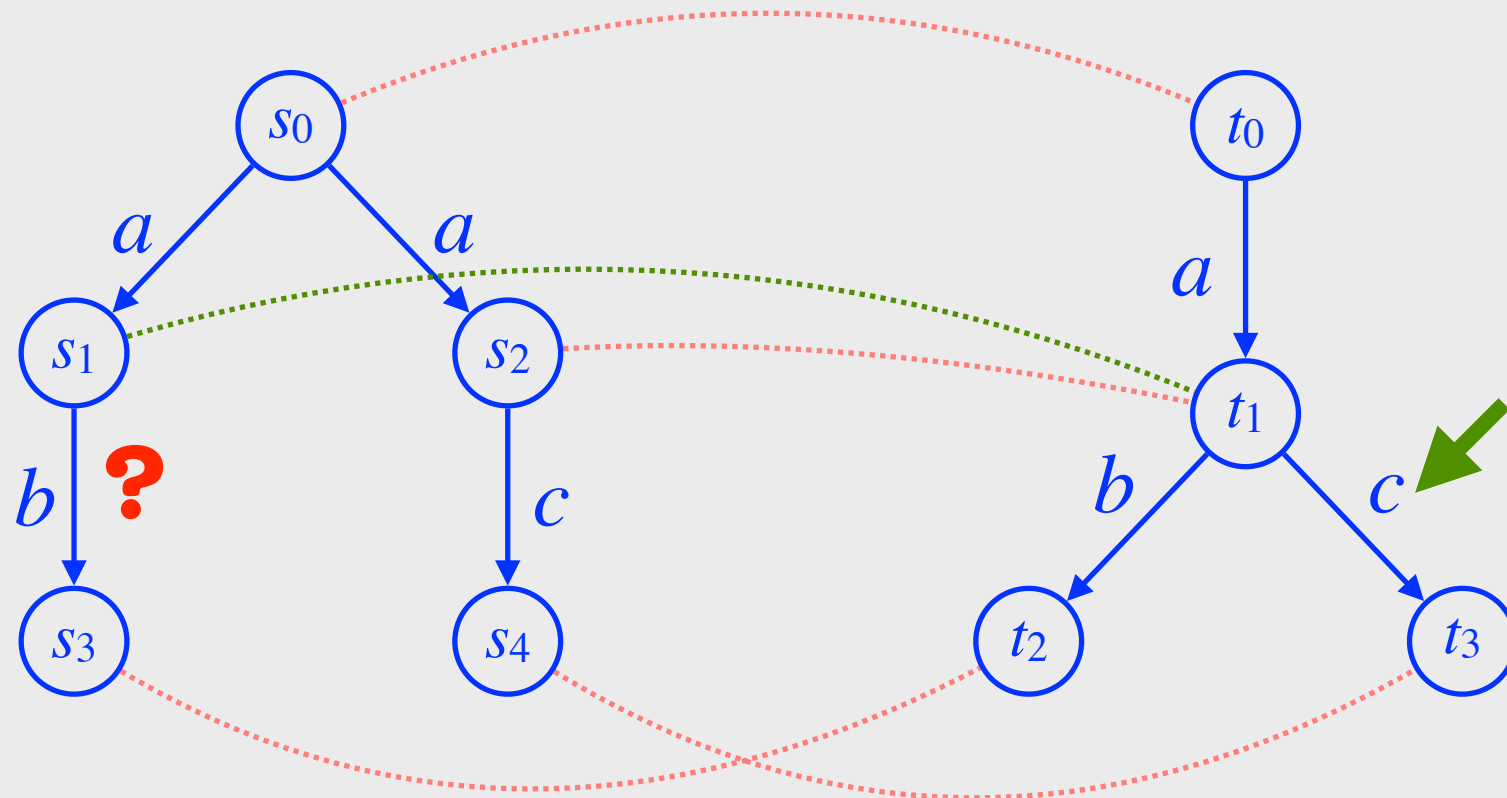
Sabemos que esta R es una simulación



¿Es R^{-1} también una simulación? **NO**

Entonces R no es
bisimulación

Sabemos que esta R es una simulación



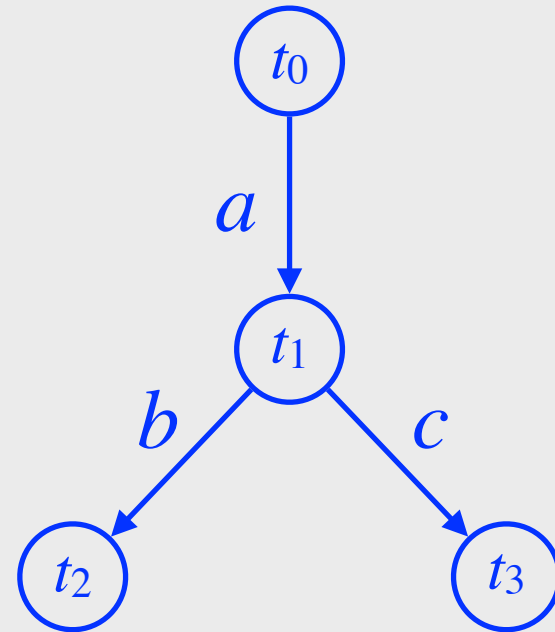
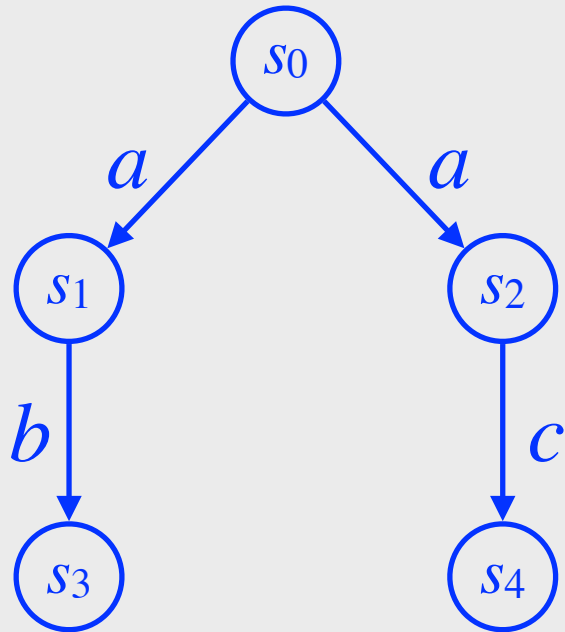
¿Es R^{-1} también una simulación?

NO

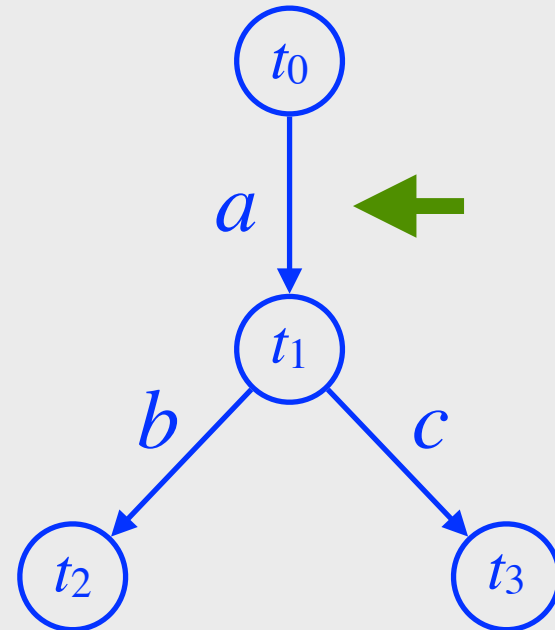
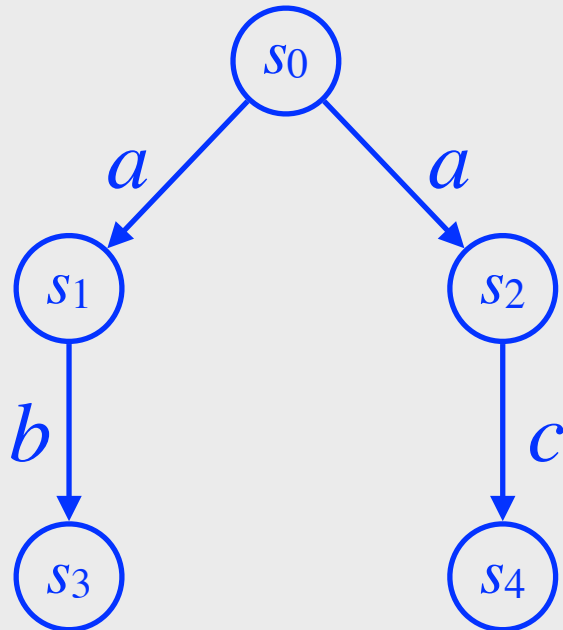
¿Hay otra relación que sea bisimulación?

Entonces R no es bisimulación

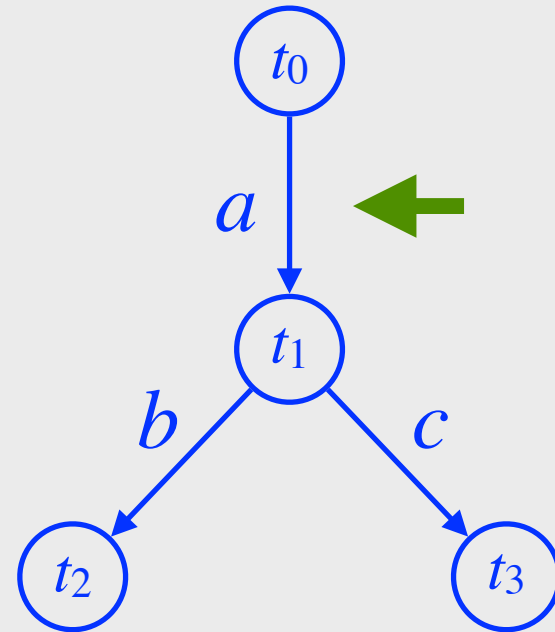
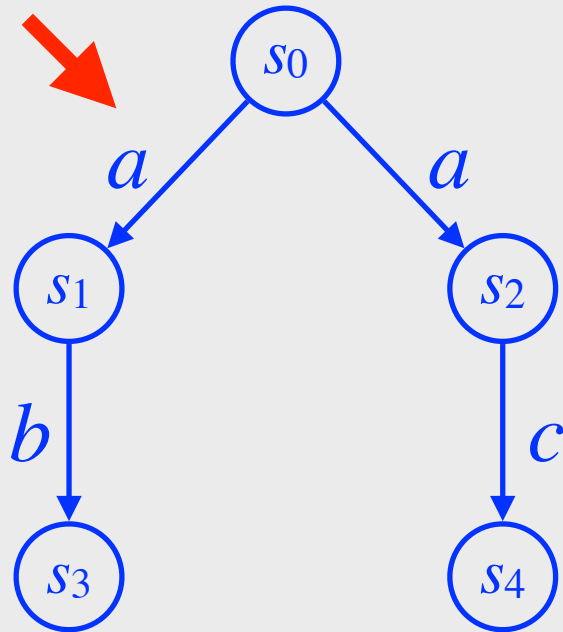
No hay una simulación al revés



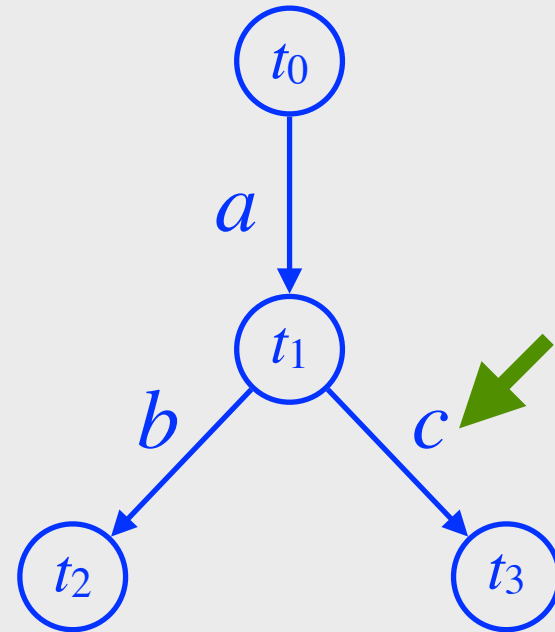
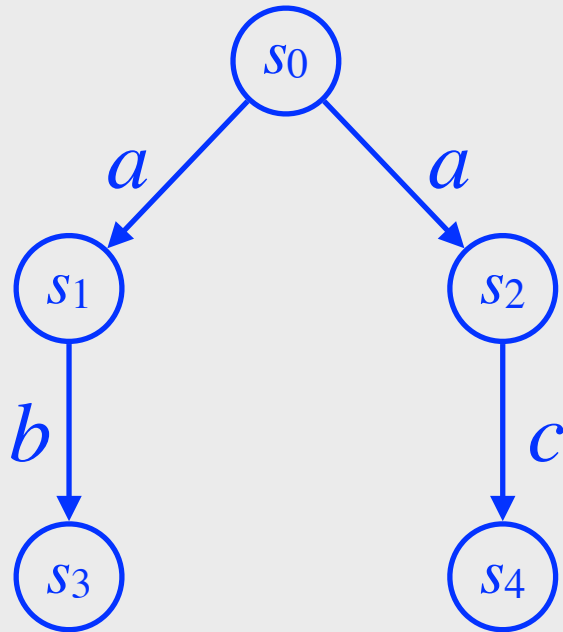
No hay una simulación al revés



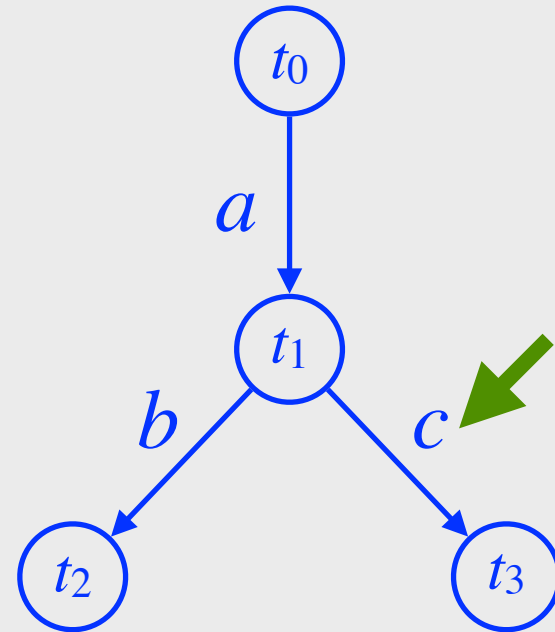
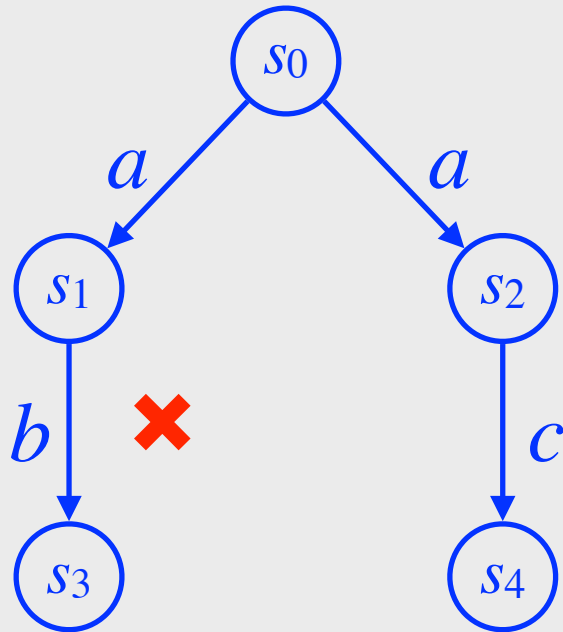
No hay una simulación al revés



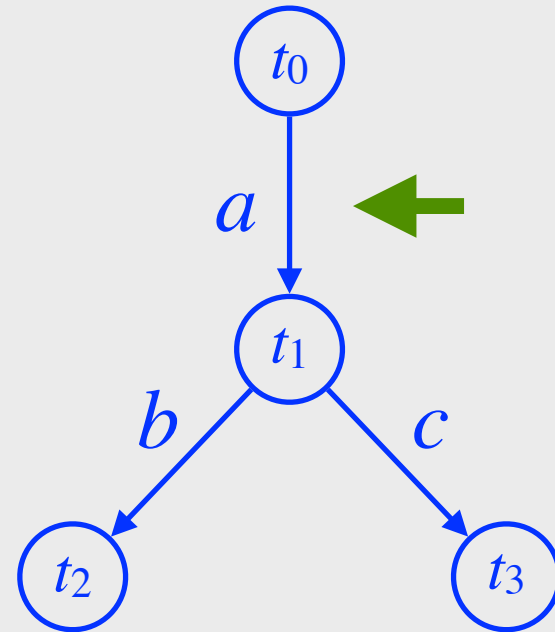
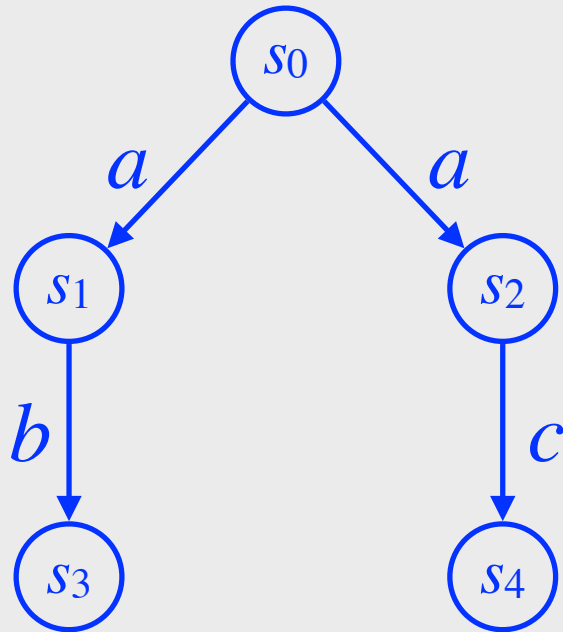
No hay una simulación al revés



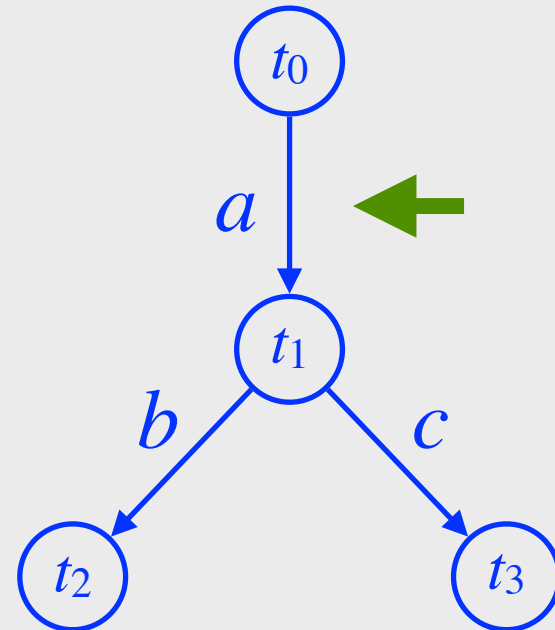
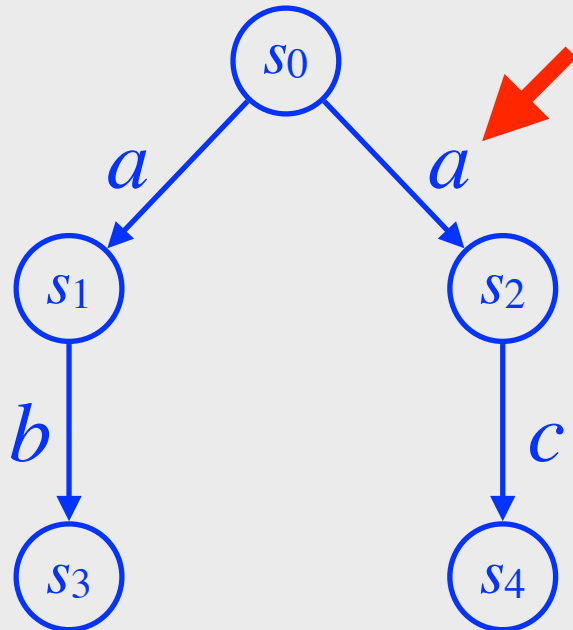
No hay una simulación al revés



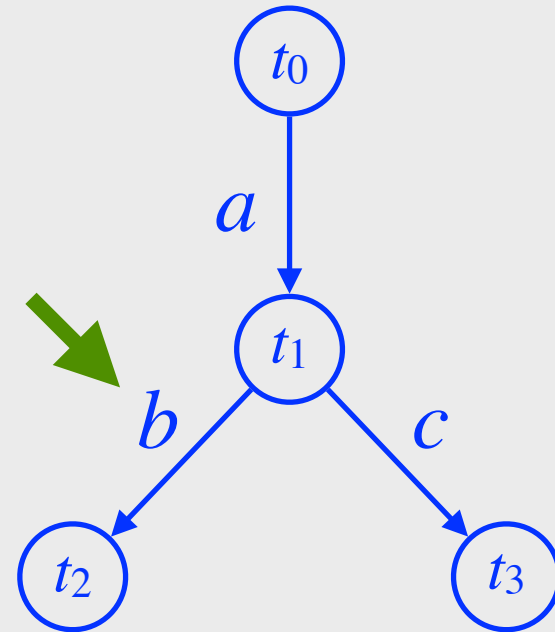
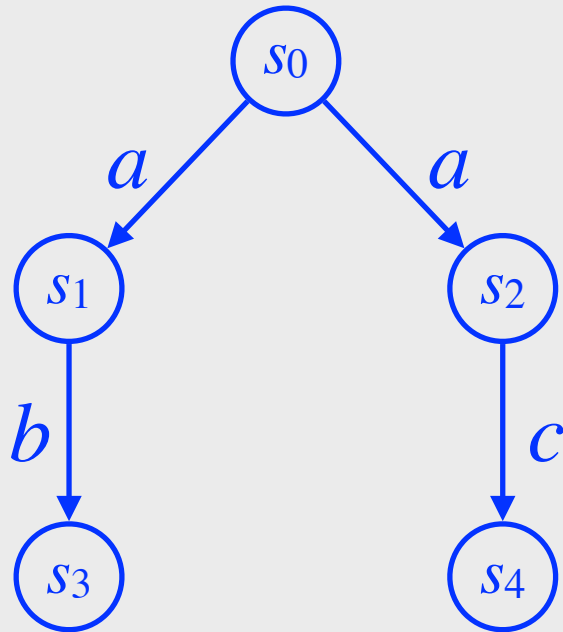
No hay una simulación al revés



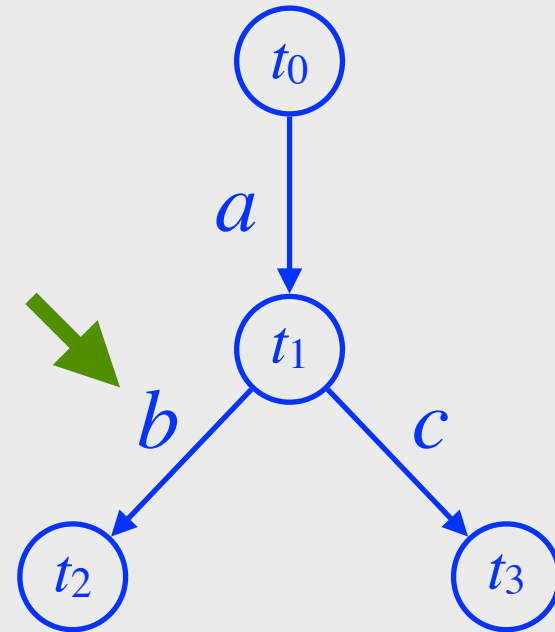
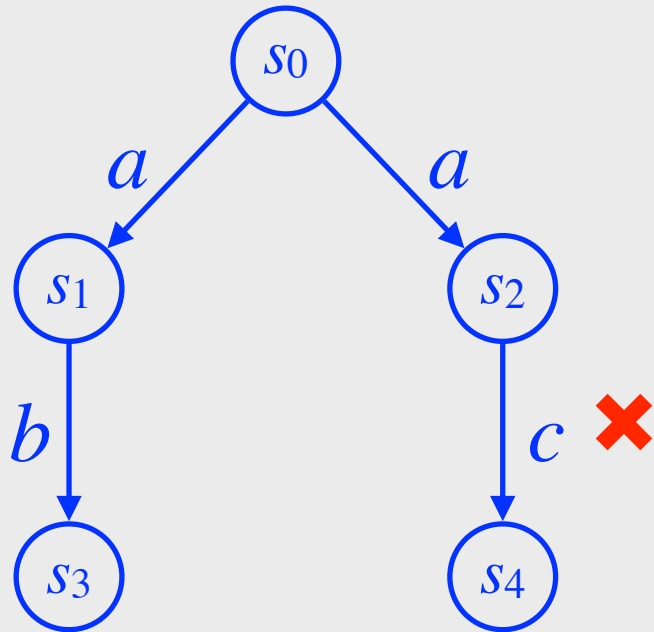
No hay una simulación al revés



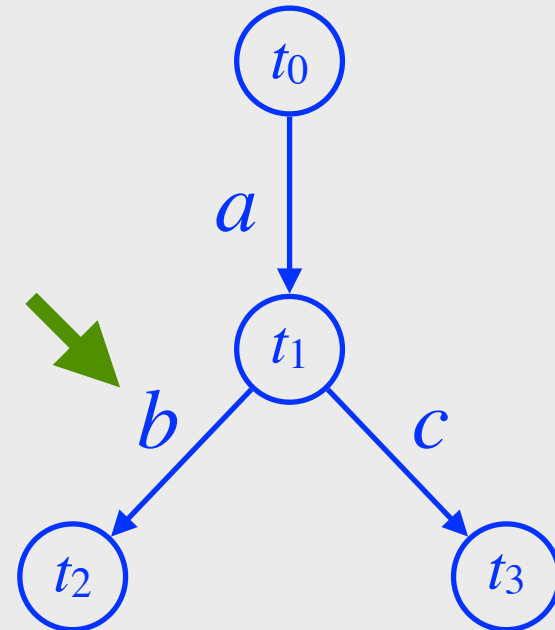
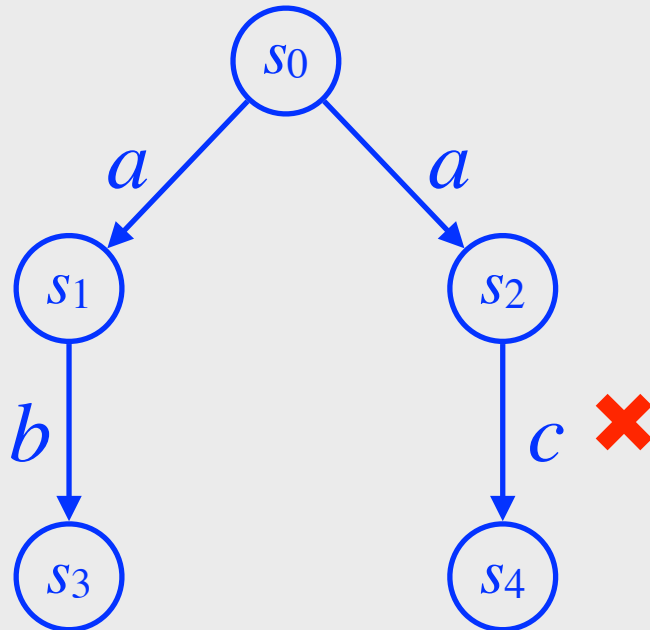
No hay una simulación al revés



No hay una simulación al revés

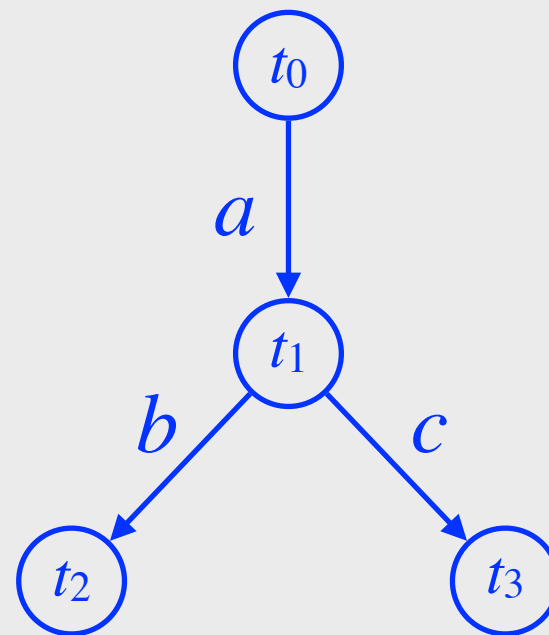
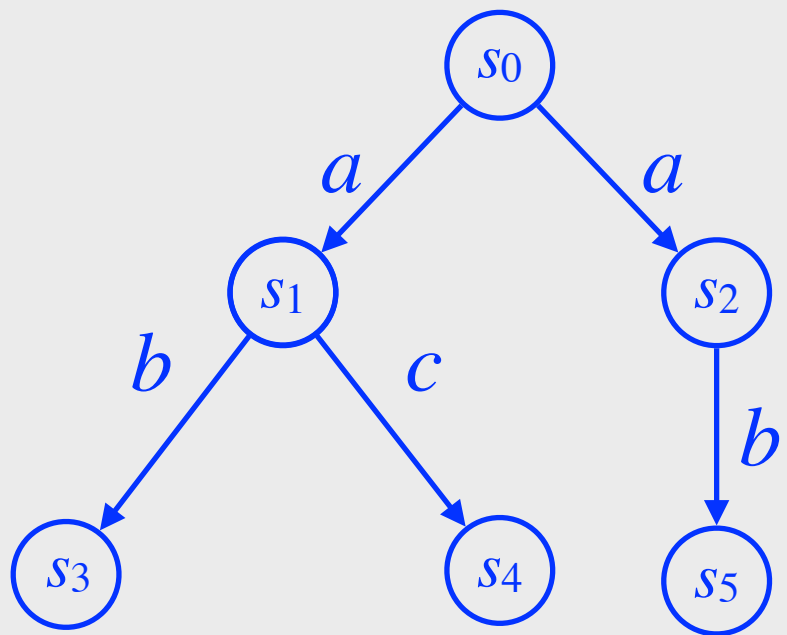


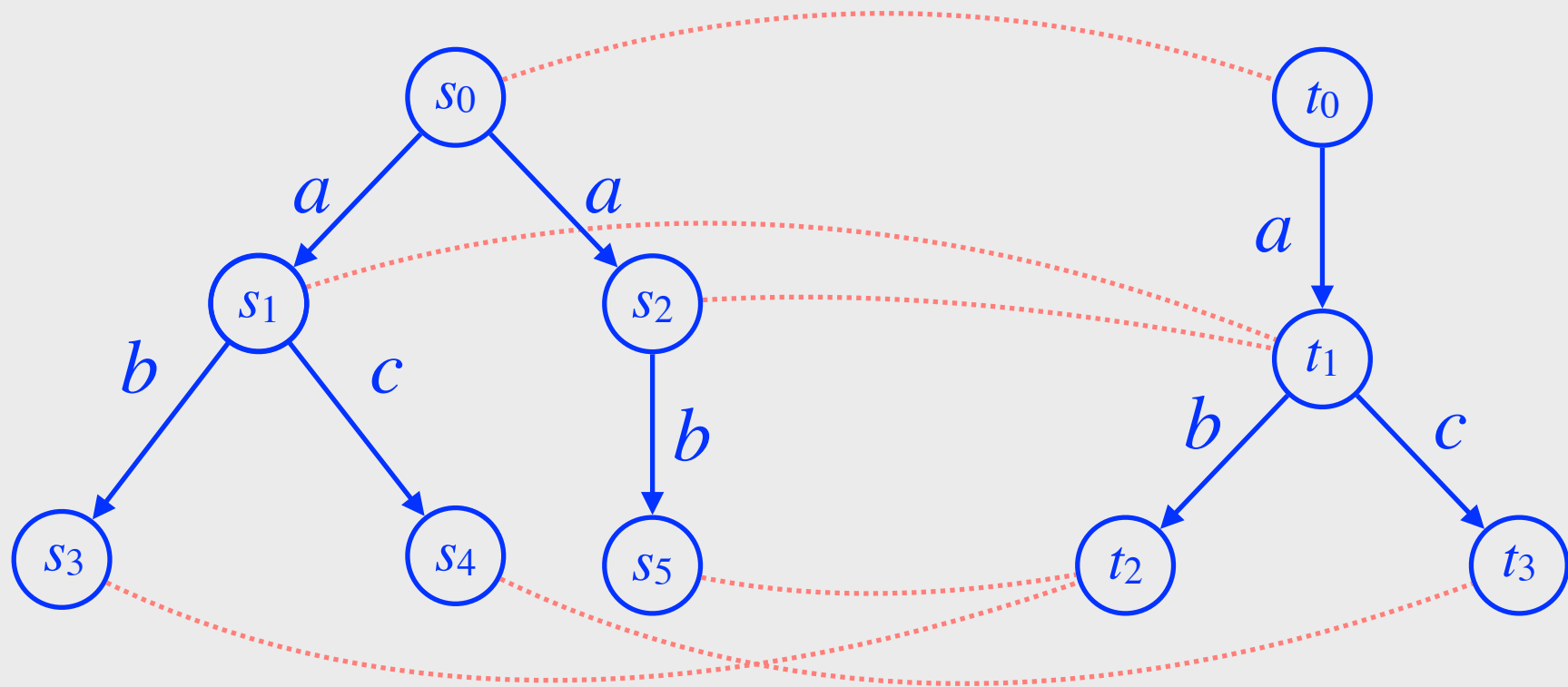
No hay una simulación al revés

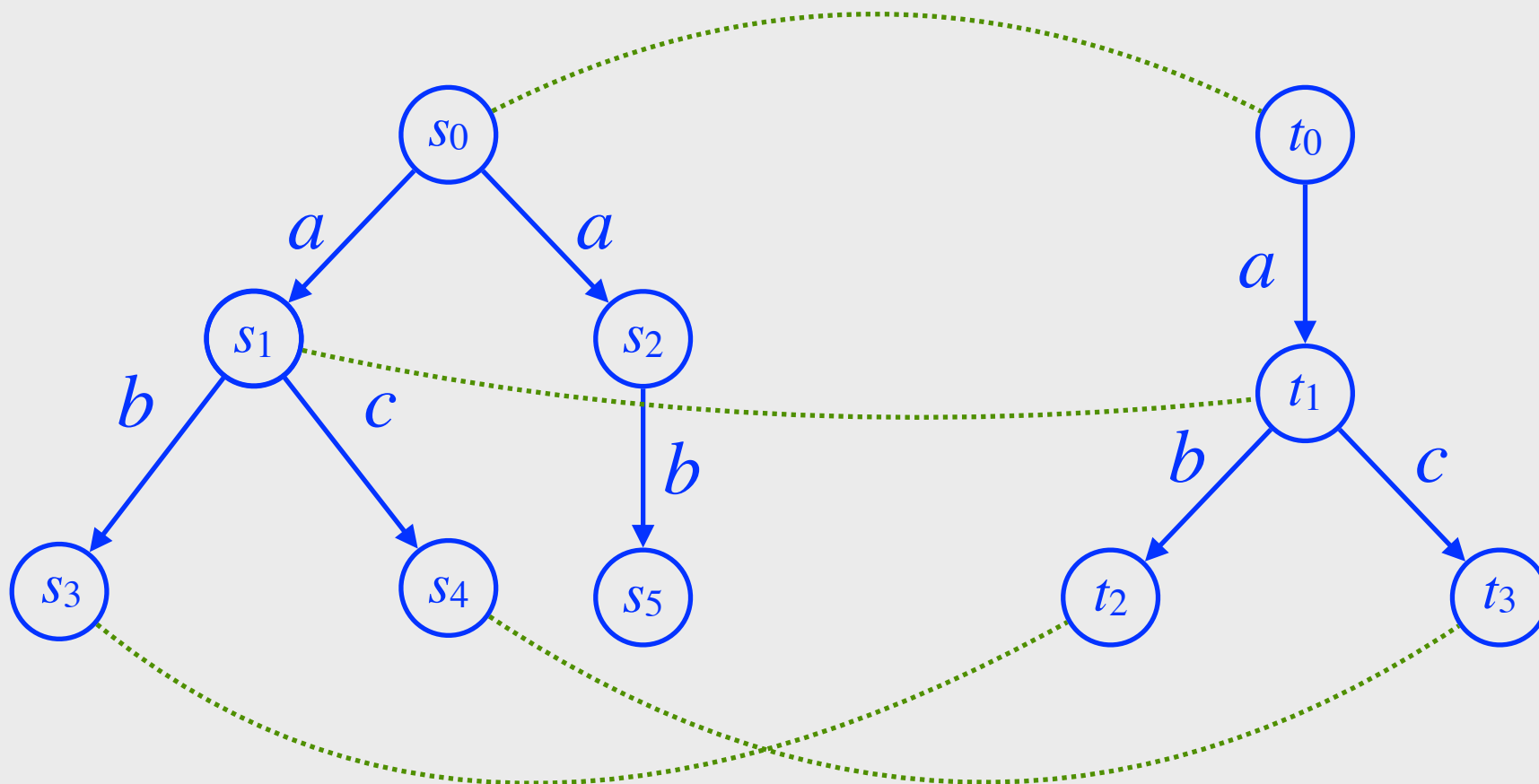


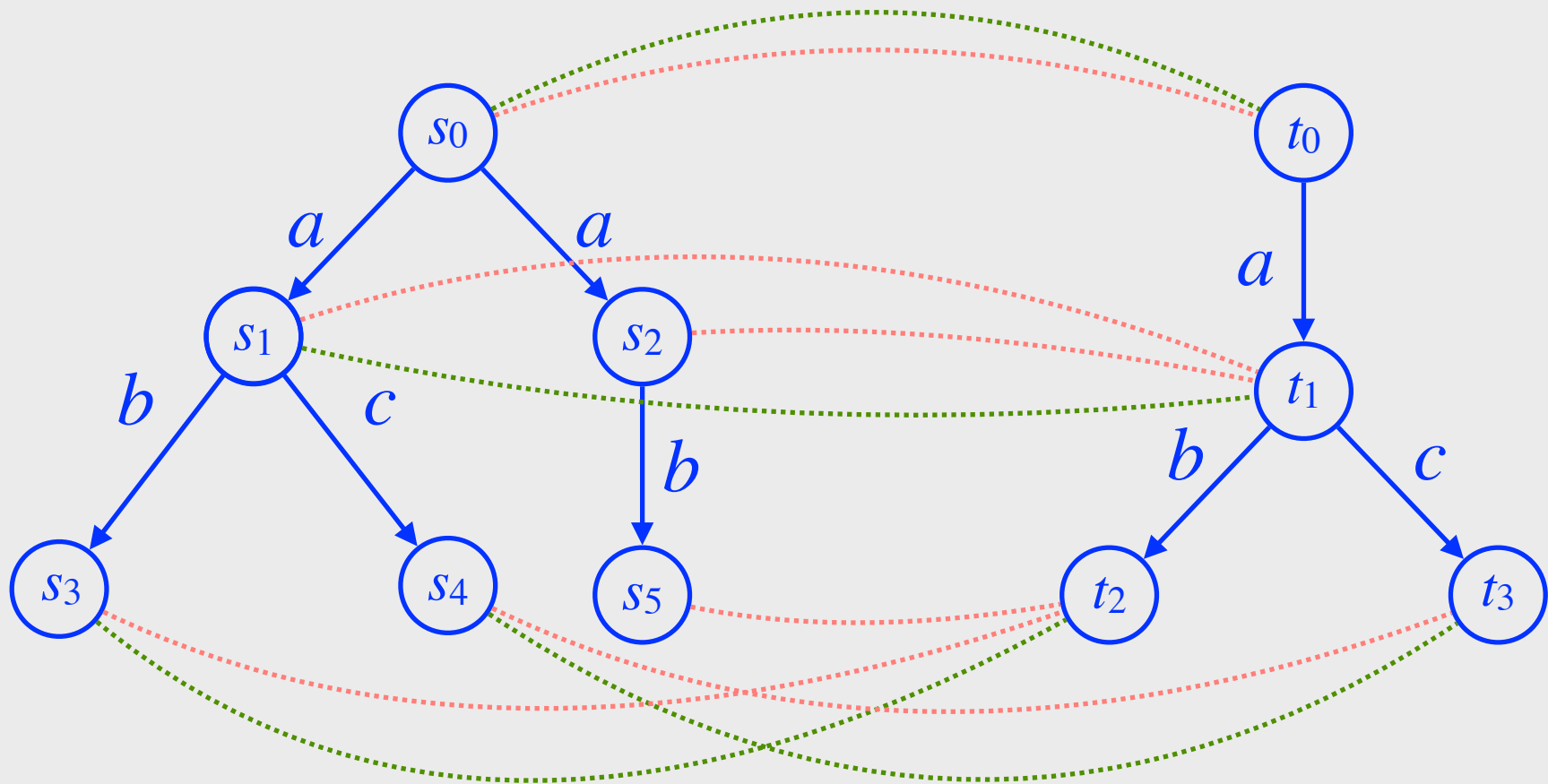
Por consiguiente no hay una bisimulación

¿Es posible que haya una simulación para cada lado pero no una bisimulación?



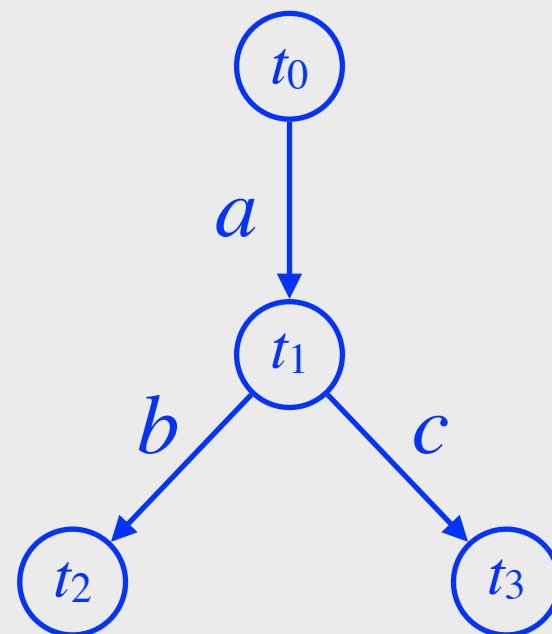
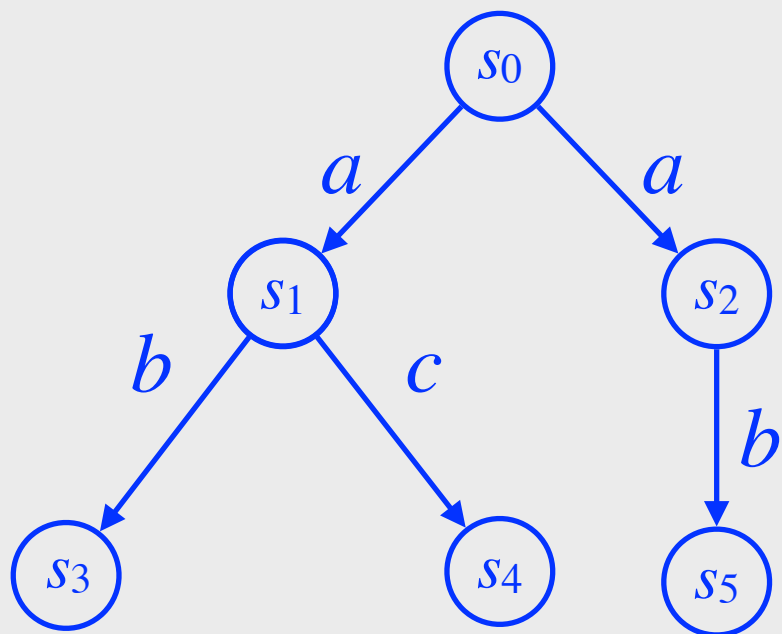




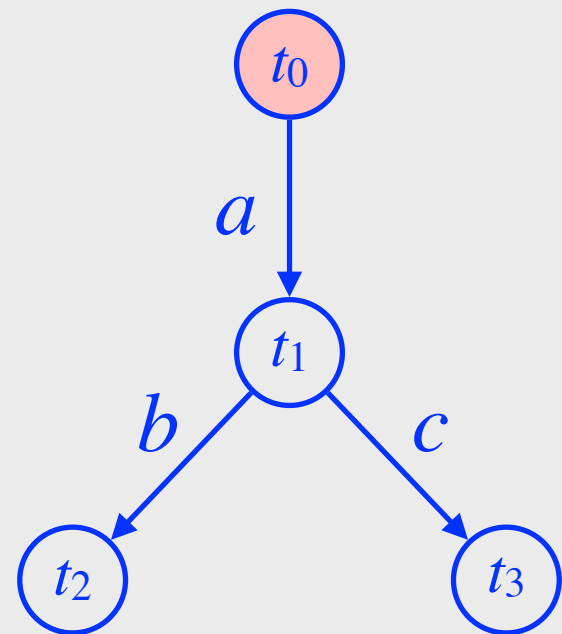
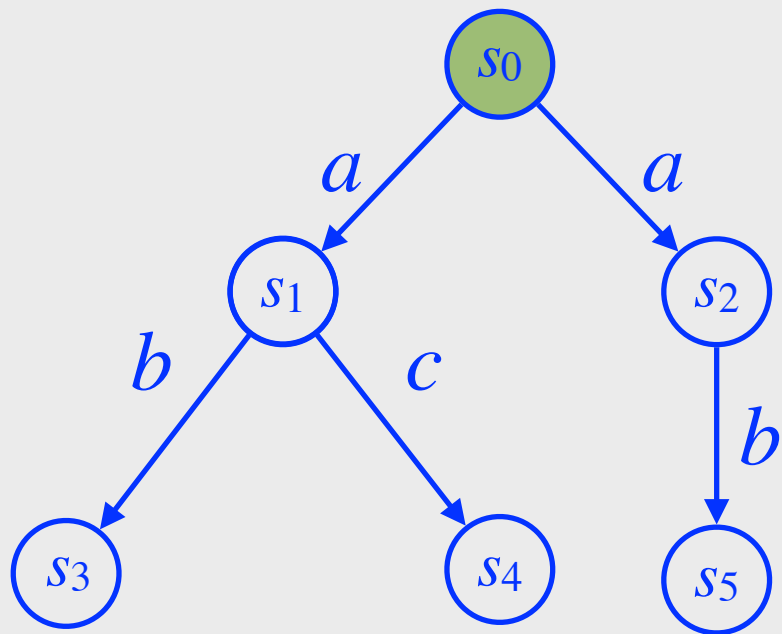


S es una simulación y T^{-1} es una simulación

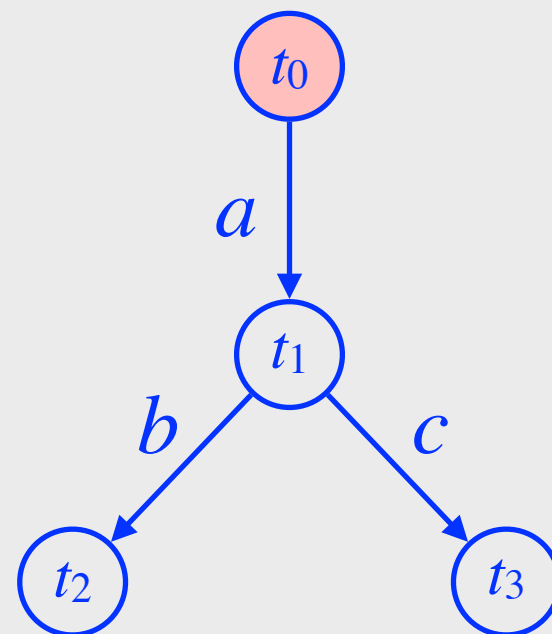
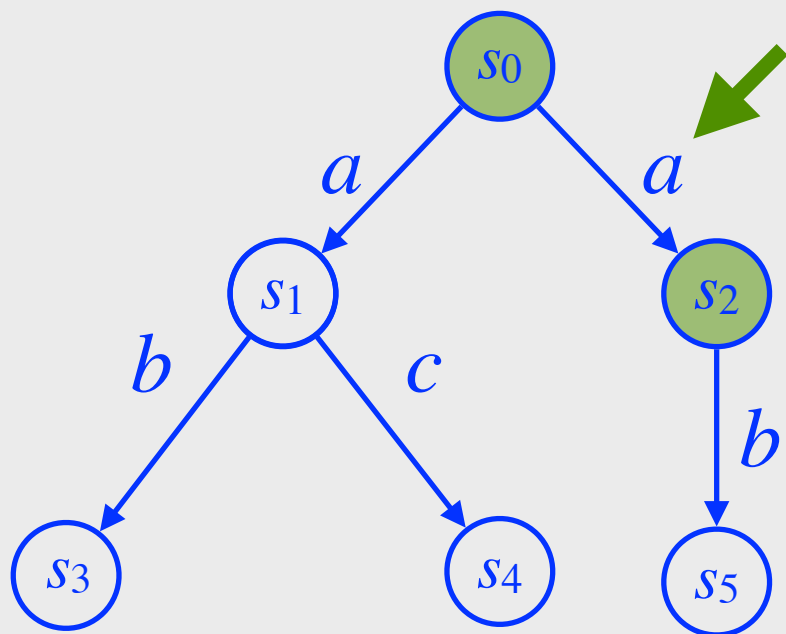
Sin embargo ...



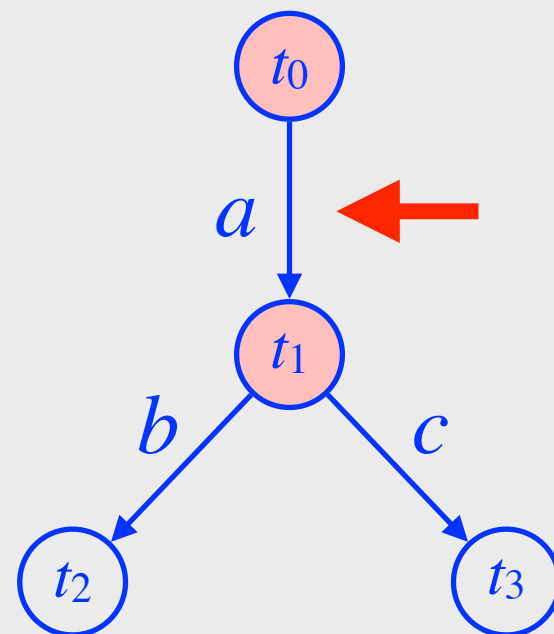
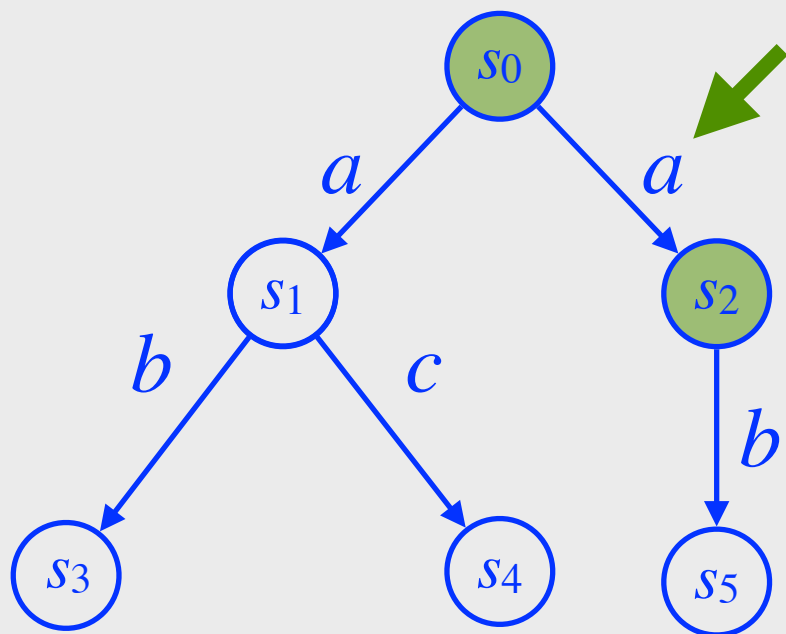
Sin embargo ...



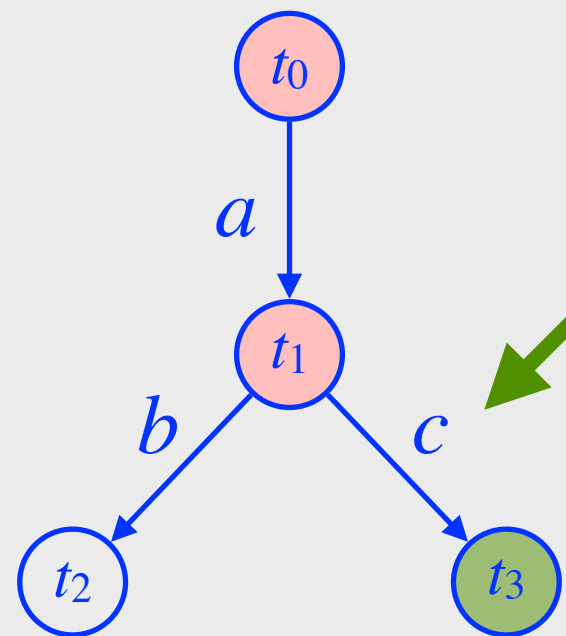
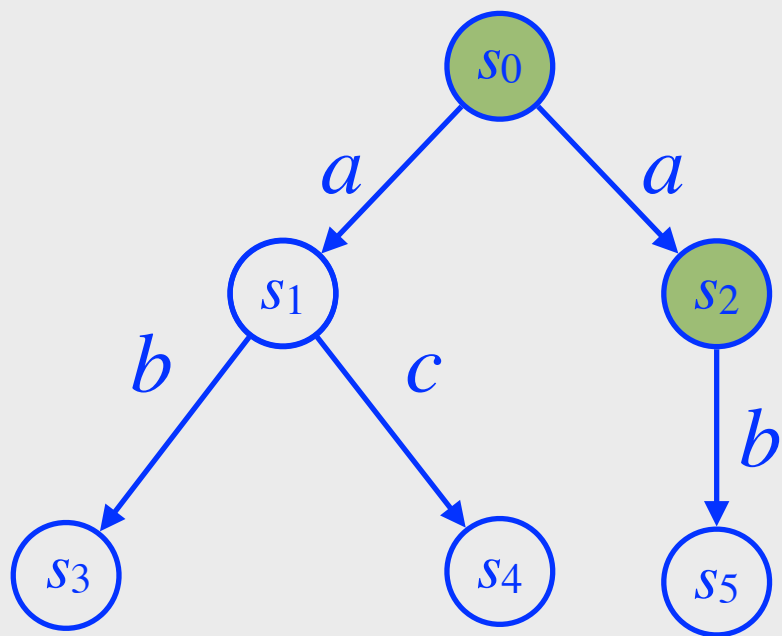
Sin embargo ...



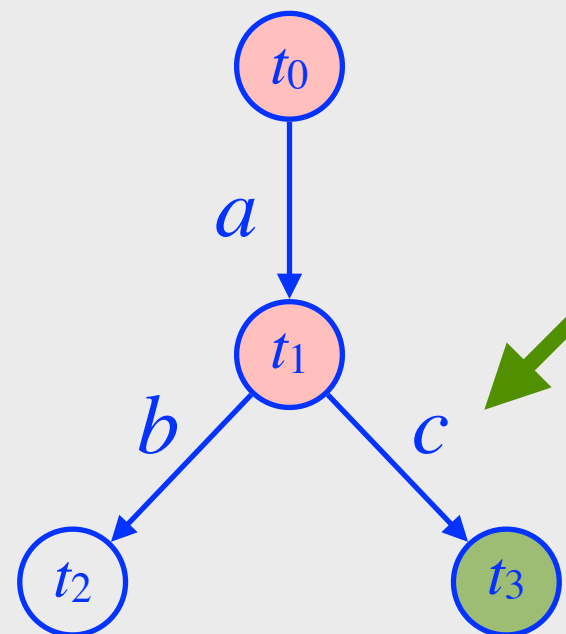
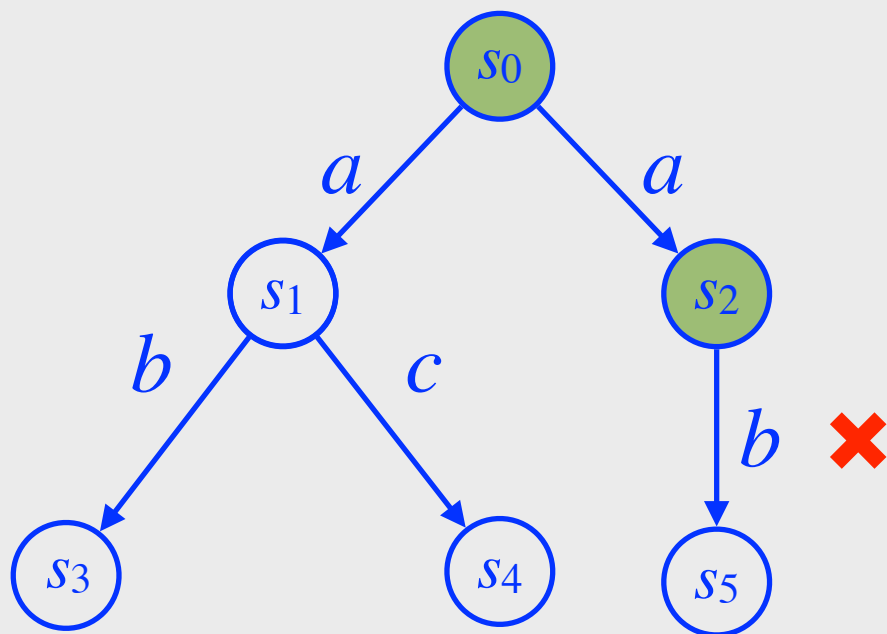
Sin embargo ...



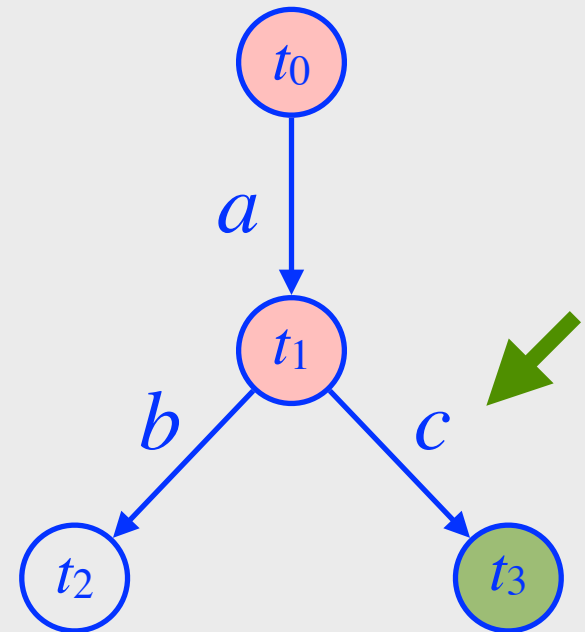
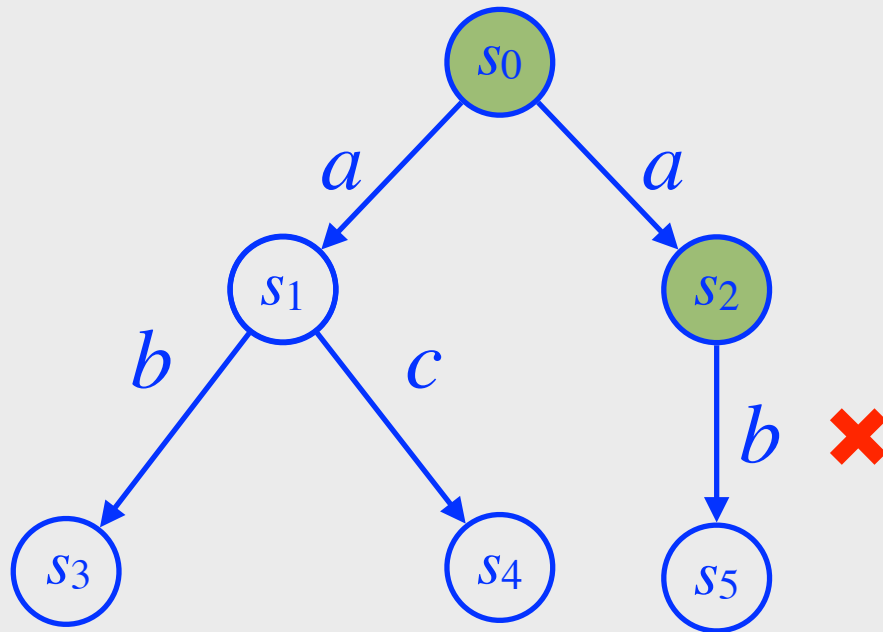
Sin embargo ...



Sin embargo ...



Sin embargo ...



Por consiguiente no son bisimilares

Bisimulación en LTS

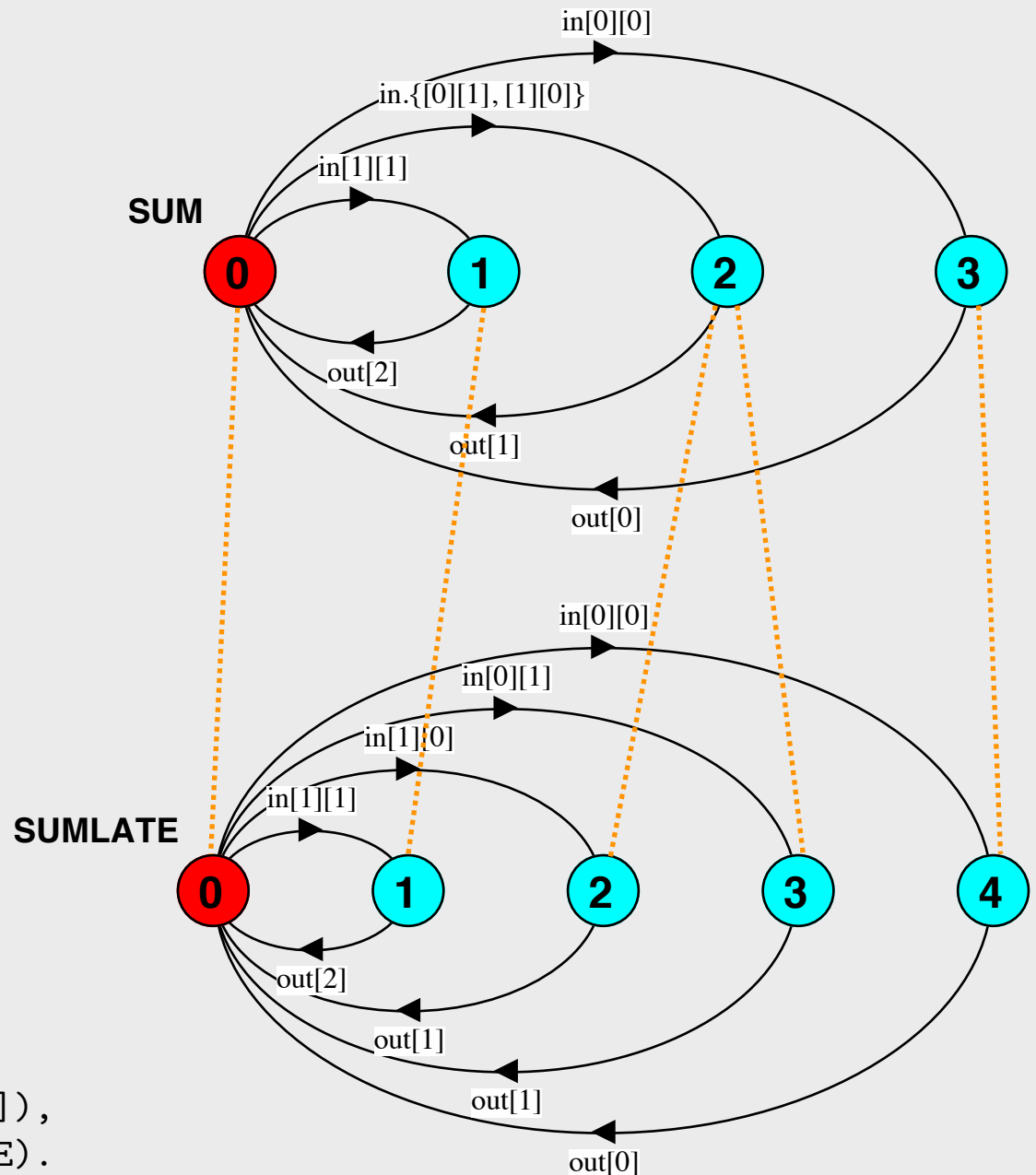
```
const N = 1
range T = 0..N
range R = 0..2*N
```

```
SUM = (in[a:T] [b:T] -> TOTAL[a+b]),
TOTAL[s:R] = (out[s] -> SUM).
```

..... indica la relación
de bisimulación

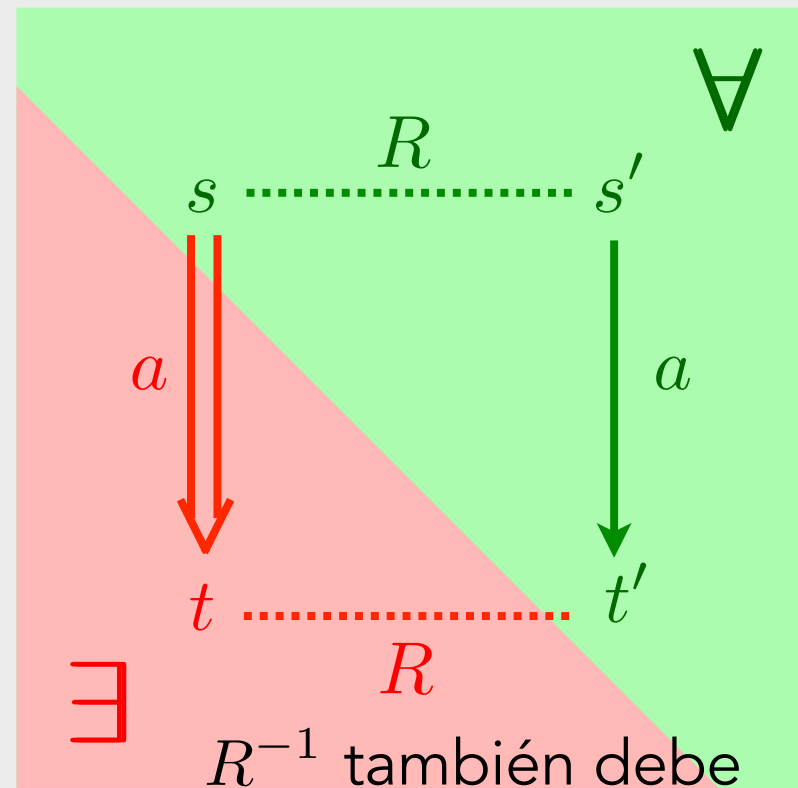
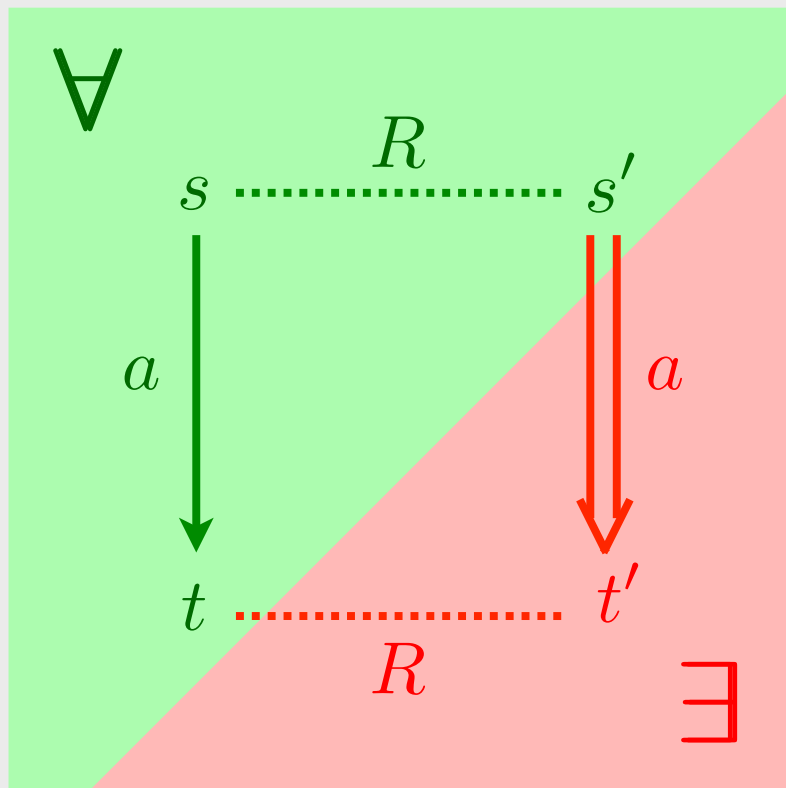
```
const N = 1
range T = 0..N
```

```
SUMLATE = (in[a:T] [b:T] -> CARRY[a] [b]),
CARRY[s:T] [t:T] = (out[s+t] -> SUMLATE).
```

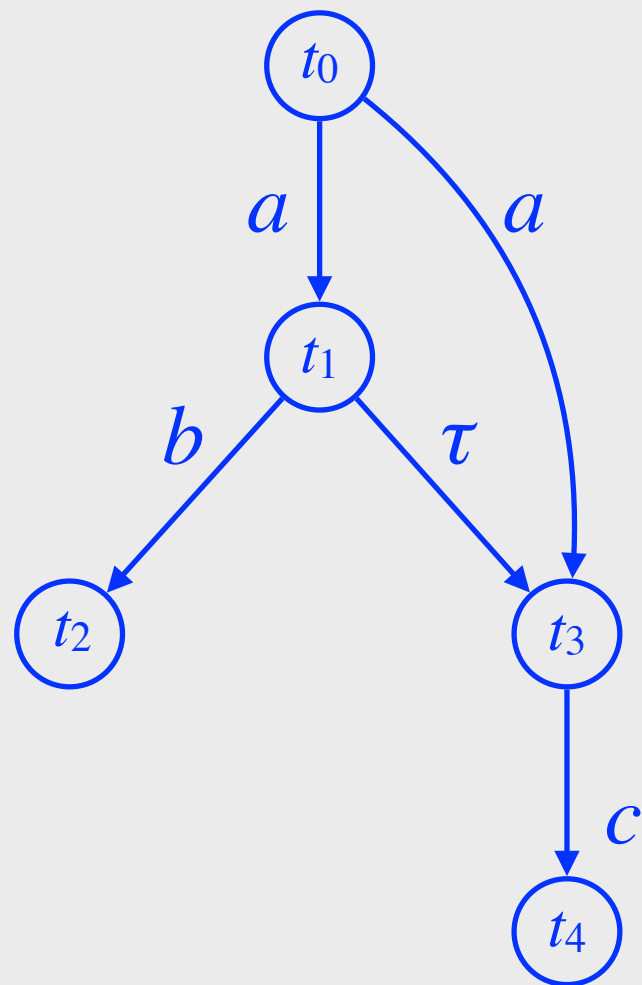
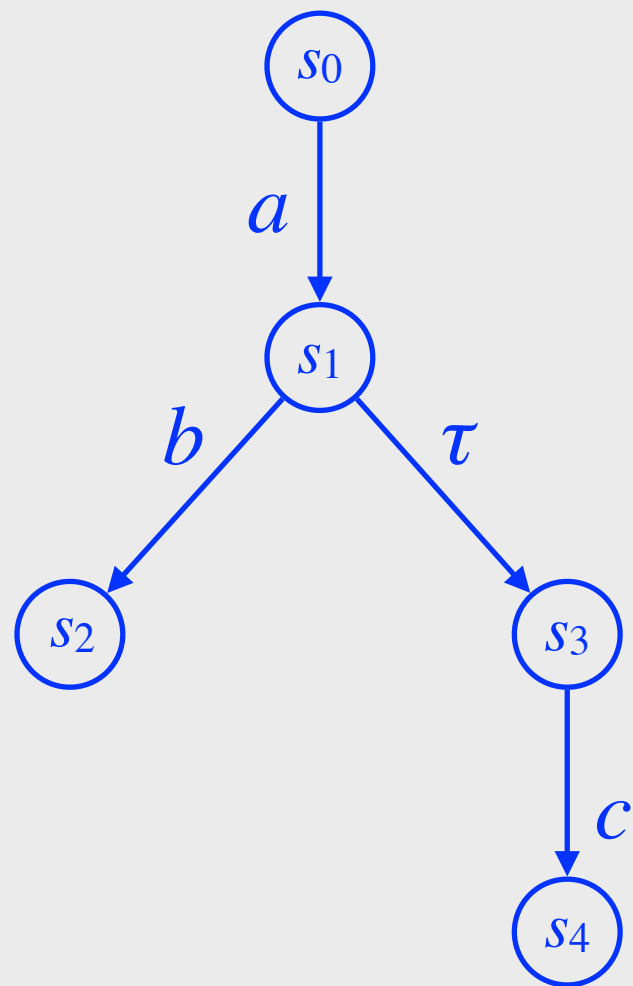


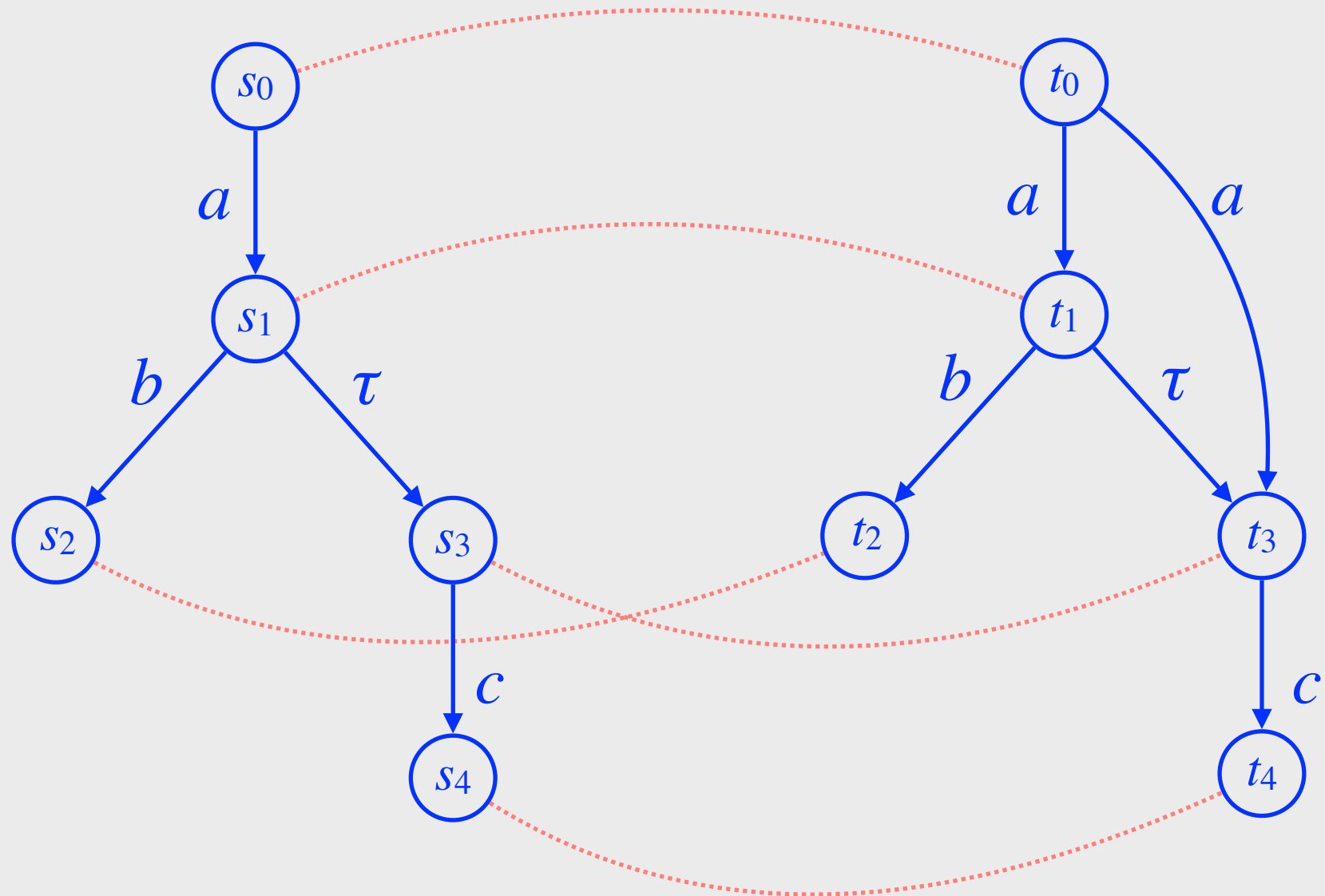
Bisimulación débil en LTS

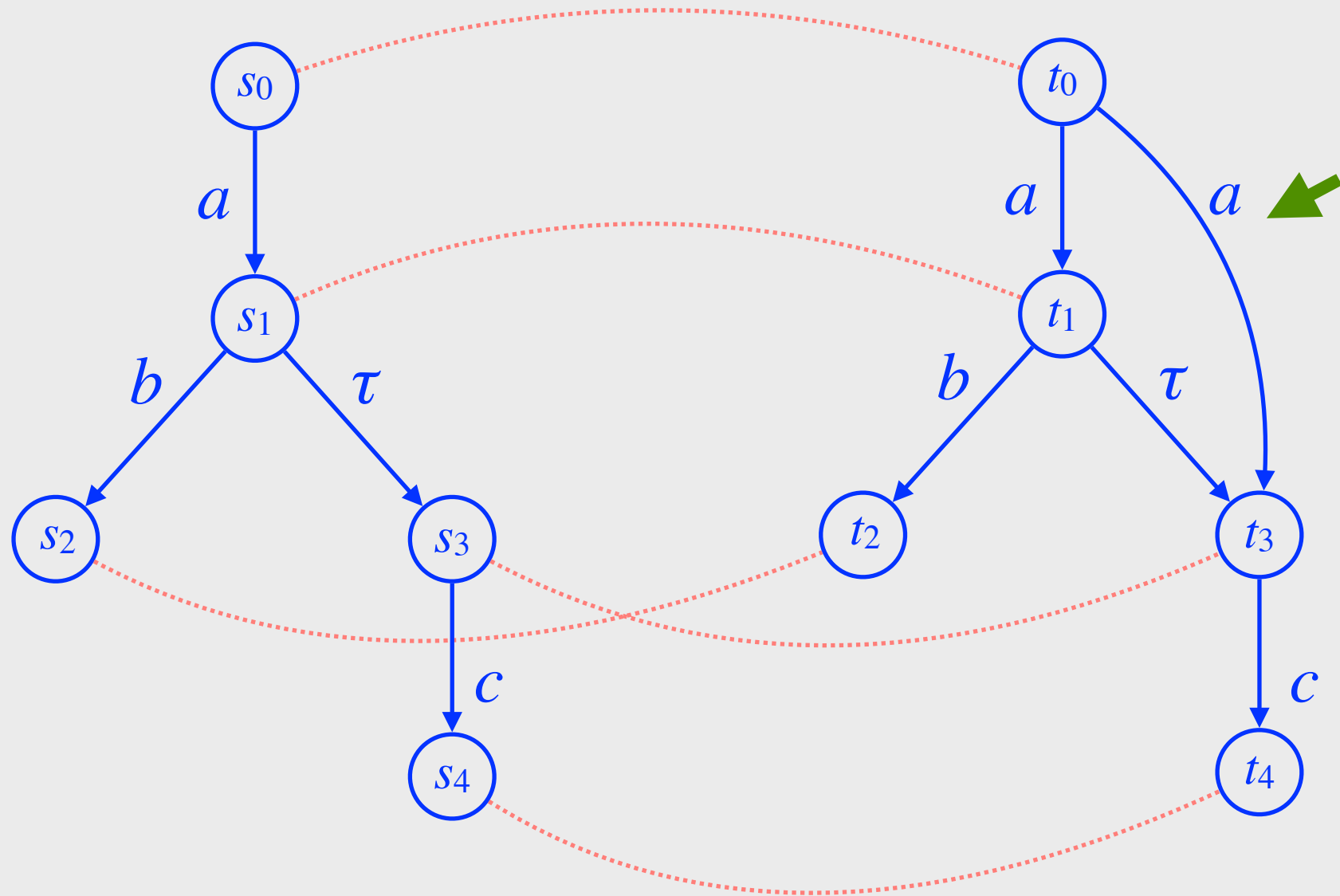
$$s \Longrightarrow t \text{ se define como } \begin{cases} s \xrightarrow{\tau}^* t & \text{si } a = \tau \\ s \xrightarrow{\tau}^* \xrightarrow{a} \xrightarrow{\tau}^* t & \text{si } a \neq \tau \end{cases}$$

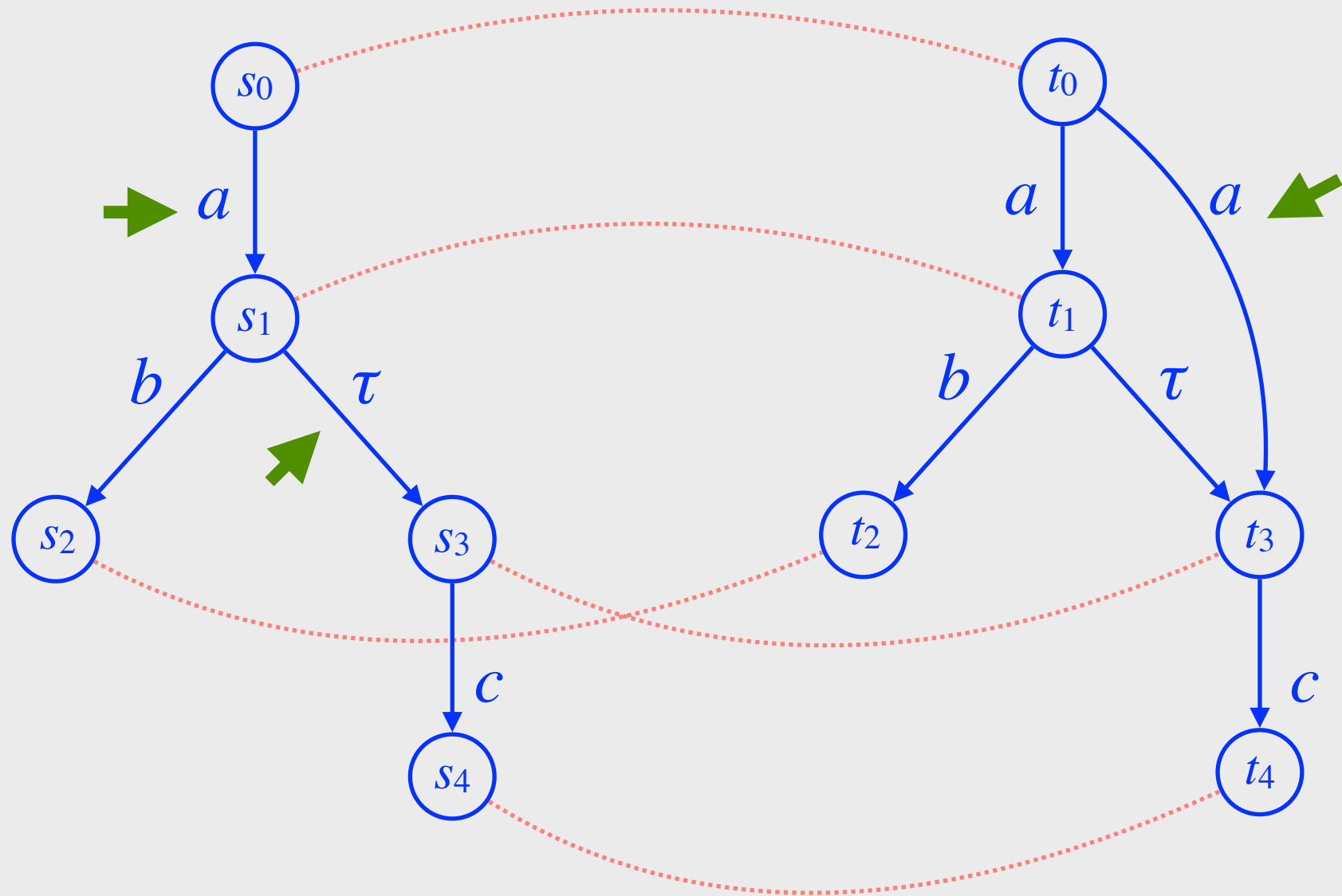


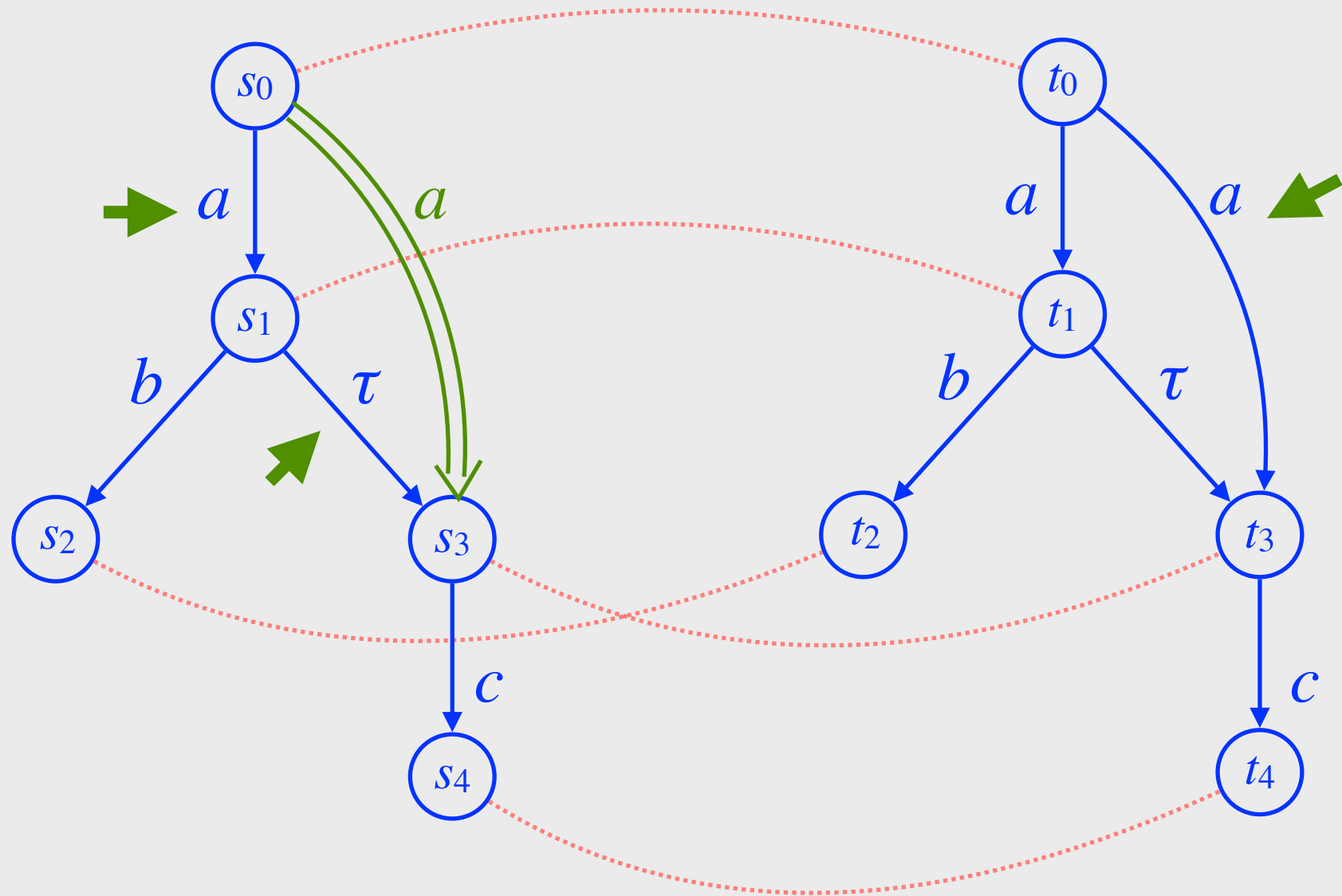
R^{-1} también debe
satisfacer esta propiedad









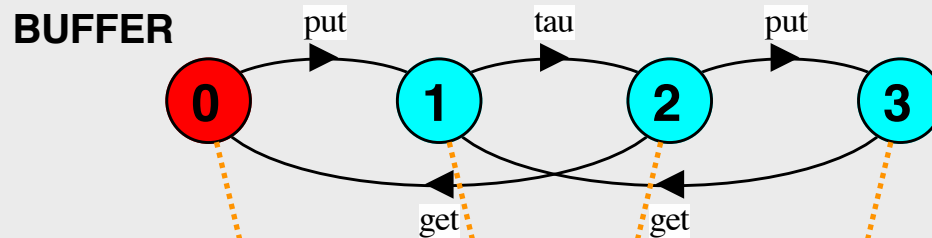


Bisimulación débil en LTS

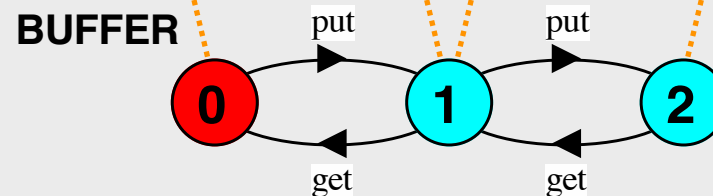
ONEBUFF = (put -> get -> ONEBUFF).

```
|| BUFFER(N=2) = ([1..N]:ONEBUFF  
  )/{put/[1].put,  
    [i:2..N].put/[i-1].get,  
    get/[N].get  
  }@{put,get}.
```

LTS de BUFFER
original



LTS de BUFFER
minimizado



Bisimulación débil en LTS

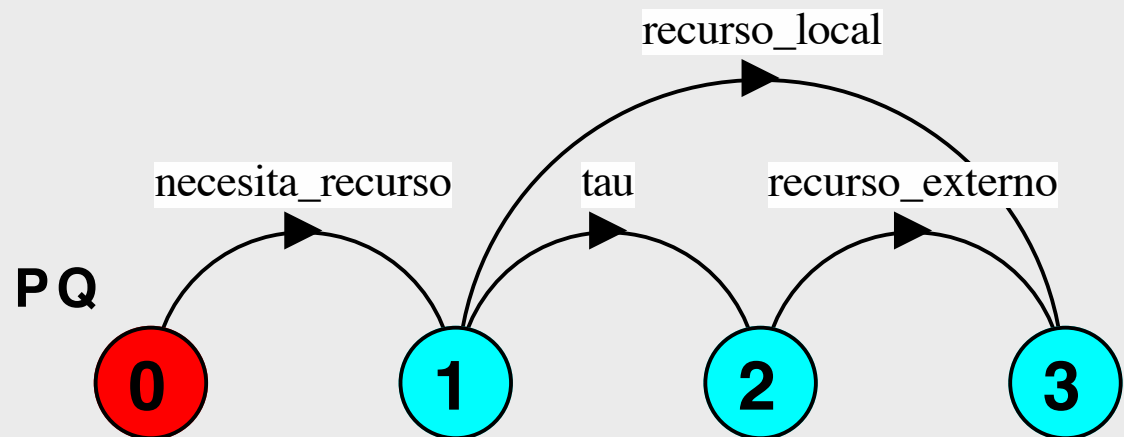
Bajo bisimulación débil, no siempre es posible eliminar las acciones tau

```
P = ( necesita_recurso -> ( recurso_local -> STOP  
                                | sync -> STOP)  
    ).
```

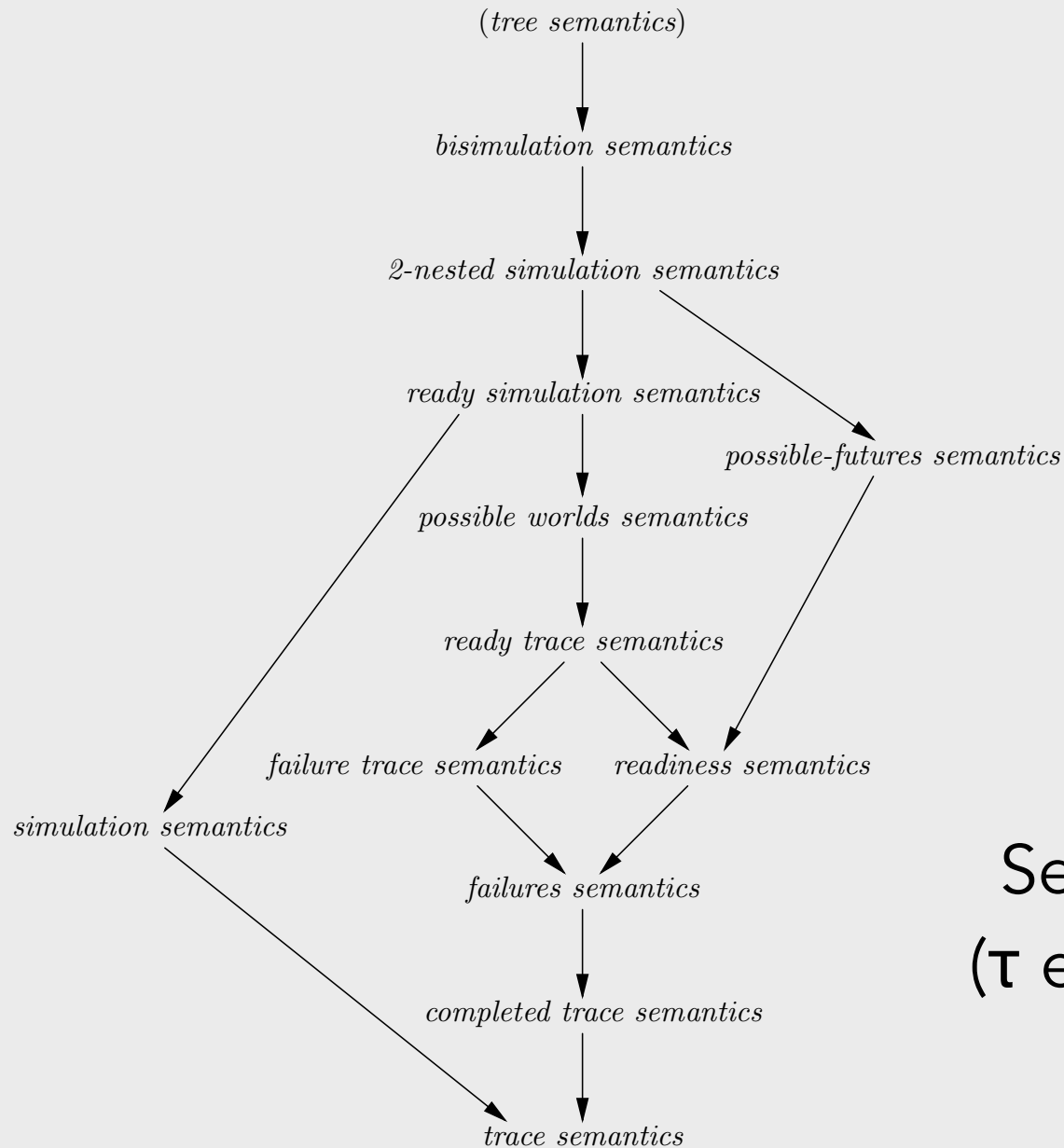
```
Q = ( sync -> recurso_externo -> STOP ).
```

```
||PQ = (P || Q) \ {sync}.
```

El LTS de PQ ya
está minimizado



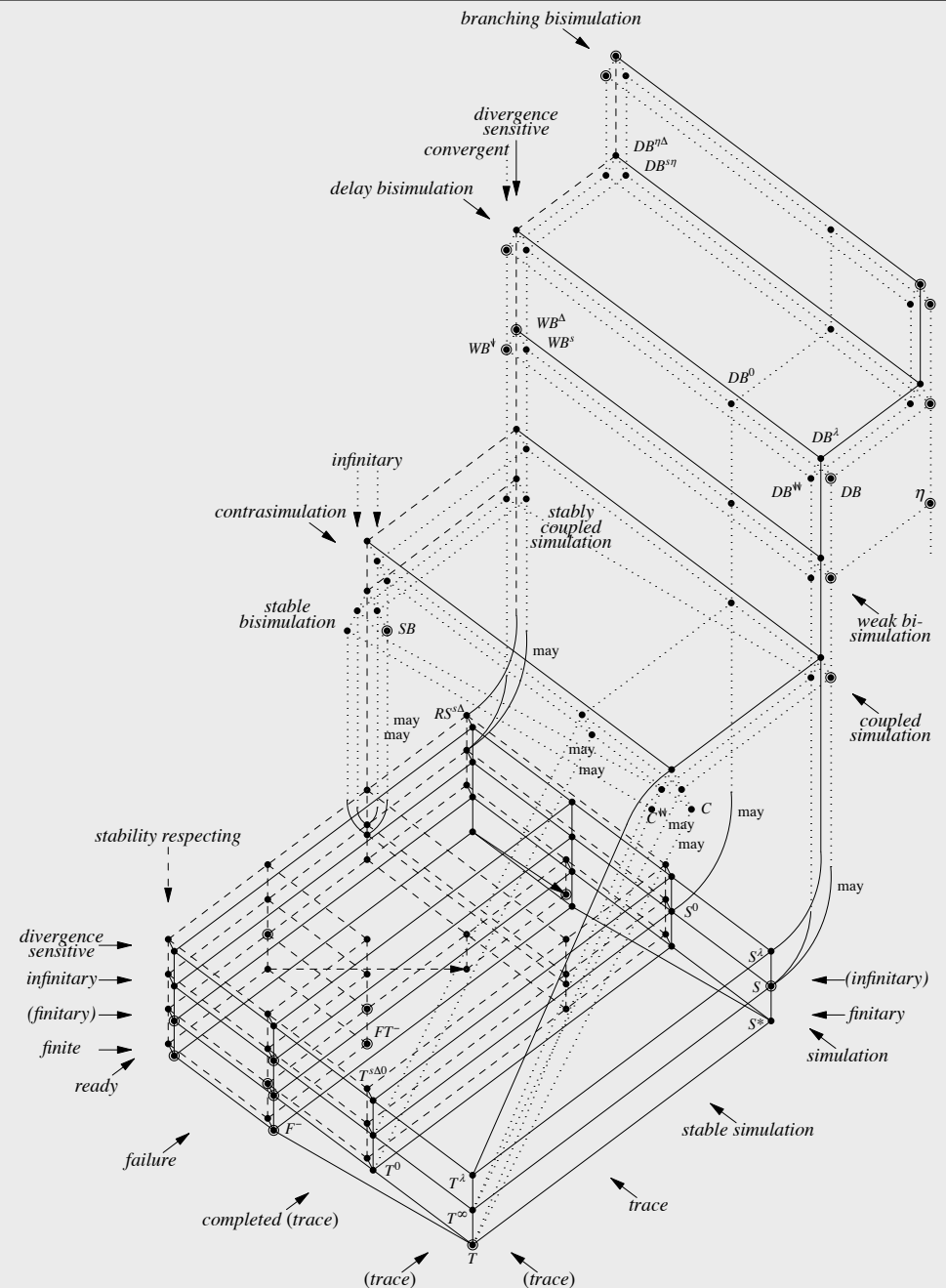
Espectros de semánticas



Semánticas sin abstracción
(τ es como cualquier evento)

Espectros de semánticas

Semánticas con abstracción
(τ es el evento sigiloso)



Comunicación mediante pasaje de mensajes

Además de la comunicación mediante recursos compartidos y la sincronización mediante TADs específicos, los procesos pueden comunicarse y/o sincronizarse a través de pasaje de mensajes. Veremos tres esquemas de pasaje de mensajes, a través de canales.

👁 **Pasaje de mensajes sincrónico**

En el pasaje de mensajes a través de canales sincrónicos, las operaciones que se consideran son:

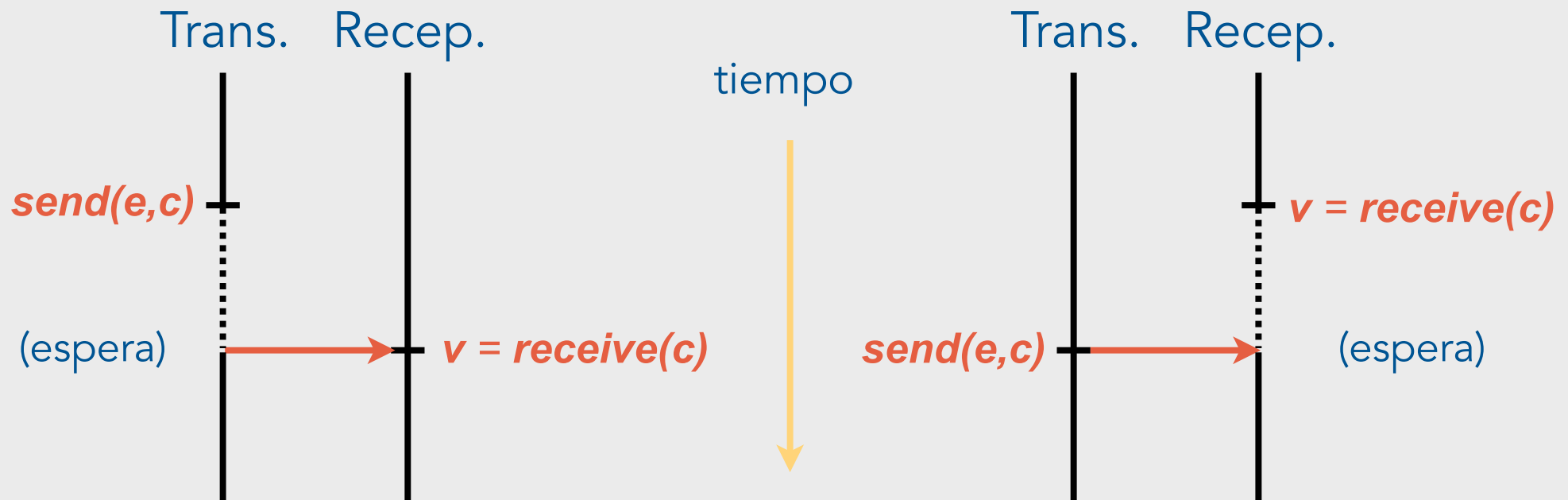
- 👁 **$send(e, c)$** envía el valor de la expresión **e** a través del canal **c** .
- 👁 **$v = receive(c)$** recibe un dato a través del canal **c** y lo almacena en **v** .

Comunicación mediante pasaje de mensajes

Pasaje de mensajes sincrónico

Cuando un proceso necesita ejecutar una operación de comunicación a través de un canal sincrónico (ya sea de envío o recepción), este se bloquea hasta tanto otro proceso realice la acción complementaria.

Luego de la comunicación, los procesos continúan independientemente.



Comunicación mediante pasaje de mensajes

Pasaje de mensajes sincrónico

Para modelar las operaciones de envío y recepción de mensajes a través de canales sincrónicos en FSP podemos usar acciones indexadas:

- ***send(e,chan)*** se implementa con ***chan.send[e]***.
- ***v = receive(chan)*** se implementa con ***chan.receive[v:M]***.

```
range M = 0..9
```

```
SENDER = SENDER[0],
```

```
SENDER[e:M] = (chan.send[e]->SENDER[(e+1)%10]).
```

```
RECEIVER = (chan.receive[v:M]->RECEIVER).
```

```
||SyncMsg = (SENDER || RECEIVER)/{chan/chan.{send,receive}}.
```

Los eventos **send** y **receive** deben sincronizarse apropiadamente

Comunicación mediante pasaje de mensajes

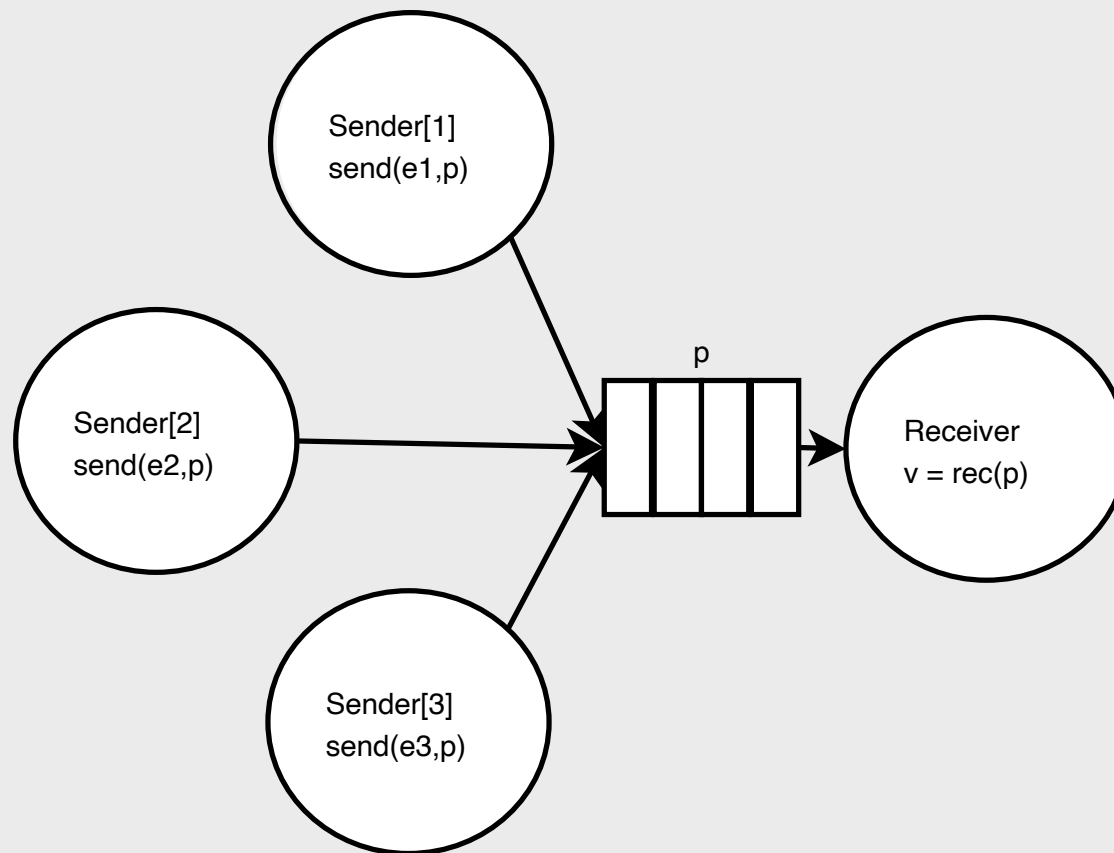
Recepción selectiva

- Permite recibir mensajes que llegan a través de distintos canales o inclusive otros eventos (ej: timeouts).
- Se implementa a través de la **elección** (con condiciones, si es necesario) y corresponde a un **select** de **Ada** u **Occam**.
- En **C**, la operación **select** se encuentra implementada en una de las librerías. (Hagan 'man select' en su xterm.)

Comunicación mediante pasaje de mensajes

Pasaje de mensajes asíncrono

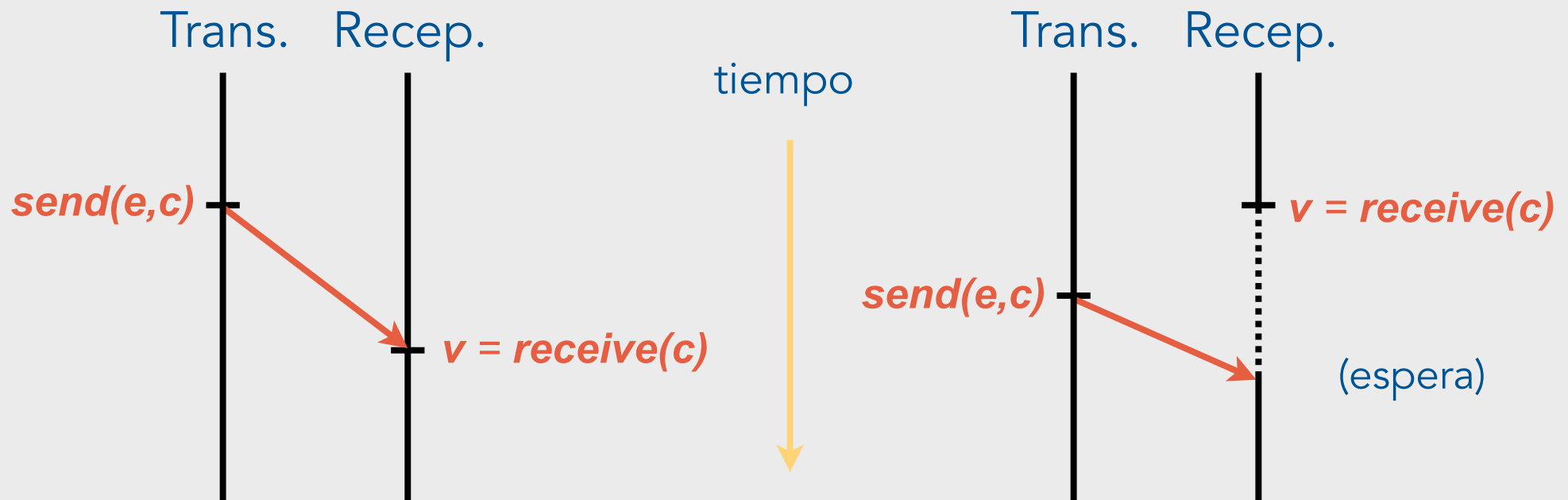
En el pasaje de mensajes asíncrono, la acción de envío no se bloquea, y los mensajes se almacenan en buffers, hasta que las acciones de recepción son ejecutadas.



Comunicación mediante pasaje de mensajes

Pasaje de mensajes asíncrono

Se implementan con las mismas primitivas **send** y **receive** que para el pasaje de mensaje síncrono, sólo que **send** no es bloqueante.



Comunicación mediante pasaje de mensajes

Pasaje de mensajes asíncrono

Para modelar las operaciones de envío y recepción de mensajes a través de canales asíncronos en FSP debemos considerar buffers acotados (debe preservarse la finitud de los modelos). Para hacer esto puede utilizarse un modelo de puertos:

```
range M = 0..9
set    S = {[M], [M] [M]}

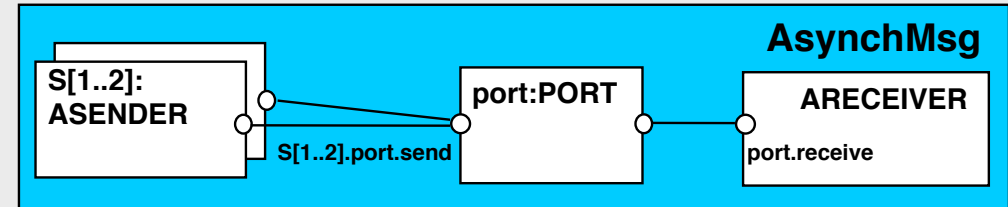
PORT          //empty state, only send permitted
  = (send[x:M]->PORT[x]),
PORT[h:M]     //one message queued to port
  = (send[x:M]->PORT[x] [h]
    |receive[h]->PORT
    ),
PORT[t:S] [h:M] //two or more messages queued to port
  = (send[x:M]->PORT[x] [t] [h]
    |receive[h]->PORT[t]
    ).
```

Comunicación mediante pasaje de mensajes

Pasaje de mensajes asíncrono

```

range M = 0..9
set S = { [M], [M] [M] }
PORT
    = (send[x:M]->PORT[x]),
PORT[v:M]
    = (send[x:M]->PORT[x][v]
      | receive[v]->PORT),
PORT[s:S][v:M]
    = (send[x:M]->PORT[x][s][v]
      | receive[v]->PORT[s]).
  
```



```

ASENDER = ASENDER[0],
ASENDER[e:M] = (port.send[e]->ASENDER[(e+1)%10]).
  
```

```

ARECEIVER = (port.receive[v:M]->ARECEIVER).
  
```

```

||AsynchMsg = (s[1..2]:ASENDER || ARECEIVER||port:PORT)
              /{s[1..2].port.send/port.send}.
  
```

```

||Abstract = AsynchMsg
              /{s[1..2].port.send/s[1..2].port.send[M],
                port.receive/port.receive[M]
              }.
  
```

Comunicación mediante pasaje de mensajes

Pasaje de mensajes asíncrono

```
range M = 0..9
set S = {[M], [M] [M]}

PORT //empty state, only send permitted
= (send[x:M]->PORT[x]),
PORT[h:M] //one message queued to port
= (send[x:M]->PORT[x] [h]
|receive[h]->PORT
),
PORT[t:S] [h:M] //two or more messages queued to port
= (send[x:M]->PORT[x] [t] [h]
|receive[h]->PORT[t]
).
```

Canal como un
buffer con overflow

```
range M = 0..9

ONEBUFF = (put[x:M] -> get[x] -> ONEBUFF).

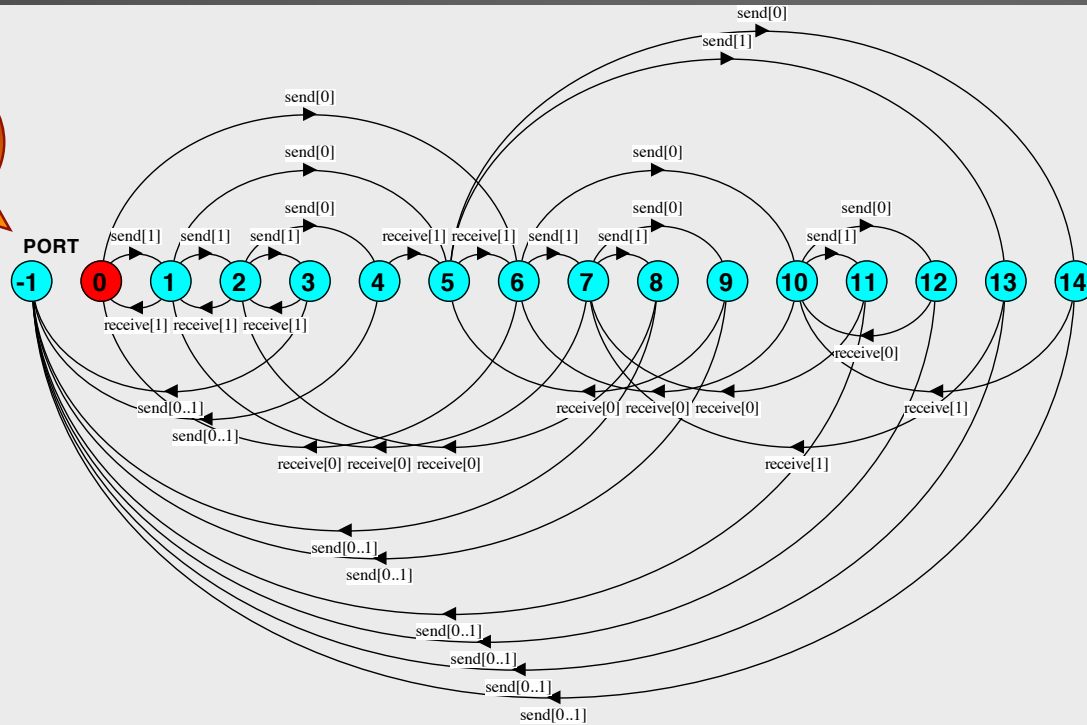
||BUFFER(N=3) = ([1..N]:ONEBUFF
)/{put/[1].put,
[i:2..N].put/[i-1].get,
get/[N].get
}@{put,get}.
```

Canal como un
buffer bloqueante

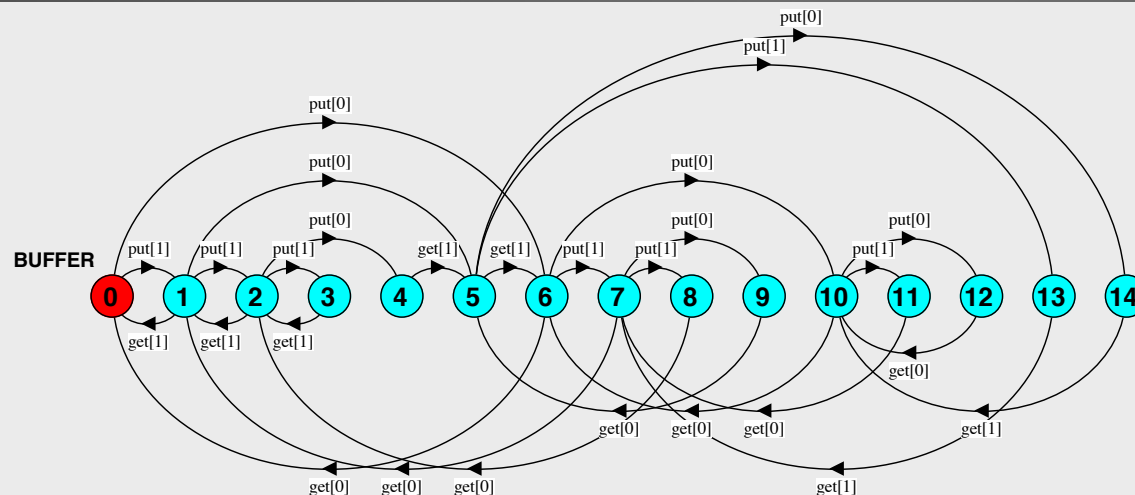
Comunicación mediante pasaje de mensajes

Pasaje de mensajes asíncrono

Estado
erróneo



Canal como un
buffer con overflow



Canal como un
buffer bloqueante

Comunicación mediante pasaje de mensajes

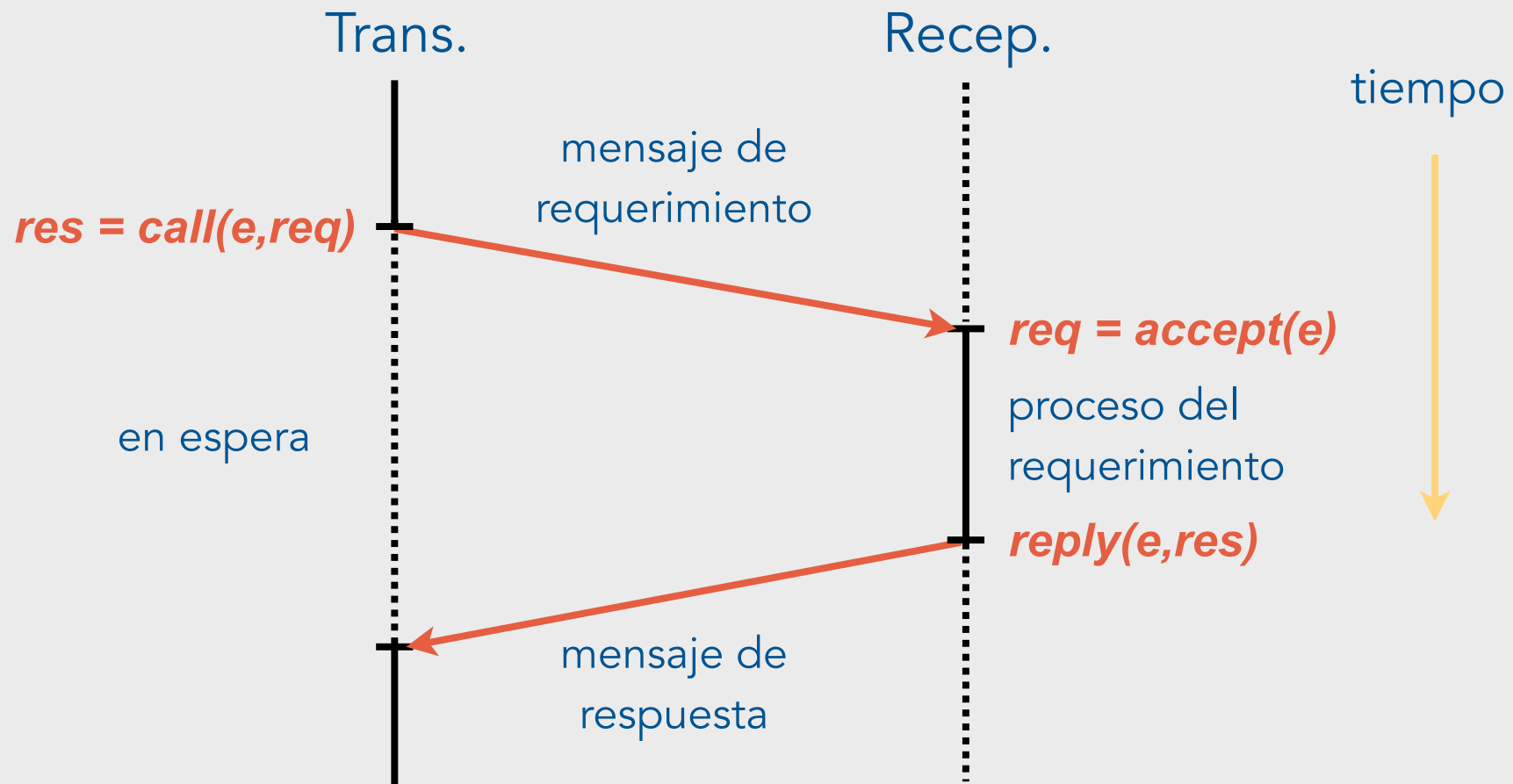
Rendezvous (o Request-Reply)

Es un protocolo de pasaje de mensajes utilizado para soportar interacción cliente-servidor.

- **$res = call(e, req)$** envía el valor **req** estableciendo el requerimiento que se encolará en la entrada **e** . Luego se bloquea hasta la ocurrencia del mensaje de respuesta, el que será recibido en la variable **res** .
- **$req = accept(e)$** recibe el valor del requerimiento en **e** y lo almacena en **req** . Si no hay requerimientos encolados en la entrada se bloquea a la espera de estos.
- **$reply(e, res)$** envía el resultado **res** como respuesta al mensaje entrado a través de **e** .

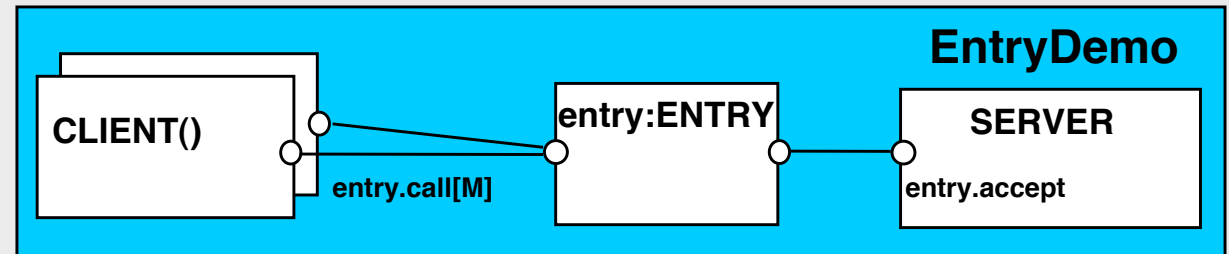
Comunicación mediante pasaje de mensajes

Rendezvous



Comunicación mediante pasaje de mensajes

Rendezvous



```
set M = {replyA,replyB}
```

```
set S = {[M], [M] [M]}
```

```
PORT = (send[x:M]->PORT[x]),
```

```
PORT[v:M] = (send[x:M]->PORT[x] [v]  
             |receive[v]->PORT),
```

```
PORT[s:S] [v:M] = (send[x:M]->PORT[x] [s] [v]  
                  |receive[v]->PORT[s]).
```

```
||ENTRY = PORT/{call/send, accept/receive}.
```

```
CLIENT(CH='reply') = (entry.call[CH]->[CH]->CLIENT).
```

```
SERVER = (entry.accept[ch:M]->[ch]->SERVER).
```

```
||EntryDemo = (CLIENT('replyA') || CLIENT('replyB')  
               || entry:ENTRY || SERVER ).
```

Ingeniería del Software II

4 - Propiedades de los sistemas
reactivos y su análisis

Propiedades de los sistemas concurrentes

Con frecuencia, los programas concurrentes suelen ser reactivos, y sus características diferentes de las de los programas secuenciales convencionales.

Por esto, las propiedades que en general se desea garantizar de programas concurrentes difieren de las propiedades de programas secuenciales. Algunas de estas son:

- No violación de **invariantes** de sistema
- Ausencia de **starvation**
- Ausencia de **deadlock**
- Garantía de **exclusion mutua** en el acceso a recursos compartidos
- Ausencia de **livelock**

Categorías de propiedades

Una clasificación clásica de las propiedades de los sistemas reactivos incluye las siguientes cuatro categorías:

- Propiedades de **alcanzabilidad** ("reachability")
- Propiedades de **seguridad** ("safety")
- Propiedades de **vitalidad** ("liveness")
- Propiedades de **equidad** ("fairness")

Propiedades de alcanzabilidad ("reachability")

"Es posible que el sistema llegue a algún estado de un conjunto dado"

Por ejemplo:

- una (o alguna) componente entra a la region crítica
- es posible llegar a un estado de ERROR (ej: deadlock, violación de exclusión mutua, violación de invariante, etc.)

En este último caso nos interesa que se cumpla la negación de la propiedad. Es usual que la propiedad de interés sea la negación de la alcanzabilidad.

Propiedades de seguridad ("safety")

"Nunca va a pasar nada malo"

Por ejemplo:

- no es posible llegar a un estado de deadlock
- se garantiza exclusión mutua
- el sistema preserva un invariante dado

Propiedades de seguridad ("safety")

"Nunca va a pasar nada malo"

Por ejemplo:

- no es posible llegar a un estado de deadlock
- se garantiza exclusión mutua
- el sistema preserva un invariante dado

En general, la negación de una propiedad safety es una propiedad de alcanzabilidad

Propiedades de vitalidad ("liveness")

"Siempre es posible que algo bueno ocurra"

Por ejemplo:

- un (sub)proceso dado termina su ejecución
- es posible alcanzar un estado de estabilidad
- si se llama al ascensor, este llegará en algún momento

Propiedades de equidad ("fairness")

"Siempre ocurrirá algo de manera frecuente"

Por ejemplo:

- siempre que un proceso esté esperando para entrar a una región crítica, entonces logrará entrar
- siempre que un proceso solicite periódicamente un recurso finalmente se le será asignado.
- un proceso dado no sufre de inanición

Pospondremos su estudio en más detalle para la próxima unidad.

Propiedades como conjuntos de trazas

La semántica de procesos (en realidad, una de las semánticas más simples para procesos) puede definirse como el conjunto de todas sus ejecuciones. Cada ejecución de un proceso puede verse como la sucesión de eventos en los cuales el proceso se involucra (y en el orden en que lo hace).

Ejemplo: Consideremos el siguiente proceso:

```
MAKER = (make->ready->MAKER) .  
USER  = (ready->use->USER) .  
  
||MAKER_USER = (MAKER || USER) .
```

La única traza posible para **MAKER** es:

```
make, ready, make, ready, make, ...
```

Algunas trazas posibles para **MAKER_USER** son:

```
make, ready, use, make, ready, use, ...  
make, ready, make, use, ready, make, ready, use, ...
```

Propiedades como conjuntos de trazas (cont.)

Al igual que los procesos, las propiedades tienen una semántica dada en términos de conjuntos de trazas. Una propiedad P se identifica con todas las sucesiones de eventos atómicos que exhiben la propiedad P .

Por ejemplo, la propiedad “no ocurren dos producciones (**make**) sucesivas” contiene las siguientes trazas:

```
use, use, use, use, use, use, use, ...
```

```
make, use, make, use, make, use, make, use, ...
```

```
ready, ready, ready, ready, ready, ...
```

Propiedades como conjuntos de trazas (cont.)

Al igual que los procesos, las propiedades tienen una semántica dada en términos de conjuntos de trazas. Una propiedad P se identifica con todas las sucesiones de eventos atómicos que exhiben la propiedad P .

Por ejemplo, la propiedad “no hay dos eventos **(make)** sucesivos” contiene las siguientes trazas:

Notar que una propiedad describe un conjunto de trazas que no necesariamente se corresponde con las trazas definidas por el comportamiento por el modelo bajo estudio

```
use, use, use, use, use, use, use, ...  
make, use, make, use, make, use, make, use, ...  
ready, ready, ready, ready, ready, ready, ...
```

Lenguajes ω -regulares

Dado un lenguaje regular $A \subseteq \Sigma^*$ definimos

$$A^\omega \stackrel{\text{def}}{=} \{\sigma_1\sigma_2\sigma_3 \dots \mid \forall i \geq 0 : \sigma_i \in A \wedge \sigma_i \neq \epsilon\}$$

es decir, A^ω es el lenguaje que contiene todas las concatenaciones infinitas de palabras no vacías de A .

Un lenguaje L se dice ω -regular si existen lenguajes regulares A_i y B_i , $0 \leq i \leq k$, tal que $\epsilon \notin B_i \neq \emptyset$ y

$$L = \bigcup_{0 \leq i \leq k} A_i \cdot B_i^\omega$$

donde $\cdot$ denota la concatenación habitual de lenguajes.

Propiedad: Los lenguajes ω -regulares son cerrados por union, intersección y complemento.

Lenguajes ω -regulares

Dado un lenguaje regular A sobre Σ

$$A^\omega \stackrel{\text{def}}{=} \{\sigma_1\sigma_2\sigma_3 \dots\}$$

También las
denominaremos **fragmentos
de trazas**

es decir, A^ω es el lenguaje formado por las concatenaciones infinitas de palabras no vacías de A .

Un lenguaje L se dice ω -regular si existen lenguajes regulares A_i y B_i , $0 \leq i \leq k$, tal que $\epsilon \notin B_i \neq \emptyset$ y

$$L = \bigcup_{0 \leq i \leq k} A_i \cdot B_i^\omega$$

donde $\cdot$ denota la concatenación habitual de lenguajes.

Propiedad: Los lenguajes ω -regulares son cerrados por union, intersección y complemento.

Lenguajes ω -regulares

Ejemplo:

La propiedad P sobre el proceso **MAKER_USER** puede escribirse como cualquiera de las siguientes expresiones ω -regulares:

$$\left((\text{make} + \varepsilon) (\text{ready} + \text{use}) \right)^\omega$$

$$\left((\text{make} + \text{ready} + \text{use})^* \text{make make} (\text{make} + \text{ready} + \text{use})^\omega \right)$$

Lenguajes ω -regulares

Ejemplo:

La propiedad P sobre el proceso **MAKER_USER** puede escribirse como cualquiera de las siguientes expresiones ω -regulares:

$$\left((\text{make} + \varepsilon) (\text{ready} + \text{use}) \right)^\omega$$

$$\left((\text{make} + \text{ready} + \text{use})^* \text{make make} (\text{make} + \text{ready} + \text{use})^\omega \right)$$

Los lenguajes ω -regulares son cerrados por complemento

Formalización de propiedades safety

Observar:

- Si una traza viola una propiedad de safety, lo hace en un “instante” finito.
- Si un prefijo de una traza (infinita) viola una propiedad de safety, no hay forma de remediarlo (cualquiera sea la forma en que se continúe, la traza no dejará de violar la propiedad).

Es decir:

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de safety si cumple

$$\forall \sigma : \sigma \notin P \Rightarrow \exists i \geq 0 : \forall \beta : \sigma[..i]\beta \notin P$$

o equivalentemente:

donde $\sigma[..i]$ denota el prefijo i -ésimo de σ .

Formalización de propiedades safety

Observar:

- Si una traza viola una propiedad de safety, lo hace en un “instante” finito.
- Si un prefijo de una traza (infinita) viola una propiedad de safety, se puede remediarlo (cualquiera sea la forma en que se continúe la traza de violar la propiedad).

Una propiedad P es de safety si toda palabra en su complemento tiene un “prefijo malo”

Es decir:

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de safety si cumple

$$\forall \sigma : \sigma \notin P \Rightarrow \exists i \geq 0 : \forall \beta : \sigma[..i]\beta \notin P$$

o equivalentemente:

donde $\sigma[..i]$ denota el prefijo i -ésimo de σ .

Formalización de propiedades safety

Observar:

- Si una traza viola una propiedad de safety, lo hace en un “instante” finito.
- Si un prefijo de una traza (infinita) viola una propiedad de safety, no hay forma de remediarlo (cualquiera sea la forma en que se continúe, la traza no dejará de violar la propiedad).

Es decir:

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de safety si cumple

$$\forall \sigma : \sigma \notin P \Rightarrow \exists i \geq 0 : \forall \beta : \sigma[..i]\beta \notin P$$

o equivalentemente:

$$\forall \sigma : \forall i \geq 0 : \exists \beta : \sigma[..i]\beta \in P \Rightarrow \sigma \in P$$

donde $\sigma[..i]$ denota el prefijo i -ésimo de σ .

Formalización de propiedades safety

Observar:

- Si una traza viola una propiedad de safety, lo hace en un “instante” finito.
- Si un prefijo de una traza (infinita) viola una propiedad de safety, no hay forma de remediarlo (cualquiera sea la forma en que se continúe, la traza no dejará de violar la propiedad).

Es decir:

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de safety si

$$\forall \sigma : \sigma \notin P \Rightarrow \exists i \geq 0 : \forall \beta$$

Una propiedad P es de safety si toda traza infinita que se pueda “aproximar” finitamente con trazas de P , también está en P

o equivalentemente:

$$\forall \sigma : \forall i \geq 0 : \exists \beta : \sigma[..i]\beta \in P \Rightarrow \sigma \in P$$

donde $\sigma[..i]$ denota el prefijo i -ésimo de σ .

Formalización de propiedades safety

Ejemplos:

$$\left((\text{make} + \varepsilon) (\text{ready} + \text{use}) \right)^\omega$$

○ Complemento:

$$(\text{make} + \text{ready} + \text{use})^* \text{make make} (\text{make} + \text{ready} + \text{use})^\omega$$



Prefijos malos

Formalización de propiedades safety

Ejemplos:

$$\left((\text{make} + \varepsilon) (\text{ready} + \text{use}) \right)^\omega$$

○ Complemento:

$$(\text{make} + \text{ready} + \text{use})^* \text{make make} (\text{make} + \text{ready} + \text{use})^\omega$$

Prefijos malos

$$\left((\text{make} + \text{ready})^* (\text{use ready}^* (\text{use} + \varepsilon) \text{ready}^* \text{make})^* \right)^\omega$$

Sólo se puede usar hasta dos productos sin que antes se haga otro

Formalización de propiedades safety

Ejemplos:

$$\left((\text{make} + \varepsilon) (\text{ready} + \text{use}) \right)^\omega$$

○ Complemento:

$$(\text{make} + \text{ready} + \text{use})^* \text{make make} (\text{make} + \text{ready} + \text{use})^\omega$$

Prefijos malos

$$\left((\text{make} + \text{ready})^* (\text{use ready}^* (\text{use} + \varepsilon) \text{ready}^* \text{make})^* \right)^\omega$$

○ Complemento:

$$(\text{make} + \text{ready} + \text{use})^* (\text{use ready}^* \text{use ready}^* \text{use}) (\text{make} + \text{ready} + \text{use})^\omega$$

Prefijos malos

Formalización de propiedades liveness

Observar:

- Ningún fragmento de traza (finito) viola una propiedad de liveness: Si el evento que se esperaba que ocurriera aún no lo hizo, puede suceder más adelante.

Es decir:

Un conjunto de trazas $P \subseteq \Sigma^\omega$ es una propiedad de liveness si cumple

$$\forall \alpha \in \Sigma^* : \exists \beta \in \Sigma^\omega : \alpha\beta \in P$$

Es decir, una propiedad P es de liveness si toda palabra en su complemento **NO** tiene un "prefijo malo"

Formalización de propiedades liveness

Ejemplo:

$(\text{make} + \text{ready})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

En algún
momento se puede
consumir

Formalización de propiedades liveness

Ejemplo:

$(\text{make} + \text{ready})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

es equivalente a:

$(\text{make} + \text{ready} + \text{use})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

Todos los
posibles prefijos
aparecen en la
propiedad

Formalización de propiedades liveness

Ejemplo:

$(\text{make} + \text{ready})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

es equivalente a:

$(\text{make} + \text{ready} + \text{use})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

Todos los posibles prefijos aparecen en la propiedad

$((\text{ready} + \text{use})^* \text{make})^\omega$

La producción se realiza indefinidamente (se producen infinitos "make" a lo largo de la ejecución)

Formalización de propiedades liveness

Ejemplo:

$(\text{make} + \text{ready})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

es equivalente a:

$(\text{make} + \text{ready} + \text{use})^* \text{use} (\text{make} + \text{ready} + \text{use})^\omega$

Todos los posibles prefijos aparecen en la propiedad

$((\text{ready} + \text{use})^* \text{make})^\omega$

es equivalente a:

$(\text{ready} + \text{use} + \text{make})^* ((\text{ready} + \text{use})^* \text{make})^\omega$

Todos los posibles prefijos aparecen en la propiedad

Safety, liveness y todas las demás

Teorema [Alpern & Schneider 85]:

Toda propiedad puede escribirse como la intersección de una propiedad de safety y una de liveness.

suponer que
 $\Sigma = \{a, b, c\}$

Ejemplo: *a* until *b*

algún evento *b* ocurre inmediatamente precedido por una serie ininterrumpida de eventos *a* (léase: "*a* ocurre hasta que ocurre *b*")

$$a^* b \Sigma^\omega = (a^* b \Sigma^\omega + a^\omega) \cap (\Sigma^* b \Sigma^\omega)$$

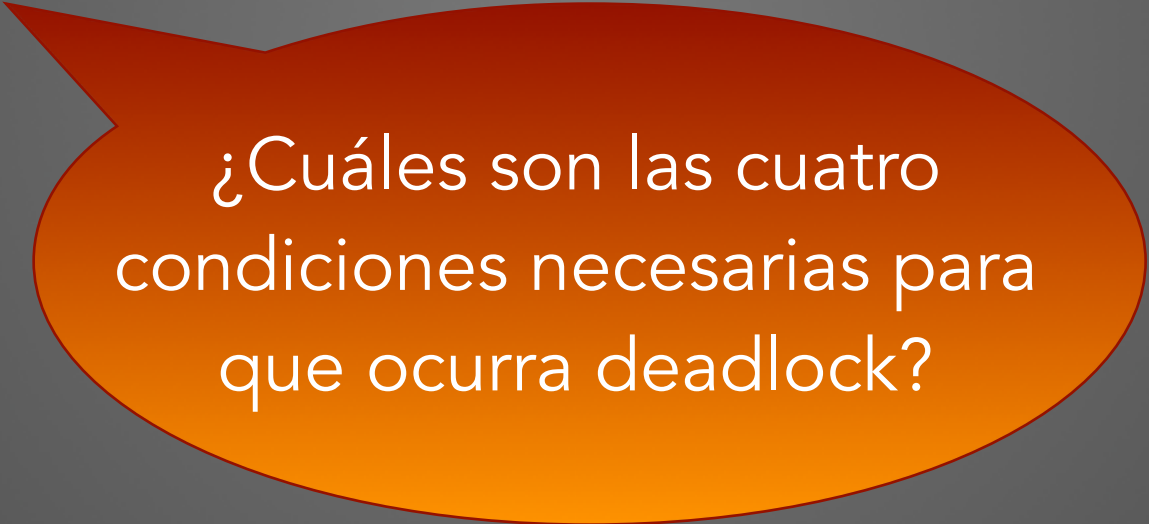
A la izquierda de la intersección se da la propiedad safety y a la derecha, la de liveness.

(¿Qué significa cada una de estas propiedades?)

Análisis de propiedades en FSP

Los distintos tipos de propiedades en FSP se verifican y/o modelan de manera distinta.

Deadlock



¿Cuáles son las cuatro condiciones necesarias para que ocurra deadlock?

Análisis de propiedades en FSP

Los distintos tipos de propiedades en FSP se verifican y/o modelan de manera distinta.

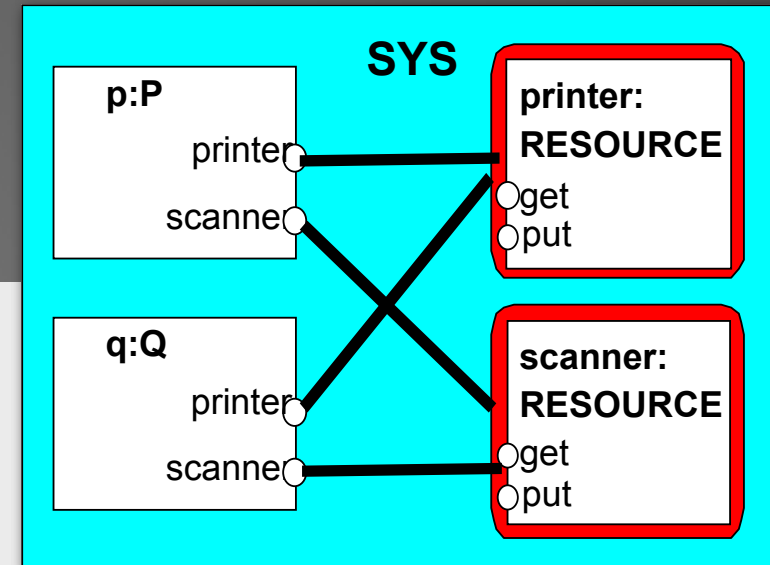
Deadlock

```
RESOURCE = (get -> put -> RESOURCE).
```

```
P = (printer.get -> scanner.get -> copy  
    -> printer.put -> scanner.put -> P).
```

```
Q = (scanner.get -> printer.get -> copy  
    -> printer.put -> scanner.put -> Q).
```

```
||SYS = (p:P || q:Q  
    || {p,q}::printer:RESOURCE  
    || {p,q}::scanner:RESOURCE  
    ).
```



Análisis de propiedades en FSP

Los distintos
modelan de m

LTSA puede chequear
deadlock. Lo hace mediante una
búsqueda exhaustiva.

se verifican y/o

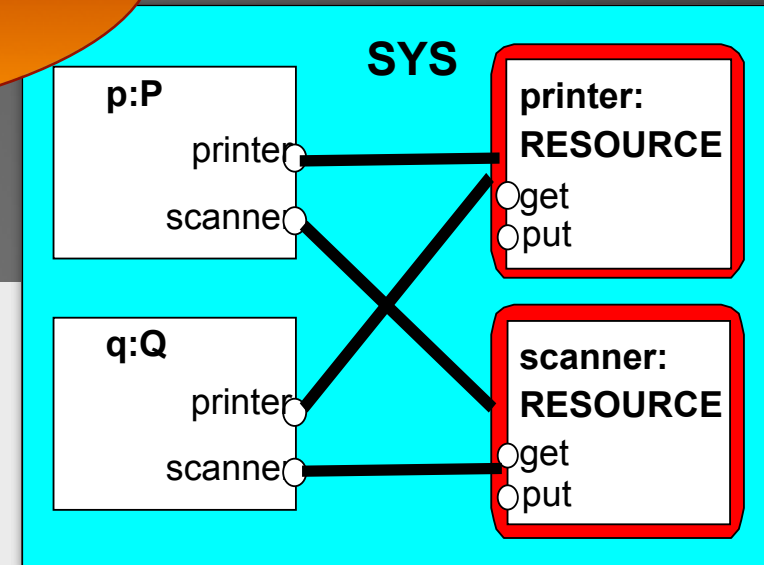
Deadlock

```
RESOURCE = (get -> put -> RESOURCE).
```

```
P = (printer.get -> scanner.get -> copy  
    -> printer.put -> scanner.put -> P).
```

```
Q = (scanner.get -> printer.get -> copy  
    -> printer.put -> scanner.put -> Q).
```

```
||SYS = (p:P || q:Q  
    || {p,q}::printer:RESOURCE  
    || {p,q}::scanner:RESOURCE  
    ).
```



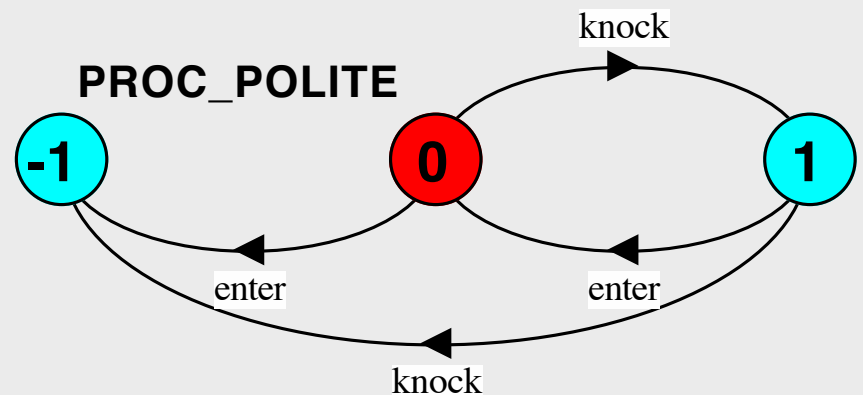
Análisis de propiedades en FSP

Safety

En FSP podemos expresar las propiedades de safety mediante procesos haciendo uso del estado de **ERROR** para indicar cuáles son las trazas que violan esta propiedad (i.e. los prefijos malos).

Ejemplo: Una persona educada golpea sólo una vez antes de entrar y no entra sin golpear.

```
PROC_POLITE = (knock->(enter->PROC_POLITE
                    | knock->ERROR
                )
              | enter->ERROR
            ).
```



Toda ejecución que lleva a ERROR es un "prefijo malo"

Análisis de propiedades en FSP

Safety (cont.)

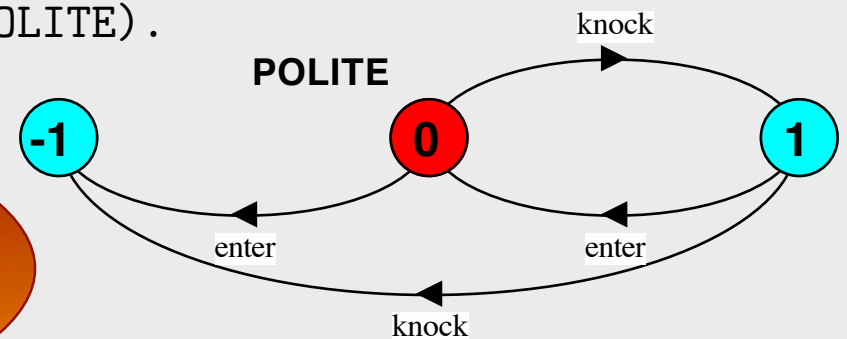
FSP provee una construcción que nos permite obviar el agregado “a mano” de los estados de error:

```
property N = P
```

indica que los eventos correspondientes al alfabeto de **P** deben ocurrir respetando el patrón de ocurrencias definido por **P** (toda ocurrencia no definida por **P** lleva al estado de **ERROR**).

En el ejemplo anterior:

```
property POLITE = (knock -> enter -> POLITE).
```



La declaración de un proceso **P** como **property** determina que la omisión de una acción en un estado de **P** interpretado como **proceso** induce una transición errónea en **P** como **propiedad**.

Análisis de propiedades en FSP

Safety (cont.)

Ejemplo

```
const Max = 3
range Int = 0..Max

SEMAPHORE(N=0) = SEMA[N] ,
SEMA[v:Int]    = (up->SEMA[v+1]
                  | when(v>0) down->SEMA[v-1]
                  ) .

LOOP = (mutex.down->enter->exit->mutex.up->LOOP) .

||SEMADEMO = (p[1..3]:LOOP
             || {p[1..3]}::mutex:SEMAPHORE(1)) .
```

Análisis de propiedades en FSP

Safety (cont.)

Ejemplo

```
const Max = 3
range Int = 0..Max
```

```
SEMAPHORE(N=0) = SEMA[N],
SEMA[v:Int]    = (up->SEMA[v+1]
                  |when(v>0) down->SEMA[v-1]
                  ).
```

```
LOOP = (mutex.down->enter->exit->mutex.up->LOOP).
```

```
||SEMADEMO = (p[1..3]:LOOP
              || {p[1..3]}::mutex:SEMAPHORE(1)).
```

```
property MUTEX
= (p[i:1..3].enter->p[i].exit->MUTEX).
```

```
||CHECK = (SEMADEMO || MUTEX).
```



Propiedad de
exclusión mututua

Análisis de propiedades en FSP

Safety (cont.)

Ejemplo

```
const Max = 3
range Int = 0..Max
```

```
SEMAPHORE(N=0) = SEMA[N],
SEMA[v:Int]    = (up->SEMA[v+1]
                  |when(v>0) down->SEMA[v-1]
                  ).
```

```
LOOP = (mutex.down->enter->exit->mutex.up->LOOP)
```

```
||SEMADEMO = (p[1..3]:LOOP
              || {p[1..3]}::mutex:SEMAPHORE(1)).
```

```
property MUTEX
= (p[i:1..3].enter->p[i].exit->MUTEX).
```

```
||CHECK = (SEMADEMO || MUTEX).
```

LTSA puede chequear safety.
También lo hace mediante
búsqueda exhaustiva.

Verificar cambiando
la inicialización del
semáforo a 2.

Propiedad de
exclusión mututua

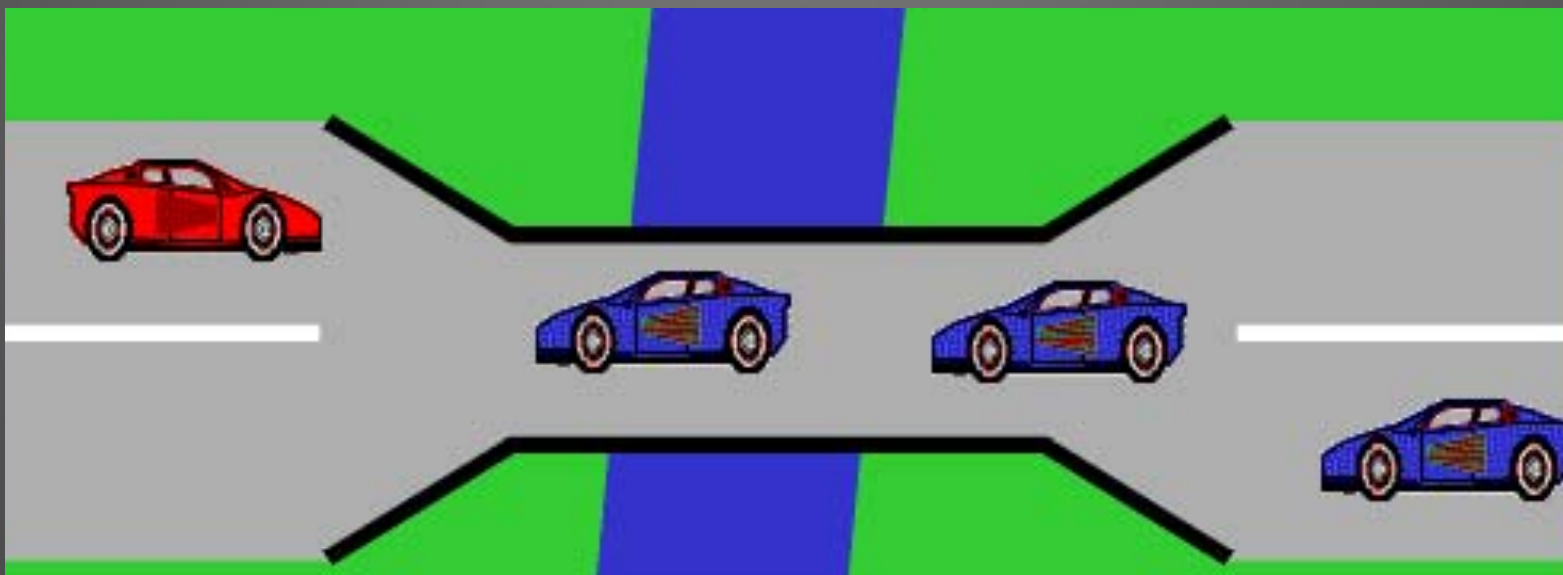
Análisis de propiedades en FSP

Safety (cont.)

Ejemplo: Un puente de una sola vía

Un puente sobre un río tiene el ancho suficiente para permitir tráfico en sólo una dirección. Por consiguiente, los autos sólo pueden cruzar el puente concurrentemente si viajan en la misma dirección.

Cuando dos autos que viajan en direcciones opuestas ingresan al puente al mismo tiempo ocurre una **violación de seguridad**.



Ejemplo: Un puente de una sola vía

```
const N = 3 // number of each type of car
range T = 0..N // type of car count
range ID= 1..N // car identities
```

```
CAR = (enter->exit->CAR).
```

```
/* cars may not overtake each other */
NOPASS1    = C[1],
C[i:ID]    = ([i].enter -> C[i%N+1]).
```

```
NOPASS2    = C[1],
C[i:ID]    = ([i].exit -> C[i%N+1]).
```

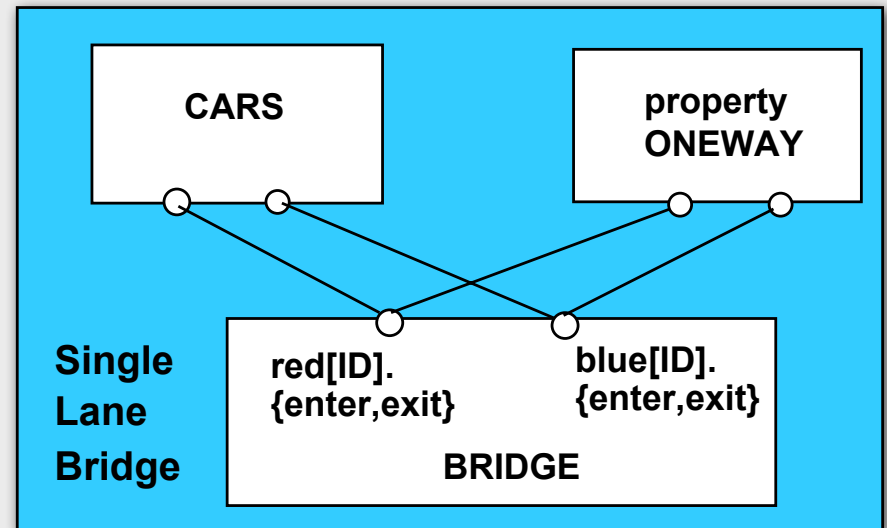
```
||CONVOY = ([ID]:CAR || NOPASS1 || NOPASS2).
```

```
||CARS = (red:CONVOY || blue:CONVOY).
```

```
property ONEWAY = ( red[ID].enter -> RED[1]
                    | blue[ID].enter -> BLUE[1]
                    ),
RED[i:ID] = (red[ID].enter -> RED[i+1]
             |when(i==1)red[ID].exit -> ONEWAY
             |when( i>1)red[ID].exit -> RED[i-1]
             ),
BLUE[i:ID] = (blue[ID].enter -> BLUE[i+1]
              |when(i==1)blue[ID].exit -> ONEWAY
              |when( i>1)blue[ID].exit -> BLUE[i-1]
              ).
```

```
BRIDGE = BRIDGE[0][0],
BRIDGE[nr:T][nb:T] =
    (when (nb==0)
      red[ID].enter -> BRIDGE[nr+1][nb]
    |red[ID].exit   -> BRIDGE[nr-1][nb]
    |when (nr==0)
      blue[ID].enter -> BRIDGE[nr][nb+1]
    |blue[ID].exit   -> BRIDGE[nr][nb-1]
    ).
```

```
||SingleLaneBridge = (CARS || BRIDGE || ONEWAY ).
```



Análisis de propiedades en FSP

Liveness

LTSA solo puede manejar un conjunto reducido de propiedades de liveness:

```
progress N = { a1, a2, ... }
```

Esto indica que, en toda ejecución fuertemente equitativa (strongly fair), alguna de las acciones **a1**, **a2**,... debe ejecutarse infinitas veces

Ejemplo:

```
COIN = ( toss -> heads -> COIN  
        | toss -> tails -> COIN ).  
progress HEADS = {heads}
```


Análisis de progreso

LTSA puede chequear progreso. En este caso, el algoritmo se basa en la búsqueda de ciclos que atrapen las acciones en N

Liveness

LTSA solo puede manejar un concepto de progreso llamado liveness:

```
progress N = { a1, a2, ... }
```

Esto indica que, en toda ejecución fuertemente equitativa (strongly fair), alguna de las acciones **a1**, **a2**,... debe ejecutarse infinitas veces

Ejemplo:

```
COIN = ( toss -> heads -> COIN
        | toss -> tails -> COIN ).
progress HEADS = {heads}
```

Análisis de progreso

Liveness

LTSA solo puede manejar un concepto de progreso llamado liveness:

```
progress N = { a1, a2, ... }
```

Esto indica que, en toda ejecución fuerte, alguna de las acciones **a1**, **a2**,... debe ejecutarse.

Ejemplo:

```
COIN = ( toss -> heads -> COIN
        | toss -> tails -> COIN ).
progress HEADS = {heads}
```

LTSA puede chequear progreso. En este caso, el algoritmo se basa en la búsqueda de ciclos que atrapen las acciones en N

Explicación: LTSA verifica bajo la suposición de **strong fairness**. Sin esta suposición, la propiedad es falsa.

LTSA la verifica verdadera

Análisis de progreso

Liveness

LTSA solo puede manejar un conjunto de acciones de progreso (liveness):

```
progress N = { a1, a2, ... }
```

Esto indica que, en toda ejecución fuerte, alguna de las acciones **a1**, **a2**,... debe ejecutarse.

Ejemplo:

```
COIN = ( toss -> heads -> COIN
        | toss -> tails -> COIN ).
progress HEADS = {heads}
```

Ejemplo: para el puente de una sola vía

```
progress BLUECROSS = {blue[ID].enter}
progress REDCROSS = {red[ID].enter}
```

LTSA puede chequear progreso. En este caso, el algoritmo se basa en la búsqueda de ciclos que atrapen las acciones en N

Explicación: LTSA verifica bajo la suposición de **strong fairness**. Sin esta suposición, la propiedad es falsa.

LTSA la verifica verdadera

LTSA las verifica correctas!! :-(

Prioridades entre acciones

LTSA asume en general que la elección es equitativa (fair), por eso no reporta error de progreso en el ejemplo de la moneda o en el puente de una sola vía.

Sin embargo, sabemos que es posible que los autos azules (o los rojos) acaparen el puente haciendo que los rojos (o los azules) esperen por siempre.

Entonces, para detectar problemas de progreso debemos imponer políticas de scheduling en las acciones que modele la situación en la que el puente (o cualquier otro sistema en general) es sometido a usos extremos.

Operador de alta prioridad: $P || C = (P || Q) \ll \{a_1, \dots, a_n\}$ especifica una composición en la cual las acciones $a_1, \dots, a_n$ tienen mayor prioridad que cualquier otra acción en $P || Q$, incluyendo τ .

Operador de baja prioridad: $P || C = (P || Q) \gg \{a_1, \dots, a_n\}$ especifica una composición en la cual las acciones $a_1, \dots, a_n$ tienen menos prioridad que cualquier otra acción en $P || Q$, incluyendo τ .

Análisis de propiedades en FSP

Liveness (cont.)

Ejemplo: Siguiendo con el puente de una sola vía deseamos modelar que el puente se encuentra congestionado.

Para ello daremos menor prioridad a los eventos de salida del puente. De esta manera se prioriza a la entrada sobre la salida forzando la congestión del puente:

```
|| CongestedBridge =  
    SingleLaneBridge >> {red[ID].exit, blue[ID].exit}.
```

Al verificar las propiedades de progreso, LTSA nos indicará ahora una situación de error.

Análisis de propiedades en FSP

Liveness (cont.)

Ejemplo: Siguiendo con el puente de ejemplo, vamos a modelar que el puente se encue

Para ello daremos menor prioridad a los procesos de cada puente. De esta manera se puede provocar la congestión del puente.

Otros model checkers permiten elegir si se desea realizar la verificación bajo **fairness** o no.

```
|| CongestedBridge =  
    SingleLaneBridge >> {red[ID].exit, blue[ID].exit}.
```

Al verificar las propiedades de progreso, LTSA nos indicará ahora una situación de error.

Ingeniería del Software II

5 - Lógica Temporal Lineal

Limitaciones de los métodos vistos hasta ahora para representar propiedades

Los métodos vistos hasta ahora están orientados a la representación del comportamiento y no a las propiedades mismas.

En particular:

- FSP provee una sintaxis para representar el comportamiento de los procesos y
- las expresiones ω -regulares proveen un lenguaje para representar el conjunto de trazas de interés.

Limitaciones de los métodos vistos hasta ahora para representar propiedades

Ejemplo: para representar la propiedad:

“Siempre que llovió, paró”

utilizando una expresión ω -regular, uno debe pensar en qué forma tienen las trazas que cumplen esa propiedad:

$$\left((\Sigma \setminus \{\text{llove}\}) + (\text{llove} \Sigma^* (\neg \text{llove})) \right)^\omega$$

CAIDADO:
notación sugestiva

Ya hemos experimentado razonamientos semejantes al escribir propiedades en FSP. (¿Cómo se representarían estas propiedades en FSP?)

Necesitamos una lógica que exprese directamente este tipo de propiedades.

Limitaciones de los métodos para representar p

Alternativamente

$$\left((\Sigma \setminus \{\text{lloveve}\})^* (\text{lloveve} \Sigma^* (\neg \text{lloveve}))^* \right)^\omega$$

Ejemplo: para representar la propiedad:

“Siempre que llovió, paró”

utilizando una expresión ω -regular, uno debe pensar en qué forma tienen las trazas que cumplen esa propiedad:

$$\left((\Sigma \setminus \{\text{lloveve}\}) + (\text{lloveve} \Sigma^* (\neg \text{lloveve})) \right)^\omega$$

CAIDADO:
notación sugestiva

Ya hemos experimentado razonamientos semejantes al escribir propiedades en FSP. (¿Cómo se representarían estas propiedades en FSP?)

Necesitamos una lógica que exprese directamente este tipo de propiedades.

Limitaciones de las lógicas usuales

Lógica proposicional: Es incapaz de reflejar el cambio de valor de verdad de las proposiciones según transcurra el tiempo.

Ejemplo: "Siempre que llovió, paro"

1er. intento: $\text{llueve} \rightarrow \neg \text{llueve}$

Esta propiedad sólo es verdadera si no llueve :-(

2do. intento: $\text{primero_llueve} \rightarrow \text{luego_para}$

Dos inconvenientes:

1. Disocia los conceptos de llover y dejar de llover.
2. ¿Cómo diferenciamos que esto ocurre siempre o que esto ocurre sólo una vez?

Otros ejemplos podrán llevar a otros problemas.

En otras palabras, la lógica proposicional carece de suficiente expresividad.

Limitaciones de las lógicas usuales

Lógica de primer orden: Contrariamente, la lógica de primer orden es demasiado expresiva.

Por un lado presenta un lenguaje demasiado complejo para representar propiedades temporales. La propiedad “Siempre que llovió, paró” se podría representar como:

$$\forall t \in \text{Tiempo} : \text{llueve}(t) \rightarrow \exists t' \in \text{Tiempo} : (t \leq t') \wedge \neg \text{llueve}(t')$$

donde **Tiempo** pueden ser, por ejemplo, los enteros no negativos o los reales no negativos.

Por otro lado, a diferencia de las lógicas proposicionales, satisfactibilidad en lógica de primer orden es indecidible y por consiguiente no permite en general el cálculo automático de si una fórmula de primer orden se satisface en un sistema dado.

Debemos encontrar algo intermedio.

Lógicas modales

En principio, la lógica modal estudia la formalización del razonamiento que involucra aserciones con modalidades, tales como “necesariamente” o “posiblemente”.

Uno de los enfoques principales consiste en complementar los operadores “clásicos” con los operadores modales:

- que expresa “necesidad”, y
- ◇ que expresa “posibilidad”.

Estos operadores modales tienen otras interpretaciones muy útiles, por ej:

- Lógica deóntica:
 - significa “obligatoriamente”
 - ◇ significa “está permitido”
- Lógica temporal:
 - significa “siempre en el futuro”
 - ◇ significa “en el algún momento en el futuro”

Lógica Temporal

Las lógicas temporales son variantes de la lógica modal que conciernen al razonamiento sobre la relación temporal de eventos.

Existen muchos tipos de lógicas temporales. Sus diferencias radican en el modelo temporal, i.e., en como cada una puede observar el paso del tiempo. Por ejemplo:

- el tiempo transcurrido entre eventos es observable
- el tiempo transcurrido entre eventos no es observable, sólo el orden temporal de los eventos.
- los instantes de tiempo son numerables
- los instantes de tiempo constituyen un conjunto denso
- el transcurso temporal esta organizado linealmente (como una sola ejecución)
- el transcurso temporal se ramifica (puede observar todas -o alguna de- las posibles ejecuciones a partir de cualquier instante)

Algunas Lógicas Temporales

Las siguientes lógicas temporales son comúnmente utilizadas para especificar las propiedades a verificar por la herramientas de model checking:

- Lógica temporal lineal proposicional (LTL o PLTL)
- Lógica de computaciones ramificadas (CTL)
- Lógica de computaciones ramificadas y temporizadas (TCTL)

LTL tiene como modelo de tiempo un conjunto numerable de instantes organizados linealmente pero no puede observar el tiempo transcurrido.

CTL tiene como modelo de tiempo un conjunto numerable de instantes organizados de forma ramificada y tampoco puede observar el tiempo transcurrido.

TCTL tiene como modelo de tiempo un conjunto numerable o denso de instantes organizados de forma ramificada y puede observar (contar) el tiempo transcurrido entre dos eventos.

Nos concentraremos en el uso de LTL.

LTL (Lógica Temporal Lineal)

Las modalidades de LTL:

Los siguientes son algunos de los operadores modales que admite la lógica LTL.

$\Box \phi$: "siempre en el futuro sucede ϕ "

$\Diamond \phi$: "en algún instante futuro sucede ϕ "

$\bigcirc \phi$: "en el siguiente instante sucede ϕ "

$\phi \mathbf{U} \psi$: " ϕ sucede hasta que suceda ψ "

$\phi \mathbf{WU} \psi$: " ϕ sucede siempre o hasta que suceda ψ "

En realidad, LTL sólo provee los operadores modales $\mathbf{U}$ y $\bigcirc$.
Todos los otros son derivados de estos dos.

Sintaxis de LTL

Sea $\mathcal{PA}$ el conjunto de todas las proposiciones atómicas. Luego

cualquier proposición atómica $p \in \mathcal{PA}$ es una fórmula LTL.

Si ϕ y ψ son fórmulas LTL,

- cualquier fórmula construida con los conectivos lógicos proposicionales

$$\neg\phi \quad \phi \wedge \psi \quad \phi \vee \psi \quad \phi \rightarrow \psi \quad \dots$$

son fórmulas LTL, y

- cualquier fórmula construida con los operadores temporales

$$\bigcirc\phi \quad \phi \mathbf{U} \psi \quad \Box\phi \quad \Diamond\phi \quad \dots$$

son fórmulas LTL.

Semántica de LTL

Recordar: Dado una teoría proposicional (i.e., un conjunto de fórmulas proposicionales que incluye los axiomas de la lógica proposicional), todo modelo de ésta puede verse como un subconjunto

$$A \subseteq \mathcal{PA}$$

de proposiciones atómicas donde las fórmulas de ese conjunto se hacen verdaderos.

En particular, diremos que

una fórmula proposicional ϕ se satisface en $A \subseteq \mathcal{PA}$ si $A \models \phi$.

Esta última noción se define inductivamente como sigue:

$A \models p$	sii	$p \in A$
$A \models \neg\phi$	sii	$A \not\models \phi$
$A \models \phi \wedge \psi$	sii	$A \models \phi$ y $A \models \psi$

Semántica de LTL (cont.)

Siguiendo el concepto anterior, y dado que LTL especifica el cambio de la validez de las proposiciones acorde cambia el tiempo, un modelo de una fórmula LTL sera un conjunto de secuencias infinitas de modelos de teorías proposicionales. Es decir:

un modelo de una fórmula LTL es un subconjunto de $(2^{\mathcal{PA}})^{\omega}$

Denotaremos que:

una fórmula LTL ϕ se satisface en una traza σ ($\sigma \in (2^{\mathcal{PA}})^{\omega}$) por $\sigma \models \phi$.

Esta última noción se define inductivamente como sigue:

Semántica de LTL (cont.)

$\sigma \models p$ sii $p \in \sigma(0)$, para todo $p \in PA$

$\sigma \models \neg\phi$ sii $\sigma \not\models \phi$

$\sigma \models \phi \wedge \psi$ sii $\sigma \models \phi$ y $\sigma \models \psi$

$\sigma \models \phi \vee \psi$ sii $\sigma \models \phi$ o $\sigma \models \psi$

$\sigma \models \bigcirc \phi$ sii $\sigma[1..] \models \phi$

$\sigma \models \Diamond \phi$ sii $\exists j \geq 0 : \sigma[j..] \models \phi$

$\sigma \models \Box \phi$ sii $\forall j \geq 0 : \sigma[j..] \models \phi$

$\sigma \models \phi \mathsf{U} \psi$ sii $\exists j \geq 0 : \sigma[j..] \models \psi$ y $\forall i : 0 \leq i < j : \sigma[i..] \models \phi$

$\sigma[i..]$ denota el sufijo i -ésimo de σ

Semántica de LTL (cont.)

El lenguaje de una fórmula LTL se define como el conjunto de todas las trazas que ésta satisface, i.e.,

$$\mathcal{L}(\phi) = \{\sigma \in (2^{\mathcal{PA}})^{\omega} \mid \sigma \models \phi\}$$

Teorema: Los lenguajes ω -regulares son estrictamente más expresivos que la lógica LTL, i.e.,

- para toda fórmula LTL ϕ , existe un lenguaje ω -regular $L \subseteq (2^{\mathcal{PA}})^{\omega}$ tal que $L = \mathcal{L}(\phi)$.
- por otro lado, el lenguaje $((\emptyset + \{p\}) \{p\})^{\omega}$ no es expresable con una fórmula LTL.

Operadores básicos en LTL

Los operadores básicos en LTL son:

$\neg$ $\wedge$ $\bigcirc$ $\mathbf{U}$

El resto de las operaciones se derivan de ellos:

- $\vee, \rightarrow, \leftrightarrow, \dots$ se obtienen de la forma usual.
- $\diamond \phi \equiv \text{true} \mathbf{U} \phi$
- $\Box \phi \equiv \neg \diamond \neg \phi$
- $\phi \mathbf{WU} \psi \equiv (\phi \mathbf{U} \psi) \vee \Box \phi$

Algunas leyes

$$\neg \Box \phi \equiv \Diamond \neg \phi$$

$$\Diamond \phi \equiv \text{true} \mathbf{U} \phi$$

$$\neg \bigcirc \phi \equiv \bigcirc \neg \phi$$

$$\Box \phi \equiv \neg \Diamond \neg \phi$$

$$\Box \Box \phi \equiv \Box \phi$$

$$\Diamond \phi \equiv \phi \vee \bigcirc \Diamond \phi$$

$$\Diamond \Diamond \phi \equiv \Diamond \phi$$

$$\Box \phi \equiv \phi \wedge \bigcirc \Box \phi$$

$$\phi \mathbf{U} (\phi \mathbf{U} \psi) \equiv \phi \mathbf{U} \psi$$

$$\phi \mathbf{U} \psi \equiv \psi \vee (\phi \wedge \bigcirc (\phi \mathbf{U} \psi))$$

$$(\phi \mathbf{U} \psi) \mathbf{U} \psi \equiv \phi \mathbf{U} \psi$$

$$(\Diamond \phi) \vee (\Diamond \psi) \equiv \Diamond (\phi \vee \psi)$$

$$\Diamond \Box \Diamond \phi \equiv \Box \Diamond \phi$$

$$(\Box \phi) \wedge (\Box \psi) \equiv \Box (\phi \wedge \psi)$$

$$\Box \Diamond \Box \phi \equiv \Diamond \Box \phi$$

$$\bigcirc (\phi \mathbf{U} \psi) \equiv (\bigcirc \phi) \mathbf{U} (\bigcirc \psi)$$

Especificación de propiedades con LTL: Safety

Las propiedades safety usualmente tienen la forma:

$\Box \phi$ Siempre ocurre ϕ (o Nunca ocurre $\neg\phi$)

Ejemplos:

- Exclusión mutua.

$$\Box(\neg crit_1 \vee \neg crit_2)$$

- Ausencia de deadlock en el problema de los filósofos

$$\Box \neg \left(\bigwedge_{i=0}^n espera_fil_i \wedge \bigwedge_{j=0}^n ocupado_tenedor_j \right)$$

- Una persona educada no entra sin golpear.

$$\neg(\neg knock \cup enter)$$

Especificación de propiedades con LTL: Safety

Las propiedades safety usualmente tienen la forma:

$\Box \phi$ Siempre ocurre ϕ (o Nunca ocurre $\neg\phi$)

Ejemplos:

- Exclusión mutua.

$$\Box(\neg crit_1 \vee \neg crit_2)$$

- Ausencia de deadlock en el problema de los filósofos

$$\Box \neg \left(\bigwedge_{i=0}^n espera_fil_i \wedge \bigwedge_{j=0}^n \neg enter_j \right)$$

que es equivalente a

- Una persona educada no entra sin ser invitada: $\neg enter \text{ WU } (knock \wedge \neg enter)$

$$\neg(\neg knock \text{ U } enter)$$

Especificación de propiedades con LTL: Liveness

Las propiedades liveness usualmente tienen la forma:

$\diamond \phi$ en algún momento en el futuro ocurre ϕ

Ejemplos:

- Un proceso dado termina en algún momento en el futuro.

$\diamond \textit{fin_proc}$

- El auto rojo nro. 0 ingresa frecuentemente al puente (progreso).

$\square \diamond \textit{red.0.enter}$

- Los procesos ingresan frecuentemente a su región crítica.

Especificación de propiedades con LTL: Liveness

Las propiedades liveness usualmente tienen la forma:

$\Diamond \phi$ en algún momento en el futuro ocurre ϕ

Ejemplos:

- Un proceso dado termina en algún momento en el futuro.

$\Diamond \textit{fin_proc}$

- El auto rojo nro. 0 ingresa frecuentemente (progreso).

$\Box \Diamond \textit{red.0.enter}$

- Los procesos ingresan frecuentemente a su región crítica.

$\Box \Diamond \textit{crit}_1 \wedge \Box \Diamond \textit{crit}_2$

Recordar:

$\Diamond \Box \Diamond \phi \equiv \Box \Diamond \phi$

Especificación de propiedades con LTL: Fairness

“Bajo ciertas condiciones un evento ocurrirá de manera frecuente”

Hay diversas formas de fairness.

Progreso (o fairness incondicional)

“Un evento ϕ debe ocurrir de manera frecuente”

Ejemplo:

$\square \diamond red.0.enter$

$\square \diamond crit_1 \wedge \square \diamond crit_2$

Especificación de propiedades con LTL: Fairness

Weak fairness (fairness condicional débil)

“Siempre ocurre que cuando un evento permanece continuamente habilitado, entonces finalmente se ejecutará”

$$\Box(\Diamond \Box \textit{habilitado}(a) \rightarrow \Diamond \textit{ejecutar}(a))$$

Más generalmente, si ϕ y ψ son dos fórmulas, weak fairness podría escribirse

$$\Box(\Diamond \Box \phi \rightarrow \Diamond \psi)$$

Luego ϕ es la condición (débil) para la ocurrencia de ψ .

La fórmula anterior se puede escribir equivalentemente de las siguientes maneras:

$$\Diamond \Box \phi \rightarrow \Box \Diamond \psi$$

$$\Box(\Box \phi \rightarrow \Diamond \psi)$$

Especificación de propiedades con LTL: Fairness

Strong fairness (fairness condicional fuerte)

En muchos casos, weak fairness no es lo suficientemente fuerte. Consideremos, por ejemplo, el siguiente programa concurrente:

```
COIN = ( toss -> heads -> COIN
        | toss -> tails -> COIN ).
```

Una traza de ejecución para COIN, que no viola weak fairness, es la siguiente:

```
toss, heads, toss, heads, toss, heads, toss, heads, ...
```

Para evitar este tipo de trazas, debe considerarse un tipo más fuerte de fairness denominado **strong fairness**.

Especificación de propiedades con LTL: Fairness

Strong fairness (fairness condicional fuerte)

“Siempre ocurre que, si un evento está frecuentemente habilitado, entonces finalmente se ejecutará”

$$\Box(\Box \Diamond \textit{habilitado}(a) \rightarrow \Diamond \textit{ejecutar}(a))$$

Como antes, podemos escribir

$$\Box(\Box \Diamond \phi \rightarrow \Diamond \psi)$$

donde ϕ será la condición (fuerte) para la ocurrencia de ψ .

La fórmula anterior se puede escribir equivalentemente como

$$\Box \Diamond \phi \rightarrow \Box \Diamond \psi$$

Especificación de propiedades con LTL:

Otras propiedades

Respuesta: Cada vez que ocurre p se emite una respuesta q .

$$\Box(p \rightarrow \Diamond q)$$

Ejemplo: Todo mensaje enviado se recibe en algún momento:

$$\Box(\text{enviar}(m) \rightarrow \Diamond \text{recibir}(m))$$

Persistencia: A partir de algún momento q se hace invariante, (i.e., q persiste).

$$\Diamond \Box q$$

La siguiente es una forma condicional de persistencia

$$\Box(p \rightarrow \Box q)$$

Ejemplo: En un protocolo de elección de líder, el nodo electo debe permanecer líder:

$$\Box(\text{electo_nodo}_i \rightarrow \Box \text{líder_nodo}_i)$$